

WetInk Next

Технический аудит и план завершения движка

Исходный код: P0-P35 • состояние на 13 августа 2026

Статус

Сборка, JVM-тесты и Android lint проходят без ошибок. Последний crash Ribbon был исправлен: причиной оказался старый несовместимый shader layout. Проект уже имеет работающие проекты, слои, autosave, thumbnails, tile persistence, Undo/Redo, STAMP и начальную геометрическую RIBBON-ленту.

Проверки: compileDebugKotlin, testDebugUnitTest, lintDebug. Lint: 0 errors, 14 warnings.

Ключевой вывод

Основа сильная, но это ещё не финальный уровень Procreate. Ближайшие риски - незавершённая Ribbon-геометрия, синхронный GPU readback для Undo на 4K и слишком крупный EngineRenderer.

Приоритетный план

P0 - стабилизация GPU и памяти

- Оставить ровно один Ribbon shader layout и подробные проверки uniforms.
- Ввести бюджет RenderTarget: RGBA16F только для document targets, screen cache и preview - RGBA8.
- Создавать caches лениво и включать fallback, если память GPU не позволяет создать target.

P1 - закончить настоящую Ribbon-геометрию

- Выделить RibbonEmitter и убрать зависимость RIBBON от CapsuleEmitter/CapsuleStrokeRenderer.
- Сделать welded banks, BEVEL, AA fringe для ROUND join/cap, miter clamp, taper и closed-loop weld.
- Заменить ArrayList в mesh build на preallocated RibbonMeshWriter и добавить device тесты углов/петель.

P2 - убрать фриззы Undo

- Заменить синхронные before/after glReadPixels в StrokeCommitter на PBO + fence pipeline.
- Читать прямоугольники или полосы с byte budget на кадр, разрезать в tiles вне GL hot path.

P3 - общая история документа

- Сделать Undo командами для stroke, clear, add/remove/duplicate/move layer и layer properties.

P4 - рефакторинг EngineRenderer

- Выделить StrokeController, DocumentLayerController и FrameComposer. Цель - 700-900 строк.

P5 - довести двигатели кистей

- Добавить time-based airbrush emission и разделить falloff profiles: hard, soft, gaussian, airbrush.
- Wet/smudge начинать только после P1-P2: отдельные ping-pong pigment/wetness targets.

P6 - закрыть UI TODO

- Brush Studio, image import, selection, transform и adjustments должны получить рабочие callbacks и Undo.

Критерии готовности

- 20 последовательных запусков/закрывов на планшете без GL crash.
- RIBBON: straight, S-curve, 20°/90° corners, circle и closed circle без белых точек, spike и seam.
- На 4096×2160 нет ощутимой паузы на UP из-за synchronous glReadPixels.
- Undo/Redo точно восстанавливает края canvas и все операции со слоями.
- Preview и committed result совпадают визуально.
- compileDebugKotlin, testDebugUnitTest и lintDebug проходят.

Приложение содержит 169 включённых исходных файлов. Далее идёт полный нумерованный снимок кода.

1. app/build.gradle.kts

```
1  plugins {
2      id("com.android.application")
3      id("org.jetbrains.kotlin.android")
4      id("org.jetbrains.kotlin.plugin.compose")
5      id("org.jetbrains.kotlin.plugin.serialization")
6  }
7
8  android {
9      namespace = "com.wetinknext"
10     compileSdk = 36
11
12     defaultConfig {
13         applicationId = "com.wetinknext"
14         minSdk = 26
15         targetSdk = 36
16         versionCode = 1
17         versionName = "0.1.0"
18         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
19     }
20
21     buildTypes {
22         release {
23             isMinifyEnabled = false
24         }
25     }
26
27     compileOptions {
28         sourceCompatibility = JavaVersion.VERSION_17
29         targetCompatibility = JavaVersion.VERSION_17
30     }
31
32     kotlinOptions {
33         jvmTarget = "17"
34     }
35
36     buildFeatures {
37         compose = true
38         buildConfig = true
39     }
40 }
41
42
43 dependencies {
44     val composeBom = platform("androidx.compose:compose-bom:2024.09.03")
45     implementation(composeBom)
46     androidTestImplementation(composeBom)
47
48     implementation("androidx.core:core-ktx:1.13.1")
49     implementation("androidx.activity:activity-compose:1.9.2")
50     implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.7")
51     implementation("androidx.compose.ui:ui")
52     implementation("androidx.compose.ui:ui-tooling-preview")
53     implementation("androidx.compose.material3:material3")
54     implementation("androidx.compose.material:material-icons-extended")
55     implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.3")
56
57     testImplementation("junit:junit:4.13.2")
58
59     debugImplementation("androidx.compose.ui:ui-tooling")
60     debugImplementation("androidx.compose.ui:ui-test-manifest")
61 }
```

2. app/src/main/AndroidManifest.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
3      <application
4          android:allowBackup="true"
5          android:icon="@mipmap/ic_launcher"
6          android:roundIcon="@mipmap/ic_launcher_round"
7          android:label="@string/app_name"
8          android:supportRtl="true"
9          android:theme="@style/Theme.WetInkNext">
10         <activity
11             android:name=".app.MainActivity"
12             android:exported="true">
13             <intent-filter>
14                 <action android:name="android.intent.action.MAIN" />
15                 <category android:name="android.intent.category.LAUNCHER" />
16             </intent-filter>
17         </activity>
18     </application>
19 </manifest>
```

3. app/src/main/java/com/wetinknext/app/AppViewModel.kt

```
1 package com.wetinknext.app
2
3 import android.content.Context
4 import android.content.SharedPreferences
5 import androidx.compose.runtime.getValue
6 import androidx.compose.runtime.mutableStateOf
7 import androidx.compose.runtime.setValue
8 import androidx.lifecycle.ViewModel
9 import androidx.lifecycle.ViewModelProvider
10 import androidx.lifecycle.viewModelScope
11 import com.wetinknext.data.project.CreateProjectRequest
12 import com.wetinknext.data.project.AutosaveCoordinator
13 import com.wetinknext.data.project.FileProjectRepository
14 import com.wetinknext.data.project.ProjectRepository
15 import com.wetinknext.data.project.ProjectSummary
16 import com.wetinknext.data.project.TrashedProjectSummary
17 import com.wetinknext.domain.document.ProjectDocument
18 import com.wetinknext.engine.undo.TileCoord
19 import com.wetinknext.engine.core.ThumbnailCapture
20 import com.wetinknext.data.project.ThumbnailEncoder
21 import com.wetinknext.engine.thumbnail.ThumbnailBuildResult
22 import com.wetinknext.domain.document.LoadedWetInkProject
23 import java.io.File
24 import kotlinx.coroutines.launch
25 import kotlinx.coroutines.Dispatchers
26 import kotlinx.coroutines.withContext
27
28 sealed interface AppRoute {
29     data object Home : AppRoute
30     data class Editor(val projectId: String) : AppRoute
31 }
32
33 data class AppUiState(
34     val projects: List<ProjectSummary> = emptyList(),
35     val trashedProjects: List<TrashedProjectSummary> = emptyList(),
36     val isLoading: Boolean = true,
37     val recoveryProject: ProjectSummary? = null,
38     val errorMessage: String? = null,
39     val isDirty: Boolean = false,
40     val isSaving: Boolean = false,
41 )
42
43 /** Owns app navigation and project catalog state, never GL resources. */
44 class AppViewModel(
45     private val repository: ProjectRepository,
46     private val session: SharedPreferences,
47 ) : ViewModel() {
48     var route: AppRoute by mutableStateOf(AppRoute.Home)
49     private set
50     var uiState: AppUiState by mutableStateOf(AppUiState())
51     private set
52     var openedDocument: ProjectDocument? by mutableStateOf(null)
53     private set
54     var openedProject: LoadedWetInkProject? by mutableStateOf(null)
55     private set
56     private val autosaveCoordinator = AutosaveCoordinator(viewModelScope)
57     private var pendingSaveDocument: ProjectDocument? = null
58     private var pendingLayerPayloads: Map<Long, ByteArray> = emptyMap()
59     private var pendingTilesAcknowledgement: (() -> Unit)? = null
60     private var pendingThumbnailWebp: ByteArray? = null
61     private var pendingLayerPreviewsWebp: Map<Long, ByteArray> = emptyMap()
62     private var latestThumbnailSequence = 0L
63
64     init {
65         refreshProjects()
66     }
67
68     fun refreshProjects() {
69         viewModelScope.launch {
70             uiState = uiState.copy(isLoading = true, errorMessage = null)
71             runCatching { repository.list() to repository.listTrash() }
72                 .onSuccess { (projects, trashedProjects) ->
73                     val interruptedId = session.getString(PREF_ACTIVE_PROJECT_ID, null)
74                     val shouldRecover = session.getBoolean(PREF_EDITOR_SESSION_OPEN, false)
75                     uiState = AppUiState(
76                         projects = projects,
77                         trashedProjects = trashedProjects,
78                         isLoading = false,
79                         recoveryProject = if (shouldRecover) {
80                             projects.firstOrNull { it.id == interruptedId }
81                         } else {
82                             null
83                         },
84                     )
85                 }
86             .onFailure { error ->
87                 uiState = uiState.copy(isLoading = false, errorMessage = error.message)
88             }
89         }
90     }
```

```

91
92 fun createProject(request: CreateProjectRequest) {
93     viewModelScope.launch {
94         uiState = uiState.copy(isLoading = true, errorMessage = null)
95         runCatching {
96             val summary = repository.create(request)
97             summary to repository.openForEditor(summary.id)
98         }.onSuccess { (summary, loadedProject) ->
99             openedProject = loadedProject
100             openedDocument = loadedProject.document
101             route = AppRoute.Editor(summary.id)
102             markEditorSession(summary.id)
103             uiState = uiState.copy(
104                 projects = (uiState.projects + summary).sortedByDescending(ProjectSummary::updatedAt),
105                 isLoading = false,
106                 recoveryProject = null,
107             )
108         }.onFailure { error ->
109             uiState = uiState.copy(isLoading = false, errorMessage = error.message)
110         }
111     }
112 }
113
114 fun openProject(projectId: String) {
115     viewModelScope.launch {
116         uiState = uiState.copy(isLoading = true, errorMessage = null)
117         runCatching {
118             val recovery = repository.openRecovery(projectId)
119             if (recovery != null) {
120                 repository.save(recovery.document, recovery.assets.layerTiles)
121                 repository.clearRecovery(projectId)
122                 recovery
123             } else {
124                 repository.openForEditor(projectId)
125             }
126         }
127         .onSuccess { loadedProject ->
128             openedProject = loadedProject
129             openedDocument = loadedProject.document
130             route = AppRoute.Editor(projectId)
131             markEditorSession(projectId)
132             uiState = uiState.copy(isLoading = false, recoveryProject = null)
133         }
134         .onFailure { error ->
135             uiState = uiState.copy(isLoading = false, errorMessage = error.message)
136         }
137     }
138 }
139
140 fun renameProject(projectId: String, name: String) {
141     viewModelScope.launch {
142         runCatching { repository.rename(projectId, name) }
143         .onSuccess { refreshProjects() }
144         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
145     }
146 }
147
148 fun duplicateProject(projectId: String) {
149     viewModelScope.launch {
150         runCatching { repository.duplicate(projectId) }
151         .onSuccess { refreshProjects() }
152         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
153     }
154 }
155
156 fun moveProjectToTrash(projectId: String) {
157     viewModelScope.launch {
158         runCatching { repository.moveToTrash(projectId) }
159         .onSuccess { refreshProjects() }
160         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
161     }
162 }
163
164 fun restoreProject(projectId: String) {
165     viewModelScope.launch {
166         runCatching { repository.restore(projectId) }
167         .onSuccess { refreshProjects() }
168         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
169     }
170 }
171
172 fun deleteProjectForever(projectId: String) {
173     viewModelScope.launch {
174         runCatching { repository.deleteForever(projectId) }
175         .onSuccess { refreshProjects() }
176         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
177     }
178 }
179
180 fun dismissRecovery() {
181     session.edit().remove(PREF_ACTIVE_PROJECT_ID).putBoolean(PREF_EDITOR_SESSION_OPEN, false).apply()
182     uiState = uiState.copy(recoveryProject = null)

```

```

183     }
184
185     /** Coalesces document metadata changes from the GL thread. */
186     fun scheduleAutosave(document: ProjectDocument) {
187         if (document.id != openedDocument?.id) return
188         openedDocument = document
189         pendingSaveDocument = document
190         schedulePendingSave()
191     }
192
193     fun scheduleTileAutosave(
194         document: ProjectDocument,
195         payloads: Map<Long, ByteArray>,
196         dirty: Map<Long, Set<TileCoord>>,
197         onSave: () -> Unit,
198     ) {
199         if (document.id != openedDocument?.id) return
200         openedDocument = document
201         pendingSaveDocument = document
202         // LayerTileStore keeps tiles dirty until this acknowledgement. Therefore
203         // the newest snapshot always contains every cancelled autosave's tiles.
204         pendingLayerPayloads = payloads
205         pendingTilesAcknowledgement = onSave
206         schedulePendingSave()
207     }
208
209     fun submitThumbnail(image: ThumbnailCapture.Rgba) {
210         val document = openedDocument ?: return
211         val projectId = document.id
212         val sequence = ++latestThumbnailSequence
213         viewModelScope.launch {
214             val encoded = withContext(Dispatchers.Default) {
215                 ThumbnailEncoder.encodeWebp(image)
216             }
217             // Encoders may finish out of order. An older stroke must never
218             // replace the preview captured for a newer committed stroke.
219             if (sequence != latestThumbnailSequence) return@launch
220             if (projectId != openedDocument?.id) {
221                 // The editor may have just been closed while encoding was in
222                 // flight. Persist this last valid preview by itself instead of
223                 // dropping it and leaving the gallery with a stale card. Read
224                 // current metadata first: the captured callback must not
225                 // overwrite a later layer/property save with stale metadata.
226                 runCatching {
227                     val currentDocument = repository.open(projectId)
228                     repository.save(currentDocument, emptyMap(), encoded)
229                 }
230                 .onSuccess { refreshProjects() }
231                 .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
232                 return@launch
233             }
234             pendingThumbnailWebp = encoded
235             pendingSaveDocument = openedDocument
236             schedulePendingSave()
237         }
238     }
239
240     /**
241     * Receives files already encoded by ThumbnailBuildScheduler. Reading their
242     * bytes on Dispatchers.IO keeps Android's main thread free; autosave then
243     * publishes the previews together with document.json in one .wetink write.
244     */
245     fun submitThumbnailBuild(result: ThumbnailBuildResult) {
246         val document = openedDocument ?: return
247         val projectId = document.id
248         viewModelScope.launch {
249             val previews = withContext(Dispatchers.IO) {
250                 val project = result.projectPreview
251                 ?.takeIf(File::isFile)
252                 ?.readBytes()
253                 val layers = result.layerPreviews.mapNotNull { (layerId, file) ->
254                     file.takeIf(File::isFile)?.readBytes()?.let { layerId to it }
255                 }.toMap()
256                 project to layers
257             }
258             if (projectId != openedDocument?.id) return@launch
259
260             previews.first?.let { pendingThumbnailWebp = it }
261             if (previews.second.isNotEmpty()) {
262                 pendingLayerPreviewsWebp = pendingLayerPreviewsWebp + previews.second
263             }
264             pendingSaveDocument = openedDocument
265             schedulePendingSave()
266         }
267     }
268
269     /**
270     * One coalesced save owns document metadata, changed layer payloads and the
271     * latest canvas thumbnail. Restarting its debounce is intentional: it keeps
272     * those three representations in the same atomic .wetink publication.
273     */
274     private fun schedulePendingSave() {

```



```

275     val document = pendingSaveDocument ?: return
276     uiState = uiState.copy(isDirty = true)
277     autosaveCoordinator.schedule {
278         val recoveryPayloads = pendingLayerPayloads
279         if (recoveryPayloads.isNotEmpty()) {
280             withContext(Dispatchers.IO) {
281                 runCatching { repository.saveRecovery(document, recoveryPayloads) }
282             }
283             .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
284         }
285
286         val documentToSave = pendingSaveDocument ?: return@schedule
287         val payloadsToSave = pendingLayerPayloads
288         val thumbnailToSave = pendingThumbnailWebp
289         val layerPreviewsToSave = pendingLayerPreviewsWebp
290         val acknowledge = pendingTilesAcknowledgement
291
292         uiState = uiState.copy(isSaving = true)
293         withContext(Dispatchers.IO) {
294             runCatching {
295                 repository.save(
296                     document = documentToSave,
297                     changedLayerTiles = payloadsToSave,
298                     thumbnailWebp = thumbnailToSave,
299                     layerPreviewsWebp = layerPreviewsToSave,
300                 )
301             }
302         }
303         .onSuccess {
304             if (pendingSaveDocument == documentToSave) {
305                 pendingSaveDocument = null
306                 pendingLayerPayloads = emptyMap()
307                 pendingTilesAcknowledgement = null
308                 if (thumbnailToSave != null) pendingThumbnailWebp = null
309                 if (layerPreviewsToSave.isNotEmpty()) pendingLayerPreviewsWebp = emptyMap()
310             }
311             repository.clearRecovery(documentToSave.id)
312             uiState = uiState.copy(isDirty = false, isSaving = false)
313             acknowledge?.invoke()
314         }
315         .onFailure { error ->
316             uiState = uiState.copy(isSaving = false, errorMessage = error.message)
317         }
318     }
319 }
320
321 fun closeEditor() {
322     autosaveCoordinator.cancel()
323     val document = openedDocument
324     val payloads = pendingLayerPayloads
325     val thumbnail = pendingThumbnailWebp
326     val layerPreviews = pendingLayerPreviewsWebp
327     val acknowledge = pendingTilesAcknowledgement
328     document?.let { openDocument ->
329         viewModelScope.launch {
330             uiState = uiState.copy(isSaving = true)
331             runCatching { repository.save(openDocument, payloads, thumbnail, layerPreviews) }
332                 .onSuccess {
333                     repository.clearRecovery(openDocument.id)
334                     acknowledge?.invoke()
335                     uiState = uiState.copy(isDirty = false, isSaving = false)
336                     refreshProjects()
337                 }
338                 .onFailure { error -> uiState = uiState.copy(isSaving = false, errorMessage = error.message) }
339         }
340     }
341     pendingSaveDocument = null
342     pendingLayerPayloads = emptyMap()
343     pendingTilesAcknowledgement = null
344     pendingThumbnailWebp = null
345     pendingLayerPreviewsWebp = emptyMap()
346     openedDocument = null
347     openedProject = null
348     route = AppRoute.Home
349     dismissRecovery()
350     refreshProjects()
351 }
352
353 private fun markEditorSession(projectId: String) {
354     session.edit()
355         .putString(PREF_ACTIVE_PROJECT_ID, projectId)
356         .putBoolean(PREF_EDITOR_SESSION_OPEN, true)
357         .apply()
358 }
359
360 companion object {
361     private const val PREFS_NAME = "wetinknext_project_session"
362     private const val PREF_ACTIVE_PROJECT_ID = "active_project_id"
363     private const val PREF_EDITOR_SESSION_OPEN = "editor_session_open"
364 }
365
366 class Factory(context: Context) : ViewModelProvider.Factory {

```

```
367     private val appContext = context.applicationContext
368
369     @Suppress("UNCHECKED_CAST")
370     override fun <T : ViewModel> create(modelClass: Class<T>): T {
371         check(modelClass.isAssignableFrom(AppViewModel::class.java))
372         return AppViewModel(
373             repository = FileProjectRepository(File(appContext.filesDir, "projects")),
374             session = appContext.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE),
375         ) as T
376     }
377 }
378 }
```

4. app/src/main/java/com/wetinknext/app/EditorScreen.kt

```
1 package com.wetinknext.app
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.rememberScrollState
7 import androidx.compose.foundation.verticalScroll
8 import androidx.compose.foundation.layout.*
9 import androidx.compose.foundation.lazy.grid.GridCells
10 import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
11 import androidx.compose.material.icons.Icons
12 import androidx.compose.material.icons.automirrored.filled.ArrowBack
13 import androidx.compose.material.icons.filled.Settings
14 import androidx.compose.material3.Icon
15 import androidx.compose.runtime.*
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.graphics.Color
19 import androidx.compose.ui.graphics.toArgb
20 import androidx.compose.ui.draw.clip
21 import androidx.compose.foundation.shape.RoundedCornerShape
22 import androidx.compose.ui.unit.dp
23 import androidx.compose.ui.unit.sp
24 import androidx.compose.ui.viewinterop.AndroidView
25 import androidx.compose.ui.platform.LocalContext
26 import com.wetinknext.domain.document.ProjectDocument
27 import com.wetinknext.engine.brush.BrushPreset
28 import com.wetinknext.engine.brush.BrushLibrary
29 import com.wetinknext.engine.core.EditorUiState
30 import com.wetinknext.engine.core.CanvasBackdropMode
31 import com.wetinknext.engine.core.PaintSurfaceView
32 import com.wetinknext.ui.animation.AnimationTimelineToolbar
33 import com.wetinknext.ui.color.ColorPanel
34 import com.wetinknext.ui.color.GlesColorState
35 import com.wetinknext.ui.components.*
36 import com.wetinknext.ui.state.LayerState
37 import com.wetinknext.ui.theme.WetInkTheme
38 import com.wetinknext.ui.theme.AppTheme
39 import com.wetinknext.ui.theme.AppThemes
40 import com.wetinknext.ui.theme.CustomThemeEditor
41 import com.wetinknext.ui.theme.ThemeController
42 import com.wetinknext.ui.theme.rememberThemeController
43
44 private enum class EditorPanel {
45     NONE,
46     BRUSH,
47     LAYERS,
48     COLOR,
49     SELECTION,
50     TRANSFORM,
51     ADJUSTMENTS,
52     ANIMATION,
53     SETTINGS,
54 }
55
56 @Composable
57 fun EditorScreen(
58     projectId: String,
59     document: ProjectDocument,
60     layerTiles: Map<Long, ByteArray> = emptyMap(),
61     layerPreviews: Map<Long, ByteArray> = emptyMap(),
62     saveStatus: String = "Сохранено",
63     onDocumentChanged: (ProjectDocument) -> Unit = {},
64     onDirtyLayerTiles: (ProjectDocument, Map<Long, ByteArray>, Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>, () -> Unit) -> Unit = { _, _, _ -> },
65     onThumbnailCaptured: (com.wetinknext.engine.core.ThumbnailCapture.Rgba) -> Unit = {},
66     onThumbnailBuildSaved: (com.wetinknext.engine.thumbnail.ThumbnailBuildResult) -> Unit = {},
67     onBack: () -> Unit = {}
68 ) {
69     val context = LocalContext.current
70     val themeController = rememberThemeController()
71     val theme = themeController.current
72     var uiState by remember { mutableStateOf(EditorUiState.empty) }
73     var openPanel by remember { mutableStateOf(EditorPanel.NONE) }
74     var isEraser by remember { mutableStateOf(false) }
75     var documentLoading by remember(projectId) { mutableStateOf(true) }
76
77     val colorState = remember { GlesColorState(context) }
78     var brushColor by remember { mutableStateOf(Color.Black) }
79     var selectedBrush by remember { mutableStateOf<BrushPreset?>(BrushLibrary.pencil6B) }
80
81     val layerState = remember { LayerState() }
82
83     val surface = remember(projectId, document.id) {
84         PaintSurfaceView(
85             context = context,
86             document = document,
87             layerTiles = layerTiles,
88             layerPreviews = layerPreviews,
89             ).also { view ->
```

```

90         view.onEditorStateChange = { uiState = it }
91         view.onProjectDocumentChange = onDocumentChanged
92         view.onDirtyLayerTiles = { changed, payloads, dirty -> onDirtyLayerTiles(changed, payloads, dirty) { view.
acknowledgeSavedTiles(dirty) } }
93         view.onThumbnailCaptured = onThumbnailCaptured
94         view.onThumbnailBuildSaved = onThumbnailBuildSaved
95         view.onDocumentSessionLoaded = { documentLoading = false }
96     }
97 }
98
99 DisposableEffect(surface, onDocumentChanged) {
100     surface.onProjectDocumentChange = onDocumentChanged
101     surface.onDirtyLayerTiles = { changed, payloads, dirty -> onDirtyLayerTiles(changed, payloads, dirty) { surface.
acknowledgeSavedTiles(dirty) } }
102     surface.onThumbnailCaptured = onThumbnailCaptured
103     surface.onThumbnailBuildSaved = onThumbnailBuildSaved
104     surface.onDocumentSessionLoaded = { documentLoading = false }
105     onDispose {
106         surface.onProjectDocumentChange = null
107         surface.onDirtyLayerTiles = null
108         surface.onThumbnailCaptured = null
109         surface.onThumbnailBuildSaved = null
110         surface.onDocumentSessionLoaded = null
111     }
112 }
113
114 LaunchedEffect(uiState) {
115     layerState.syncFromEditor(uiState)
116 }
117
118 // Initialize state
119 LaunchedEffect(surface) {
120     surface.requestState()
121     surface.applyBrushPreset(BrushLibrary.pencil6B)
122     selectedBrush = BrushLibrary.pencil6B
123 }
124
125 LaunchedEffect(theme.canvasBackdrop, theme.canvasGrid, themeController.backdropMode) {
126     surface.setCanvasBackdrop(theme.canvasBackdrop.toArgb(), theme.canvasGrid.toArgb(), themeController.backdropMode)
127 }
128
129 WetInkTheme(theme = theme, fontMode = themeController.font) {
130     Box(
131         modifier = Modifier
132             .fillMaxSize()
133             .background(theme.appBg),
134     ) {
135         AndroidView(
136             modifier = Modifier.fillMaxSize(),
137             factory = { surface },
138         )
139
140         if (documentLoading) {
141             Box(
142                 modifier = Modifier.fillMaxSize().background(theme.appBg.copy(alpha = .92f)),
143                 contentAlignment = Alignment.Center,
144             ) {
145                 androidx.compose.material3.Text("Загрузка документа...", color = theme.textPrimary)
146             }
147         }
148
149         // Save status remains in state for diagnostics; it must not overlap the canvas.
150
151         Box(
152             modifier = Modifier
153                 .align(Alignment.TopStart)
154                 .padding(12.dp),
155         ) {
156             CanvasActionButton(
157                 icon = Icons.AutoMirrored.Filled.ArrowBack,
158                 description = "На главную",
159                 theme = theme,
160                 onClick = onBack,
161             )
162         }
163
164         // Top Toolbar
165         Box(
166             modifier = Modifier
167                 .align(Alignment.TopEnd)
168         ) {
169             TopToolbar(
170                 theme = theme,
171                 currentColor = brushColor,
172                 canUndo = uiState.canUndo,
173                 canRedo = uiState.canRedo,
174                 isSelectionActive = openPanel == EditorPanel.SELECTION,
175                 isTransformActive = openPanel == EditorPanel.TRANSFORM,
176                 isAnimationActive = openPanel == EditorPanel.ANIMATION,
177                 isAdjustmentsActive = openPanel == EditorPanel.ADJUSTMENTS,
178                 onUndoClick = { surface.undo() },
179                 onRedoClick = { surface.redo() },

```

```

180         onSelectionClick = {
181             openPanel = if (openPanel == EditorPanel.SELECTION) EditorPanel.NONE else EditorPanel.SELECTION
182         },
183         onTransformClick = {
184             openPanel = if (openPanel == EditorPanel.TRANSFORM) EditorPanel.NONE else EditorPanel.TRANSFORM
185         },
186         onAnimationClick = {
187             openPanel = if (openPanel == EditorPanel.ANIMATION) EditorPanel.NONE else EditorPanel.ANIMATION
188         },
189         onAdjustmentsClick = {
190             openPanel = if (openPanel == EditorPanel.ADJUSTMENTS) EditorPanel.NONE else EditorPanel.ADJUSTMENTS
191         },
192         onSettingsClick = {
193             openPanel = if (openPanel == EditorPanel.SETTINGS) EditorPanel.NONE else EditorPanel.SETTINGS
194         },
195         onLayersClick = {
196             openPanel = if (openPanel == EditorPanel.LAYERS) EditorPanel.NONE else EditorPanel.LAYERS
197         },
198         onColorClick = {
199             openPanel = if (openPanel == EditorPanel.COLOR) EditorPanel.NONE else EditorPanel.COLOR
200         },
201     )
202 }
203
204 // Left controls keep the same compact stack as the reference editor.
205 Column(
206     modifier = Modifier
207         .align(Alignment.TopStart)
208         .padding(top = 100.dp),
209     horizontalAlignment = Alignment.Start,
210 ) {
211     SideToolbar(
212         theme = theme,
213         brushSize = uiState.brushSizePx,
214         brushOpacity = uiState.brushOpacity,
215         isEraser = isEraser,
216         onBrushSizeChange = { surface.setBrushSize(it) },
217         onBrushOpacityChange = { surface.setBrushOpacity(it) },
218         onBrushClick = {
219             openPanel = if (openPanel == EditorPanel.BRUSH) EditorPanel.NONE else EditorPanel.BRUSH
220         },
221         onEraserClick = {
222             isEraser = it
223             surface.setEraserEnabled(it)
224         }
225     )
226     LeftCanvasActions(
227         theme = theme,
228         onClearLayerClick = { surface.clearActiveLayer() },
229         onMirrorClick = { surface.toggleCanvasMirror() },
230     )
231 }
232
233 // Panel Host
234 EditorPanelHost(
235     panel = openPanel,
236     theme = theme,
237     surface = surface,
238     layerState = layerState,
239     colorState = colorState,
240     brushColor = brushColor,
241     selectedBrush = selectedBrush,
242     themeController = themeController,
243     onBrushSelected = { preset ->
244         selectedBrush = preset
245         surface.applyBrushPreset(preset)
246     },
247     onColorChange = {
248         brushColor = it
249         surface.setBrushColor(it)
250     },
251     onDismiss = { openPanel = EditorPanel.NONE }
252 )
253 }
254 }
255 }
256
257 @Composable
258 private fun EditorPanelHost(
259     panel: EditorPanel,
260     theme: com.wetinknext.ui.theme.AppTheme,
261     surface: PaintSurfaceView?,
262     layerState: LayerState,
263     colorState: GlesColorState,
264     brushColor: Color,
265     selectedBrush: BrushPreset?,
266     themeController: ThemeController,
267     onBrushSelected: (BrushPreset) -> Unit,
268     onColorChange: (Color) -> Unit,
269     onDismiss: () -> Unit
270 ) {
271     Box(modifier = Modifier.fillMaxSize()) {

```

```

272 when (panel) {
273     EditorPanel.NONE -> Unit
274
275     EditorPanel.SETTINGS -> {
276         Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 70.dp, end = 16.dp)) {
277             ThemeSettingsPanel(
278                 theme = theme,
279                 themeController = themeController,
280                 onDismiss = onDismiss,
281             )
282         }
283     }
284
285     EditorPanel.BRUSH -> {
286         Box(modifier = Modifier.align(Alignment.CenterStart).padding(start = 72.dp)) {
287             BrushPanel(
288                 currentBrush = selectedBrush ?: BrushLibrary.pencil6B,
289                 onBrushSelect = onBrushSelected,
290                 onBrushStudioOpen = { /* TODO */ },
291                 theme = theme,
292                 onDismiss = onDismiss,
293                 onDuplicate = { /* TODO */ },
294                 onDelete = { /* TODO */ }
295             )
296         }
297     }
298
299     EditorPanel.LAYERS -> {
300         Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 80.dp, end = 16.dp)) {
301             LayersPanel(
302                 state = layerState,
303                 theme = theme,
304                 onDismiss = onDismiss,
305                 onAddLayer = { surface?.addLayer() },
306                 onSelectLayer = { surface?.setActiveLayer(it) },
307                 onVisibleChange = { id, visible -> surface?.setLayerVisible(id, visible) },
308                 onOpacityChange = { id, opacity -> surface?.setLayerOpacity(id, opacity) },
309                 onDuplicateLayer = { id -> surface?.duplicateLayer(id) },
310                 onRemoveLayer = { id -> surface?.removeLayer(id) }
311             )
312         }
313     }
314
315     EditorPanel.COLOR -> {
316         Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 80.dp, end = 16.dp)) {
317             ColorPanel(
318                 state = colorState,
319                 color = brushColor,
320                 theme = theme,
321                 onColorChange = onColorChange,
322                 onAddFromPhoto = { /* TODO */ },
323                 onDismiss = onDismiss
324             )
325         }
326     }
327
328     EditorPanel.SELECTION -> {
329         Box(modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 32.dp)) {
330             SelectionToolbar(
331                 theme = theme,
332                 currentShape = SelectionShapeUi.FREEHAND,
333                 onShapeChange = { /* TODO */ },
334                 onDone = onDismiss
335             )
336         }
337     }
338
339     EditorPanel.TRANSFORM -> {
340         Box(modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 32.dp)) {
341             TransformMenuView(
342                 theme = theme,
343                 currentMode = TransformModeUi.UNIFORM,
344                 actions = object : TransformActions {
345                     override fun reset() { /* TODO */ }
346                     override fun cancel() { onDismiss() }
347                     override fun apply() { onDismiss() }
348                     override fun flipHorizontal() { /* TODO */ }
349                     override fun flipVertical() { /* TODO */ }
350                     override fun setMode(mode: TransformModeUi) { /* TODO */ }
351                 }
352             )
353         }
354     }
355
356     EditorPanel.ADJUSTMENTS -> {
357         Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 80.dp, end = 16.dp)) {
358             AdjustmentsPanel(
359                 theme = theme,
360                 onClose = onDismiss,
361                 onAction = { /* TODO */ }
362             )
363         }
364     }

```

```

364     }
365
366     EditorPanel.ANIMATION -> {
367         Box(modifier = Modifier.align(Alignment.BottomCenter)) {
368             AnimationTimelineToolbar(
369                 theme = theme,
370                 document = com.wetinknext.domain.animation.AnimationDocument(enabled = true),
371                 currentFrameId = 0L,
372                 onFrameSelect = {},
373                 onFrameMove = { _, _ -> },
374                 onAddFrame = {},
375                 onDuplicateFrame = {},
376                 onDeleteFrame = {},
377                 onHoldChange = { _, _ -> },
378                 onRoleToggle = { _, _ -> },
379                 onAssembleLayers = {},
380                 onLayersClick = {},
381                 onClose = onDismiss
382             )
383         }
384     }
385 }
386
387 }
388
389 @Composable
390 private fun ThemeSettingsPanel(
391     theme: AppTheme,
392     themeController: ThemeController,
393     onDismiss: () -> Unit,
394 ) {
395     var preferencesOpen by remember { mutableStateOf(false) }
396     var customThemeOpen by remember { mutableStateOf(false) }
397     Column(
398         modifier = Modifier
399             .width(320.dp)
400             .heightIn(max = 620.dp)
401             .clip(RoundedCornerShape(22.dp))
402             .background(theme.panelBg)
403             .verticalScroll(rememberScrollState())
404             .padding(10.dp),
405         verticalArrangement = Arrangement.spacedBy(12.dp),
406     ) {
407         Row(verticalAlignment = Alignment.CenterVertically) {
408             androidx.compose.material3.Text(
409                 text = if (preferencesOpen) "Предпочтения" else "Меню",
410                 color = theme.textPrimary,
411                 style = androidx.compose.material3.MaterialTheme.typography.titleLarge,
412                 modifier = Modifier.weight(1f),
413             )
414             androidx.compose.material3.Text(
415                 text = "x",
416                 color = theme.textSecondary,
417                 style = androidx.compose.material3.MaterialTheme.typography.headlineSmall,
418                 modifier = Modifier.size(28.dp).clickable(onClick = onDismiss),
419             )
420         }
421
422         Row(
423             modifier = Modifier.fillMaxWidth().clip(RoundedCornerShape(10.dp))
424                 .background(theme.panelBgVariant.copy(alpha = .5f)).padding(3.dp),
425             horizontalArrangement = Arrangement.spacedBy(2.dp),
426         ) {
427             listOf("Меню", "Холст", "Экспорт", "Сведения").forEachIndexed { index, label ->
428                 Box(
429                     modifier = Modifier
430                         .weight(1f).clip(RoundedCornerShape(8.dp))
431                         .background(if (index == 0) theme.panelBgVariant else Color.Transparent)
432                         .padding(vertical = 8.dp),
433                     contentAlignment = Alignment.Center,
434                 ) {
435                     androidx.compose.material3.Text(
436                         text = label,
437                         color = if (index == 0) theme.textPrimary else theme.textSecondary,
438                         fontSize = 11.sp,
439                         maxLines = 1,
440                         softWrap = false,
441                     )
442                 }
443             }
444         }
445
446         if (customThemeOpen) {
447             CustomThemeEditor(
448                 colors = themeController.customColors,
449                 theme = theme,
450                 backdropMode = themeController.backdropMode,
451                 onChange = themeController::updateCustomColors,
452                 onBackdropModeChange = themeController::selectBackdropMode,
453                 onDismiss = { customThemeOpen = false },
454             )
455         } else if (!preferencesOpen) {

```

```

456         androidx.compose.material3.Text(
457             text = "Предпочтения",
458             color = theme.textPrimary,
459             modifier = Modifier.fillMaxWidth().clickable { preferencesOpen = true }.padding(vertical = 14.dp, horizontal
= 4.dp),
460         )
461     } else {
462         androidx.compose.material3.Text(
463             text = "Тема",
464             color = theme.textSecondary,
465             style = androidx.compose.material3.MaterialTheme.typography.labelMedium,
466             modifier = Modifier.padding(start = 10.dp),
467         )
468         val choices = AppThemes.all + themeController.customTheme.copy(id = "custom", displayName = "Своя")
469         choices.chunked(5).forEach { row ->
470             Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
471                 row.forEach { candidate ->
472                     val selected = candidate.id == themeController.current.id
473                     Box(
474                         modifier = Modifier.size(48.dp).clip(androidx.compose.foundation.shape.CircleShape)
475                         .background(if (selected) Color.White else Color.Transparent)
476                         .clickable {
477                             if (candidate.id == "custom") {
478                                 themeController.selectCustom()
479                                 customThemeOpen = true
480                             } else themeController.select(candidate)
481                         }
482                         .padding(2.dp),
483                         contentAlignment = Alignment.Center,
484                     ) {
485                         Box(
486                             modifier = Modifier.fillMaxSize().clip(androidx.compose.foundation.shape.CircleShape)
487                             .background(if (candidate.id == "custom") theme.panelBgVariant else candidate.appBg),
488                             contentAlignment = Alignment.Center,
489                         ) {
490                             if (candidate.id == "custom") {
491                                 Icon(
492                                     imageVector = Icons.Filled.Settings,
493                                     contentDescription = "Настроить свою тему",
494                                     tint = theme.textPrimary,
495                                     modifier = Modifier.size(21.dp),
496                                 )
497                             }
498                             else Box(Modifier.size(24.dp).clip(androidx.compose.foundation.shape.CircleShape).background(
candidate.accent))
499                         }
500                     }
501                 }
502             }
503         }
504     }
505 }
506 }
507 }
508
509 private fun PaintSurfaceView.applyBrushPreset(
510     preset: BrushPreset
511 ) {
512     val settings = preset.settings
513
514     applyBrush(settings)
515
516     val grainPath = settings.grainAssetPath
517         ?.trim()
518         ?.takeIf { it.isNotEmpty() }
519
520     if (grainPath == null) {
521         clearGrainTexture()
522     } else {
523         loadGrainTexture(
524             path = grainPath,
525             scale = settings.grainScale,
526             canvasLocked = settings.grainCanvasLocked,
527             depth = settings.textureDepth,
528             contrast = settings.textureContrast,
529         )
530     }
531
532     val shapePath = settings.shapeAssetPath?.trim()?.takeIf { it.isNotEmpty() }
533     if (shapePath == null) {
534         clearShapeTexture()
535     } else {
536         loadShapeTexture(path = shapePath, rgbToAlpha = settings.rgbToAlpha)
537     }
538 }

```


5. app/src/main/java/com/wetinknext/app/MainActivity.kt

```
1 package com.wetinknext.app
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.viewModels
7 import androidx.core.view.WindowCompat
8 import androidx.core.view.WindowInsetsCompat
9 import androidx.core.view.WindowInsetsControllerCompat
10
11 class MainActivity : ComponentActivity() {
12     private val appViewModel: AppViewModel by viewModels {
13         AppViewModel.Factory(this)
14     }
15
16     override fun onCreate(savedInstanceState: Bundle?) {
17         super.onCreate(savedInstanceState)
18         WindowCompat.setDecorFitsSystemWindows(window, false)
19         WindowInsetsControllerCompat(window, window.decorView).apply {
20             hide(WindowInsetsCompat.Type.statusBars())
21             systemBarsBehavior = WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE
22         }
23         setContent { AppRoot(appViewModel) }
24     }
25 }
```

6. app/src/main/java/com/wetinknext/app/WetInkApp.kt

```
1 package com.wetinknext.app
2
3 import androidx.activity.compose.BackHandler
4 import androidx.compose.material3.AlertDialog
5 import androidx.compose.material3.Text
6 import androidx.compose.material3.TextButton
7 import androidx.compose.runtime.Composable
8 import androidx.compose.runtime.LaunchedEffect
9 import androidx.compose.ui.text.font.FontWeight
10 import com.wetinknext.data.project.CreateProjectRequest
11 import com.wetinknext.data.project.ProjectSummary
12 import com.wetinknext.ui.home.HomeProject
13 import com.wetinknext.ui.home.HomeScreen
14 import com.wetinknext.ui.home.DeletedHomeProject
15 import com.wetinknext.ui.theme.WetInkTheme
16 import com.wetinknext.ui.theme.rememberThemeController
17
18 @Composable
19 fun AppRoot(viewModel: AppViewModel) {
20     when (val currentRoute = viewModel.route) {
21         AppRoute.Home -> ProjectHomeRoute(viewModel)
22         is AppRoute.Editor -> {
23             val document = viewModel.openedDocument
24             val loadedProject = viewModel.openedProject
25             if (document?.id == currentRoute.projectId && loadedProject?.document?.id == currentRoute.projectId) {
26                 BackHandler { viewModel.closeEditor() }
27                 EditorScreen(
28                     projectId = currentRoute.projectId,
29                     document = document,
30                     layerTiles = loadedProject.assets.layerTiles,
31                     layerPreviews = loadedProject.assets.layerPreviewsWebp,
32                     saveStatus = when {
33                         viewModel.uiState.isSaving -> "Сохраняется..."
34                         viewModel.uiState.isDirty -> "Есть несохранённые изменения"
35                         else -> "Сохранено"
36                     },
37                     onDocumentChanged = viewModel::scheduleAutosave,
38                     onDirtyLayerTiles = viewModel::scheduleTileAutosave,
39                     onThumbnailCaptured = viewModel::submitThumbnail,
40                     onThumbnailBuildSaved = viewModel::submitThumbnailBuild,
41                     onBack = viewModel::closeEditor,
42                 )
43             } else {
44                 LaunchedEffect(currentRoute.projectId) {
45                     viewModel.openProject(currentRoute.projectId)
46                 }
47             }
48         }
49     }
50 }
51
52 /** Kept for existing previews and callers while MainActivity uses AppRoot. */
53 @Composable
54 fun WetInkApp(viewModel: AppViewModel) = AppRoot(viewModel)
55
56 @Composable
57 private fun ProjectHomeRoute(viewModel: AppViewModel) {
58     val themeController = rememberThemeController()
59     val theme = themeController.current
60     val state = viewModel.uiState
61
62     WetInkTheme(theme = theme, fontMode = themeController.font) {
63         HomeScreen(
64             theme = theme,
65             projects = state.projects.map(ProjectSummary::toHomeProject),
66             deletedProjects = state.trashedProjects.map { trashed ->
67                 DeletedHomeProject(
68                     project = trashed.project.toHomeProject(),
69                     deletedAtMs = trashed.deletedAt,
70                 )
71             },
72             onOpenProject = { viewModel.openProject(it.id) },
73             onCreateProject = { name, width, height, dpi, colorProfile ->
74                 viewModel.createProject(
75                     CreateProjectRequest(
76                         name = name,
77                         width = width,
78                         height = height,
79                         dpi = dpi,
80                         colorProfile = colorProfile,
81                     )
82                 ),
83             },
84             onRenameProject = { project, name -> viewModel.renameProject(project.id, name) },
85             onDuplicateProject = { project -> viewModel.duplicateProject(project.id) },
86             onDeleteProject = { project -> viewModel.moveProjectToTrash(project.id) },
87             onRestoreProject = { deleted -> viewModel.restoreProject(deleted.project.id) },
88             onDeleteForever = { deleted -> viewModel.deleteProjectForever(deleted.project.id) },
89         )
90     }
```

```

91     state.recoveryProject?.let { project ->
92         AlertDialog(
93             onDismissRequest = viewModel::dismissRecovery,
94             title = { Text("Восстановить работу?" ) },
95             text = { Text("Приложение было закрыто во время работы с «${project.name}»." ) },
96             confirmButton = {
97                 TextButton(onClick = { viewModel.openProject(project.id) }) {
98                     Text("Открыть", fontWeight = FontWeight.SemiBold)
99                 }
100             },
101             dismissButton = {
102                 TextButton(onClick = viewModel::dismissRecovery) {
103                     Text("Не сейчас")
104                 }
105             },
106         )
107     }
108 }
109 }
110
111 private fun ProjectSummary.toHomeProject(): HomeProject = HomeProject(
112     id = id,
113     name = name,
114     canvasWidth = width,
115     canvasHeight = height,
116     updatedAtMs = updatedAt,
117     thumbnailBytes = thumbnailBytes,
118     thumbnailVersion = thumbnailVersion,
119 )

```

7. app/src/main/java/com/wetinknext/data/project/AutosaveCoordinator.kt

```
1 package com.wetinknext.data.project
2
3 import kotlinx.coroutines.CoroutineScope
4 import kotlinx.coroutines.Job
5 import kotlinx.coroutines.delay
6 import kotlinx.coroutines.launch
7
8 /**
9  * Coalesces completed document edits into one disk publication.
10  *
11  * Callers only schedule after transactional changes (UP, clear, layer or
12  * property operations), never for high-frequency MOVE events.
13  */
14 class AutosaveCoordinator(
15     private val scope: CoroutineScope,
16     private val debounceMillis: Long = DEFAULT_DEBOUNCE_MILLIS,
17 ) {
18     private var pendingJob: Job? = null
19
20     fun schedule(save: suspend () -> Unit) {
21         pendingJob?.cancel()
22         pendingJob = scope.launch {
23             delay(debounceMillis)
24             save()
25         }
26     }
27
28     fun cancel() {
29         pendingJob?.cancel()
30         pendingJob = null
31     }
32
33     companion object {
34         const val DEFAULT_DEBOUNCE_MILLIS = 700L
35     }
36 }
```

8. app/src/main/java/com/wetinknext/data/project/DirectoryProjectStore.kt

```
1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.LoadedWetInkProject
4 import com.wetinknext.domain.document.ProjectAssets
5 import com.wetinknext.domain.document.ProjectDocument
6 import com.wetinknext.domain.document.ProjectManifest
7 import com.wetinknext.domain.document.StoredLayer
8 import java.io.File
9 import java.io.IOException
10 import java.nio.file.AtomicMoveNotSupportedException
11 import java.nio.file.Files
12 import java.nio.file.StandardCopyOption
13 import kotlinx.serialization.decodeFromString
14 import kotlinx.serialization.encodeToString
15 import kotlinx.serialization.json.Json
16
17 /** New on-disk project format: directory metadata plus independently replaceable layer files. */
18 class DirectoryProjectStore(
19     private val clock: () -> Long = System::currentTimeMillis,
20     private val json: Json = Json { encodeDefaults = true; ignoreUnknownKeys = true; prettyPrint = true },
21 ) {
22     private var lastTimestamp = Long.MIN_VALUE
23
24     fun save(directory: File, document: ProjectDocument, assets: ProjectAssets): LoadedWetInkProject {
25         require(directory.name == document.id) { "Project directory/id mismatch" }
26         val expected = document.layers.map { it.id }.toSet()
27         require(assets.layerTiles.keys == expected) { "Every layer needs one payload" }
28         ensureDirectory(directory)
29         val timestamp = nextTimestamp()
30         val saved = document.copy(
31             updatedAt = timestamp,
32             thumbnailVersion = if (assets.thumbnailWebp != null) timestamp else document.thumbnailVersion,
33         )
34         saved.layers.forEach { layer -> writeAtomic(File(directory, layer.pixelFile), checkNotNull(assets.layerTiles[layer.id])) }
35         assets.thumbnailWebp?.let { writeAtomic(File(directory, THUMBNAI), it) }
36         assets.layerPreviewsWebp.forEach { (id, bytes) -> writeAtomic(File(directory, previewPath(id)), bytes) }
37         deleteRemovedLayerFiles(directory, saved.layers.map { it.id }.toSet())
38         writeAtomic(File(directory, DOCUMENT), json.encodeToString(saved).encodeToByteArray())
39         val manifest = ProjectManifest(
40             documentId = saved.id, name = saved.name, width = saved.width, height = saved.height,
41             thumbnailPath = assets.thumbnailWebp?.let { THUMBNAI }, thumbnailVersion = saved.thumbnailVersion,
42             createdAt = saved.createdAt, updatedAt = saved.updatedAt,
43             layers = saved.layers.map { StoredLayer(it.id, it.pixelFile, checkNotNull(assets.layerTiles[it.id]).size.toLong()) }
44         )
45         writeAtomic(File(directory, MANIFEST), json.encodeToString(manifest).encodeToByteArray())
46         return LoadedWetInkProject(saved, manifest, assets)
47     }
48
49     fun load(directory: File): LoadedWetInkProject {
50         val manifest = readManifest(directory)
51         val document = readDocument(directory)
52         require(manifest.documentId == document.id) { "Manifest/document id mismatch" }
53         val tiles = document.layers.associate { layer ->
54             val bytes = requireFile(File(directory, layer.pixelFile)).readBytes()
55             val stored = manifest.layers.firstOrNull { it.id == layer.id }?: error("Missing layer metadata")
56             require(stored.byteSize == bytes.size.toLong()) { "Layer size mismatch" }
57             layer.id to bytes
58         }
59         val previews = document.layers.mapNotNull { layer ->
60             File(directory, previewPath(layer.id)).takeIf(File::isFile)?.readBytes()?.let { layer.id to it }
61         }.toMap()
62         val thumbnail = manifest.thumbnailPath?.let { File(directory, it).takeIf(File::isFile)?.readBytes() }
63         return LoadedWetInkProject(document, manifest, ProjectAssets(tiles, thumbnail, previews))
64     }
65
66     /** Keeps gallery sorting deterministic when two saves share one millisecond. */
67     @Synchronized
68     private fun nextTimestamp(): Long {
69         val now = clock()
70         val next = maxOf(now, lastTimestamp + 1L)
71         lastTimestamp = next
72         return next
73     }
74
75     fun readManifest(directory: File): ProjectManifest =
76         json.decodeFromString(requireFile(File(directory, MANIFEST)).readText())
77
78     fun readDocument(directory: File): ProjectDocument =
79         json.decodeFromString(requireFile(File(directory, DOCUMENT)).readText())
80
81     fun readThumbnail(directory: File): ByteArray? = File(directory, THUMBNAI).takeIf(File::isFile)?.readBytes()
82
83     fun delete(directory: File) {
84         require(directory.isDirectory) { "Project directory does not exist" }
85         directory.walkBottomUp().forEach { if (!it.delete()) throw IOException("Cannot delete ${it.path}") }
86     }
87
88     private fun deleteRemovedLayerFiles(directory: File, active: Set<Long>) {

```

```

89         listOf(LAYERS, PREVIEWS).forEach { folder ->
90             File(directory, folder).listFiles().orEmpty().forEach { file ->
91                 val id = file.name.removePrefix("layer-").substringBefore('.').toLongOrNull()
92                 ?: file.name.substringBefore('.').toLongOrNull()
93                 if (id != null && id !in active) file.delete()
94             }
95         }
96     }
97
98     private fun writeAtomic(file: File, bytes: ByteArray) {
99         ensureDirectory(checkNotNull(file.parentFile))
100         val tmp = File(checkNotNull(file.parentFile), ".${file.name}.tmp")
101         try {
102             tmp.outputStream().use { it.write(bytes) }
103             try { Files.move(tmp.toPath(), file.toPath(), StandardCopyOption.ATOMIC_MOVE, StandardCopyOption.
REPLACE_EXISTING) }
104             catch (_: AtomicMoveNotSupportedException) { Files.move(tmp.toPath(), file.toPath(), StandardCopyOption.
REPLACE_EXISTING) }
105             } finally { if (tmp.exists()) tmp.delete() }
106         }
107
108     private fun ensureDirectory(file: File) {
109         if (!file.exists() && !file.mkdirs()) throw IOException("Cannot create ${file.path}")
110         require(file.isDirectory) { "Expected directory: ${file.path}" }
111     }
112
113     private fun requireFile(file: File): File = file.also { require(it.isFile) { "Missing ${it.path}" } }
114
115     private companion object {
116         const val MANIFEST = "manifest.json"
117         const val DOCUMENT = "document.json"
118         const val THUMBNAIL = "thumbnail.webp"
119         const val LAYERS = "layers"
120         const val PREVIEWS = "thumbnails"
121         fun previewPath(layerId: Long) = "$PREVIEWS/layer-$layerId.webp"
122     }
123 }

```

9. app/src/main/java/com/wetinknext/data/project/FileProjectRepository.kt

```
1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.ProjectAssets
4 import com.wetinknext.domain.document.ProjectDocument
5 import com.wetinknext.domain.document.ProjectManifest
6 import com.wetinknext.domain.document.WetInkProjectContainer
7 import java.io.File
8 import java.io.IOException
9 import java.nio.file.AtomicMoveNotSupportedException
10 import java.nio.file.Files
11 import java.nio.file.StandardCopyOption
12 import java.util.Base64
13 import java.util.UUID
14
15 /** File-backed project catalog. Each project remains one independent .wetink file. */
16 class FileProjectRepository(
17     private val projectsDirectory: File,
18     private val container: WetInkProjectContainer = WetInkProjectContainer(),
19     private val directoryProjectStore: DirectoryProjectStore = DirectoryProjectStore(),
20 ) : ProjectRepository {
21     override suspend fun create(request: CreateProjectRequest): ProjectSummary {
22         val document = ProjectDocument.newUntitled(
23             name = request.name,
24             width = request.width,
25             height = request.height,
26             dpi = request.dpi,
27             colorProfile = request.colorProfile,
28         )
29         val assets = ProjectAssets(
30             layerTiles = document.layers.associate { it.id to ByteArray(0) },
31             thumbnailWebp = DEFAULT_THUMBNAIL_WEBP,
32         )
33         val saved = directoryStore.save(projectDirectory(document.id), document, assets)
34         return saved.manifest.toSummary()
35     }
36
37     override suspend fun list(): List<ProjectSummary> {
38         ensureDirectories()
39         return listProjects(projectsDirectory)
40     }
41
42     override suspend fun listTrash(): List<TrashedProjectSummary> {
43         ensureDirectories()
44         return trashDirectory.listFiles()
45             .orEmpty()
46             .asSequence()
47             .filter { file -> file.isDirectory || (file.isFile && file.extension.equals(FILE_EXTENSION, ignoreCase = true) &&
!file.name.endsWith(".recovery.$FILE_EXTENSION")) }
48             .mapNotNull { file ->
49                 runCatching {
50                     TrashedProjectSummary(readSummary(file), file.lastModified())
51                 }.getOrNull()
52             }
53             .sortedByDescending(TrashedProjectSummary::deletedAt)
54             .toList()
55     }
56
57     override suspend fun open(id: String): ProjectDocument = loadProject(id).document
58
59     override suspend fun openForEditor(id: String) = loadProject(id)
60
61     override suspend fun save(document: ProjectDocument) {
62         val previous = loadProject(document.id)
63         saveProject(document, previous.assets.retainDocumentLayers(document))
64     }
65
66     override suspend fun save(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>) {
67         if (changedLayerTiles.isEmpty()) return save(document)
68         val previous = loadProject(document.id)
69         val allowed = document.layers.map { it.id }.toSet()
70         require(changedLayerTiles.keys.all(allowed::contains)) { "Unknown layer tile payload" }
71         saveProject(
72             document,
73             previous.assets.retainDocumentLayers(document).copy(
74                 layerTiles = previous.assets.layerTiles
75                     .filterKeys(allowed::contains)
76                     .plus(changedLayerTiles),
77             ),
78         )
79     }
80
81     override suspend fun save(
82         document: ProjectDocument,
83         changedLayerTiles: Map<Long, ByteArray>,
84         thumbnailWebp: ByteArray?,
85         layerPreviewsWebp: Map<Long, ByteArray>,
86     ) {
87         val previous = loadProject(document.id)
88         val layerIds = document.layers.map { it.id }.toSet()
89         saveProject(
```

```

90         document,
91         previous.assets.copy(
92             layerTiles = (previous.assets.layerTiles + changedLayerTiles)
93                 .filterKeys(layerIds::contains),
94             thumbnailWebp = thumbnailWebp ?: previous.assets.thumbnailWebp,
95             layerPreviewsWebp = (previous.assets.layerPreviewsWebp + layerPreviewsWebp)
96                 .filterKeys(layerIds::contains),
97         ),
98     )
99 }
100
101 override suspend fun saveRecovery(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>) {
102     val current = loadProject(document.id)
103     container.save(recoveryFile(document.id), document, current.assets.copy(layerTiles = current.assets.layerTiles +
104 changedLayerTiles))
105 }
106
107 override suspend fun clearRecovery(id: String) {
108     val file = recoveryFile(id)
109     if (file.exists() && !file.delete()) throw IOException("Cannot clear recovery snapshot")
110 }
111
112 override suspend fun openRecovery(id: String) = recoveryFile(id).takeIf(File::isFile)?.let(container::load)
113
114 override suspend fun rename(id: String, name: String) {
115     val document = open(id)
116     save(document.copy(name = name))
117 }
118
119 /** Creates a separate editable copy; undo history intentionally is not persisted. */
120 override suspend fun duplicate(id: String): ProjectSummary {
121     val source = loadProject(id)
122     val copyId = UUID.randomUUID().toString()
123     val copiedDocument = source.document.copy(
124         id = copyId,
125         name = duplicateName(source.document.name),
126         createdAt = System.currentTimeMillis(),
127         updatedAt = System.currentTimeMillis(),
128     )
129     val saved = directoryStore.save(projectDirectory(copyId), copiedDocument, source.assets)
130     return saved.manifest.toSummary(saved.assets.thumbnailWebp)
131 }
132
133 override suspend fun moveToTrash(id: String) {
134     val source = projectLocation(id)
135     require(source.exists()) { "Project does not exist: $id" }
136     val target = trashedProjectLocation(id, source.isDirectory)
137     atomicMove(source, target)
138     target.setLastModified(System.currentTimeMillis())
139 }
140
141 override suspend fun restore(id: String) {
142     val source = trashedProjectLocation(id, true).takeIf(File::exists)?: trashedProjectLocation(id, false)
143     require(source.exists()) { "Trashed project does not exist: $id" }
144     atomicMove(source, if (source.isDirectory) projectDirectory(id) else legacyProjectFile(id))
145 }
146
147 override suspend fun deleteForever(id: String) {
148     val file = trashedProjectLocation(id, true).takeIf(File::exists)?: trashedProjectLocation(id, false)
149     require(file.exists()) { "Trashed project does not exist: $id" }
150     if (file.isDirectory) directoryStore.delete(file) else if (!file.delete()) throw IOException("Cannot delete project
151 permanently: ${file.path}")
152 }
153
154 private fun projectDirectory(id: String): File {
155     ensureDirectories()
156     return File(projectsDirectory, safeId(id))
157 }
158
159 private fun legacyProjectFile(id: String): File = File(projectsDirectory, "${safeId(id)}.FILE_EXTENSION")
160
161 private fun trashedProjectLocation(id: String, directory: Boolean): File {
162     ensureDirectories()
163     return File(trashDirectory, if (directory) safeId(id) else "${safeId(id)}.FILE_EXTENSION")
164 }
165
166 private fun projectLocation(id: String): File = projectDirectory(id).takeIf(File::exists)?: legacyProjectFile(id)
167 private fun recoveryFile(id: String): File = File(projectsDirectory, "${safeId(id)}.recovery.FILE_EXTENSION")
168
169 private fun ensureDirectories() {
170     if (!projectsDirectory.exists() && !projectsDirectory.mkdirs()) {
171         throw IOException("Cannot create projects directory: ${projectsDirectory.path}")
172     }
173     if (!trashDirectory.exists() && !trashDirectory.mkdirs()) {
174         throw IOException("Cannot create project trash: ${trashDirectory.path}")
175     }
176 }
177
178 private fun listProjects(directory: File): List<ProjectSummary> =
179     directory.listFiles()
180         .orEmpty()
181         .asSequence()

```



```

180         .filter { file -> file.isDirectory || (file.isFile && file.extension.equals(FILE_EXTENSION, ignoreCase = true) &&
!file.name.endsWith(".recovery.$FILE_EXTENSION")) }
181     }.mapNotNull { file -> runCatching { readSummary(file) }.getOrNull() }
182     }.sortedByDescending(ProjectSummary::updatedAt)
183     }.toList()
184
185     private fun atomicMove(source: File, target: File) {
186         try {
187             Files.move(
188                 source.toPath(),
189                 target.toPath(),
190                 StandardCopyOption.ATOMIC_MOVE,
191                 StandardCopyOption.REPLACE_EXISTING,
192             )
193         } catch (error: AtomicMoveNotSupportedException) {
194             throw IOException("Atomic project move is not supported", error)
195         }
196     }
197
198     private fun loadProject(id: String) = projectDirectory(id).takeIf(File::isDirectory)
199     ?.let(directoryStore::load)
200     ?: container.load(legacyProjectFile(id))
201
202     /** First regular save migrates an old ZIP project to the directory format. */
203     private fun saveProject(document: ProjectDocument, assets: ProjectAssets) =
204         directoryStore.save(projectDirectory(document.id), document, assets).also {
205             legacyProjectFile(document.id).takeIf(File::isFile)?.delete()
206         }
207
208     private fun readSummary(location: File): ProjectSummary = if (location.isDirectory) {
209         directoryStore.readManifest(location).toSummary(directoryStore.readThumbnail(location))
210     } else {
211         container.readManifest(location).toSummary(container.readThumbnail(location))
212     }
213
214     private fun safeId(id: String): String {
215         require(id.isNotBlank() && id.all { it.isLetterOrDigit() || it == '-' || it == '_' }) {
216             "Unsafe project id"
217         }
218         return id
219     }
220
221     private fun duplicateName(name: String): String = "$name copy".take(80)
222
223     /** Drops binary assets for layers removed from the persisted document. */
224     private fun ProjectAssets.retainDocumentLayers(document: ProjectDocument): ProjectAssets {
225         val allowed = document.layers.map { it.id }.toSet()
226         return copy(
227             layerTiles = layerTiles.filterKeys(allowed::contains),
228             layerPreviewsWebp = layerPreviewsWebp.filterKeys(allowed::contains),
229         )
230     }
231
232     private fun ProjectManifest.toSummary(thumbnail: ByteArray? = null): ProjectSummary = ProjectSummary(
233         id = documentId,
234         name = name,
235         width = width,
236         height = height,
237         updatedAt = updatedAt,
238         thumbnailPath = thumbnailPath,
239         thumbnailBytes = thumbnail,
240         thumbnailVersion = thumbnailVersion,
241     )
242
243     private val trashDirectory: File
244         get() = File(projectsDirectory, TRASH_DIRECTORY)
245
246     private companion object {
247         const val FILE_EXTENSION = "wetink"
248         const val TRASH_DIRECTORY = ".trash"
249         // Valid transparent 1x1 WebP until a GL-generated project preview is available.
250         val DEFAULT_THUMBNAIL_WEBP: ByteArray = Base64.getDecoder().decode(
251             "UklGRiIAAABXRJQVIA4IBYAAADQAwCdASoBAAEAAUAmJaQAA3AA/vuUAAA=",
252         )
253     }
254 }

```

10. app/src/main/java/com/wetinknext/data/project/ProjectRepository.kt

```
1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.ProjectDocument
4 import com.wetinknext.domain.document.LoadedWetInkProject
5 import com.wetinknext.domain.document.ProjectAssets
6
7 interface ProjectRepository {
8     suspend fun create(request: CreateProjectRequest): ProjectSummary
9     suspend fun list(): List<ProjectSummary>
10    suspend fun listTrash(): List<TrashedProjectSummary>
11    suspend fun open(id: String): ProjectDocument
12    /** Reads document metadata and all layer payloads needed to build a GPU session. */
13    suspend fun openForEditor(id: String): LoadedWetInkProject
14    suspend fun save(document: ProjectDocument)
15    /** Atomically replaces only the supplied layer payloads; other layer data is retained. */
16    suspend fun save(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>)
17    suspend fun save(
18        document: ProjectDocument,
19        changedLayerTiles: Map<Long, ByteArray>,
20        thumbnailWebp: ByteArray?,
21        layerPreviewsWebp: Map<Long, ByteArray> = emptyMap(),
22    )
23    suspend fun saveRecovery(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>)
24    suspend fun clearRecovery(id: String)
25    suspend fun openRecovery(id: String): LoadedWetInkProject?
26    suspend fun rename(id: String, name: String)
27    suspend fun duplicate(id: String): ProjectSummary
28    suspend fun moveToTrash(id: String)
29    suspend fun restore(id: String)
30    suspend fun deleteForever(id: String)
31 }
32
33 data class CreateProjectRequest(
34     val name: String,
35     val width: Int = ProjectDocument.DEFAULT_WIDTH,
36     val height: Int = ProjectDocument.DEFAULT_HEIGHT,
37     val dpi: Int = ProjectDocument.DEFAULT_DPI,
38     val colorProfile: String = ProjectDocument.SRGB_PROFILE,
39 )
40
41 /** Metadata for the gallery. No layer tile data is loaded to create this item. */
42 data class ProjectSummary(
43     val id: String,
44     val name: String,
45     val width: Int,
46     val height: Int,
47     val updatedAt: Long,
48     val thumbnailPath: String?,
49     val thumbnailBytes: ByteArray? = null,
50     /** Cache version for the gallery WebP; metadata loading remains tile-free. */
51     val thumbnailVersion: Long = 0L,
52 )
53
54 data class TrashedProjectSummary(
55     val project: ProjectSummary,
56     val deletedAt: Long,
57 )
```

11. app/src/main/java/com/wetinknext/data/project/ThumbnailEncoder.kt

```
1 package com.wetinknext.data.project
2
3 import android.graphics.Bitmap
4 import android.os.Build
5 import com.wetinknext.engine.core.ThumbnailCapture
6 import java.io.ByteArrayOutputStream
7 import kotlin.math.pow
8 import kotlin.math.roundToInt
9
10 /** CPU-only encoder. Call from a background coroutine, never from the GL thread. */
11 object ThumbnailEncoder {
12     fun encodeWebp(image: ThumbnailCapture.Rgba): ByteArray {
13         val pixels = IntArray(image.width * image.height)
14         for (destinationY in 0 until image.height) {
15             // ThumbnailRenderer already flips glReadPixels into Bitmap order.
16             var source = destinationY * image.width * 4
17             val destination = destinationY * image.width
18             for (x in 0 until image.width) {
19                 val redLinear = (image.pixels[source++].toInt() and 0xFF) / 255f
20                 val greenLinear = (image.pixels[source++].toInt() and 0xFF) / 255f
21                 val blueLinear = (image.pixels[source++].toInt() and 0xFF) / 255f
22                 val a = image.pixels[source++].toInt() and 0xFF
23                 val r = (linearToSrgb(redLinear) * 255f).roundToInt().coerceIn(0, 255)
24                 val g = (linearToSrgb(greenLinear) * 255f).roundToInt().coerceIn(0, 255)
25                 val b = (linearToSrgb(blueLinear) * 255f).roundToInt().coerceIn(0, 255)
26                 pixels[destination + x] = (a shl 24) or (r shl 16) or (g shl 8) or b
27             }
28         }
29         val bitmap = Bitmap.createBitmap(pixels, image.width, image.height, Bitmap.Config.ARGB_8888)
30         return ByteArrayOutputStream().use { output ->
31             check(bitmap.compress(webpFormat(), 88, output)) { "WebP encoding failed" }
32             bitmap.recycle()
33             output.toByteArray()
34         }
35     }
36
37     @Suppress("DEPRECATION")
38     private fun webpFormat(): Bitmap.CompressFormat =
39         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {
40             Bitmap.CompressFormat.WEBP_LOSSY
41         } else {
42             Bitmap.CompressFormat.WEBP
43         }
44
45     private fun linearToSrgb(value: Float): Float {
46         val linear = value.coerceIn(0f, 1f)
47         return if (linear <= 0.0031308f) linear * 12.92f
48         else (1.055 * linear.toDouble().pow(1.0 / 2.4) - 0.055).toFloat()
49     }
50 }
```

12. app/src/main/java/com/wetinknext/domain/animation/AnimationModels.kt

```
1 package com.wetinknext.domain.animation
2
3 import kotlinx.serialization.Serializable
4
5 @Serializable
6 enum class AnimationPlaybackMode {
7     LOOP,
8     PING_PONG,
9     ONE_SHOT,
10 }
11
12 @Serializable
13 enum class AnimationFrameRole {
14     NORMAL,
15     BACKGROUND,
16     FOREGROUND,
17 }
18
19 @Serializable
20 data class OnionSkinSettings(
21     val enabled: Boolean = false, // Disabled by default until renderer supports it
22     val previousFrames: Int = 1,
23     val nextFrames: Int = 1,
24     val opacity: Float = 0.35f,
25     val previousColorArgb: Int = 0xFFFFF4FA3.toInt(),
26     val nextColorArgb: Int = 0xFF4A8DFF.toInt(),
27     val blendPrimaryFrame: Boolean = false,
28 )
29
30 @Serializable
31 data class AnimationFrame(
32     val id: Long,
33     val layerIds: List<Long>,
34     val holdFrames: Int = 1,
35     val role: AnimationFrameRole = AnimationFrameRole.NORMAL,
36 )
37
38 @Serializable
39 data class AnimationDocument(
40     val enabled: Boolean = false,
41     val framesPerSecond: Int = 12,
42     val playbackMode: AnimationPlaybackMode = AnimationPlaybackMode.LOOP,
43     val onionSkin: OnionSkinSettings = OnionSkinSettings(),
44     val frames: List<AnimationFrame> = emptyList(),
45 )
```

13. app/src/main/java/com/wetinknext/domain/animation/AnimationRenderPlan.kt

```
1 package com.wetinknext.domain.animation
2
3 /**
4  * Defines which frames and roles should be rendered in the current view.
5  */
6 data class AnimationRenderPlan(
7     val backgroundFrame: AnimationFrame?,
8     val currentFrame: AnimationFrame?,
9     val foregroundFrame: AnimationFrame?,
10    val previousOnionFrames: List<AnimationFrame>,
11    val nextOnionFrames: List<AnimationFrame>,
12 )
13
14 /**
15  * Logic to decide which layers are visible based on the animation document.
16  * This is a pure function for easy testing.
17  */
18 fun buildAnimationRenderPlan(
19     document: AnimationDocument,
20     currentFrameId: Long,
21 ): AnimationRenderPlan {
22     if (!document.enabled) {
23         return AnimationRenderPlan(null, null, null, emptyList(), emptyList())
24     }
25
26     val frames = document.frames
27     val currentIndex = frames.indexOfFirst { it.id == currentFrameId }
28
29     val bgFrame = frames.find { it.role == AnimationFrameRole.BACKGROUND }
30     val fgFrame = frames.find { it.role == AnimationFrameRole.FOREGROUND }
31     val current = if (currentIndex >= 0) frames[currentIndex] else null
32
33     val prevOnions = mutableListOf<AnimationFrame>()
34     val nextOnions = mutableListOf<AnimationFrame>()
35
36     if (document.onionSkin.enabled && currentIndex >= 0) {
37         // Previous frames
38         for (i in 1..document.onionSkin.previousFrames) {
39             val idx = currentIndex - i
40             if (idx >= 0) {
41                 val f = frames[idx]
42                 if (f.role == AnimationFrameRole.NORMAL) prevOnions.add(f)
43             }
44         }
45         // Next frames
46         for (i in 1..document.onionSkin.nextFrames) {
47             val idx = currentIndex + i
48             if (idx < frames.size) {
49                 val f = frames[idx]
50                 if (f.role == AnimationFrameRole.NORMAL) nextOnions.add(f)
51             }
52         }
53     }
54
55     return AnimationRenderPlan(
56         backgroundFrame = bgFrame,
57         currentFrame = current,
58         foregroundFrame = fgFrame,
59         previousOnionFrames = prevOnions,
60         nextOnionFrames = nextOnions
61     )
62 }
```

14. app/src/main/java/com/wetinknext/domain/animation/AnimationTimeline.kt

```

1 package com.wetinknext.domain.animation
2
3 /**
4  * Creates a default set of animation frames from a list of layer IDs.
5  * Each layer becomes its own frame. System "Background" (usually ID 1) should be filtered out by the caller.
6  */
7 fun createFramesFromLayers(layerIds: List<Long>): List<AnimationFrame> {
8     return layerIds.map { layerId ->
9         AnimationFrame(
10             id = layerId, // Use layerId as initial frame ID for simplicity in 1:1 mapping
11             layerIds = listOf(layerId)
12         )
13     }
14 }
15
16 /**
17  * Ensures the animation document is valid given the currently existing layers.
18  * Removes dead layer references, empty frames, and enforces range limits.
19  */
20 fun normalizedAnimationDocument(
21     document: AnimationDocument,
22     existingLayerIds: Set<Long>,
23 ): AnimationDocument {
24     return normalizedAnimationDocument(
25         document = document,
26         layersInDocumentOrder = existingLayerIds.map { AnimationLayerInfo(id = it) },
27     )
28 }
29
30 /** Minimal layer data needed by the timeline. Kept independent from OpenGL resources. */
31 data class AnimationLayerInfo(
32     val id: Long,
33     val isCanvasBackgroundLayer: Boolean = false,
34     val isClipping: Boolean = false,
35 )
36
37 /**
38  * Normalizes frames against the real document order. Canvas background never becomes a frame.
39  * Layer IDs inside every frame are also kept in document order, which is required for clipping.
40  */
41 fun normalizedAnimationDocument(
42     document: AnimationDocument,
43     layersInDocumentOrder: List<AnimationLayerInfo>,
44 ): AnimationDocument {
45     val order = layersInDocumentOrder.mapIndexed { index, layer -> layer.id to index }.toMap()
46     val animatableIds = layersInDocumentOrder
47         .filterNot { it.isCanvasBackgroundLayer }
48         .map { it.id }
49     val usedLayerIds = mutableSetOf<Long>()
50
51     val normalizedFrames = document.frames.mapNotNull { frame ->
52         val validLayerIds = frame.layerIds
53             .filter { id -> id in order && id !in usedLayerIds }
54             .sortedBy { order.getOrDefault(it, 0) }
55
56         if (validLayerIds.isEmpty()) {
57             null
58         } else {
59             usedLayerIds.addAll(validLayerIds)
60             frame.copy(
61                 layerIds = validLayerIds,
62                 holdFrames = frame.holdFrames.coerceIn(1, 120)
63             )
64         }
65     }
66     // A layer can belong to only one frame. Any newly created document layer gets its own frame.
67     val missingLayerIds = animatableIds.filter { it !in usedLayerIds }
68     val newFrames = missingLayerIds.map { layerId ->
69         AnimationFrame(id = layerId, layerIds = listOf(layerId))
70     }
71     return document.copy(
72         framesPerSecond = document.framesPerSecond.coerceIn(1, 60),
73         frames = normalizedFrames + newFrames
74     )
75 }
76
77 /**
78  * Combines multiple layers into a single animation frame.
79  */
80 fun groupLayersIntoFrame(
81     document: AnimationDocument,
82     layerIdsToGroup: List<Long>,
83     newFrameId: Long = System.nanoTime()
84 ): AnimationDocument {
85     val orderedIds = layerIdsToGroup.distinct()
86     val idsSet = orderedIds.toSet()
87     if (idsSet.isEmpty()) return document
88
89     val insertionIndex = document.frames.indexOfFirst { frame -> frame.layerIds.any { it in idsSet } }
90     val cleanedFrames = document.frames.map { frame ->

```

```

91     frame.copy(layerIds = frame.layerIds.filter { it !in idsSet })
92   }.filter { it.layerIds.isNotEmpty() }.toMutableList()
93   val newFrame = AnimationFrame(id = newFrameId, layerIds = orderedIds)
94   val targetIndex = if (insertionIndex < 0) cleanedFrames.size else insertionIndex.coerceAtMost(cleanedFrames.size)
95   cleanedFrames.add(targetIndex, newFrame)
96   return document.copy(frames = cleanedFrames)
97 }
98
99 sealed interface AnimationFrameGroupingValidation {
100   data class Valid(val layerIdsInDocumentOrder: List<Long>) : AnimationFrameGroupingValidation
101   data object EmptySelection : AnimationFrameGroupingValidation
102   data class MissingClippingBase(val clippingLayerId: Long) : AnimationFrameGroupingValidation
103   data class MissingClippingLayer(val baseLayerId: Long, val clippingLayerId: Long) : AnimationFrameGroupingValidation
104 }
105
106 /**
107  * A clipping layer and its base must live in one animation frame: otherwise a frame would render
108  * with a different compositing result than the document. The selection is returned in document order.
109  */
110 fun validateAnimationFrameGrouping(
111   selectedLayerIds: Set<Long>,
112   layersInDocumentOrder: List<AnimationLayerInfo>,
113 ): AnimationFrameGroupingValidation {
114   val layers = layersInDocumentOrder.filterNot { it.isCanvasBackgroundLayer }
115   val selected = selectedLayerIds.intersect(layers.map { it.id }.toSet())
116   if (selected.isEmpty()) return AnimationFrameGroupingValidation.EmptySelection
117
118   val baseByClippingId = mutableMapOf<Long, Long>()
119   layers.forEachIndexed { index, layer ->
120     if (layer.isClipping) {
121       val base = layers.subList(0, index).lastOrNull { !it.isClipping }
122       ?: return AnimationFrameGroupingValidation.MissingClippingBase(layer.id)
123       baseByClippingId[layer.id] = base.id
124       if (layer.id in selected && base.id !in selected) {
125         return AnimationFrameGroupingValidation.MissingClippingLayer(layer.id, base.id)
126       }
127     }
128   }
129   baseByClippingId.forEach { (clippingId, baseId) ->
130     if (baseId in selected && clippingId !in selected) {
131       return AnimationFrameGroupingValidation.MissingClippingLayer(baseId, clippingId)
132     }
133   }
134   return AnimationFrameGroupingValidation.Valid(layers.filter { it.id in selected }.map { it.id })
135 }
136
137 /**
138  * Expands frames into a flat list of frame IDs to be played back, accounting for holdFrames.
139  */
140 fun expandedPlaybackFrameIds(
141   frames: List<AnimationFrame>,
142 ): List<Long> {
143   return frames.flatMap { frame ->
144     List(frame.holdFrames) { frame.id }
145   }
146 }
147
148 data class PlaybackStep(
149   val nextIndex: Int,
150   val nextDirection: Int, // 1 for forward, -1 for backward
151   val shouldStop: Boolean = false
152 )
153
154 /**
155  * Calculates the next index in the playback sequence.
156  */
157 fun nextPlaybackIndex(
158   currentIndex: Int,
159   direction: Int,
160   frameCount: Int,
161   mode: AnimationPlaybackMode,
162 ): PlaybackStep {
163   if (frameCount <= 0) return PlaybackStep(0, 1, true)
164   if (frameCount == 1) return PlaybackStep(0, 1, mode == AnimationPlaybackMode.ONE_SHOT)
165
166   var nextDir = direction
167   var nextIdx = currentIndex + direction
168   var stop = false
169
170   when (mode) {
171     AnimationPlaybackMode.LOOP -> {
172       if (nextIdx >= frameCount) nextIdx = 0
173       if (nextIdx < 0) nextIdx = frameCount - 1
174     }
175     AnimationPlaybackMode.PING_PONG -> {
176       if (nextIdx >= frameCount) {
177         nextIdx = (frameCount - 2).coerceAtLeast(0)
178         nextDir = -1
179       } else if (nextIdx < 0) {
180         nextIdx = (1).coerceAtMost(frameCount - 1)
181         nextDir = 1
182       }
183     }
184   }

```

```
183     }
184     AnimationPlaybackMode.ONE_SHOT -> {
185         if (nextIdx >= frameCount) {
186             nextIdx = frameCount - 1
187             stop = true
188         }
189     }
190 }
191
192 return PlaybackStep(nextIdx, nextDir, stop)
193 }
```


15. app/src/main/java/com/wetinknext/domain/document/ProjectDocument.kt

```

1 package com.wetinknext.domain.document
2
3 import com.wetinknext.domain.animation.AnimationDocument
4 import com.wetinknext.engine.brush.BrushPreset
5 import com.wetinknext.engine.canvas.BlendMode
6 import kotlinx.serialization.Serializable
7 import java.util.UUID
8
9 /**
10  * Persisted project structure. Pixel data deliberately lives outside this JSON
11  * document: [LayerDocument.pixelFile] points at its future tile/container data.
12  */
13 @Serializable
14 data class ProjectDocument(
15     val id: String,
16     val name: String,
17     val width: Int,
18     val height: Int,
19     val dpi: Int,
20     val colorProfile: String,
21     val createdAt: Long,
22     val updatedAt: Long,
23     val layers: List<LayerDocument>,
24     val brushes: List<BrushPreset> = emptyList(),
25     val animation: AnimationDocument = AnimationDocument(),
26     val activeLayerId: Long? = null,
27     /** Changes only when the project-wide preview is rebuilt. */
28     val thumbnailVersion: Long = 0L,
29     val version: Int = CURRENT_VERSION,
30 ) {
31     init {
32         require(id.isNotBlank()) { "Project id must not be blank" }
33         require(name.isNotBlank()) { "Project name must not be blank" }
34         require(width > 0 && height > 0) { "Project dimensions must be positive" }
35         require(dpi in 1..2400) { "Project DPI must be within 1..2400" }
36         require(colorProfile.isNotBlank()) { "Project color profile must not be blank" }
37         require(layers.isNotEmpty()) { "A project must contain at least one layer" }
38         require(layers.map(LayerDocument::id).distinct().size == layers.size) {
39             "Layer ids must be unique"
40         }
41         require(activeLayerId == null || layers.any { it.id == activeLayerId }) {
42             "Active layer must belong to the project"
43         }
44         require(version >= 1) { "Project version must be positive" }
45         require(thumbnailVersion >= 0L) { "Project thumbnail version must not be negative" }
46     }
47
48     fun withUpdatedTimestamp(nowMillis: Long): ProjectDocument = copy(updatedAt = nowMillis)
49
50     companion object {
51         const val CURRENT_VERSION = 1
52         const val DEFAULT_WIDTH = 1500
53         const val DEFAULT_HEIGHT = 2000
54         const val DEFAULT_DPI = 300
55         const val SRGB_PROFILE = "sRGB IEC61966-2.1"
56         const val DISPLAY_P3_PROFILE = "Display P3"
57         const val MIN_DIMENSION = 256
58         const val MAX_DIMENSION = 8192
59         const val MIN_DPI = 72
60         const val MAX_DPI = 1200
61         const val MAX_INITIAL_GPU_BYTES = 256L * 1024L * 1024L
62         private const val INITIAL_RENDER_TARGET_COUNT = 3L
63         private const val RGBA16F_BYTES_PER_PIXEL = 8L
64         val SUPPORTED_COLOR_PROFILES = setOf(SRGB_PROFILE, DISPLAY_P3_PROFILE)
65
66         fun newUntitled(
67             name: String = "Untitled",
68             width: Int = DEFAULT_WIDTH,
69             height: Int = DEFAULT_HEIGHT,
70             dpi: Int = DEFAULT_DPI,
71             colorProfile: String = SRGB_PROFILE,
72             nowMillis: Long = System.currentTimeMillis(),
73             id: String = UUID.randomUUID().toString(),
74         ): ProjectDocument {
75             val normalizedProfile = normalizeColorProfile(colorProfile)
76             validateNewCanvas(width, height, dpi, normalizedProfile)
77             return ProjectDocument(
78                 id = id,
79                 name = name,
80                 width = width,
81                 height = height,
82                 dpi = dpi,
83                 colorProfile = normalizedProfile,
84                 createdAt = nowMillis,
85                 updatedAt = nowMillis,
86                 layers = listOf(
87                     LayerDocument(
88                         id = BACKGROUND_LAYER_ID,
89                         name = "Фон",
90                         visible = true,

```

```

91         locked = true,
92         opacity = 1f,
93         blendMode = BlendMode.NORMAL,
94         pixelFile = pixelFileFor(BACKGROUND_LAYER_ID),
95     ),
96     LayerDocument(
97         id = FIRST_PAINT_LAYER_ID,
98         name = "Слой 1",
99         visible = true,
100        locked = false,
101        opacity = 1f,
102        blendMode = BlendMode.NORMAL,
103        pixelFile = pixelFileFor(FIRST_PAINT_LAYER_ID),
104    ),
105 ),
106 activeLayerId = FIRST_PAINT_LAYER_ID,
107 )
108 }
109
110 /** Validates the initial two layer targets plus the transient stroke target. */
111 fun validateNewCanvas(width: Int, height: Int, dpi: Int, colorProfile: String) {
112     require(width in MIN_DIMENSION..MAX_DIMENSION) { "Canvas width must be within $MIN_DIMENSION..$MAX_DIMENSION px" }
113     require(height in MIN_DIMENSION..MAX_DIMENSION) { "Canvas height must be within $MIN_DIMENSION..$MAX_DIMENSION
px" }
114     require(dpi in MIN_DPI..MAX_DPI) { "Canvas DPI must be within $MIN_DPI..$MAX_DPI" }
115     require(normalizeColorProfile(colorProfile) in SUPPORTED_COLOR_PROFILES) { "Unsupported color profile:
$colorProfile" }
116     val estimatedBytes = width.toLong() * height * INITIAL_RENDER_TARGET_COUNT * RGBA16F_BYTES_PER_PIXEL
117     require(estimatedBytes <= MAX_INITIAL_GPU_BYTES) {
118         "Canvas requires ${estimatedBytes / (1024 * 1024)} MB GPU memory; limit is ${MAX_INITIAL_GPU_BYTES / (1024 *
1024)} MB"
119     }
120 }
121
122 /** Keeps older UI values (`sRGB`) compatible with the persisted profile identifier. */
123 fun normalizeColorProfile(value: String): String = when (value.trim()) {
124     "sRGB", SRGB_PROFILE -> SRGB_PROFILE
125     "Display P3", DISPLAY_P3_PROFILE -> DISPLAY_P3_PROFILE
126     else -> value.trim()
127 }
128
129 fun pixelFileFor(layerId: Long): String = "layers/$layerId.tiles"
130
131 private const val BACKGROUND_LAYER_ID = 1L
132 private const val FIRST_PAINT_LAYER_ID = 2L
133 }
134 }
135
136 @Serializable
137 data class LayerDocument(
138     val id: Long,
139     val name: String,
140     val visible: Boolean,
141     val locked: Boolean,
142     val opacity: Float,
143     val blendMode: BlendMode,
144     val pixelFile: String,
145     /** Increments whenever the runtime layer pixels change; useful for preview invalidation. */
146     val thumbnailVersion: Long = 0L,
147 ) {
148     init {
149         require(id >= 0L) { "Layer id must not be negative" }
150         require(name.isNotBlank()) { "Layer name must not be blank" }
151         require(opacity in 0f..1f) { "Layer opacity must be within 0..1" }
152         require(pixelFile.isNotBlank()) { "Layer pixel file must not be blank" }
153         require(thumbnailVersion >= 0L) { "Layer thumbnail version must not be negative" }
154     }
155 }

```

16. app/src/main/java/com/wetinknext/domain/document/WetInkProjectContainer.kt

```

1  package com.wetinknext.domain.document
2
3  import kotlinx.serialization.Serializable
4  import kotlinx.serialization.decodeFromString
5  import kotlinx.serialization.encodeToString
6  import kotlinx.serialization.json.Json
7  import java.io.BufferedInputStream
8  import java.io.BufferedOutputStream
9  import java.io.ByteArrayOutputStream
10 import java.io.File
11 import java.io.IOException
12 import java.nio.file.AtomicMoveNotSupportedException
13 import java.nio.file.Files
14 import java.nio.file.StandardCopyOption
15 import java.util.zip.ZipEntry
16 import java.util.zip.ZipFile
17 import java.util.zip.ZipOutputStream
18
19 @Serializable
20 data class ProjectManifest(
21     val format: String = FORMAT,
22     val formatVersion: Int = FORMAT_VERSION,
23     val documentId: String,
24     val name: String,
25     val width: Int,
26     val height: Int,
27     val documentPath: String = DOCUMENT_ENTRY,
28     val thumbnailPath: String? = null,
29     val thumbnailVersion: Long = 0L,
30     val createdAt: Long,
31     val updatedAt: Long,
32     val layers: List<StoredLayer>,
33 ) {
34     companion object {
35         const val FORMAT = "wetink"
36         const val FORMAT_VERSION = 1
37         const val DOCUMENT_ENTRY = "document.json"
38     }
39 }
40
41 @Serializable
42 data class StoredLayer(
43     val id: Long,
44     val path: String,
45     val byteSize: Long,
46 )
47
48 /** Binary payloads deliberately sit next to the JSON document, not inside it. */
49 data class ProjectAssets(
50     val layerFiles: Map<Long, ByteArray>,
51     val thumbnailWebp: ByteArray? = null,
52     val layerPreviewsWebp: Map<Long, ByteArray> = emptyMap(),
53 )
54
55 data class LoadedWetInkProject(
56     val document: ProjectDocument,
57     val manifest: ProjectManifest,
58     val assets: ProjectAssets,
59 )
60
61 /**
62  * Project ZIP reader/writer. A project is published only by an atomic rename
63  * from `name.wetink.tmp` to `name.wetink`, so a failed save cannot corrupt the
64  * previously published project.
65  */
66 class WetInkProjectContainer(
67     private val clockMillis: () -> Long = System::currentTimeMillis,
68     private val json: Json = Json {
69         encodeDefaults = true
70         ignoreUnknownKeys = true
71         prettyPrint = true
72     },
73 ) {
74     fun save(
75         projectFile: File,
76         document: ProjectDocument,
77         assets: ProjectAssets,
78     ): LoadedWetInkProject {
79         require(projectFile.extension.equals(FILE_EXTENSION, ignoreCase = true)) {
80             "Project file must use .$FILE_EXTENSION extension"
81         }
82         validateAssets(document, assets)
83
84         val savedTimestamp = clockMillis()
85         val savedDocument = document.copy(
86             updatedAt = savedTimestamp,
87             thumbnailVersion = if (assets.thumbnailWebp != null) savedTimestamp else document.thumbnailVersion,
88         )
89         val manifest = ProjectManifest(
90             documentId = savedDocument.id,

```

```

91         name = savedDocument.name,
92         width = savedDocument.width,
93         height = savedDocument.height,
94         thumbnailPath = assets.thumbnailWebp?.let { THUMBNAIL_ENTRY },
95         thumbnailVersion = savedDocument.thumbnailVersion,
96         createdAt = savedDocument.createdAt,
97         updatedAt = savedDocument.updatedAt,
98         layers = savedDocument.layers.map { layer ->
99             StoredLayer(
100                 id = layer.id,
101                 path = layer.pixelFile,
102                 byteSize = checkNotNull(assets.layerTiles[layer.id]).size.toLong(),
103             )
104         },
105     )
106
107     val parent = projectFile.parentFile ?: throw IOException("Project file requires a parent folder")
108     if (!parent.exists() && !parent.mkdirs()) throw IOException("Cannot create ${parent.path}")
109     val temporaryFile = File(parent, "${projectFile.name}.tmp")
110     if (temporaryFile.exists() && !temporaryFile.delete()) {
111         throw IOException("Cannot replace stale temporary project ${temporaryFile.path}")
112     }
113
114     var published = false
115     try {
116         BufferedOutputStream(temporaryFile.outputStream()).use { output ->
117             ZipOutputStream(output).use { zip ->
118                 // Write large layer data first; metadata becomes visible only
119                 // after every payload has been written and the ZIP is closed.
120                 savedDocument.layers.forEach { layer ->
121                     writeEntry(zip, layer.pixelFile, checkNotNull(assets.layerTiles[layer.id]))
122                 }
123                 writeEntry(zip, ProjectManifest.DOCUMENT_ENTRY, json.encodeToString(savedDocument).encodeToByteArray())
124                 assets.thumbnailWebp?.let { writeEntry(zip, THUMBNAIL_ENTRY, it) }
125                 assets.layerPreviewsWebp.forEach { (layerId, preview) ->
126                     writeEntry(zip, previewPathFor(layerId), preview)
127                 }
128                 writeEntry(zip, MANIFEST_ENTRY, json.encodeToString(manifest).encodeToByteArray())
129             }
130         }
131         if (temporaryFile.length() <= 0L) throw IOException("Temporary project is empty")
132         atomicPublish(temporaryFile, projectFile)
133         published = true
134     } finally {
135         if (!published && temporaryFile.exists()) temporaryFile.delete()
136     }
137
138     return LoadedWetInkProject(savedDocument, manifest, assets)
139 }
140
141 fun load(projectFile: File): LoadedWetInkProject {
142     require(projectFile.isFile) { "Project does not exist: ${projectFile.path}" }
143     ZipFile(projectFile).use { zip ->
144         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
145         require(manifest.format == ProjectManifest.FORMAT) { "Unsupported project format" }
146         require(manifest.formatVersion == ProjectManifest.FORMAT_VERSION) { "Unsupported project version" }
147
148         val document = readJson<ProjectDocument>(zip, manifest.documentPath)
149         require(document.id == manifest.documentId) { "Manifest/document id mismatch" }
150         require(document.updatedAt == manifest.updatedAt) { "Manifest/document timestamp mismatch" }
151         require(manifest.layers.map { it.id } == document.layers.map { it.id }) {
152             "Manifest/document layer order mismatch"
153         }
154
155         val tiles = document.layers.associate { layer ->
156             val stored = manifest.layers.first { it.id == layer.id }
157             require(stored.path == layer.pixelFile) { "Layer path mismatch for ${layer.id}" }
158             val bytes = readEntry(zip, layer.pixelFile)
159             require(bytes.size.toLong() == stored.byteSize) { "Layer size mismatch for ${layer.id}" }
160             layer.id to bytes
161         }
162         val previews = document.layers.mapNotNull { layer ->
163             val entry = zip.getEntry(previewPathFor(layer.id)) ?: return@mapNotNull null
164             layer.id to readEntry(zip, entry)
165         }.toMap()
166         val thumbnail = manifest.thumbnailPath?.let { readEntry(zip, it) }
167
168         return LoadedWetInkProject(
169             document = document,
170             manifest = manifest,
171             assets = ProjectAssets(tiles, thumbnail, previews),
172         )
173     }
174 }
175
176 /** Reads just the compact manifest; no document JSON or tile payload is loaded. */
177 fun readManifest(projectFile: File): ProjectManifest {
178     require(projectFile.isFile) { "Project does not exist: ${projectFile.path}" }
179     ZipFile(projectFile).use { zip ->
180         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
181         require(manifest.format == ProjectManifest.FORMAT) { "Unsupported project format" }
182         require(manifest.formatVersion == ProjectManifest.FORMAT_VERSION) { "Unsupported project version" }

```

```

183         return manifest
184     }
185 }
186
187 fun readThumbnail(projectFile: File): ByteArray? {
188     ZipFile(projectFile).use { zip ->
189         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
190         return manifest.thumbnailPath?.let { readEntry(zip, it) }
191     }
192 }
193
194 /** Opens only project metadata. Tile bytes remain in the ZIP until requested. */
195 fun readDocument(projectFile: File): ProjectDocument {
196     require(projectFile.isFile) { "Project does not exist: ${projectFile.path}" }
197     ZipFile(projectFile).use { zip ->
198         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
199         require(manifest.format == ProjectManifest.FORMAT) { "Unsupported project format" }
200         require(manifest.formatVersion == ProjectManifest.FORMAT_VERSION) { "Unsupported project version" }
201         val document = readJson<ProjectDocument>(zip, manifest.documentPath)
202         require(document.id == manifest.documentId) { "Manifest/document id mismatch" }
203         require(document.updatedAt == manifest.updatedAt) { "Manifest/document timestamp mismatch" }
204         return document
205     }
206 }
207
208 private fun validateAssets(document: ProjectDocument, assets: ProjectAssets) {
209     val expectedIds = document.layers.map { it.id }.toSet()
210     require(assets.layerTiles.keys == expectedIds) { "Every document layer requires one tile payload" }
211     require(assets.layerPreviewsWebp.keys.all(expectedIds::contains)) {
212         "Preview references an unknown layer"
213     }
214     document.layers.forEach { layer ->
215         require(isSafeLayerPath(layer.pixelFile)) { "Unsafe layer path: ${layer.pixelFile}" }
216     }
217 }
218
219 private fun atomicPublish(temporaryFile: File, projectFile: File) {
220     try {
221         Files.move(
222             temporaryFile.toPath(),
223             projectFile.toPath(),
224             StandardCopyOption.ATOMIC_MOVE,
225             StandardCopyOption.REPLACE_EXISTING,
226         )
227     } catch (error: AtomicMoveNotSupportedException) {
228         throw IOException("Atomic project publish is not supported for ${projectFile.path}", error)
229     }
230 }
231
232 private fun writeEntry(zip: ZipOutputStream, path: String, bytes: ByteArray) {
233     zip.putNextEntry(ZipEntry(path))
234     zip.write(bytes)
235     zip.closeEntry()
236 }
237
238 private inline fun <reified T> readJson(zip: ZipFile, path: String): T =
239     json.decodeFromString(readEntry(zip, path).decodeToString())
240
241 private fun readEntry(zip: ZipFile, path: String): ByteArray {
242     val entry = zip.getEntry(path) ?: throw IOException("Missing project entry: $path")
243     return readEntry(zip, entry)
244 }
245
246 private fun readEntry(zip: ZipFile, entry: ZipEntry): ByteArray {
247     if (entry.size > MAX_ENTRY_BYTES) throw IOException("Project entry is too large: ${entry.name}")
248     BufferedInputStream(zip.getInputStream(entry)).use { input ->
249         val output = ByteArrayOutputStream(entry.size.coerceAtLeast(0L).coerceAtMost(MAX_ENTRY_BYTES).toInt())
250         val buffer = ByteArray(READ_BUFFER_BYTES)
251         var total = 0L
252         while (true) {
253             val read = input.read(buffer)
254             if (read < 0) break
255             total += read
256             if (total > MAX_ENTRY_BYTES) {
257                 throw IOException("Project entry is too large: ${entry.name}")
258             }
259             output.write(buffer, 0, read)
260         }
261         return output.toByteArray()
262     }
263 }
264
265 private fun isSafeLayerPath(path: String): Boolean =
266     path.startsWith("layers/") && !path.contains("..") && !path.startsWith('/')
267
268 private companion object {
269     const val FILE_EXTENSION = "wetink"
270     const val MANIFEST_ENTRY = "manifest.json"
271     const val THUMBNAIL_ENTRY = "thumbnail.webp"
272     const val MAX_ENTRY_BYTES = 256L * 1024L * 1024L
273     const val READ_BUFFER_BYTES = 16 * 1024
274 }

```

```
275         fun previewPathFor(layerId: Long): String = "previews/$layerId.webp"
276     }
277 }
```

17. app/src/main/java/com/wetinknext/engine/brush/BlendController.kt

```
1 package com.wetinknext.engine.brush
2
3 import android.opengl.GLES30
4
5 class BlendController {
6
7     fun begin(policy: BlendPolicy) {
8         GLES30.glEnable(GLES30.GL_BLEND)
9         // NON_BUILDUP needs a dedicated accumulation mask. Until that pass is
10        // introduced, both policies use the correct premultiplied source-over.
11        GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
12        GLES30.glBlendFunc(
13            GLES30.GL_ONE,
14            GLES30.GL_ONE_MINUS_SRC_ALPHA,
15        )
16    }
17
18    fun end() {
19        GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
20        GLES30.glBlendFunc(
21            GLES30.GL_ONE,
22            GLES30.GL_ONE_MINUS_SRC_ALPHA,
23        )
24        GLES30.glDisable(GLES30.GL_BLEND)
25    }
26 }
```

18. app/src/main/java/com/wetinknext/engine/brush/BrushDynamics.kt

```

1 package com.wetinknext.engine.brush
2
3 import kotlin.math.atan2
4 import kotlin.math.cos
5 import kotlin.math.pow
6 import kotlin.math.sqrt
7 import kotlin.math.sin
8
9 object BrushDynamics {
10
11     fun tiltMagnitude(
12         tiltX: Float,
13         tiltY: Float,
14     ): Float {
15         return sqrt(
16             tiltX * tiltX +
17             tiltY * tiltY,
18         ).coerceIn(0f, 1f)
19     }
20
21     /** Resolves all local, deterministic properties of one STAMP dab. */
22     fun resolve(
23         settings: BrushSettings,
24         pressure: Float,
25         tiltX: Float,
26         tiltY: Float,
27         velocityPxPerSecond: Float,
28         random01: Float,
29         out: ResolvedDab,
30     ) {
31         val p = pressure
32             .coerceIn(0f, 1f)
33             .let { it.toDouble().pow(settings.pressureGamma.toDouble()).toFloat() }
34
35         val tilt = tiltMagnitude(tiltX, tiltY)
36
37         val pressureSize = if (settings.pressureToSize) {
38             settings.minSizeRatio + (1f - settings.minSizeRatio) * p
39         } else {
40             1f
41         }
42         val velocityNorm = (velocityPxPerSecond / VELOCITY_REFERENCE_PX_PER_SECOND)
43             .coerceIn(0f, 1f)
44         val velocitySize = 1f - settings.velocityToSize * velocityNorm
45         val tiltSize = 1f + settings.tiltToSize * tilt
46         val random = random01.coerceIn(0f, 1f)
47         val jitterSize = 1f + ((random * 2f) - 1f) * settings.sizeJitter
48         out.radius = (
49             settings.baseRadiusPx * pressureSize * velocitySize * tiltSize * jitterSize
50         ).coerceAtLeast(0.25f)
51
52         val pressureCoverage = if (settings.pressureToOpacity) p else 1f
53         val velocityCoverage = 1f - settings.velocityToOpacity * velocityNorm
54
55         // Tilt must not make a dab more opaque.
56         val tiltCoverage = 1f - (0.45f * settings.tiltToOpacity * tilt)
57         val jitterCoverage = 1f + ((random * 2f) - 1f) * settings.opacityJitter
58         out.coverage = (
59             pressureCoverage * velocityCoverage * tiltCoverage * jitterCoverage
60         ).coerceIn(0f, 1f)
61
62         out.flow = settings.flow.coerceIn(0f, 1f)
63         out.hardness = settings.hardness.coerceIn(0f, 1f)
64         out.rotationRad = tiltAngle(tiltX, tiltY) * settings.tiltToRotation
65
66         val scatterRadius = settings.scatter.coerceAtLeast(0f) * out.radius
67         val scatterAngle = random * (Math.PI.toFloat() * 2f)
68         out.scatterX = cos(scatterAngle) * scatterRadius
69         out.scatterY = sin(scatterAngle) * scatterRadius
70     }
71
72     private fun tiltAngle(tiltX: Float, tiltY: Float): Float = atan2(tiltY, tiltX)
73
74     private const val VELOCITY_REFERENCE_PX_PER_SECOND = 2_400f
75 }
76
77 data class ResolvedDab(
78     var radius: Float = 0f,
79     var rotationRad: Float = 0f,
80     var coverage: Float = 1f,
81     var flow: Float = 1f,
82     var hardness: Float = 1f,
83     var scatterX: Float = 0f,
84     var scatterY: Float = 0f,
85 )

```


19. app/src/main/java/com/wetinknext/engine/brush/BrushLibrary.kt

```

1  package com.wetinknext.engine.brush
2
3  import androidx.compose.material.icons.Icons
4  import androidx.compose.material.icons.filled.Brush
5  import androidx.compose.material.icons.filled.Create
6  import androidx.compose.material.icons.filled.Star
7  import androidx.compose.ui.graphics.vector.ImageVector
8
9  data class BrushUiCategory(
10     val name: String,
11     val icon: ImageVector,
12     val brushes: List<BrushPreset>,
13 )
14
15 object BrushLibrary {
16     /**
17      * Soft, low-flow airbrush. Kept in the stamp pipeline because its look is
18      * built from many translucent dabs, rather than a single ribbon contour.
19      */
20     val airbrush = BrushPreset(
21         id = "airbrush",
22         settings = BrushSettings(
23             name = "Airbrush",
24             category = "Paint",
25             renderMode = BrushRenderMode.STAMP,
26             baseRadiusPx = 80f,
27             opacity = 0.18f,
28             flow = 0.08f,
29             spacing = 0.035f,
30             spacingUsesDiameter = true,
31             pressureToSize = false,
32             pressureToOpacity = true,
33             pressureGamma = 1.15f,
34             hardness = 0f,
35             antiAliasLevel = 3,
36             blendPolicy = BlendPolicy.NORMAL_BUILDUP,
37             useTempStrokeBuffer = true,
38         ),
39     )
40
41     val pencil6B = BrushPreset(
42         id = "pencil_6b",
43         settings = BrushSettings(
44             name = "Pencil 6B",
45             category = "Pencil",
46             renderMode = BrushRenderMode.STAMP,
47             baseRadiusPx = 8f,
48             opacity = 0.82f,
49             flow = 0.65f,
50             spacing = 0.08f,
51             spacingUsesDiameter = true,
52             hardness = 0.72f,
53             smoothing = 0.08f,
54             streamline = 0.04f,
55             pressureToSize = true,
56             pressureToOpacity = true,
57             pressureGamma = 1.25f,
58             minSizeRatio = 0.30f,
59             tiltToSize = 0.55f,
60             grainAssetPath = "asset:brush/pencil_6b_grain.png",
61             grainCanvasLocked = true,
62             grainScale = 5.5f,
63             textureDepth = 0.65f,
64             textureContrast = 1.35f,
65             blendPolicy = BlendPolicy.NORMAL_BUILDUP,
66         ),
67     )
68
69     val gPen = BrushPreset(
70         id = "g_pen",
71         settings = BrushSettings(
72             name = "G-Pen",
73             category = "Ink",
74             renderMode = BrushRenderMode.RIBBON,
75             baseRadiusPx = 4.5f,
76             opacity = 1f,
77             flow = 1f,
78             smoothing = 0.35f,
79             streamline = 0.22f,
80             pressureToSize = true,
81             pressureToOpacity = false,
82             pressureGamma = 1.7f,
83             minSizeRatio = 0.06f,
84             blendPolicy = BlendPolicy.NON_BUILDUP,
85         )
86     )
87
88     /** Baseline ink preset for checking the capsule path without geometric taper. */
89     val pen = BrushPreset(
90         id = "pen",

```

```

91         settings = gPen.settings.copy(
92             name = "Pen",
93             ribbon = gPen.settings.ribbon.copy(
94                 taperStartPx = 0f,
95                 taperEndPx = 0f,
96             ),
97         ),
98     )
99
100     val allCategories = listOf(
101         BrushUiCategory(
102             name = "Paint",
103             icon = Icons.Default.Brush,
104             brushes = listOf(airbrush),
105         ),
106         BrushUiCategory(
107             name = "Pencil",
108             icon = Icons.Default.Create,
109             brushes = listOf(pencil6B)
110         ),
111         BrushUiCategory(
112             name = "Ink",
113             icon = Icons.Default.Brush,
114             brushes = listOf(gPen, pen)
115         ),
116         BrushUiCategory(
117             name = "Favorites",
118             icon = Icons.Default.Star,
119             brushes = emptyList()
120         )
121     )
122 }

```

20. app/src/main/java/com/wetinknext/engine/brush/BrushPreset.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlinx.serialization.Serializable
4
5 @Serializable
6 data class BrushPreset(
7     val id: String,
8     val settings: BrushSettings,
9 )
```

21. app/src/main/java/com/wetinknext/engine/brush/BrushRepository.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlinx.serialization.encodeToString
4 import kotlinx.serialization.json.Json
5
6 class BrushRepository(
7     private val json: Json = defaultJson,
8 ) {
9     fun encode(preset: BrushPreset): String =
10         json.encodeToString(preset)
11
12     fun decode(value: String): BrushPreset =
13         json.decodeFromString<BrushPreset>(value).let { preset ->
14             preset.copy(settings = preset.settings.resolved())
15         }
16
17     companion object {
18         val defaultJson = Json {
19             encodeDefaults = true
20             ignoreUnknownKeys = true
21             explicitNulls = false
22         }
23     }
24 }
```

22. app/src/main/java/com/wetinknext/engine/brush/BrushSettings.kt

```

1  package com.wetinknext.engine.brush
2
3  import kotlinx.serialization.Serializable
4
5  @Serializable
6  enum class BrushRenderMode { STAMP, RIBBON, WET }
7
8  @Serializable
9  enum class BlendPolicy { NORMAL_BUILDUP, NON_BUILDUP }
10
11 @Serializable
12 enum class RibbonCap { ROUND, BUTT }
13
14 @Serializable
15 enum class RibbonJoin { MITER, ROUND, BEVEL }
16
17 @Serializable
18 data class RibbonSettings(
19     val minWidthRatio: Float = 0.06f,
20     val taperStartPx: Float = 8f,
21     val taperEndPx: Float = 14f,
22     val miterLimit: Float = 3f,
23     val aaWidthPx: Float = 1f,
24     val cap: RibbonCap = RibbonCap.ROUND,
25     val join: RibbonJoin = RibbonJoin.ROUND,
26     val minPointDistancePx: Float = 0.5f,
27 )
28
29 @Serializable
30 data class WetSettings(val wetness: Float = 0f, val spread: Float = 0f, val bleed: Float = 0f)
31
32 @Serializable
33 data class BrushSettings(
34     val name: String = "Debug Stamp",
35     val category: String = "Debug",
36     val renderMode: BrushRenderMode = BrushRenderMode.STAMP,
37     /**
38      * Internal brush radius in document pixels.
39      * UI displays diameter = baseRadiusPx * 2.
40      */
41     val baseRadiusPx: Float = 14f,
42     val opacity: Float = 1f,
43     val flow: Float = 1f,
44     val spacing: Float = 0.16f,
45     val spacingUsesDiameter: Boolean = true,
46     val hardness: Float = 1f,
47     val colorArgb: Long = 0xFF000000L,
48     val smoothing: Float = 0f,
49     val streamline: Float = 0f,
50     val antiAliasLevel: Int = 2,
51     val useTempStrokeBuffer: Boolean = false,
52     val pressureToSize: Boolean = true,
53     val pressureToOpacity: Boolean = true,
54     val pressureGamma: Float = 1f,
55     val minSizeRatio: Float = 0.05f,
56     val velocityToSize: Float = 0f,
57     val velocityToOpacity: Float = 0f,
58     val tiltToSize: Float = 0f,
59     val tiltToOpacity: Float = 0f,
60     val tiltToRotation: Float = 0f,
61     val rotationJitter: Float = 0f,
62     val sizeJitter: Float = 0f,
63     val opacityJitter: Float = 0f,
64     val scatter: Float = 0f,
65     val shapeAssetPath: String? = null,
66     val grainAssetPath: String? = null,
67     val grainAsset: Float = 1f,
68     val grainCanvasLocked: Boolean = true,
69     val rgbToAlpha: Boolean = false,
70     val textureContrast: Float = 1f,
71     val textureDepth: Float = 1f,
72     val pixelPen: Boolean = false,
73     val squareStroke: Boolean = false,
74     val noAntialias: Boolean = false,
75     val blendPolicy: BlendPolicy = BlendPolicy.NORMAL_BUILDUP,
76     val ribbon: RibbonSettings = RibbonSettings(),
77     val wet: WetSettings = WetSettings(),
78 ) {
79     /** Maps the persisted brush policy to the explicit runtime stroke mode. */
80     val strokeRenderMode: StrokeRenderMode
81         get() = when (blendPolicy) {
82             BlendPolicy.NORMAL_BUILDUP -> StrokeRenderMode.NORMAL_BUILDUP
83             BlendPolicy.NON_BUILDUP -> StrokeRenderMode.NON_BUILDUP
84         }
85
86     /** Current engine-supported runtime stroke mode for this brush. */
87     val effectiveStrokeRenderMode: StrokeRenderMode
88         get() = strokeRenderMode
89
90     /** Returns a settings copy safe for every engine path. */

```

```

91 fun resolved(): BrushSettings {
92     val safeRadius = baseRadiusPx.coerceIn(0.25f, 4096f)
93     val safeOpacity = opacity.coerceIn(0f, 1f)
94     val safeFlow = flow.coerceIn(0f, 1f)
95     val safeSpacing = spacing.coerceIn(0.001f, 4f)
96     val safeGamma = pressureGamma.coerceIn(0.05f, 8f)
97     val safeMinSize = minSizeRatio.coerceIn(0.01f, 1f)
98
99     val safeVelocityToSize = velocityToSize.coerceIn(0f, 1f)
100    val safeVelocityToOpacity = velocityToOpacity.coerceIn(0f, 1f)
101    val safeTiltToSize = tiltToSize.coerceIn(0f, 1f)
102    val safeTiltToOpacity = tiltToOpacity.coerceIn(0f, 1f)
103    val safeTiltToRotation = tiltToRotation.coerceIn(0f, 1f)
104    val safeSizeJitter = sizeJitter.coerceIn(0f, 1f)
105    val safeOpacityJitter = opacityJitter.coerceIn(0f, 1f)
106    val safeRotationJitter = rotationJitter.coerceIn(0f, 1f)
107    val safeScatter = scatter.coerceAtLeast(0f)
108
109    val safeGrainScale = grainScale.coerceIn(0.0001f, 256f)
110    val safeTextureDepth = textureDepth.coerceIn(0f, 1f)
111    val safeTextureContrast = textureContrast.coerceIn(0f, 4f)
112    val safeAa = antiAliasLevel.coerceIn(0, 3)
113
114    val safeRibbon = ribbon.copy(
115        minWidthRatio = ribbon.minWidthRatio.coerceIn(0.01f, 1f),
116        taperStartPx = ribbon.taperStartPx.coerceAtLeast(0f),
117        taperEndPx = ribbon.taperEndPx.coerceAtLeast(0f),
118        miterLimit = ribbon.miterLimit.coerceIn(1f, 32f),
119        aaWidthPx = ribbon.aaWidthPx.coerceIn(0f, 16f),
120        minPointDistancePx = ribbon.minPointDistancePx.coerceIn(0.05f, 32f),
121    )
122    val safeWet = wet.copy(
123        wetness = wet.wetness.coerceIn(0f, 1f),
124        spread = wet.spread.coerceIn(0f, 1f),
125        bleed = wet.bleed.coerceIn(0f, 1f),
126    )
127
128    return copy(
129        baseRadiusPx = safeRadius,
130        opacity = safeOpacity,
131        flow = safeFlow,
132        spacing = safeSpacing,
133        pressureGamma = safeGamma,
134        minSizeRatio = safeMinSize,
135        velocityToSize = safeVelocityToSize,
136        velocityToOpacity = safeVelocityToOpacity,
137        tiltToSize = safeTiltToSize,
138        tiltToOpacity = safeTiltToOpacity,
139        tiltToRotation = safeTiltToRotation,
140        sizeJitter = safeSizeJitter,
141        opacityJitter = safeOpacityJitter,
142        rotationJitter = safeRotationJitter,
143        scatter = safeScatter,
144        grainScale = safeGrainScale,
145        textureDepth = safeTextureDepth,
146        textureContrast = safeTextureContrast,
147        antiAliasLevel = safeAa,
148        ribbon = safeRibbon,
149        wet = safeWet,
150    )
151 }
152 }

```

23. app/src/main/java/com/wetinknext/engine/brush/BrushTexture.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.GlCheck
5  import java.nio.ByteBuffer
6
7  class BrushTexture {
8      var textureId: Int = 0
9      private set
10
11     var width: Int = 0
12     private set
13
14     var height: Int = 0
15     private set
16
17     fun createFromRgba(
18         width: Int,
19         height: Int,
20         rgba: ByteBuffer,
21     ) {
22         GlCheck.checkOnGThread()
23         release()
24
25         val ids = IntArray(1)
26         GLES30.glGenTextures(1, ids, 0)
27         textureId = ids[0]
28
29         check(textureId != 0) { "Failed to create brush texture" }
30
31         this.width = width
32         this.height = height
33
34         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, textureId)
35
36         GLES30.glTexParameteri(
37             GLES30.GL_TEXTURE_2D,
38             GLES30.GL_TEXTURE_MIN_FILTER,
39             GLES30.GL_LINEAR,
40         )
41         GLES30.glTexParameteri(
42             GLES30.GL_TEXTURE_2D,
43             GLES30.GL_TEXTURE_MAG_FILTER,
44             GLES30.GL_LINEAR,
45         )
46         GLES30.glTexParameteri(
47             GLES30.GL_TEXTURE_2D,
48             GLES30.GL_TEXTURE_WRAP_S,
49             GLES30.GL_REPEAT,
50         )
51         GLES30.glTexParameteri(
52             GLES30.GL_TEXTURE_2D,
53             GLES30.GL_TEXTURE_WRAP_T,
54             GLES30.GL_REPEAT,
55         )
56
57         GLES30.glPixelStorei(
58             GLES30.GL_UNPACK_ALIGNMENT,
59             1,
60         )
61
62         GLES30.glTexImage2D(
63             GLES30.GL_TEXTURE_2D,
64             0,
65             GLES30.GL_RGBA,
66             width,
67             height,
68             0,
69             GLES30.GL_RGBA,
70             GLES30.GL_UNSIGNED_BYTE,
71             rgba,
72         )
73
74         GLES30.glPixelStorei(
75             GLES30.GL_UNPACK_ALIGNMENT,
76             4,
77         )
78
79         GLES30.glBindTexture(
80             GLES30.GL_TEXTURE_2D,
81             0,
82         )
83     }
84
85     fun release() {
86         GlCheck.checkOnGThread()
87         if (textureId != 0) {
88             GLES30.glDeleteTextures(
89                 1,
90                 intArrayOf(textureId),

```

```
91         0,  
92     )  
93 }  
94  
95     textureId = 0  
96     width = 0  
97     height = 0  
98 }  
99  
100 }
```


24. app/src/main/java/com/wetinknext/engine/brush/CapsuleEmitter.kt

```

1 package com.wetinknext.engine.brush
2
3 import com.wetinknext.engine.input.InputBatch
4 import kotlin.math.hypot
5 import kotlin.math.max
6 import kotlin.math.pow
7
8 /**
9  * Преобразует поток InputSample в цепочку капсул.
10  *
11  * Один сегмент:
12  *   A = x0, y0, radius0
13  *   B = x1, y1, radius1
14  *
15  * Капсула с одинаковыми радиусами является обычной capsule.
16  * При разном давлении shader строит round-cone.
17  *
18  * Важно:
19  * - не хранит InputSample;
20  * - не создаёт List/FloatArray на MOVE;
21  * - reusable state принадлежит GL/render pipeline;
22  * - pointerId фиксируется на DOWN;
23  * - давление применяется к радиусу;
24  * - opacity намеренно не применяется здесь, она применяется одним blit-pass.
25  */
26 class CapsuleEmitter(
27     private var settings: BrushSettings,
28 ) {
29     /** Retains the same resolved samples for the welded RIBBON path. */
30     private val ribbonGeometry = RibbonGeometryBuilder()
31     private var ribbonMesh: RibbonGeometryBuilder.Mesh? = null
32     private var ribbonMeshDirty = true
33     var ribbonMeshVersion: Long = 0L
34     private set
35     private var strokeSettings: BrushSettings? = null
36     private val stabilizer = Stabilizer()
37     private val pressureFilter = PressureFilter()
38     private val resolvedDab = ResolvedDab()
39
40     private var active = false
41     private var pointerId = -1
42
43     private var hasLastPoint = false
44     private var lastX = 0f
45     private var lastY = 0f
46     private var lastRadius = 0f
47     private var lastCoverage = 1f
48     // Reserved for a future oriented tip; the round capsule deliberately ignores it.
49     private var lastOrientation = 0f
50     private var lastTiltX = 0f
51     private var lastTiltY = 0f
52
53     // Окно точек для интерполяции P0, P1, P2, P3
54     private var p0x = 0f; private var p0y = 0f; private var p0r = 0f
55     private var p1x = 0f; private var p1y = 0f; private var p1r = 0f
56     private var p2x = 0f; private var p2y = 0f; private var p2r = 0f
57     private var p3x = 0f; private var p3y = 0f; private var p3r = 0f
58     private var p0c = 1f; private var p1c = 1f
59     private var p2c = 1f; private var p3c = 1f
60     private var pointsInWindow = 0
61
62     private var emittedSegments = 0
63
64     var hasStroke: Boolean = false
65     private set
66
67     var totalLength: Float = 0f
68     private set
69
70     /** Physical input distance, retained for stroke diagnostics and future dynamics. */
71     private var strokeDistance = 0f
72
73     var minX: Float = 0f
74     private set
75
76     var minY: Float = 0f
77     private set
78
79     var maxX: Float = 0f
80     private set
81
82     var maxY: Float = 0f
83     private set
84
85     var hasBounds: Boolean = false
86     private set
87
88     fun updateSettings(value: BrushSettings) {
89         // Active stroke reads strokeSettings; this value is for the next begin().
90         settings = value.resolved()

```

```

91     }
92
93     private fun activeSettings(): BrushSettings {
94         check(active) { "Capsule emitter has no active stroke" }
95         return strokeSettings ?: error("Missing stroke settings")
96     }
97
98     /**
99     * Полный сброс текущего stroke.
100    * overflowCount рендера здесь не сбрасывается.
101    */
102    fun reset() {
103        active = false
104        strokeSettings = null
105        pointerId = -1
106
107        hasLastPoint = false
108        lastX = 0f
109        lastY = 0f
110        lastRadius = 0f
111        lastCoverage = 1f
112        lastOrientation = 0f
113        lastTiltX = 0f
114        lastTiltY = 0f
115
116        pointsInWindow = 0
117
118        emittedSegments = 0
119        hasStroke = false
120        totalLength = 0f
121        strokeDistance = 0f
122
123        minX = 0f
124        minY = 0f
125        maxX = 0f
126        maxY = 0f
127        hasBounds = false
128
129        stabilizer.reset()
130        pressureFilter.reset()
131        ribbonGeometry.clear()
132        ribbonMesh = null
133        ribbonMeshDirty = true
134        ribbonMeshVersion++
135    }
136
137    /**
138    * Начало stroke.
139    *
140    * Первый sample добавляется как вырожденная капсула.
141    * В fragment shader она превращается в чистый диск.
142    */
143    fun begin(
144        batch: InputBatch,
145        out: CapsuleStrokeRenderer,
146        strokeSettings: BrushSettings = settings,
147    ) {
148        reset()
149        out.beginStroke()
150        val resolvedSettings = strokeSettings.resolved()
151        this.strokeSettings = resolvedSettings
152
153        if (batch.sampleCount <= 0) return
154
155        val sample = batch.samples[0]
156
157        active = true
158        pointerId = sample.pointerId
159
160        stabilizer.strength = (
161            resolvedSettings.smoothing * SMOOTHING_WEIGHT +
162            resolvedSettings.streamline * STREAMLINE_WEIGHT
163        ).coerceIn(0f, 1f)
164
165        stabilizer.process(
166            timestampNanos = sample.timestampNanos,
167            rawX = sample.canvasX,
168            rawY = sample.canvasY,
169        )
170
171        val x = stabilizer.x
172        val y = stabilizer.y
173        val filteredPressure = pressureFilter.filter(sample.timestampNanos, sample.pressure)
174        BrushDynamics.resolve(
175            settings = resolvedSettings,
176            pressure = filteredPressure,
177            tiltX = sample.tiltX,
178            tiltY = sample.tiltY,
179            velocityPxPerSecond = 0f,
180            random01 = 0.5f,
181            out = resolvedDab,
182        )

```

```

183         val radius = resolvedDab.radius
184         val coverage = resolvedDab.coverage
185
186         lastX = x
187         lastY = y
188         lastRadius = radius
189         lastCoverage = coverage
190         lastOrientation = resolvedDab.rotationRad
191         lastTiltX = sample.tiltX
192         lastTiltY = sample.tiltY
193         out.setStrokeRotation(resolvedDab.rotationRad)
194         hasLastPoint = true
195         hasStroke = true
196         ribbonGeometry.addPoint(x, y, radius, coverage, resolvedDab.rotationRad)
197         ribbonMeshDirty = true
198         ribbonMeshVersion++
199
200         // Инициализация окна
201         p0x = x; p0y = y; p0r = radius
202         p1x = x; p1y = y; p1r = radius
203         p2x = x; p2y = y; p2r = radius
204         p0c = coverage; p1c = coverage; p2c = coverage
205         pointsInWindow = 1
206
207         // Вырожденный сегмент: круглый dab/tap.
208         out.addSegment(
209             x0 = x,
210             y0 = y,
211             radius0 = radius,
212             coverage0 = coverage,
213             x1 = x,
214             y1 = y,
215             radius1 = radius,
216             coverage1 = coverage,
217         )
218         emittedSegments++
219
220         includeBounds(x, y, radius)
221     }
222
223     /**
224     * Обрабатывает MOVE batch.
225     *
226     * Historical samples уже должны находиться внутри batch
227     * раньше текущего sample, это обеспечивает StrokeInputCapturer.
228     */
229     fun append(
230         batch: InputBatch,
231         out: CapsuleStrokeRenderer,
232     ) {
233         if (!active || batch.sampleCount <= 0) return
234         val settings = activeSettings()
235
236         for (index in 0 until batch.sampleCount) {
237             val sample = batch.samples[index]
238
239             if (sample.pointerId != pointerId) {
240                 continue
241             }
242
243             stabilizer.process(
244                 timestampNanos = sample.timestampNanos,
245                 rawX = sample.canvasX,
246                 rawY = sample.canvasY,
247             )
248
249             val x = stabilizer.x
250             val y = stabilizer.y
251             val filteredPressure = pressureFilter.filter(sample.timestampNanos, sample.pressure)
252             BrushDynamics.resolve(
253                 settings = settings,
254                 pressure = filteredPressure,
255                 tiltX = sample.tiltX,
256                 tiltY = sample.tiltY,
257                 velocityPxPerSecond = stabilizer.velocity,
258                 random01 = 0.5f,
259                 out = resolvedDab,
260             )
261             val radius = resolvedDab.radius
262             val coverage = resolvedDab.coverage
263             lastOrientation = resolvedDab.rotationRad
264             lastTiltX = sample.tiltX
265             lastTiltY = sample.tiltY
266             out.setStrokeRotation(resolvedDab.rotationRad)
267
268             emitPoint(
269                 x = x,
270                 y = y,
271                 radius = radius,
272                 coverage = coverage,
273                 out = out,
274             )

```

```

275     }
276 }
277
278 /**
279  * Завершает stroke.
280  *
281  * Последний текущий sample должен быть передан через append()
282  * до вызова finish().
283  */
284 fun finish(
285     out: CapsuleStrokeRenderer,
286     cancel: Boolean,
287 ) {
288     if (!active) return
289     val settings = activeSettings()
290
291     if (cancel) {
292         reset()
293         out.clearStrokeData()
294         return
295     }
296
297     // Принудительно завершаем кривую, используя последнюю точку как P3
298     if (pointsInWindow >= 2 && settings.smoothing > 1e-4f) {
299         interpolate(
300             p0x, p0y, p0r,
301             p0c,
302             p1x, p1y, p1r, p1c,
303             p2x, p2y, p2r, p2c,
304             p2x, p2y, p2r, p2c,
305             out,
306         )
307     }
308
309     // The end taper is only knowable after UP, when the complete mesh exists.
310     // Preview stays untapered; commit redraws every segment with these final radii.
311     out.applyTaper { distanceFromStart, meshLength ->
312         taperScale(distanceFromStart, meshLength, settings)
313     }
314
315     active = false
316     pointerId = -1
317     ribbonMeshDirty = true
318     ribbonMeshVersion++
319 }
320
321 /** Mesh is built after UP, when end taper can be evaluated against total length. */
322 fun buildRibbonMesh(): RibbonGeometryBuilder.Mesh? {
323     if (ribbonMeshDirty) {
324         ribbonMesh = strokeSettings?.let { ribbonGeometry.build(it.ribbon) }
325         ribbonMeshDirty = false
326     }
327     return ribbonMesh
328 }
329
330 private fun emitPoint(
331     x: Float,
332     y: Float,
333     radius: Float,
334     coverage: Float,
335     out: CapsuleStrokeRenderer,
336 ) {
337     val settings = activeSettings()
338     val dx = x - lastX
339     val dy = y - lastY
340     val distance = hypot(dx, dy)
341     strokeDistance += distance
342
343     if (distance < minimumPointDistance()) {
344         return
345     }
346
347     if (settings.smoothing <= 1e-4f) {
348         // Raw mode: прямые сегменты без задержки на окно
349         out.addSegment(lastX, lastY, lastRadius, lastCoverage, x, y, radius, coverage)
350         emittedSegments++
351         totalLength += distance
352         includeBounds(lastX, lastY, lastRadius)
353         includeBounds(x, y, radius)
354     } else {
355         // Interpolated mode: Catmull-Rom через окно
356         when (pointsInWindow) {
357             1 -> {
358                 p2x = x; p2y = y; p2r = radius; p2c = coverage
359                 pointsInWindow = 2
360             }
361             2 -> {
362                 p3x = x; p3y = y; p3r = radius; p3c = coverage
363                 interpolate(p0x, p0y, p0r, p0c, p1x, p1y, p1r, p1c, p2x, p2y, p2r, p2c, p3x, p3y, p3r, p3c, out)
364                 // Сдвиг окна
365                 p0x = p1x; p0y = p1y; p0r = p1r; p0c = p1c
366                 p1x = p2x; p1y = p2y; p1r = p2r; p1c = p2c

```

```

367         p2x = p3x; p2y = p3y; p2r = p3r; p2c = p3c
368     }
369 }
370 }
371
372 hasStroke = true
373 ribbonGeometry.addPoint(x, y, radius, coverage, lastOrientation)
374 ribbonMeshDirty = true
375 ribbonMeshVersion++
376 lastX = x
377 lastY = y
378 lastRadius = radius
379 lastCoverage = coverage
380 }
381
382 private fun taperScale(
383     distanceFromStart: Float,
384     totalLength: Float,
385     settings: BrushSettings,
386 ): Float {
387     val start = settings.ribbon.taperStartPx
388     val end = settings.ribbon.taperEndPx
389
390     val startFactor = if (start <= 0f) {
391         1f
392     } else {
393         (distanceFromStart / start).coerceIn(0f, 1f)
394     }
395
396     val distanceToEnd = totalLength - distanceFromStart
397     val endFactor = if (end <= 0f) {
398         1f
399     } else {
400         (distanceToEnd / end).coerceIn(0f, 1f)
401     }
402
403     return minOf(startFactor, endFactor)
404         .coerceIn(settings.ribbon.minWidthRatio, 1f)
405 }
406
407
408 /**
409  * Адаптивная интерполяция сегмента P1-P2 с использованием P0 и P3 как контрольных точек.
410  */
411 private fun interpolate(
412     p0x: Float, p0y: Float, p0r: Float, p0c: Float,
413     p1x: Float, p1y: Float, p1r: Float, p1c: Float,
414     p2x: Float, p2y: Float, p2r: Float, p2c: Float,
415     p3x: Float, p3y: Float, p3r: Float, p3c: Float,
416     out: CapsuleStrokeRenderer
417 ) {
418     val dist = hypot(p2x - p1x, p2y - p1y)
419     if (dist < 1e-4f) return
420
421     // Вычисляем контрольные точки для кубической кривой Безье (Catmull-Rom -> Bezier)
422     val c1x = p1x + (p2x - p0x) / 6f
423     val c1y = p1y + (p2y - p0y) / 6f
424     val c2x = p2x - (p3x - p1x) / 6f
425     val c2y = p2y - (p3y - p1y) / 6f
426
427     // Адаптивное количество шагов: примерно каждые 0.25..0.5 радиуса
428     val avgRadius = (p1r + p2r) * 0.5f
429     val stepPx = max(1f, avgRadius * INTERPOLATION_STEP_RATIO)
430     val steps = max(1, (dist / stepPx).toInt().coerceAtMost(MAX_INTERPOLATION_STEPS))
431
432     var prevX = p1x
433     var prevY = p1y
434     var prevR = p1r
435     var prevCoverage = p1c
436
437     for (i in 1..steps) {
438         val t = i.toFloat() / steps
439         val mt = 1f - t
440
441         // Кубическое Безье для позиции
442         val x = mt * mt * mt * p1x + 3f * mt * mt * t * c1x + 3f * mt * t * t * c2x + t * t * t * p2x
443         val y = mt * mt * mt * p1y + 3f * mt * mt * t * c1y + 3f * mt * t * t * c2y + t * t * t * p2y
444
445         // Линейная интерполяция радиуса
446         val r = p1r + (p2r - p1r) * t
447         val coverage = p1c + (p2c - p1c) * t
448
449         out.addSegment(prevX, prevY, prevR, prevCoverage, x, y, r, coverage)
450         emittedSegments++
451         totalLength += hypot(x - prevX, y - prevY)
452         includeBounds(x, y, r)
453
454         prevX = x
455         prevY = y
456         prevR = r
457         prevCoverage = coverage
458     }

```

```

459     }
460
461     /**
462     * Для капсул не нужен такой плотный шаг, как для stamp-кисти:
463     * segment shader сам покрывает весь отрезок между A и B.
464     */
465     private fun minimumPointDistance(): Float {
466         val settings = activeSettings()
467         val configured = settings.ribbon.minPointDistancePx
468             .coerceIn(MIN_POINT_DISTANCE_MIN, MAX_POINT_DISTANCE)
469
470         val radiusBased = settings.baseRadiusPx * POINT_STEP_RADIUS_RATIO
471
472         return max(
473             configured,
474             radiusBased.coerceAtMost(MAX_POINT_DISTANCE),
475         )
476     }
477
478     private fun includeBounds(
479         x: Float,
480         y: Float,
481         radius: Float,
482     ) {
483         val left = x - radius
484         val top = y - radius
485         val right = x + radius
486         val bottom = y + radius
487
488         if (!hasBounds) {
489             minX = left
490             minY = top
491             maxX = right
492             maxY = bottom
493             hasBounds = true
494             return
495         }
496
497         if (left < minX) minX = left
498         if (top < minY) minY = top
499         if (right > maxX) maxX = right
500         if (bottom > maxY) maxY = bottom
501     }
502
503     companion object {
504         private const val SMOOTHING_WEIGHT = 0.7f
505         private const val STREAMLINE_WEIGHT = 0.3f
506
507         private const val MIN_RADIUS_PX = 0.25f
508
509         private const val MIN_POINT_DISTANCE_MIN = 0.05f
510         private const val MAX_POINT_DISTANCE = 32f
511         private const val POINT_STEP_RADIUS_RATIO = 0.08f
512
513         private const val INTERPOLATION_STEP_RATIO = 0.4f
514         private const val MAX_INTERPOLATION_STEPS = 64
515     }
516 }

```

25. app/src/main/java/com/wetinknext/engine/brush/CapsuleStrokeRenderer.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.opengl.GLES30
4  import com.wetinknext.BuildConfig
5  import com.wetinknext.engine.gl.GlCheck
6  import com.wetinknext.engine.gl.GlProgram
7  import com.wetinknext.engine.gl.RenderTarget
8  import com.wetinknext.engine.gl.ShaderLib
9  import java.nio.ByteBuffer
10 import java.nio.ByteOrder
11
12 /**
13  * GPU-renderer для круглого штриха.
14  *
15  * Один instance:
16  *   x0, y0, radius0,
17  *   x1, y1, radius1
18  *
19  * Геометрия каждого сегмента является AABB, а настоящая форма
20  * рассчитывается в capsuleFragment через signed distance.
21  *
22  * Важно:
23  * - target должен быть очищен перед началом нового штриха;
24  * - рисование использует premultiplied source-over blending;
25  * - цвет должен быть постоянным для всего strokeTarget;
26  * - выход shader: premultiplied linear RGBA.
27  */
28 class CapsuleStrokeRenderer(
29     private val maxSegments: Int = DEFAULT_MAX_SEGMENTS,
30 ) {
31     private var program: GlProgram? = null
32
33     private var vaoId = 0
34     private var cornerVboId = 0
35     private var instanceVboId = 0
36
37     private var uCanvasToClip = -1
38     private var uColorLinear = -1
39     private val blendController = BlendController()
40     private var uCanvasSize = -1
41     private var uGrainTex = -1
42     private var uGrainActive = -1
43     private var uGrainScale = -1
44     private var uGrainCanvasLocked = -1
45     private var uTextureDepth = -1
46     private var uTextureContrast = -1
47     private var uFlow = -1
48     private var uCoverageOnly = -1
49
50     private var grainTextureId = 0
51     private var grainScale = 1f
52     private var grainCanvasLocked = true
53     private var textureDepth = 1f
54     private var textureContrast = 1f
55     // Reserved for a future oriented marker/chisel implementation.
56     private var strokeRotation = 0f
57
58     private val checkGlErrors: Boolean
59         get() = BuildConfig.DEBUG
60
61     /**
62     * Формат:
63     *   x0, y0, r0, x1, y1, r1
64     */
65     private val instanceData =
66         ByteBuffer
67             .allocateDirect(
68                 maxSegments * FLOATS_PER_SEGMENT * Float.SIZE_BYTES,
69             )
70             .order(ByteOrder.nativeOrder())
71             .asFloatBuffer()
72
73     var segmentCount: Int = 0
74         private set
75
76     private var uploadedSegmentCount = 0
77
78     /**
79     * Не сбрасывается на каждый новый stroke.
80     * Иначе диагностика потерь снова будет скрыта.
81     */
82     var overflowCount: Long = 0L
83         private set
84
85     val isEmpty: Boolean
86         get() = segmentCount == 0
87
88     val hasPendingSegments: Boolean
89         get() = segmentCount > uploadedSegmentCount
90

```

```

91 fun create() {
92     GLCheck.checkOnGThread()
93     release()
94
95     program = GLProgram(
96         vertexSource = ShaderLib.capsuleVertex,
97         fragmentSource = ShaderLib.capsuleFragment,
98     )
99
100     val currentProgram = checkNotNull(program)
101     currentProgram.use()
102
103     uCanvasToClip = GLES30.glGetUniformLocation(
104         currentProgram.id,
105         "uCanvasToClip",
106     )
107     uColorLinear = GLES30.glGetUniformLocation(
108         currentProgram.id,
109         "uColorLinear",
110     )
111     uCanvasSize = GLES30.glGetUniformLocation(
112         currentProgram.id,
113         "uCanvasSize",
114     )
115     uGrainTex = GLES30.glGetUniformLocation(
116         currentProgram.id,
117         "uGrainTex",
118     )
119     uGrainScale = GLES30.glGetUniformLocation(
120         currentProgram.id,
121         "uGrainScale",
122     )
123     uGrainActive = GLES30.glGetUniformLocation(
124         currentProgram.id,
125         "uGrainActive",
126     )
127     uGrainCanvasLocked = GLES30.glGetUniformLocation(
128         currentProgram.id,
129         "uGrainCanvasLocked",
130     )
131     uTextureDepth = GLES30.glGetUniformLocation(
132         currentProgram.id,
133         "uTextureDepth",
134     )
135     uTextureContrast = GLES30.glGetUniformLocation(
136         currentProgram.id,
137         "uTextureContrast",
138     )
139     uFlow = GLES30.glGetUniformLocation(
140         currentProgram.id,
141         "uFlow",
142     )
143     uCoverageOnly = GLES30.glGetUniformLocation(
144         currentProgram.id,
145         "uCoverageOnly",
146     )
147
148     check(uCanvasToClip >= 0) {
149         "Capsule shader uniform uCanvasToClip is missing"
150     }
151     check(uColorLinear >= 0) {
152         "Capsule shader uniform uColorLinear is missing"
153     }
154     check(uCanvasSize >= 0) {
155         "Capsule shader uniform uCanvasSize is missing"
156     }
157     check(uFlow >= 0) {
158         "Capsule shader uniform uFlow is missing"
159     }
160     check(uCoverageOnly >= 0) {
161         "Capsule shader uniform uCoverageOnly is missing"
162     }
163
164     val corners = floatArrayOf(
165         -1f, -1f,
166         1f, -1f,
167         -1f, 1f,
168         1f, 1f,
169     )
170
171     val cornerData =
172         ByteBuffer
173             .allocateDirect(corners.size * Float.SIZE_BYTES)
174             .order(ByteOrder.nativeOrder())
175             .asFloatBuffer()
176
177     cornerData.put(corners)
178     cornerData.position(0)
179
180     val ids = IntArray(1)
181
182     GLES30.glGenVertexArrays(1, ids, 0)

```



```

183         vaoId = ids[0]
184
185         GLES30.glGenBuffers(1, ids, 0)
186         cornerVboId = ids[0]
187
188         GLES30.glGenBuffers(1, ids, 0)
189         instanceVboId = ids[0]
190
191         check(vaoId != 0) { "Failed to create capsule VAO" }
192         check(cornerVboId != 0) { "Failed to create capsule corner VBO" }
193         check(instanceVboId != 0) { "Failed to create capsule instance VBO" }
194
195         GLES30.glBindVertexArray(vaoId)
196
197         // Углы AABB-квада.
198         GLES30.glBindBuffer(
199             GLES30.GL_ARRAY_BUFFER,
200             cornerVboId,
201         )
202         GLES30.glBufferData(
203             GLES30.GL_ARRAY_BUFFER,
204             corners.size * Float.SIZE_BYTES,
205             cornerData,
206             GLES30.GL_STATIC_DRAW,
207         )
208         GLES30.glEnableVertexAttribArray(ATTR_CORNER)
209         GLES30.glVertexAttribPointer(
210             ATTR_CORNER,
211             2,
212             GLES30.GL_FLOAT,
213             false,
214             2 * Float.SIZE_BYTES,
215             0,
216         )
217
218         // Данные сегментов.
219         GLES30.glBindBuffer(
220             GLES30.GL_ARRAY_BUFFER,
221             instanceVboId,
222         )
223         GLES30.glBufferData(
224             GLES30.GL_ARRAY_BUFFER,
225             maxSegments * STRIDE_BYTES,
226             null,
227             GLES30.GL_DYNAMIC_DRAW,
228         )
229
230         // A: x0, y0, radius0
231         GLES30.glEnableVertexAttribArray(ATTR_SEGMENT_A)
232         GLES30.glVertexAttribPointer(
233             ATTR_SEGMENT_A,
234             4,
235             GLES30.GL_FLOAT,
236             false,
237             STRIDE_BYTES,
238             0,
239         )
240         GLES30.glVertexAttribDivisor(
241             ATTR_SEGMENT_A,
242             1,
243         )
244
245         // B: x1, y1, radius1
246         GLES30.glEnableVertexAttribArray(ATTR_SEGMENT_B)
247         GLES30.glVertexAttribPointer(
248             ATTR_SEGMENT_B,
249             4,
250             GLES30.GL_FLOAT,
251             false,
252             STRIDE_BYTES,
253             4 * Float.SIZE_BYTES,
254         )
255         GLES30.glVertexAttribDivisor(
256             ATTR_SEGMENT_B,
257             1,
258         )
259
260         GLES30.glBindBuffer(
261             GLES30.GL_ARRAY_BUFFER,
262             0,
263         )
264         GLES30.glBindVertexArray(0)
265
266         GLES30.glUseProgram(0)
267
268         checkNoGLError("CapsuleStrokeRenderer.create")
269     }
270
271     /**
272     * Начинает новый stroke.
273     *
274     * Старый target должен быть очищен отдельно:

```

```

275     * strokeTarget.clear(0f, 0f, 0f, 0f)
276     */
277     fun beginStroke() {
278         segmentCount = 0
279         uploadedSegmentCount = 0
280         strokeRotation = 0f
281
282         instanceData.clear()
283         instanceData.limit(
284             maxSegments * FLOATS_PER_SEGMENT,
285         )
286     }
287
288     /**
289     * Добавляет один сегмент в CPU-buffer.
290     *
291     * Если capacity исчерпана, сегмент не теряется молча:
292     * overflowCount увеличивается.
293     */
294     fun addSegment(
295         x0: Float,
296         y0: Float,
297         radius0: Float,
298         coverage0: Float,
299         x1: Float,
300         y1: Float,
301         radius1: Float,
302         coverage1: Float,
303     ): Boolean {
304         if (segmentCount >= maxSegments) {
305             overflowCount++
306             return false
307         }
308
309         val offset = segmentCount * FLOATS_PER_SEGMENT
310
311         instanceData.put(offset, x0)
312         instanceData.put(offset + 1, y0)
313         instanceData.put(offset + 2, radius0.coerceAtLeast(0.01f))
314         instanceData.put(offset + 3, coverage0.coerceIn(0f, 1f))
315
316         instanceData.put(offset + 4, x1)
317         instanceData.put(offset + 5, y1)
318         instanceData.put(offset + 6, radius1.coerceAtLeast(0.01f))
319         instanceData.put(offset + 7, coverage1.coerceIn(0f, 1f))
320
321         segmentCount++
322         return true
323     }
324
325     /**
326     * Retains pen orientation for a future non-round marker/chisel path.
327     * Round capsule geometry deliberately does not use the value yet.
328     */
329     fun setStrokeRotation(rotationRad: Float) {
330         strokeRotation = rotationRad
331     }
332
333     /**
334     * Applies geometric taper to all accumulated segments after the stroke is complete.
335     * The callback receives mesh arc-distance and the complete mesh length, so the end
336     * taper never depends on an estimate made during MOVE.
337     */
338     fun applyTaper(scaleAt: (distanceFromStart: Float, totalLength: Float) -> Float) {
339         if (segmentCount == 0) return
340
341         var totalLength = 0f
342         for (index in 0 until segmentCount) {
343             val offset = index * FLOATS_PER_SEGMENT
344             totalLength += kotlin.math.hypot(
345                 instanceData.get(offset + 4) - instanceData.get(offset),
346                 instanceData.get(offset + 5) - instanceData.get(offset + 1),
347             )
348         }
349
350         var distanceFromStart = 0f
351         for (index in 0 until segmentCount) {
352             val offset = index * FLOATS_PER_SEGMENT
353             val segmentLength = kotlin.math.hypot(
354                 instanceData.get(offset + 4) - instanceData.get(offset),
355                 instanceData.get(offset + 5) - instanceData.get(offset + 1),
356             )
357
358             val startScale = scaleAt(distanceFromStart, totalLength)
359             val endScale = scaleAt(distanceFromStart + segmentLength, totalLength)
360             instanceData.put(offset + 2, (instanceData.get(offset + 2) * startScale).coerceAtLeast(0.01f))
361             instanceData.put(offset + 6, (instanceData.get(offset + 6) * endScale).coerceAtLeast(0.01f))
362             distanceFromStart += segmentLength
363         }
364     }
365
366     /**

```

```

367     * Загружает и рисует только новые сегменты.
368     *
369     * Используется для preview во время MOVE.
370     * Перед первым вызовом target должен быть очищен.
371     */
372     fun drawPending(
373         target: RenderTarget,
374         width: Int,
375         height: Int,
376         canvasToClip: FloatArray,
377         colorLinear: FloatArray,
378         blendPolicy: BlendPolicy,
379         flow: Float,
380     ) {
381         val first = uploadedSegmentCount
382         val count = segmentCount - uploadedSegmentCount
383
384         if (count <= 0) return
385
386         val drawn = drawRange(
387             target = target,
388             width = width,
389             height = height,
390             documentWidth = width,
391             documentHeight = height,
392             canvasToClip = canvasToClip,
393             colorLinear = colorLinear,
394             firstSegment = first,
395             count = count,
396             blendPolicy = blendPolicy,
397             flow = flow,
398             coverageOnly = false,
399         )
400
401         if (drawn) {
402             uploadedSegmentCount = segmentCount
403         }
404     }
405
406     /**
407     * Полностью рисует все накопленные сегменты.
408     *
409     * Используется при commit, если target был очищен.
410     */
411     fun drawAll(
412         target: RenderTarget,
413         width: Int,
414         height: Int,
415         canvasToClip: FloatArray,
416         colorLinear: FloatArray,
417         blendPolicy: BlendPolicy,
418         flow: Float,
419     ) {
420         if (segmentCount <= 0) return
421
422         drawRange(
423             target = target,
424             width = width,
425             height = height,
426             documentWidth = width,
427             documentHeight = height,
428             canvasToClip = canvasToClip,
429             colorLinear = colorLinear,
430             firstSegment = 0,
431             count = segmentCount,
432             blendPolicy = blendPolicy,
433             flow = flow,
434             coverageOnly = false,
435         )
436     }
437
438     /** Draws only new segments into an RGBA8 screen-sized preview target. */
439     fun drawPendingPreview(
440         target: RenderTarget,
441         previewWidth: Int,
442         previewHeight: Int,
443         documentWidth: Int,
444         documentHeight: Int,
445         canvasToClip: FloatArray,
446         colorLinear: FloatArray,
447         blendPolicy: BlendPolicy,
448         flow: Float,
449     ) {
450         val first = uploadedSegmentCount
451         val count = segmentCount - uploadedSegmentCount
452         if (count <= 0) return
453
454         val drawn = drawRange(
455             target = target,
456             width = previewWidth,
457             height = previewHeight,
458             documentWidth = documentWidth,

```

```

459         documentHeight = documentHeight,
460         canvasToClip = canvasToClip,
461         colorLinear = colorLinear,
462         firstSegment = first,
463         count = count,
464         blendPolicy = blendPolicy,
465         flow = flow,
466         coverageOnly = false,
467     )
468     if (drawn) uploadedSegmentCount = segmentCount
469 }
470
471 /** Adds only new capsule segments to an RGBA8 GL_MAX coverage mask. */
472 fun drawPendingCoveragePreview(
473     target: RenderTarget,
474     previewWidth: Int,
475     previewHeight: Int,
476     documentWidth: Int,
477     documentHeight: Int,
478     canvasToClip: FloatArray,
479     flow: Float,
480 ) {
481     val first = uploadedSegmentCount
482     val count = segmentCount - first
483     if (count <= 0) return
484     val drawn = drawRange(
485         target = target,
486         width = previewWidth,
487         height = previewHeight,
488         documentWidth = documentWidth,
489         documentHeight = documentHeight,
490         canvasToClip = canvasToClip,
491         colorLinear = ZERO_COLOR,
492         firstSegment = first,
493         count = count,
494         blendPolicy = BlendPolicy.NON_BUILDUP,
495         flow = flow,
496         coverageOnly = true,
497     )
498     if (drawn) uploadedSegmentCount = segmentCount
499 }
500
501 /** Rebuilds every segment into the document-sized union coverage mask. */
502 fun drawAllCoverage(
503     target: RenderTarget,
504     width: Int,
505     height: Int,
506     canvasToClip: FloatArray,
507     flow: Float,
508 ) {
509     if (segmentCount <= 0) return
510     drawRange(
511         target = target,
512         width = width,
513         height = height,
514         documentWidth = width,
515         documentHeight = height,
516         canvasToClip = canvasToClip,
517         colorLinear = ZERO_COLOR,
518         firstSegment = 0,
519         count = segmentCount,
520         blendPolicy = BlendPolicy.NON_BUILDUP,
521         flow = flow,
522         coverageOnly = true,
523     )
524 }
525
526 /**
527  * Полностью перерисовывает сегменты в диапазоне [firstSegment, firstSegment + count).
528  *
529  * Важный момент: glVertexAttribPointer вызывается здесь повторно,
530  * потому что offset у instanced-атрибутов меняется для каждого диапазона.
531  */
532 private fun drawRange(
533     target: RenderTarget,
534     width: Int,
535     height: Int,
536     documentWidth: Int,
537     documentHeight: Int,
538     canvasToClip: FloatArray,
539     colorLinear: FloatArray,
540     firstSegment: Int,
541     count: Int,
542     blendPolicy: BlendPolicy,
543     flow: Float,
544     coverageOnly: Boolean,
545 ): Boolean {
546     val currentProgram = program ?: return false
547
548     require(canvasToClip.size >= 16) {
549         "canvasToClip must contain at least 16 floats"
550     }

```

```

551     require(colorLinear.size >= 3) {
552         "colorLinear must contain at least 3 floats"
553     }
554     require(firstSegment >= 0)
555     require(count >= 0)
556     require(firstSegment + count <= segmentCount)
557
558     if (count == 0) return true
559
560     target.bind()
561
562     GLES30.glViewport(
563         0,
564         0,
565         width,
566         height,
567     )
568
569     GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
570
571     if (coverageOnly) {
572         GLES30.glEnable(GLES30.GL_BLEND)
573         GLES30.glBlendEquation(GLES30.GL_MAX)
574         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE)
575     } else {
576         blendController.begin(blendPolicy)
577     }
578
579     currentProgram.use()
580
581     GLES30.glUniformMatrix4fv(
582         uCanvasToClip,
583         1,
584         false,
585         canvasToClip,
586         0,
587     )
588
589     GLES30.glUniform3f(
590         uColorLinear,
591         colorLinear[0].coerceIn(0f, 1f),
592         colorLinear[1].coerceIn(0f, 1f),
593         colorLinear[2].coerceIn(0f, 1f),
594     )
595
596     GLES30.glUniform2f(
597         uCanvasSize,
598         documentWidth.toFloat(),
599         documentHeight.toFloat(),
600     )
601
602     GLES30.glUniform1f(
603         uGrainScale,
604         grainScale,
605     )
606
607     GLES30.glUniform1i(
608         uGrainActive,
609         if (grainTextureId != 0) 1 else 0,
610     )
611
612     GLES30.glUniform1i(
613         uGrainCanvasLocked,
614         if (grainCanvasLocked) 1 else 0,
615     )
616
617     GLES30.glUniform1f(
618         uTextureDepth,
619         textureDepth,
620     )
621
622     GLES30.glUniform1f(
623         uTextureContrast,
624         textureContrast,
625     )
626
627     GLES30.glUniform1f(
628         uFlow,
629         flow.coerceIn(0f, 1f),
630     )
631     GLES30.glUniform1i(uCoverageOnly, if (coverageOnly) 1 else 0)
632
633     if (grainTextureId != 0) {
634         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
635         GLES30.glBindTexture(
636             GLES30.GL_TEXTURE_2D,
637             grainTextureId,
638         )
639         GLES30.glUniform1i(uGrainTex, 2)
640     }
641
642     /*

```

```

643     * FloatBuffer должен содержать только передаваемый диапазон.
644     * glBufferSubData прочитает remaining() элементов.
645     */
646     val sourceStart = firstSegment * FLOATS_PER_SEGMENT
647     val sourceEnd =
648         sourceStart + count * FLOATS_PER_SEGMENT
649
650     instanceData.position(sourceStart)
651     instanceData.limit(sourceEnd)
652
653     GLES30.glBindVertexArray(vaoId)
654     GLES30.glBindBuffer(
655         GLES30.GL_ARRAY_BUFFER,
656         instanceVboId,
657     )
658
659     GLES30.glBufferSubData(
660         GLES30.GL_ARRAY_BUFFER,
661         0,
662         count * STRIDE_BYTES,
663         instanceData,
664     )
665
666     /*
667     * Для ES 3.0 нет glDrawArraysInstancedBaseInstance.
668     * Поэтому offset атрибутов сдвигается вручную.
669     *
670     * VAO должен быть привязан во время glVertexAttribPointer.
671     */
672     GLES30.glVertexAttribPointer(
673         ATTR_SEGMENT_A,
674         4,
675         GLES30.GL_FLOAT,
676         false,
677         STRIDE_BYTES,
678         0,
679     )
680
681     GLES30.glVertexAttribPointer(
682         ATTR_SEGMENT_B,
683         4,
684         GLES30.GL_FLOAT,
685         false,
686         STRIDE_BYTES,
687         4 * Float.SIZE_BYTES,
688     )
689
690     GLES30.glDrawArraysInstanced(
691         GLES30.GL_TRIANGLE_STRIP,
692         0,
693         CORNER_VERTEX_COUNT,
694         count,
695     )
696
697     if (grainTextureId != 0) {
698         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
699         GLES30.glBindTexture(
700             GLES30.GL_TEXTURE_2D,
701             0,
702         )
703     }
704
705     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
706
707     GLES30.glBindBuffer(
708         GLES30.GL_ARRAY_BUFFER,
709         0,
710     )
711     GLES30.glBindVertexArray(0)
712
713     if (coverageOnly) {
714         GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
715         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
716         GLES30.glDisable(GLES30.GL_BLEND)
717     } else {
718         blendController.end()
719     }
720
721     GLES30.glBindFramebuffer(
722         GLES30.GL_FRAMEBUFFER,
723         0,
724     )
725
726     instanceData.clear()
727     instanceData.limit(
728         maxSegments * FLOATS_PER_SEGMENT,
729     )
730
731     if (checkGLErrors) {
732         checkNoGLError("CapsuleStrokeRenderer.drawRange")
733     }
734     return true

```

```

735     }
736
737     /**
738     * Сбрасывает только GPU/CPU состояние текущего renderer.
739     * Счётчик overflow сохраняется.
740     */
741     fun clearStrokeData() {
742         segmentCount = 0
743         uploadedSegmentCount = 0
744         instanceData.clear()
745         instanceData.limit(
746             maxSegments * FLOATS_PER_SEGMENT,
747         )
748     }
749
750     fun setGrainTexture(
751         textureId: Int,
752         scale: Float,
753         canvasLocked: Boolean,
754         depth: Float,
755         contrast: Float,
756     ) {
757         grainTextureId = textureId
758         grainScale = scale.coerceAtLeast(0.0001f)
759         grainCanvasLocked = canvasLocked
760         textureDepth = depth.coerceIn(0f, 1f)
761         textureContrast = contrast.coerceIn(0f, 2f)
762     }
763
764     fun clearGrainTexture() {
765         grainTextureId = 0
766         grainScale = 1f
767         grainCanvasLocked = true
768         textureDepth = 1f
769         textureContrast = 1f
770     }
771
772     /**
773     * Все GL-объекты удаляются только на GL-потоке.
774     */
775     fun release() {
776         GLCheck.checkOnGLThread()
777         program?.release()
778         program = null
779
780         if (instanceVboId != 0) {
781             GLES30.glDeleteBuffers(
782                 1,
783                 intArrayOf(instanceVboId),
784                 0,
785             )
786         }
787
788         if (cornerVboId != 0) {
789             GLES30.glDeleteBuffers(
790                 1,
791                 intArrayOf(cornerVboId),
792                 0,
793             )
794         }
795
796         if (vaoId != 0) {
797             GLES30.glDeleteVertexArrays(
798                 1,
799                 intArrayOf(vaoId),
800                 0,
801             )
802         }
803
804         vaoId = 0
805         cornerVboId = 0
806         instanceVboId = 0
807
808         uCanvasToClip = -1
809         uColorLinear = -1
810         uCanvasSize = -1
811         uGrainTex = -1
812         uGrainScale = -1
813         uGrainActive = -1
814         uGrainCanvasLocked = -1
815         uTextureDepth = -1
816         uTextureContrast = -1
817         uFlow = -1
818         uCoverageOnly = -1
819         grainTextureId = 0
820
821         segmentCount = 0
822         uploadedSegmentCount = 0
823
824         instanceData.clear()
825         instanceData.limit(
826             maxSegments * FLOATS_PER_SEGMENT,

```

```

827     )
828 }
829
830 private fun checkNoGLError(label: String) {
831     val error = GLES30.glGetError()
832     check(error == GLES30.GL_NO_ERROR) {
833         "$label: GL error 0x${error.toString(16)}"
834     }
835 }
836
837 companion object {
838
839     private val ZERO_COLOR = floatArrayOf(0f, 0f, 0f)
840
841     private const val ATTR_CORNER = 0
842     private const val ATTR_SEGMENT_A = 1
843     private const val ATTR_SEGMENT_B = 2
844
845     private const val CORNER_VERTEX_COUNT = 4
846
847     const val FLOATS_PER_SEGMENT = 8
848     const val STRIDE_BYTES =
849         FLOATS_PER_SEGMENT * Float.SIZE_BYTES
850
851     const val DEFAULT_MAX_SEGMENTS = 16_384
852 }
853 }

```


26. app/src/main/java/com/wetinknext/engine/brush/ClosureDebug.kt

```
1 package com.wetinknext.engine.brush
2
3 import android.util.Log
4 import com.wetinknext.BuildConfig
5
6 /** One debug log per finished stroke; never used from the render hot path. */
7 object ClosureDebug {
8     const val TAG = "RibbonClosure"
9     fun publish(distance: Float, threshold: Float, closed: Boolean, samples: Int) {
10         if (!BuildConfig.DEBUG) return
11         runCatching {
12             Log.d(TAG, "dist=%.2f thr=%.2f closed=%s n=%d".format(distance, threshold, closed, samples))
13         }
14     }
15 }
```

27. app/src/main/java/com/wetinknext/engine/brush/ColorSpaces.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.pow
4
5 object ColorSpaces {
6     fun srgbToLinear(value: Float): Float { val v=value.coerceIn(0f,1f); return if(v<=.04045f)v/12.92f else ((v+.055f)/1.
055f).pow(2.4f) }
7     fun linearToSrgb(value: Float): Float { val v=value.coerceIn(0f,1f); return if(v<=.0031308f)v*12.92f else 1.055f*v.pow(1f/
2.4f)-.055f }
8     fun srgb8ToLinear(Argb: Long,out:FloatArray){require(out.size>=3);out[0]=srgbToLinear(((Argb shr 16)and 0xFF)/255f);out[
1]=srgbToLinear(((Argb shr 8)and 0xFF)/255f);out[2]=srgbToLinear((Argb and 0xFF)/255f)}
9 }
```

28. app/src/main/java/com/wetinknext/engine/brush/DabBuffer.kt

```

1  package com.wetinknext.engine.brush
2
3  import com.wetinknext.engine.core.DirtyRect
4  import java.nio.ByteBuffer
5  import java.nio.ByteOrder
6
7  class DabBuffer(val capacity: Int = DEFAULT_CAPACITY) {
8      init { require(capacity > 0) }
9      val floats = ByteBuffer.allocateDirect(capacity * FLOATS_PER_DAB * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).
asFloatBuffer()
10     var count = 0; private set; var overflowCount = 0L; private set
11     fun clear() { count = 0; overflowCount = 0L; floats.clear() }
12     fun add(
13         x: Float,
14         y: Float,
15         radius: Float,
16         rotation: Float,
17         coverage: Float,
18         flow: Float,
19         hardness: Float,
20     ): Boolean {
21         if (count >= capacity) {
22             overflowCount++
23             return false
24         }
25         floats.put(x)
26         floats.put(y)
27         floats.put(radius)
28         floats.put(rotation)
29         floats.put(coverage.coerceIn(0f, 1f))
30         floats.put(flow.coerceIn(0f, 1f))
31         floats.put(hardness.coerceIn(0f, 1f))
32         count++
33         return true
34     }
35     fun prepareForUpload() { floats.position(0); floats.limit(count * FLOATS_PER_DAB) }
36     fun prepareForUpload(firstDab: Int, dabCount: Int) {
37         require(firstDab >= 0)
38         require(dabCount >= 0)
39         require(firstDab + dabCount <= count)
40         floats.position(firstDab * FLOATS_PER_DAB)
41         floats.limit((firstDab + dabCount) * FLOATS_PER_DAB)
42     }
43     /** Restores the direct buffer for appending after a GL upload. */
44     fun finishUpload() {
45         floats.limit(capacity * FLOATS_PER_DAB)
46         floats.position(count * FLOATS_PER_DAB)
47     }
48
49     /**
50      * Includes the exact circular footprint of each emitted dab plus a caller
51      * supplied safety fringe for analytic AA and textured-tip filtering.
52      */
53     fun includeDirtyRect(rect: DirtyRect, extraMargin: Float) {
54         val safeExtraMargin = extraMargin.coerceAtLeast(0f)
55         for (index in 0 until count) {
56             val offset = index * FLOATS_PER_DAB
57             val x = floats.get(offset)
58             val y = floats.get(offset + 1)
59             val radius = floats.get(offset + 2).coerceAtLeast(0f) + safeExtraMargin
60             rect.include(
61                 left = x - radius,
62                 top = y - radius,
63                 right = x + radius,
64                 bottom = y + radius,
65             )
66         }
67     }
68     companion object { const val FLOATS_PER_DAB = 7; const val DEFAULT_CAPACITY = 8192 }
69 }

```

29. app/src/main/java/com/wetinknext/engine/brush/DabRenderer.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.GlCheck
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.RenderTarget
7  import com.wetinknext.engine.gl.ShaderLib
8  import java.nio.ByteBuffer
9  import java.nio.ByteOrder
10
11 /** Instanced circular dab renderer. All colour values are premultiplied linear output. */
12 class DabRenderer(private val maxDabs: Int) {
13     private var program: GlProgram? = null
14     private var vaoId = 0
15     private var quadBufferId = 0
16     private var instanceBufferId = 0
17     private var uCanvasToClip = -1
18     private var uColorLinear = -1
19     private var uCanvasSize = -1
20     private var uGrainTex = -1
21     private var uGrainActive = -1
22     private var uGrainScale = -1
23     private var uGrainCanvasLocked = -1
24     private var uTextureDepth = -1
25     private var uTextureContrast = -1
26     private var uStrokeOpacity = -1
27     private var uCoverageOnly = -1
28     private var uShapeTex = -1
29     private var uShapeActive = -1
30     private var uReverseShape = -1
31     private var uRgbToAlpha = -1
32
33     private var grainTextureId = 0
34     private var grainScale = 1f
35     private var grainCanvasLocked = true
36     private var textureDepth = 1f
37     private var textureContrast = 1f
38     private var shapeTextureId = 0
39     private var reverseShape = false
40     private var rgbToAlpha = false
41
42     private var uploadedDabCount = 0
43     private val blendController = BlendController()
44
45     /** Number of dabs already written into the current stroke target. */
46     val uploadedCount: Int get() = uploadedDabCount
47
48     fun create() {
49         GlCheck.checkOnGThread()
50         release()
51         uploadedDabCount = 0
52         program = GlProgram(ShaderLib.dabVertex, ShaderLib.dabFragment)
53         val currentProgram = checkNotNull(program)
54         currentProgram.use()
55         uCanvasToClip = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasToClip")
56         uColorLinear = GLES30.glGetUniformLocation(currentProgram.id, "uColorLinear")
57         uCanvasSize = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasSize")
58         uGrainTex = GLES30.glGetUniformLocation(currentProgram.id, "uGrainTex")
59         uGrainActive = GLES30.glGetUniformLocation(currentProgram.id, "uGrainActive")
60         uGrainScale = GLES30.glGetUniformLocation(currentProgram.id, "uGrainScale")
61         uGrainCanvasLocked = GLES30.glGetUniformLocation(currentProgram.id, "uGrainCanvasLocked")
62         uTextureDepth = GLES30.glGetUniformLocation(currentProgram.id, "uTextureDepth")
63         uTextureContrast = GLES30.glGetUniformLocation(currentProgram.id, "uTextureContrast")
64         uStrokeOpacity = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeOpacity")
65         uCoverageOnly = GLES30.glGetUniformLocation(currentProgram.id, "uCoverageOnly")
66         uShapeTex = GLES30.glGetUniformLocation(currentProgram.id, "uShapeTex")
67         uShapeActive = GLES30.glGetUniformLocation(currentProgram.id, "uShapeActive")
68         uReverseShape = GLES30.glGetUniformLocation(currentProgram.id, "uReverseShape")
69         uRgbToAlpha = GLES30.glGetUniformLocation(currentProgram.id, "uRgbToAlpha")
70
71         check(
72             uCanvasToClip >= 0 && uColorLinear >= 0 && uStrokeOpacity >= 0 &&
73             uCoverageOnly >= 0,
74         ) { "Dab shader uniforms missing" }
75
76         val quad = floatArrayOf(-1f, -1f, 1f, -1f, -1f, 1f, 1f, 1f)
77         val quadData = ByteBuffer.allocateDirect(quad.size * Float.SIZE_BYTES)
78             .order(ByteOrder.nativeOrder())
79             .asFloatBuffer()
80         quadData.put(quad).position(0)
81
82         val ids = IntArray(1)
83         GLES30.glGenVertexArrays(1, ids, 0)
84         vaoId = ids[0]
85         GLES30.glGenBuffers(1, ids, 0)
86         quadBufferId = ids[0]
87         GLES30.glGenBuffers(1, ids, 0)
88         instanceBufferId = ids[0]
89
90         GLES30.glBindVertexArray(vaoId)

```

```

91     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, quadBufferId)
92     GLES30.glBufferData(
93         GLES30.GL_ARRAY_BUFFER,
94         quad.size * Float.SIZE_BYTES,
95         quadData,
96         GLES30.GL_STATIC_DRAW,
97     )
98     GLES30.glEnableVertexAttribArray(0)
99     GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
100
101     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, instanceBufferId)
102     GLES30.glBufferData(
103         GLES30.GL_ARRAY_BUFFER,
104         maxDabs * DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES,
105         null,
106         GLES30.GL_DYNAMIC_DRAW,
107     )
108     GLES30.glEnableVertexAttribArray(1)
109     GLES30.glVertexAttribPointer(1, 4, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 0)
110     GLES30.glVertexAttribDivisor(1, 1)
111     GLES30.glEnableVertexAttribArray(2)
112     GLES30.glVertexAttribPointer(2, 3, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 16)
113     GLES30.glVertexAttribDivisor(2, 1)
114     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0)
115     GLES30.glBindVertexArray(0)
116 }
117
118 /** Draws all buffered dabs into a document-space RenderTarget. */
119 fun drawInto(
120     target: RenderTarget,
121     width: Int,
122     height: Int,
123     canvasToFbo: FloatArray,
124     dabs: DabBuffer,
125     colorLinear: FloatArray,
126     blendPolicy: BlendPolicy,
127     strokeOpacity: Float = 1f,
128 ) {
129     if (dabs.count == 0) return
130     require(colorLinear.size >= 3)
131
132     dabs.prepareForUpload(firstDab = 0, dabCount = dabs.count)
133     try {
134         drawRange(
135             target = target,
136             width = width,
137             height = height,
138             documentWidth = width,
139             documentHeight = height,
140             canvasToFbo = canvasToFbo,
141             dabs = dabs,
142             firstDab = 0,
143             count = dabs.count,
144             colorLinear = colorLinear,
145             blendPolicy = blendPolicy,
146             strokeOpacity = strokeOpacity,
147             coverageOnly = false,
148         )
149     } finally {
150         dabs.finishUpload()
151     }
152 }
153
154 fun release() {
155     GLCheck.checkOnGThread()
156     program?.release()
157     program = null
158     uploadedDabCount = 0
159     grainTextureId = 0
160     shapeTextureId = 0
161     reverseShape = false
162     rgbToAlpha = false
163     if (instanceBufferId != 0) GLES30.glDeleteBuffers(1, intArrayOf(instanceBufferId), 0)
164     if (quadBufferId != 0) GLES30.glDeleteBuffers(1, intArrayOf(quadBufferId), 0)
165     if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0)
166     vaoId = 0
167     quadBufferId = 0
168     instanceBufferId = 0
169     uCanvasToClip = -1
170     uColorLinear = -1
171     uStrokeOpacity = -1
172     uCoverageOnly = -1
173 }
174
175 fun beginStroke() {
176     uploadedDabCount = 0
177 }
178
179 fun clearStrokeData() {
180     uploadedDabCount = 0
181 }
182

```

```

183 fun setGrainTexture(
184     textureId: Int,
185     scale: Float,
186     canvasLocked: Boolean,
187     depth: Float,
188     contrast: Float,
189 ) {
190     grainTextureId = textureId
191     grainScale = scale.coerceAtLeast(0.0001f)
192     grainCanvasLocked = canvasLocked
193     textureDepth = depth.coerceIn(0f, 1f)
194     textureContrast = contrast.coerceIn(0f, 2f)
195 }
196
197 fun clearGrainTexture() {
198     grainTextureId = 0
199     grainScale = 1f
200     grainCanvasLocked = true
201     textureDepth = 1f
202     textureContrast = 1f
203 }
204
205 /** Sets a tip mask sampled once per dab in local dab coordinates. */
206 fun setShapeTexture(textureId: Int, reverse: Boolean, rgbToAlpha: Boolean) {
207     shapeTextureId = textureId
208     reverseShape = reverse
209     this.rgbToAlpha = rgbToAlpha
210 }
211
212 fun clearShapeTexture() {
213     shapeTextureId = 0
214     reverseShape = false
215     rgbToAlpha = false
216 }
217
218 /**
219  * Draws all dabs into a coverage target. The fragment coverage-only path
220  * is wired in the next NON_BUILDUP step; keeping upload ownership here
221  * prevents the mask pass from disturbing incremental colour preview data.
222  */
223 fun drawCoverageInto(
224     target: RenderTarget,
225     width: Int,
226     height: Int,
227     canvasToFbo: FloatArray,
228     dabs: DabBuffer,
229 ) {
230     if (dabs.count == 0) return
231
232     dabs.prepareForUpload(firstDab = 0, dabCount = dabs.count)
233     try {
234         drawRange(
235             target = target,
236             width = width,
237             height = height,
238             documentWidth = width,
239             documentHeight = height,
240             canvasToFbo = canvasToFbo,
241             dabs = dabs,
242             firstDab = 0,
243             count = dabs.count,
244             colorLinear = ZERO_COLOR,
245             blendPolicy = BlendPolicy.NON_BUILDUP,
246             strokeOpacity = 1f,
247             coverageOnly = true,
248         )
249     } finally {
250         dabs.finishUpload()
251     }
252 }
253
254 fun drawPendingInto(
255     target: RenderTarget,
256     width: Int,
257     height: Int,
258     canvasToFbo: FloatArray,
259     dabs: DabBuffer,
260     colorLinear: FloatArray,
261     blendPolicy: BlendPolicy,
262     strokeOpacity: Float = 1f,
263 ) {
264     val first = uploadedDabCount
265     val count = dabs.count - first
266     if (count <= 0) return
267
268     dabs.prepareForUpload(first, count)
269     try {
270         drawRange(
271             target = target,
272             width = width,
273             height = height,
274             documentWidth = width,

```

```

275         documentHeight = height,
276         canvasToFbo = canvasToFbo,
277         dabs = dabs,
278         firstDab = first,
279         count = count,
280         colorLinear = colorLinear,
281         blendPolicy = blendPolicy,
282         strokeOpacity = strokeOpacity,
283         coverageOnly = false,
284     )
285     uploadedDabCount = dabs.count
286 } finally {
287     dabs.finishUpload()
288 }
289 }
290
291 /**
292  * Incremental preview into a screen-sized target. Coordinates and texture
293  * grain remain document-space; only rasterisation uses screen dimensions.
294  */
295 fun drawPendingPreviewInto(
296     target: RenderTarget,
297     previewWidth: Int,
298     previewHeight: Int,
299     documentWidth: Int,
300     documentHeight: Int,
301     canvasToClip: FloatArray,
302     dabs: DabBuffer,
303     colorLinear: FloatArray,
304     blendPolicy: BlendPolicy,
305     strokeOpacity: Float = 1f,
306 ) {
307     val first = uploadedDabCount
308     val count = dabs.count - first
309     if (count <= 0) return
310
311     dabs.prepareForUpload(first, count)
312     try {
313         drawRange(
314             target = target,
315             width = previewWidth,
316             height = previewHeight,
317             documentWidth = documentWidth,
318             documentHeight = documentHeight,
319             canvasToFbo = canvasToClip,
320             dabs = dabs,
321             firstDab = first,
322             count = count,
323             colorLinear = colorLinear,
324             blendPolicy = blendPolicy,
325             strokeOpacity = strokeOpacity,
326             coverageOnly = false,
327         )
328         uploadedDabCount = dabs.count
329     } finally {
330         dabs.finishUpload()
331     }
332 }
333
334 /** Adds only new dabs to a screen-sized NON_BUILDUP union coverage mask. */
335 fun drawPendingCoveragePreviewInto(
336     target: RenderTarget,
337     previewWidth: Int,
338     previewHeight: Int,
339     documentWidth: Int,
340     documentHeight: Int,
341     canvasToClip: FloatArray,
342     dabs: DabBuffer,
343 ) {
344     val first = uploadedDabCount
345     val count = dabs.count - first
346     if (count <= 0) return
347
348     dabs.prepareForUpload(first, count)
349     try {
350         drawRange(
351             target = target,
352             width = previewWidth,
353             height = previewHeight,
354             documentWidth = documentWidth,
355             documentHeight = documentHeight,
356             canvasToFbo = canvasToClip,
357             dabs = dabs,
358             firstDab = first,
359             count = count,
360             colorLinear = ZERO_COLOR,
361             blendPolicy = BlendPolicy.NON_BUILDUP,
362             strokeOpacity = 1f,
363             coverageOnly = true,
364         )
365         uploadedDabCount = dabs.count
366     } finally {

```

```

367         dabs.finishUpload()
368     }
369 }
370
371 private fun drawRange(
372     target: RenderTarget,
373     width: Int,
374     height: Int,
375     documentWidth: Int,
376     documentHeight: Int,
377     canvasToFbo: FloatArray,
378     dabs: DabBuffer,
379     firstDab: Int,
380     count: Int,
381     colorLinear: FloatArray,
382     blendPolicy: BlendPolicy,
383     strokeOpacity: Float,
384     coverageOnly: Boolean,
385 ) {
386     val currentProgram = program ?: return
387     target.bind()
388     GLES30.glViewport(0, 0, width, height)
389     if (coverageOnly) {
390         // A coverage mask is a union of dabs: overlaps retain their maximum
391         // alpha instead of source-over accumulating darkness.
392         GLES30.glEnable(GLES30.GL_BLEND)
393         GLES30.glBlendEquation(GLES30.GL_MAX)
394         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE)
395     } else {
396         blendController.begin(blendPolicy)
397     }
398     currentProgram.use()
399     GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToFbo, 0)
400     GLES30.glUniform3f(uColorLinear, colorLinear[0], colorLinear[1], colorLinear[2])
401
402     // The GLSL coverage output is added in the following NON_BUILDUP pass.
403     // Keep the explicit mode here so every draw call already has a stable API.
404     @Suppress("UNUSED_VARIABLE")
405     val coveragePass = coverageOnly
406
407     GLES30.glUniform2f(
408         uCanvasSize,
409         documentWidth.toFloat(),
410         documentHeight.toFloat(),
411     )
412     GLES30.glUniform1i(uGrainActive, if (grainTextureId != 0) 1 else 0)
413     GLES30.glUniform1f(uGrainScale, grainScale)
414     GLES30.glUniform1i(uGrainCanvasLocked, if (grainCanvasLocked) 1 else 0)
415     GLES30.glUniform1f(uTextureDepth, textureDepth)
416     GLES30.glUniform1f(uTextureContrast, textureContrast)
417     GLES30.glUniform1f(uStrokeOpacity, strokeOpacity.coerceIn(0f, 1f))
418     GLES30.glUniform1i(uCoverageOnly, if (coverageOnly) 1 else 0)
419     GLES30.glUniform1i(uShapeActive, if (shapeTextureId != 0) 1 else 0)
420     GLES30.glUniform1i(uReverseShape, if (reverseShape) 1 else 0)
421     GLES30.glUniform1i(uRgbToAlpha, if (rgbToAlpha) 1 else 0)
422
423     if (grainTextureId != 0) {
424         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
425         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, grainTextureId)
426         GLES30.glUniform1i(uGrainTex, 2)
427     }
428     if (shapeTextureId != 0) {
429         GLES30.glActiveTexture(GLES30.GL_TEXTURE3)
430         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, shapeTextureId)
431         GLES30.glUniform1i(uShapeTex, 3)
432     }
433
434     GLES30.glBindVertexArray(vaoId)
435     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, instanceBufferId)
436     GLES30.glBufferSubData(
437         GLES30.GL_ARRAY_BUFFER,
438         0,
439         count * DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES,
440         dabs.floats,
441     )
442
443     // DabBuffer has already positioned the source range. Upload it at the
444     // VBO origin, otherwise firstDab is applied twice during a preview.
445     GLES30.glVertexAttribPointer(
446         1, 4, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 0
447     )
448     GLES30.glVertexAttribPointer(
449         2, 3, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 16
450     )
451
452     GLES30.glDrawArraysInstanced(GLES30.GL_TRIANGLE_STRIP, 0, 4, count)
453
454     if (grainTextureId != 0) {
455         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
456         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
457     }
458     if (shapeTextureId != 0) {

```



```

459         GLES30.glActiveTexture(GLES30.GL_TEXTURE3)
460         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
461     }
462     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
463
464     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0)
465     GLES30.glBindVertexArray(0)
466     if (coverageOnly) {
467         GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
468         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
469         GLES30.glDisable(GLES30.GL_BLEND)
470     } else {
471         blendController.end()
472     }
473     GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
474 }
475
476 private companion object {
477     val ZERO_COLOR = floatArrayOf(0f, 0f, 0f)
478 }
479 }

```

30. app/src/main/java/com/wetinknext/engine/brush/OneEuroFilter.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.PI
4 import kotlin.math.sqrt
5
6 class OneEuroFilter(private var minCutoff: Float = 1.6f, private var beta: Float = 0.25f, private var dCutoff: Float = 1f) {
7     private var initialized = false; private var lastTimestampNanos = 0L
8     private var x = 0f; private var y = 0f; private var dx = 0f; private var dy = 0f
9     fun reset() { initialized = false; lastTimestampNanos = 0L; x = 0f; y = 0f; dx = 0f; dy = 0f }
10    fun filter(timestampNanos: Long, rawX: Float, rawY: Float, out: FloatArray) {
11        require(out.size >= 2)
12        if (!initialized) { initialized = true; lastTimestampNanos = timestampNanos; x = rawX; y = rawY; out[0] = x; out[1] =
y; return }
13        var dt = (timestampNanos - lastTimestampNanos) / 1_000_000_000f
14        if (dt < MIN_DT) dt = MIN_DT
15        lastTimestampNanos = timestampNanos
16        val rate = 1f / dt; val alphaD = alpha(rate, dCutoff)
17        dx = alphaD * ((rawX - x) * rate) + (1f - alphaD) * dx; dy = alphaD * ((rawY - y) * rate) + (1f - alphaD) * dy
18        val alpha = alpha(rate, (minCutoff + beta * sqrt(dx * dx + dy * dy)).coerceAtLeast(0.01f))
19        x = alpha * rawX + (1f - alpha) * x; y = alpha * rawY + (1f - alpha) * y; out[0] = x; out[1] = y
20    }
21    private fun alpha(rate: Float, cutoff: Float): Float { val tau = 1f / (2f * PI.toFloat() * cutoff); return 1f / (1f + tau
* rate) }
22    private companion object { const val MIN_DT = 1f / 1000f }
23 }
```

31. app/src/main/java/com/wetinknext/engine/brush/PressureFilter.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.exp
4
5 /**
6  * Time-based stylus pressure smoothing.
7  * A zero pressure sample while the pointer is still in contact is treated as a
8  * device glitch; Android sends the real zero value on UP after the final point.
9  */
10 class PressureFilter {
11     private var initialized = false
12     private var lastTimestampNanos = 0L
13     private var value = 0f
14
15     fun reset() { initialized = false; lastTimestampNanos = 0L; value = 0f }
16
17     fun filter(timestampNanos: Long, rawPressure: Float): Float {
18         val raw = rawPressure.coerceIn(0f, 1f)
19         if (!initialized) { initialized = true; lastTimestampNanos = timestampNanos; value = raw; return value }
20         val dt = ((timestampNanos - lastTimestampNanos) / NANOS_PER_SECOND).coerceIn(MIN_DT, MAX_DT)
21         lastTimestampNanos = timestampNanos
22         val stableRaw = if (raw <= ZERO_GLITCH_THRESHOLD && value > ZERO_GLITCH_THRESHOLD) value else raw
23         val tau = if (stableRaw >= value) RISE_TAU_SECONDS else FALL_TAU_SECONDS
24         val alpha = 1f - exp((-dt / tau).toDouble()).toFloat()
25         value += (stableRaw - value) * alpha
26         return value.coerceIn(0f, 1f)
27     }
28
29     private companion object {
30         const val NANOS_PER_SECOND = 1_000_000_000f
31         const val MIN_DT = 1f / 1000f
32         const val MAX_DT = 1f / 20f
33         const val RISE_TAU_SECONDS = .018f
34         const val FALL_TAU_SECONDS = .055f
35         const val ZERO_GLITCH_THRESHOLD = .01f
36     }
37 }
```

32. app/src/main/java/com/wetinknext/engine/brush/RibbonGeometryBuilder.kt

```

1  package com.wetinknext.engine.brush
2
3  import kotlin.math.PI
4  import kotlin.math.abs
5  import kotlin.math.acos
6  import kotlin.math.atan2
7  import kotlin.math.ceil
8  import kotlin.math.cos
9  import kotlin.math.hypot
10 import kotlin.math.max
11 import kotlin.math.min
12 import kotlin.math.sin
13
14 /**
15  * Builds one continuous ribbon mesh from sampled stroke points.
16  *
17  * Positions are retained in preallocated arrays while input is arriving. The
18  * resulting mesh shares each bank vertex between neighbour quads; this is the
19  * important difference from the old independent capsule SDF segments.
20  */
21 class RibbonGeometryBuilder(
22     private val maxPoints: Int = 8_192,
23 ) {
24     private val positions = FloatArray(maxPoints * 2)
25     private val radii = FloatArray(maxPoints)
26     private val coverage = FloatArray(maxPoints)
27     private val rotations = FloatArray(maxPoints)
28     private val cumulativeLength = FloatArray(maxPoints)
29
30     var count: Int = 0
31     private set
32
33     fun clear() { count = 0 }
34
35     fun addPoint(
36         x: Float,
37         y: Float,
38         radius: Float,
39         coverageValue: Float,
40         rotationRad: Float = 0f,
41     ): Boolean {
42         if (count >= maxPoints) return false
43         if (count > 0) {
44             val previous = (count - 1) * 2
45             if (hypot(x - positions[previous], y - positions[previous + 1]) < POINT_EPSILON) {
46                 radii[count - 1] = radius.coerceAtLeast(0.25f)
47                 coverage[count - 1] = coverageValue.coerceIn(0f, 1f)
48                 rotations[count - 1] = rotationRad
49                 return true
50             }
51         }
52         val offset = count * 2
53         positions[offset] = x
54         positions[offset + 1] = y
55         radii[count] = radius.coerceAtLeast(0.25f)
56         coverage[count] = coverageValue.coerceIn(0f, 1f)
57         rotations[count] = rotationRad
58         cumulativeLength[count] = if (count == 0) 0f else cumulativeLength[count - 1] +
59             hypot(x - positions[offset - 2], y - positions[offset - 1])
60         count++
61         return true
62     }
63
64     /** Vertex format: x, y, coverage. Indices are triangles. */
65     data class Mesh(val vertices: FloatArray, val indices: IntArray, val bounds: FloatArray)
66
67     fun build(settings: RibbonSettings): Mesh? {
68         if (count == 0) return null
69         if (count == 1) return buildDot(settings)
70
71         val total = cumulativeLength[count - 1].coerceAtLeast(POINT_EPSILON)
72         val tangentX = FloatArray(count)
73         val tangentY = FloatArray(count)
74         val leftX = FloatArray(count); val leftY = FloatArray(count)
75         val rightX = FloatArray(count); val rightY = FloatArray(count)
76         val effectiveRadius = FloatArray(count)
77         for (i in 0 until count) {
78             tangentAt(i, tangentX, tangentY)
79             effectiveRadius[i] = radii[i] * taperScale(cumulativeLength[i], total, settings)
80         }
81
82         for (i in 0 until count) {
83             val p = i * 2
84             val nx = -tangentY[i]; val ny = tangentX[i]
85             var ox = nx * effectiveRadius[i]; var oy = ny * effectiveRadius[i]
86             if (i in 1 until count - 1 && settings.join == RibbonJoin.MITER) {
87                 val miter = miterOffset(
88                     tangentX[i - 1], tangentY[i - 1], tangentX[i], tangentY[i],
89                     effectiveRadius[i], settings.miterLimit,
90                 )

```

```

91         if (miter != null) { ox = miter.first; oy = miter.second }
92     }
93     leftX[i] = positions[p] + ox; leftY[i] = positions[p + 1] + oy
94     rightX[i] = positions[p] - ox; rightY[i] = positions[p + 1] - oy
95 }
96
97 val vertices = ArrayList<Float>(count * 12)
98 val indices = ArrayList<Int>(count * 18)
99 val bounds = floatArrayOf(Float.POSITIVE_INFINITY, Float.POSITIVE_INFINITY, Float.NEGATIVE_INFINITY, Float.
NEGATIVE_INFINITY)
100 fun add(x: Float, y: Float, a: Float): Int {
101     vertices += x; vertices += y; vertices += a
102     bounds[0] = min(bounds[0], x); bounds[1] = min(bounds[1], y)
103     bounds[2] = max(bounds[2], x); bounds[3] = max(bounds[3], y)
104     return vertices.size / 3 - 1
105 }
106 val aa = settings.aaWidthPx.coerceAtLeast(0f)
107 val li = IntArray(count); val ri = IntArray(count)
108 val lo = IntArray(count); val ro = IntArray(count)
109 for (i in 0 until count) {
110     val p = i * 2; val nx = -tangentY[i]; val ny = tangentX[i]
111     li[i] = add(leftX[i], leftY[i], coverage[i]); ri[i] = add(rightX[i], rightY[i], coverage[i])
112     lo[i] = add(positions[p] + nx * (effectiveRadius[i] + aa), positions[p + 1] + ny * (effectiveRadius[i] + aa), 0f)
113     ro[i] = add(positions[p] - nx * (effectiveRadius[i] + aa), positions[p + 1] - ny * (effectiveRadius[i] + aa), 0f)
114 }
115 fun tri(a: Int, b: Int, c: Int) { indices += a; indices += b; indices += c }
116 for (i in 0 until count - 1) {
117     tri(li[i], ri[i], li[i + 1]); tri(ri[i], ri[i + 1], li[i + 1])
118     if (aa > 0f) {
119         tri(lo[i], li[i], lo[i + 1]); tri(li[i], li[i + 1], lo[i + 1])
120         tri(ri[i], ro[i], ri[i + 1]); tri(ro[i], ro[i + 1], ri[i + 1])
121     }
122 }
123 if (settings.join == RibbonJoin.ROUND) {
124     for (i in 1 until count - 1) addRoundJoin(i, tangentX, tangentY, effectiveRadius, coverage, aa, ::add, ::tri)
125 }
126 if (settings.cap == RibbonCap.ROUND) {
127     addCap(0, true, tangentX[0], tangentY[0], effectiveRadius[0], coverage[0], aa, ::add, ::tri)
128     addCap(count - 1, false, tangentX[count - 1], tangentY[count - 1], effectiveRadius[count - 1], coverage[count -
1], aa, ::add, ::tri)
129 }
130 return Mesh(vertices.toFloatArray(), indices.toIntArray(), bounds)
131 }
132
133 private fun buildDot(settings: RibbonSettings): Mesh {
134     val savedCount = count
135     // A degenerate stroke is a genuine ROUND cap fan, not a capsule AABB.
136     val vertices = ArrayList<Float>(); val indices = ArrayList<Int>()
137     val x = positions[0]; val y = positions[1]; val r = radii[0]; val aa = settings.aaWidthPx
138     fun add(px: Float, py: Float, a: Float) = (vertices.also { it += px; it += py; it += a }.size / 3 - 1)
139     val c = add(x, y, coverage[0]); var previous = -1
140     for (i in 0..DOT_SEGMENTS) {
141         val a = i * 2f * PI.toFloat() / DOT_SEGMENTS
142         val v = add(x + cos(a) * r, y + sin(a) * r, coverage[0])
143         if (previous >= 0) { indices += c; indices += previous; indices += v }
144         previous = v
145     }
146     return Mesh(vertices.toFloatArray(), indices.toIntArray(), floatArrayOf(x-r-aa,y-r-aa,x+r+aa,y+r+aa))
147 }
148
149 private fun tangentAt(i: Int, tx: FloatArray, ty: FloatArray) {
150     val before = (i - 1).coerceAtLeast(0) * 2; val after = (i + 1).coerceAtMost(count - 1) * 2
151     val dx = positions[after] - positions[before]; val dy = positions[after + 1] - positions[before + 1]
152     val d = hypot(dx, dy).coerceAtLeast(POINT_EPSILON); tx[i] = dx / d; ty[i] = dy / d
153 }
154
155 private fun miterOffset(ax: Float, ay: Float, bx: Float, by: Float, r: Float, limit: Float): Pair<Float, Float>? {
156     val nx = -ay; val ny = ax; val nbx = -by; val nby = bx
157     val mx = nax + nbx; val my = nay + nby; val ml = hypot(mx, my)
158     if (ml < POINT_EPSILON) return null
159     val ux = mx / ml; val uy = my / ml; val den = ux * nbx + uy * nby
160     if (abs(den) < POINT_EPSILON) return null
161     val length = r / den
162     return if (abs(length) <= r * limit) Pair(ux * length, uy * length) else null
163 }
164
165 private fun addRoundJoin(i: Int, tx: FloatArray, ty: FloatArray, radii: FloatArray, alpha: FloatArray, aa: Float, add: (
Float, Float, Float) -> Int, tri: (Int, Int, Int) -> Unit) {
166     val cross = tx[i - 1] * ty[i] - ty[i - 1] * tx[i]
167     if (abs(cross) < POINT_EPSILON) return
168     val px = positions[i*2]; val py = positions[i*2+1]; val sign = if (cross > 0f) -1f else 1f
169     val start = atan2(tx[i-1] * sign, -ty[i-1] * sign); val end = atan2(tx[i] * sign, -ty[i] * sign)
170     var delta = end - start; while (delta > PI) delta -= (2 * PI).toFloat(); while (delta < -PI) delta += (2 * PI).
toFloat()
171     val steps = ceil(abs(delta) / 0.35f).toInt().coerceIn(1, 12); val center = add(px, py, alpha[i]); val last = -1
172     for (s in 0..steps) { val a = start + delta*s/steps; val v = add(px*cos(a)*radii[i],py*sin(a)*radii[i],alpha[i]); if(
last>=0) tri(center,last,v); last=v }
173 }
174
175 private fun addCap(i: Int, start: Boolean, tx: Float, ty: Float, r: Float, alpha: Float, aa: Float, add: (Float, Float,
Float) -> Int, tri: (Int, Int, Int) -> Unit) {
176     val px=positions[i*2]; val py=positions[i*2+1]; val base=atan2(ty,tx)+(if(start) PI.toFloat() else 0f); val c=add(px,

```

```

py,alpha); var last=-1
177     for(s in 0..CAP_SEGMENTS){ val a=base-PI.toFloat()/2f+PI.toFloat()*s/CAP_SEGMENTS; val v=add(px+cos(a)*r,py+sin(a)*r,
alpha); if(last>=0)tri(c,last,v);last=v }
178     }
179
180     private fun taperScale(distance: Float, total: Float, settings: RibbonSettings): Float {
181         val start=if(settings.taperStartPx<=0f)1f else (distance/settings.taperStartPx).coerceIn(0f,1f)
182         val end=if(settings.taperEndPx<=0f)1f else ((total-distance)/settings.taperEndPx).coerceIn(0f,1f)
183         return min(start,end).coerceIn(settings.minWidthRatio,1f)
184     }
185
186     private companion object { const val POINT_EPSILON=0.0001f; const val CAP_SEGMENTS=24; const val DOT_SEGMENTS=32 }
187 }

```

33. app/src/main/java/com/wetinknext/engine/brush/RibbonMeshRenderer.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.GlCheck
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.RenderTarget
7  import com.wetinknext.engine.gl.ShaderLib
8  import java.nio.ByteBuffer
9  import java.nio.ByteOrder
10 import java.nio.FloatBuffer
11 import java.nio.IntBuffer
12
13 /** GPU half of [RibbonGeometryBuilder]. One draw call renders a welded mesh. */
14 class RibbonMeshRenderer {
15     private var program: GlProgram? = null
16     private var vao = 0; private var vertices = 0; private var indices = 0
17     private var uMatrix = -1; private var uColor = -1; private var uCoverageOnly = -1; private var uFlow = -1
18     private var vertexUpload: FloatBuffer? = null
19     private var indexUpload: IntBuffer? = null
20
21     fun create() {
22         GlCheck.checkOnGThread(); release()
23         // Use the dedicated coverage-mask aware shader, not the legacy
24         // three-attribute ribbon experiment retained in ShaderLib.
25         program = GlProgram(ShaderLib.ribbonVertex, ShaderLib.ribbonFragment)
26         val p = checkNotNull(program); p.use()
27         uMatrix=GLES30.glGetUniformLocation(p.id,"uCanvasToClip")
28         uColor=GLES30.glGetUniformLocation(p.id,"uColorLinear")
29         uCoverageOnly=GLES30.glGetUniformLocation(p.id,"uCoverageOnly")
30         uFlow=GLES30.glGetUniformLocation(p.id,"uFlow")
31         check((uMatrix>=0 && uColor>=0 && uCoverageOnly>=0 && uFlow>=0))
32         val ids=IntArray(1); GLES30.glGenVertexArrays(1,ids,0); vao=ids[0]
33         GLES30.glGenBuffers(1,ids,0); vertices=ids[0]; GLES30.glGenBuffers(1,ids,0); indices=ids[0]
34         GLES30.glBindVertexArray(vao); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER,vertices)
35         GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0,2,GLES30.GL_FLOAT,false,12,0)
36         GLES30.glEnableVertexAttribArray(1); GLES30.glVertexAttribPointer(1,1,GLES30.GL_FLOAT,false,12,8)
37         GLES30.glBindBuffer(GLES30.GL_ELEMENT_ARRAY_BUFFER,indices); GLES30.glBindVertexArray(0)
38     }
39     fun draw(target: RenderTarget, width:Int, height:Int, matrix:FloatArray, mesh:RibbonGeometryBuilder.Mesh, color:
FloatArray, flow: Float, coverageOnly:Boolean) {
40         val p=program?:return; if(mesh.indices.isEmpty()) return
41         target.bind(); GLES30.glViewport(0,0,width,height); p.use()
42         GLES30.glUniformMatrix4fv(uMatrix,1,false,matrix,0); GLES30.glUniform3fv(uColor,1,color,0); GLES30.glUniform1i(
uCoverageOnly,if(coverageOnly)1 else 0); GLES30.glUniform1f(uFlow, flow.coerceIn(0f, 1f))
43         val vb = obtainVertexBuffer(mesh.vertices.size); vb.clear(); vb.put(mesh.vertices).position(0)
44         val ib = obtainIndexBuffer(mesh.indices.size); ib.clear(); ib.put(mesh.indices).position(0)
45         GLES30.glBindVertexArray(vao); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER,vertices); GLES30.glBufferData(GLES30.
GL_ARRAY_BUFFER,mesh.vertices.size*4,vb,GLES30.GL_STREAM_DRAW)
46         GLES30.glBindBuffer(GLES30.GL_ELEMENT_ARRAY_BUFFER,indices); GLES30.glBufferData(GLES30.GL_ELEMENT_ARRAY_BUFFER,mesh.
indices.size*4,ib,GLES30.GL_STREAM_DRAW)
47         if(coverageOnly){GLES30.glEnable(GLES30.GL_BLEND);GLES30.glBlendEquation(GLES30.GL_MAX);GLES30.glBlendFunc(GLES30.
GL_ONE,GLES30.GL_ONE)}else{GLES30.glEnable(GLES30.GL_BLEND);GLES30.glBlendEquation(GLES30.GL_FUNC_ADD);GLES30.glBlendFunc(GLES30.
GL_ONE,GLES30.GL_ONE_MINUS_SRC_ALPHA)}
48         GLES30.glDrawElements(GLES30.GL_TRIANGLES,mesh.indices.size,GLES30.GL_UNSIGNED_INT,0)
49         GLES30.glBindVertexArray(0); GLES30.glBlendEquation(GLES30.GL_FUNC_ADD); GLES30.glBlendFunc(GLES30.GL_ONE,GLES30.
GL_ONE_MINUS_SRC_ALPHA); GLES30.glDisable(GLES30.GL_BLEND)
50     }
51     private fun obtainVertexBuffer(size: Int): FloatBuffer {
52         val current = vertexUpload
53         if (current != null && current.capacity() >= size) return current
54         val capacity = maxOf(size, (current?.capacity() ?: 256) * 2)
55         return ByteBuffer.allocateDirect(capacity * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer().also {
vertexUpload = it }
56     }
57     private fun obtainIndexBuffer(size: Int): IntBuffer {
58         val current = indexUpload
59         if (current != null && current.capacity() >= size) return current
60         val capacity = maxOf(size, (current?.capacity() ?: 256) * 2)
61         return ByteBuffer.allocateDirect(capacity * Int.SIZE_BYTES).order(ByteOrder.nativeOrder()).asIntBuffer().also {
indexUpload = it }
62     }
63     fun release(){ if(vao!=0)GLES30.glDeleteVertexArrays(1,intArrayOf(vao),0); if(vertices!=0)GLES30.glDeleteBuffers(1,
intArrayOf(vertices),0); if(indices!=0)GLES30.glDeleteBuffers(1,intArrayOf(indices),0); vao=0;vertices=0;indices=0;
vertexUpload=null;indexUpload=null;program?.release();program=null }
64 }

```

34. app/src/main/java/com/wetinknext/engine/brush/Stabilizer.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.sqrt
4
5 class Stabilizer(private val filter: OneEuroFilter = OneEuroFilter()) {
6     var strength = 0f; var x = 0f; private set; var y = 0f; private set; var velocity = 0f; private set
7     private var hasLast = false; private var lastX = 0f; private var lastY = 0f; private var lastT = 0L; private val filtered
8     = FloatArray(2)
9     fun reset() { filter.reset(); x = 0f; y = 0f; velocity = 0f; hasLast = false; lastX = 0f; lastY = 0f; lastT = 0L }
10    fun process(timestampNanos: Long, rawX: Float, rawY: Float) {
11        if (strength <= 0f) { x = rawX; y = rawY } else { filter.filter(timestampNanos, rawX, rawY, filtered); x = filtered[
12            0]; y = filtered[1] }
13        if (hasLast) { var dt = (timestampNanos - lastT) / 1_000_000_000f; if (dt < 1f / 1000f) dt = 1f / 1000f; val d = sqrt(
14            (x-lastX)*(x-lastX)+(y-lastY)*(y-lastY)); velocity += 0.2f * (d / dt - velocity) }
15        hasLast = true; lastX = x; lastY = y; lastT = timestampNanos
16    }
17 }
```


35. app/src/main/java/com/wetinknext/engine/brush/StampEmitter.kt

```

1 package com.wetinknext.engine.brush
2
3 import com.wetinknext.engine.input.InputBatch
4 import kotlin.math.sqrt
5
6 class StampEmitter(initialSettings: BrushSettings) {
7     var settings: BrushSettings = initialSettings
8     private set
9     private var strokeSettings: BrushSettings? = null
10
11     fun updateSettings(newSettings: BrushSettings) {
12         // Active stroke reads strokeSettings; this value is for the next begin().
13         settings = newSettings.resolved()
14     }
15
16     private val stabilizer = Stabilizer()
17     private val pressureFilter = PressureFilter()
18     private val resolvedDab = ResolvedDab()
19     private var active = false
20     private var pointerId = -1
21     private var hasLast = false
22     private var lastX = 0f
23     private var lastY = 0f
24     private var lastPressure = 0f
25     private var lastTiltX = 0f
26     private var lastTiltY = 0f
27     private var lastOrientationRad = 0f
28     private var lastVelocity = 0f
29     private var randomState = 0x13579BDF
30     private var strokeSeed = 0
31     private var carriedDistance = 0f
32     private var movedDuringStroke = false
33     private var pointCount = 0
34     private var p0x = 0f; private var p0y = 0f; private var p0p = 0f
35     private var p1x = 0f; private var p1y = 0f; private var p1p = 0f
36     private var p2x = 0f; private var p2y = 0f; private var p2p = 0f
37     private var p3x = 0f; private var p3y = 0f; private var p3p = 0f
38
39     private fun activeSettings(): BrushSettings {
40         check(active) { "Stamp emitter has no active stroke" }
41         return strokeSettings ?: error("Missing stroke settings")
42     }
43
44     fun reset() {
45         active = false
46         pointerId = -1
47         hasLast = false
48         carriedDistance = 0f
49         movedDuringStroke = false
50         pointCount = 0
51         strokeSettings = null
52         lastOrientationRad = 0f
53         lastVelocity = 0f
54         stabilizer.reset()
55         pressureFilter.reset()
56     }
57
58     fun begin(
59         batch: InputBatch,
60         out: DabBuffer,
61         strokeSettings: BrushSettings = settings,
62     ) {
63         reset()
64         strokeSeed++
65         randomState = 0x13579BDF xor strokeSeed
66         val resolvedSettings = strokeSettings.resolved()
67         this.strokeSettings = resolvedSettings
68         if (batch.isEmpty()) return
69         stabilizer.strength = (resolvedSettings.smoothing * .7f + resolvedSettings.streamline * .3f).coerceIn(0f, 1f)
70         val s = batch.samples[0]
71         active = true
72         pointerId = s.pointerId
73         stabilizer.process(s.timestampNanos, s.canvasX, s.canvasY)
74         lastX = stabilizer.x
75         lastY = stabilizer.y
76         lastPressure = pressureFilter.filter(s.timestampNanos, s.pressure)
77         lastTiltX = s.tiltX
78         lastTiltY = s.tiltY
79         lastOrientationRad = s.orientationRad
80         lastVelocity = 0f
81         hasLast = true
82         p0x = lastX; p0y = lastY; p0p = lastPressure
83         p1x = lastX; p1y = lastY; p1p = lastPressure
84         p2x = lastX; p2y = lastY; p2p = lastPressure
85         p3x = lastX; p3y = lastY; p3p = lastPressure
86         pointCount = 1
87     }
88
89     fun append(batch: InputBatch, out: DabBuffer) {
90         if (!active) return

```

```

91     val settings = activeSettings()
92     for (i in 0 until batch.sampleCount) {
93         val s = batch.samples[i]
94         if (s.pointerId == pointerId) {
95             stabilizer.process(s.timestampNanos, s.canvasX, s.canvasY)
96             val pressure = pressureFilter.filter(s.timestampNanos, s.pressure)
97             val velocity = stabilizer.velocity
98             lastOrientationRad = s.orientationRad
99             if (settings.smoothing <= .0001f) {
100                 addPoint(stabilizer.x, stabilizer.y, pressure, s.tiltX, s.tiltY, velocity, s.orientationRad, out)
101             } else {
102                 lastVelocity = velocity
103                 lastTiltX = s.tiltX
104                 lastTiltY = s.tiltY
105                 appendSmoothedPoint(stabilizer.x, stabilizer.y, pressure, out)
106             }
107         }
108     }
109 }
110
111 fun finish(out: DabBuffer, cancel: Boolean) {
112     if (!active) return
113     val settings = activeSettings()
114     if (cancel) {
115         reset()
116         return
117     }
118     if (settings.smoothing > .0001f && pointCount >= 3) {
119         flushSmoothedTail(out)
120     }
121     if (!movedDuringStroke || carriedDistance > .001f) {
122         addDab(
123             x = lastX,
124             y = lastY,
125             pressure = lastPressure,
126             tiltX = lastTiltX,
127             tiltY = lastTiltY,
128             velocity = lastVelocity,
129             orientationRad = lastOrientationRad,
130             out = out,
131         )
132     }
133     reset()
134 }
135
136 private fun appendSmoothedPoint(x: Float, y: Float, pressure: Float, out: DabBuffer) {
137     p0x = p1x; p0y = p1y; p0p = p1p
138     p1x = p2x; p1y = p2y; p1p = p2p
139     p2x = p3x; p2y = p3y; p2p = p3p
140     p3x = x; p3y = y; p3p = pressure
141     pointCount = (pointCount + 1).coerceAtMost(4)
142
143     when (pointCount) {
144         2 -> addPoint(x, y, pressure, lastTiltX, lastTiltY, lastVelocity, lastOrientationRad, out)
145         4 -> interpolateCatmullRom(out)
146     }
147 }
148
149 /** Processes the last delayed Catmull-Rom segment with a duplicated endpoint. */
150 private fun flushSmoothedTail(out: DabBuffer) {
151     p0x = p1x; p0y = p1y; p0p = p1p
152     p1x = p2x; p1y = p2y; p1p = p2p
153     p2x = p3x; p2y = p3y; p2p = p3p
154     p3x = p2x; p3y = p2y; p3p = p2p
155     pointCount = 4
156     interpolateCatmullRom(out)
157 }
158
159 private fun interpolateCatmullRom(out: DabBuffer) {
160     val settings = activeSettings()
161     val dx = p2x - p1x
162     val dy = p2y - p1y
163     val distance = sqrt(dx * dx + dy * dy)
164     if (distance < .0001f) return
165
166     val steps = (distance / maxOf(1f, settings.baseRadiusPx * .25f))
167         .toInt()
168         .coerceIn(2, 64)
169
170     for (index in 1..steps) {
171         val t = index.toFloat() / steps
172         val t2 = t * t
173         val t3 = t2 * t
174         val x = .5f * (
175             2f * p1x + (-p0x + p2x) * t +
176             (2f * p0x - 5f * p1x + 4f * p2x - p3x) * t2 +
177             (-p0x + 3f * p1x - 3f * p2x + p3x) * t3
178         )
179         val y = .5f * (
180             2f * p1y + (-p0y + p2y) * t +
181             (2f * p0y - 5f * p1y + 4f * p2y - p3y) * t2 +
182             (-p0y + 3f * p1y - 3f * p2y + p3y) * t3

```

```

183         )
184         val pressure = p1p + (p2p - p1p) * t
185         addPoint(x, y, pressure, lastTiltX, lastTiltY, lastVelocity, lastOrientationRad, out)
186     }
187 }
188
189 private fun addPoint(x: Float, y: Float, p: Float, tx: Float, ty: Float, velocity: Float, orientationRad: Float, out:
DabBuffer) {
190     val settings = activeSettings()
191     val dx = x - lastX
192     val dy = y - lastY
193     val distance = sqrt(dx * dx + dy * dy)
194     if (distance <= .0001f) return
195     if (distance > 1.5f) movedDuringStroke = true
196
197     var travelled = 0f
198     while (travelled < distance) {
199         val tProbe = (travelled / distance).coerceIn(0f, 1f)
200         val probePressure = lastPressure + (p - lastPressure) * tProbe
201         val probeTiltX = lastTiltX + (tx - lastTiltX) * tProbe
202         val probeTiltY = lastTiltY + (ty - lastTiltY) * tProbe
203
204         BrushDynamics.resolve(
205             settings = settings,
206             pressure = probePressure,
207             tiltX = probeTiltX,
208             tiltY = probeTiltY,
209             velocityPxPerSecond = lastVelocity + (velocity - lastVelocity) * tProbe,
210             // This probe only selects spacing. Do not consume the stroke
211             // RNG before the dab that will actually be emitted.
212             random01 = 0.5f,
213             out = resolvedDab,
214         )
215
216         val step = spacingForRadius(resolvedDab.radius)
217         // Radius can shrink with pressure, leaving more carried distance
218         // than the new step. Emit at the current point and reset it.
219         val needed = (step - carriedDistance).coerceAtLeast(0f)
220
221         if (travelled + needed > distance) {
222             carriedDistance += distance - travelled
223             break
224         }
225
226         travelled += needed
227         val t = (travelled / distance).coerceIn(0f, 1f)
228         addDab(
229             x = lastX + dx * t,
230             y = lastY + dy * t,
231             pressure = lastPressure + (p - lastPressure) * t,
232             tiltX = lastTiltX + (tx - lastTiltX) * t,
233             tiltY = lastTiltY + (ty - lastTiltY) * t,
234             velocity = lastVelocity + (velocity - lastVelocity) * t,
235             orientationRad = orientationRad,
236             out = out,
237         )
238         carriedDistance = 0f
239     }
240     lastX = x
241     lastY = y
242     lastPressure = p
243     lastTiltX = tx
244     lastTiltY = ty
245     lastVelocity = velocity
246     lastOrientationRad = orientationRad
247 }
248
249 private fun spacingForRadius(radius: Float): Float {
250     val settings = activeSettings()
251     val unit = if (settings.spacingUsesDiameter) radius * 2f else radius
252     return (unit * settings.spacing).coerceIn(.25f, 256f)
253 }
254
255 private fun addDab(
256     x: Float,
257     y: Float,
258     pressure: Float,
259     tiltX: Float,
260     tiltY: Float,
261     velocity: Float,
262     orientationRad: Float,
263     out: DabBuffer,
264 ) {
265     val settings = activeSettings()
266     BrushDynamics.resolve(
267         settings = settings,
268         pressure = pressure,
269         tiltX = tiltX,
270         tiltY = tiltY,
271         velocityPxPerSecond = velocity,
272         random01 = nextRandom01(),
273         out = resolvedDab,

```

```

274     )
275     out.add(
276         x = x + resolvedDab.scatterX,
277         y = y + resolvedDab.scatterY,
278         radius = resolvedDab.radius,
279         rotation = resolvedDab.rotationRad,
280         coverage = resolvedDab.coverage,
281         flow = resolvedDab.flow,
282         hardness = resolvedDab.hardness,
283     )
284 }
285
286 /** Deterministic LCG: undo/replay never depends on a global random source. */
287 private fun nextRandom01(): Float {
288     randomState = randomState * 1_664_525 + 1_013_904_223
289     return ((randomState ushr 8) and 0x00FF_FFFF) / 16_777_216f
290 }
291
292 }

```

36. app/src/main/java/com/wetinknext/engine/brush/StrokeRenderMode.kt

```
1 package com.wetinknext.engine.brush
2
3 /**
4  * Declares what a temporary stroke texture represents on this pass.
5  *
6  * NORMAL_BUILDUP stores premultiplied colour accumulated with source-over.
7  * NON_BUILDUP stores union coverage and receives colour only in the combine
8  * pass. Keeping this explicit prevents a renderer from guessing by policy.
9  */
10 enum class StrokeRenderMode {
11     NORMAL_BUILDUP,
12     NON_BUILDUP,
13 }
```

37. app/src/main/java/com/wetinknext/engine/brush/TextureLoader.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.content.Context
4  import android.graphics.Bitmap
5  import android.graphics.BitmapFactory
6  import java.nio.ByteBuffer
7  import java.nio.ByteOrder
8  import java.util.concurrent.ExecutorService
9  import java.util.concurrent.Executors
10
11  data class LoadedBrushTexture(
12      val path: String,
13      val width: Int,
14      val height: Int,
15      val rgba: ByteBuffer,
16  )
17
18  class TextureLoader(
19      private val context: Context,
20      private val executor: ExecutorService =
21          Executors.newSingleThreadExecutor(),
22  ) {
23      fun loadAsync(
24          path: String,
25          onLoad: (LoadedBrushTexture) -> Unit,
26          onError: (Throwable) -> Unit,
27      ) {
28          executor.execute {
29              try {
30                  val loaded = decode(path)
31                  onLoad(loaded)
32              } catch (error: Throwable) {
33                  onError(error)
34              }
35          }
36      }
37
38      private fun decode(path: String): LoadedBrushTexture {
39          val source = decodeBitmap(path)
40
41          val bitmap = if (source.config == Bitmap.Config.ARGB_8888) {
42              source
43          } else {
44              source.copy(Bitmap.Config.ARGB_8888, false)
45          }
46
47          val width = bitmap.width
48          val height = bitmap.height
49
50          val rgba = ByteBuffer
51              .allocateDirect(width * height * 4)
52              .order(ByteOrder.nativeOrder())
53
54          // Bitmap stores ARGB_8888 pixels as packed ARGB ints. Uploading that
55          // buffer directly makes the byte order platform-dependent; GLES expects
56          // the explicit RGBA byte sequence declared in BrushTexture.
57          val pixels = IntArray(width * height)
58          bitmap.getPixels(
59              pixels,
60              0,
61              width,
62              0,
63              0,
64              width,
65              height,
66          )
67          for (pixel in pixels) {
68              rgba.put(((pixel ushr 16) and 0xFF).toByte())
69              rgba.put(((pixel ushr 8) and 0xFF).toByte())
70              rgba.put((pixel and 0xFF).toByte())
71              rgba.put(((pixel ushr 24) and 0xFF).toByte())
72          }
73          rgba.flip()
74
75          if (bitmap != source && !bitmap.isRecycled) {
76              bitmap.recycle()
77          }
78
79          if (!source.isRecycled && source != bitmap) {
80              source.recycle()
81          }
82
83          return LoadedBrushTexture(
84              path = path,
85              width = width,
86              height = height,
87              rgba = rgba,
88          )
89      }
90  }

```

```

91     private fun decodeBitmap(path: String): Bitmap {
92         val bitmap = if (path.startsWith("asset:")) {
93             val assetPath = path.removePrefix("asset:")
94
95             context.assets.open(assetPath).use { input ->
96                 BitmapFactory.decodeStream(input)
97             }
98         } else {
99             BitmapFactory.decodeFile(path)
100         }
101
102         return requireNotNull(bitmap) {
103             "Cannot decode brush texture: $path"
104         }
105     }
106
107     fun shutdown() {
108         executor.shutdown()
109     }
110 }

```

38. app/src/main/java/com/wetinknext/engine/canvas/BlendMode.kt

```
1 package com.wetinknext.engine.canvas
2
3 import kotlinx.serialization.Serializable
4
5 /** Stored per layer now; shader implementations beyond NORMAL are scheduled after P6. */
6 @Serializable
7 enum class BlendMode(val id: Int) {
8     NORMAL(0),
9     MULTIPLY(1),
10    SCREEN(2),
11    OVERLAY(3),
12    DARKEN(4),
13    LIGHTEN(5),
14    ADD(6),
15 }
```


39. app/src/main/java/com/wetinknext/engine/canvas/CanvasBackdropRenderer.kt

```

1  package com.wetinknext.engine.canvas
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.GlCheck
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.ShaderLib
7  import java.nio.ByteBuffer
8  import java.nio.ByteOrder
9
10 /** Draws the editor workspace pattern in one GPU pass. */
11 class CanvasBackdropRenderer {
12     private var program: GlProgram? = null
13     private var vertexArrayId = 0
14     private var vertexBufferId = 0
15     private var backgroundColorUniform = -1
16     private var gridColorUniform = -1
17     private var modeUniform = -1
18
19     fun create() {
20         GlCheck.checkOnGThread()
21         release()
22
23         program = GlProgram(ShaderLib.fullscreenVertex, ShaderLib.canvasBackdropFragment)
24         val currentProgram = checkNotNull(program)
25         currentProgram.use()
26         backgroundColorUniform = GLES30.glGetUniformLocation(currentProgram.id, "uBackgroundColor")
27         gridColorUniform = GLES30.glGetUniformLocation(currentProgram.id, "uGridColor")
28         modeUniform = GLES30.glGetUniformLocation(currentProgram.id, "uMode")
29         check(backgroundColorUniform >= 0 && gridColorUniform >= 0 && modeUniform >= 0) {
30             "Canvas backdrop uniforms missing"
31         }
32
33         val vertices = floatArrayOf(-1f, -1f, 1f, -1f, 1f, 1f, 1f, 1f)
34         val vertexData = ByteBuffer
35             .allocateDirect(vertices.size * Float.SIZE_BYTES)
36             .order(ByteOrder.nativeOrder())
37             .asFloatBuffer()
38             .apply { put(vertices); position(0) }
39         val ids = IntArray(1)
40         GLES30.glGenVertexArrays(1, ids, 0)
41         vertexArrayId = ids[0]
42         GLES30.glGenBuffers(1, ids, 0)
43         vertexBufferId = ids[0]
44         GLES30.glBindVertexArray(vertexArrayId)
45         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vertexBufferId)
46         GLES30.glBufferData(
47             GLES30.GL_ARRAY_BUFFER,
48             vertices.size * Float.SIZE_BYTES,
49             vertexData,
50             GLES30.GL_STATIC_DRAW,
51         )
52         GLES30.glEnableVertexAttribArray(0)
53         GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
54         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0)
55         GLES30.glBindVertexArray(0)
56         GlCheck.noError("CanvasBackdropRenderer create")
57     }
58
59     fun render(
60         viewportWidth: Int,
61         viewportHeight: Int,
62         backgroundColor: FloatArray,
63         gridColor: FloatArray,
64         mode: Int,
65     ) {
66         val currentProgram = program ?: return
67         if (viewportWidth <= 0 || viewportHeight <= 0 || vertexArrayId == 0) return
68
69         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
70         GLES30.glViewport(0, 0, viewportWidth, viewportHeight)
71         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
72         GLES30.glDisable(GLES30.GL_BLEND)
73         currentProgram.use()
74         GLES30.glUniform3fv(backgroundColorUniform, 1, backgroundColor, 0)
75         GLES30.glUniform3fv(gridColorUniform, 1, gridColor, 0)
76         GLES30.glUniform1i(modeUniform, mode)
77         GLES30.glBindVertexArray(vertexArrayId)
78         GLES30.glDrawArrays(GLES30.GL_TRIANGLE_STRIP, 0, 4)
79         GLES30.glBindVertexArray(0)
80     }
81
82     fun release() {
83         if (vertexBufferId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vertexBufferId), 0)
84         if (vertexArrayId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vertexArrayId), 0)
85         vertexArrayId = 0
86         vertexBufferId = 0
87         program?.release()
88         program = null
89         backgroundColorUniform = -1
90         gridColorUniform = -1
91     }
92 }

```

```
91         modeUniform = -1
92     }
93
94     companion object {
95         const val MODE_GRID = 1
96         const val MODE_CHECKERBOARD = 2
97     }
98 }
```

40. app/src/main/java/com/wetinknext/engine/canvas/Compositor.kt

```

1 package com.wetinknext.engine.canvas
2
3 import android.opengl.GLES30
4 import com.wetinknext.engine.gl.CanvasGeometry
5 import com.wetinknext.engine.gl.GlCheck
6 import com.wetinknext.engine.gl.GlProgram
7 import com.wetinknext.engine.gl.RenderTarget
8 import com.wetinknext.engine.gl.ShaderLib
9 import com.wetinknext.engine.brush.StrokeRenderMode
10
11 /** Presents visible layers bottom-to-top, inserting the active preview at its layer position. */
12 class Compositor {
13     private var program: GlProgram? = null
14     private var uCanvasToClip = -1
15     private var uCanvasSize = -1
16     private var uLayerTex = -1
17     private var uStrokeTex = -1
18     private var uScreenStrokeTex = -1
19     private var uStrokeCoverageTex = -1
20     private var uStrokeActive = -1
21     private var uStrokeIsScreenSpace = -1
22     private var uStrokeMode = -1
23     private var uStrokeErase = -1
24     private var uOpacity = -1
25     private var uStrokeOpacity = -1
26     private var uStrokeColorLinear = -1
27
28     fun create() {
29         GlCheck.checkOnGThread()
30         release()
31         program = GlProgram(ShaderLib.compositorVertex, ShaderLib.compositorFragment)
32         val currentProgram = checkNotNull(program)
33         currentProgram.use()
34         uCanvasToClip = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasToClip")
35         uCanvasSize = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasSize")
36         uLayerTex = GLES30.glGetUniformLocation(currentProgram.id, "uLayerTex")
37         uStrokeTex = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeTex")
38         uScreenStrokeTex = GLES30.glGetUniformLocation(currentProgram.id, "uScreenStrokeTex")
39         uStrokeCoverageTex = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeCoverageTex")
40         uStrokeActive = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeActive")
41         uStrokeIsScreenSpace = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeIsScreenSpace")
42         uStrokeMode = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeMode")
43         uStrokeErase = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeErase")
44         uOpacity = GLES30.glGetUniformLocation(currentProgram.id, "uOpacity")
45         uStrokeOpacity = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeOpacity")
46         uStrokeColorLinear = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeColorLinear")
47         check(uCanvasToClip >= 0 && uCanvasSize >= 0 && uLayerTex >= 0 &&
48             uStrokeTex >= 0 && uScreenStrokeTex >= 0 && uStrokeCoverageTex >= 0 &&
49             uStrokeActive >= 0 && uStrokeIsScreenSpace >= 0 && uStrokeMode >= 0 &&
50             uStrokeErase >= 0 && uOpacity >= 0 && uStrokeOpacity >= 0 && uStrokeColorLinear >= 0) {
51             "Compositor uniforms missing: canvasToClip=$uCanvasToClip, " +
52                 "canvasSize=$uCanvasSize, layerTex=$uLayerTex, strokeTex=$uStrokeTex, " +
53                 "screenStrokeTex=$uScreenStrokeTex, strokeActive=$uStrokeActive, " +
54                 "strokeIsScreenSpace=$uStrokeIsScreenSpace, strokeErase=$uStrokeErase, " +
55                 "opacity=$uOpacity, strokeOpacity=$uStrokeOpacity"
56         }
57         GlCheck.noError("Compositor create")
58     }
59
60     fun render(
61         destination: RenderTarget? = null,
62         geometry: CanvasGeometry,
63         layers: LayerStack,
64         activeLayerId: Long,
65         strokeTextureId: Int,
66         strokeCoverageTextureId: Int = 0,
67         strokeIsScreenSpace: Boolean = false,
68         strokeMode: StrokeRenderMode = StrokeRenderMode.NORMAL_BUILDUP,
69         strokeErase: Boolean,
70         strokeColorLinear: FloatArray = DEFAULT_STROKE_COLOR,
71         canvasToClip: FloatArray,
72         strokeOpacity: Float,
73         firstLayerIndex: Int = 0,
74         lastLayerExclusive: Int = Int.MAX_VALUE,
75     ) {
76         val currentProgram = program ?: return
77         destination?.bind()
78         currentProgram.use()
79         GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToClip, 0)
80         GLES30.glUniform2f(uCanvasSize, layers.canvasWidth.toFloat(), layers.canvasHeight.toFloat())
81         GLES30.glEnable(GLES30.GL_BLEND)
82         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
83
84         for ((index, layer) in layers.allLayers().withIndex()) {
85             if (index < firstLayerIndex || index >= lastLayerExclusive) continue
86             if (!layer.created || !layer.isVisible) continue
87             val hasStroke = layer.id == activeLayerId &&
88                 (strokeTextureId != 0 || strokeCoverageTextureId != 0)
89
90             GLES30.glActiveTexture(GLES30.GL_TEXTURE0)

```

```

91         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, layer.target.textureId)
92         GLES30.glUniform1i(uLayerTex, 0)
93         if (hasStroke) {
94             val textureUnit = if (strokeIsScreenSpace) GLES30.GL_TEXTURE2 else GLES30.GL_TEXTURE1
95             GLES30.glActiveTexture(textureUnit)
96             GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, strokeTextureId)
97             GLES30.glUniform1i(
98                 if (strokeIsScreenSpace) uScreenStrokeTex else uStrokeTex,
99                 if (strokeIsScreenSpace) 2 else 1,
100             )
101             if (strokeMode == StrokeRenderMode.NON_BUILDUP) {
102                 GLES30.glActiveTexture(GLES30.GL_TEXTURE3)
103                 GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, strokeCoverageTextureId)
104                 GLES30.glUniform1i(uStrokeCoverageTex, 3)
105             }
106         }
107         GLES30.glUniform1i(uStrokeActive, if (hasStroke) 1 else 0)
108         GLES30.glUniform1i(uStrokeIsScreenSpace, if (hasStroke && strokeIsScreenSpace) 1 else 0)
109         GLES30.glUniform1i(uStrokeMode, if (hasStroke && strokeMode == StrokeRenderMode.NON_BUILDUP) 1 else 0)
110         GLES30.glUniform1i(uStrokeErase, if (hasStroke && strokeErase) 1 else 0)
111         GLES30.glUniform1f(uOpacity, layer.opacity.coerceIn(0f, 1f))
112         GLES30.glUniform1f(uStrokeOpacity, strokeOpacity.coerceIn(0f, 1f))
113         GLES30.glUniform3f(uStrokeColorLinear, strokeColorLinear[0], strokeColorLinear[1], strokeColorLinear[2])
114         geometry.draw()
115     }
116
117     GLES30.glDisable(GLES30.GL_BLEND)
118     GLES30.glActiveTexture(GLES30.GL_TEXTURE1)
119     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
120     GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
121     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
122     GLES30.glActiveTexture(GLES30.GL_TEXTURE3)
123     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
124     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
125     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
126 }
127
128 /**
129  * Draws one layer into the currently bound target.
130  *
131  * This is deliberately separate from [render]: a layer-preview needs the
132  * layer bitmap even when the layer is currently hidden in the project. Its
133  * opacity is still applied, while a blend mode has no second layer to blend
134  * against in this one-layer path.
135  */
136 fun renderLayer(
137     geometry: CanvasGeometry,
138     layer: PaintLayer,
139     canvasWidth: Int = layer.target.width,
140     canvasHeight: Int = layer.target.height,
141     canvasToClip: FloatArray,
142 ) {
143     if (!layer.created || canvasWidth <= 0 || canvasHeight <= 0) return
144     val currentProgram = program ?: return
145     currentProgram.use()
146     GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToClip, 0)
147     GLES30.glUniform2f(uCanvasSize, canvasWidth.toFloat(), canvasHeight.toFloat())
148     GLES30.glEnable(GLES30.GL_BLEND)
149     GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
150
151     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
152     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, layer.target.textureId)
153     GLES30.glUniform1i(uLayerTex, 0)
154     GLES30.glUniform1i(uStrokeActive, 0)
155     GLES30.glUniform1i(uStrokeIsScreenSpace, 0)
156     GLES30.glUniform1i(uStrokeMode, 0)
157     GLES30.glUniform1i(uStrokeErase, 0)
158     GLES30.glUniform1f(uOpacity, layer.opacity.coerceIn(0f, 1f))
159     GLES30.glUniform1f(uStrokeOpacity, 1f)
160     GLES30.glUniform3f(uStrokeColorLinear, 0f, 0f, 0f)
161     geometry.draw()
162
163     GLES30.glDisable(GLES30.GL_BLEND)
164     GLES30.glActiveTexture(GLES30.GL_TEXTURE1)
165     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
166     GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
167     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
168     GLES30.glActiveTexture(GLES30.GL_TEXTURE3)
169     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
170     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
171     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
172 }
173
174 fun release() {
175     GLCheck.checkOnGThread()
176     program?.release()
177     program = null
178     uCanvasToClip = -1
179     uCanvasSize = -1
180     uLayerTex = -1
181     uStrokeTex = -1
182     uScreenStrokeTex = -1

```

```
183         uStrokeCoverageTex = -1
184         uStrokeActive = -1
185         uStrokeIsScreenSpace = -1
186         uStrokeMode = -1
187         uStrokeErase = -1
188         uOpacity = -1
189         uStrokeOpacity = -1
190         uStrokeColorLinear = -1
191     }
192
193     private companion object {
194         val DEFAULT_STROKE_COLOR = floatArrayOf(0f, 0f, 0f)
195     }
196 }
```

41. app/src/main/java/com/wetinknext/engine/canvas/LayerStack.kt

```

1 package com.wetinknext.engine.canvas
2
3 import com.wetinknext.engine.core.Camera
4 import com.wetinknext.engine.gl.GlCaps
5 import com.wetinknext.domain.document.ProjectDocument
6
7 /**
8  * Render-thread-owned ordered collection of document layers.
9  * Index 0 is the bottom-most layer and the final item is the top-most layer.
10 */
11 class LayerStack {
12     val camera = Camera()
13
14     private val layers = mutableListOf<PaintLayer>()
15     private var nextId = 1L
16     private var documentUsesHalfFloat = false
17
18     var activeLayerId: Long = NO_LAYER
19     private set
20     var canvasWidth: Int = 0
21     private set
22     var canvasHeight: Int = 0
23     private set
24
25     val count: Int get() = layers.size
26
27     fun allLayers(): List<PaintLayer> = layers
28
29     fun activeLayer(): PaintLayer? = findLayerById(activeLayerId)
30
31     fun findLayerById(id: Long): PaintLayer? = layers.firstOrNull { it.id == id }
32
33     fun indexOfLayer(id: Long): Int = layers.indexOfFirst { it.id == id }
34
35     /** Creates the runtime stack from persisted layer metadata. */
36     fun create(caps: GlCaps, document: ProjectDocument) {
37         release()
38         canvasWidth = document.width
39         canvasHeight = document.height
40         documentUsesHalfFloat = caps.supportsHalfFloatColorBuffer
41
42         document.layers.forEachIndexed { index, source ->
43             val layer = PaintLayer(source.id, source.name).also {
44                 it.isVisible = source.visible
45                 it.isLocked = source.locked
46                 it.opacity = source.opacity
47                 it.blendMode = source.blendMode
48                 it.version = source.thumbnailVersion
49                 it.create(canvasWidth, canvasHeight, documentUsesHalfFloat)
50             }
51             // Tile restoration is a later persistence step. Preserve the new
52             // document's white background until a stored pixel payload is loaded.
53             if (index == 0 && source.locked) {
54                 layer.target.clear(1f, 1f, 1f, 1f)
55             }
56             layers += layer
57         }
58
59         activeLayerId = document.activeLayerId ?: layers.last().id
60         nextId = (layers.maxOf { it.id } + 1L).coerceAtLeast(1L)
61     }
62
63     /** Creates the locked opaque white background and one transparent drawing layer. */
64     fun create(caps: GlCaps, width: Int, height: Int) {
65         require(width > 0 && height > 0)
66         release()
67         canvasWidth = width
68         canvasHeight = height
69         documentUsesHalfFloat = caps.supportsHalfFloatColorBuffer
70
71         val background = newLayer("Фон", documentUsesHalfFloat).also {
72             it.isLocked = true
73             it.target.clear(1f, 1f, 1f, 1f)
74         }
75         layers += background
76
77         val drawing = newLayer("Слой 1", documentUsesHalfFloat)
78         layers += drawing
79         activeLayerId = drawing.id
80     }
81
82     fun addLayer(name: String, insertAt: Int = layers.size): PaintLayer {
83         check(canvasWidth > 0 && canvasHeight > 0) { "LayerStack has not been created" }
84         val layer = newLayer(name, documentUsesHalfFloat)
85         layers.add(insertAt.coerceIn(0, layers.size), layer)
86         activeLayerId = layer.id
87         return layer
88     }
89
90     /** A document must retain at least one layer; locked layers cannot be removed. */

```

```

91     fun removeLayer(id: Long): PaintLayer? {
92         val index = layers.indexOfFirst { it.id == id }
93         if (index < 0 || layers.size <= 1) return null
94         val layer = layers[index]
95         if (layer.isLocked) return null
96
97         layers.removeAt(index)
98         layer.release()
99         if (activeLayerId == id) {
100             activeLayerId = layers[index.coerceAtMost(layers.lastIndex)].id
101         }
102         return layer
103     }
104
105     fun setActive(id: Long): Boolean {
106         if (findLayerById(id) == null) return false
107         activeLayerId = id
108         return true
109     }
110
111     fun moveLayer(id: Long, delta: Int): Boolean {
112         val index = layers.indexOfFirst { it.id == id }
113         if (index < 0) return false
114         val newIndex = (index + delta).coerceIn(0, layers.lastIndex)
115         if (newIndex == index) return false
116         val layer = layers.removeAt(index)
117         layers.add(newIndex, layer)
118         return true
119     }
120
121     fun release() {
122         layers.forEach(PaintLayer::release)
123         layers.clear()
124         activeLayerId = NO_LAYER
125         canvasWidth = 0
126         canvasHeight = 0
127         documentUsesHalfFloat = false
128     }
129
130     private fun newLayer(name: String, useHalfFloat: Boolean): PaintLayer =
131         PaintLayer(nextId++, name).also { it.create(canvasWidth, canvasHeight, useHalfFloat) }
132
133     private companion object {
134         const val NO_LAYER = -1L
135     }
136 }

```

42. app/src/main/java/com/wetinknext/engine/canvas/LinearPresentRenderer.kt

```

1  package com.wetinknext.engine.canvas
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.CanvasGeometry
5  import com.wetinknext.engine.gl.GlCheck
6  import com.wetinknext.engine.gl.GlProgram
7  import com.wetinknext.engine.gl.ShaderLib
8
9  /** Final pass: presents a premultiplied-linear canvas texture to the sRGB screen. */
10 class LinearPresentRenderer {
11     private var program: GlProgram? = null
12     private var uCanvasToClip = -1
13     private var uCanvasSize = -1
14     private var uTexture = -1
15
16     fun create() {
17         GlCheck.checkOnG1Thread()
18         release()
19         program = GlProgram(ShaderLib.compositorVertex, ShaderLib.linearPresentFragment)
20         val currentProgram = checkNotNull(program)
21         currentProgram.use()
22         uCanvasToClip = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasToClip")
23         uCanvasSize = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasSize")
24         uTexture = GLES30.glGetUniformLocation(currentProgram.id, "uTexture")
25         check(uCanvasToClip >= 0 && uCanvasSize >= 0 && uTexture >= 0) {
26             "Linear present uniforms missing"
27         }
28         GlCheck.noError("LinearPresentRenderer create")
29     }
30
31     fun render(
32         sourceTextureId: Int,
33         geometry: CanvasGeometry,
34         canvasToClipMatrix: FloatArray,
35         canvasWidth: Int,
36         canvasHeight: Int,
37         viewportWidth: Int,
38         viewportHeight: Int,
39     ) {
40         if (sourceTextureId == 0) return
41         val currentProgram = program ?: return
42         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
43         GLES30.glViewport(0, 0, viewportWidth, viewportHeight)
44         // A hidden background layer leaves transparent document pixels. Blend
45         // them over the editor backdrop instead of replacing it with black.
46         GLES30.glEnable(GLES30.GL_BLEND)
47         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
48         currentProgram.use()
49         GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToClipMatrix, 0)
50         GLES30.glUniform2f(uCanvasSize, canvasWidth.toFloat(), canvasHeight.toFloat())
51         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
52         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, sourceTextureId)
53         GLES30.glUniform1i(uTexture, 0)
54         geometry.draw()
55         GLES30.glDisable(GLES30.GL_BLEND)
56         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
57     }
58
59     fun release() {
60         GlCheck.checkOnG1Thread()
61         program?.release()
62         program = null
63         uCanvasToClip = -1
64         uCanvasSize = -1
65         uTexture = -1
66     }
67 }

```


43. app/src/main/java/com/wetinknext/engine/canvas/LinearTextureBlitter.kt

```

1  package com.wetinknext.engine.canvas
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.GlCheck
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.ShaderLib
7  import java.nio.ByteBuffer
8  import java.nio.ByteOrder
9
10 /**
11  * Blends one premultiplied-linear screen texture into the currently bound
12  * framebuffer. Unlike [ScreenPresentRenderer], this deliberately performs no
13  * colour-space conversion: it is used only while building a linear composite.
14  */
15 class LinearTextureBlitter {
16     private var program: GlProgram? = null
17     private var vertexArrayId = 0
18     private var vertexBufferId = 0
19     private var textureUniform = -1
20
21     fun create() {
22         GlCheck.checkOnGLThread()
23         release()
24
25         program = GlProgram(ShaderLib.fullscreenVertex, ShaderLib.linearCopyFragment)
26         val currentProgram = checkNotNull(program)
27         currentProgram.use()
28         textureUniform = GLES30.glGetUniformLocation(currentProgram.id, "uTexture")
29         check(textureUniform >= 0) { "Linear copy uniform uTexture is missing" }
30
31         val vertices = floatArrayOf(-1f, -1f, 1f, -1f, -1f, 1f, 1f, 1f)
32         val data = ByteBuffer
33             .allocateDirect(vertices.size * Float.SIZE_BYTES)
34             .order(ByteOrder.nativeOrder())
35             .asFloatBuffer()
36             .apply { put(vertices); position(0) }
37         val ids = IntArray(1)
38         GLES30.glGenVertexArrays(1, ids, 0)
39         vertexArrayId = ids[0]
40         GLES30.glGenBuffers(1, ids, 0)
41         vertexBufferId = ids[0]
42         GLES30.glBindVertexArray(vertexArrayId)
43         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vertexBufferId)
44         GLES30.glBufferData(
45             GLES30.GL_ARRAY_BUFFER,
46             vertices.size * Float.SIZE_BYTES,
47             data,
48             GLES30.GL_STATIC_DRAW,
49         )
50         GLES30.glEnableVertexAttribArray(0)
51         GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
52         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0)
53         GLES30.glBindVertexArray(0)
54     }
55
56     /** The caller owns the framebuffer and viewport. */
57     fun blit(sourceTextureId: Int) {
58         val currentProgram = program ?: return
59         if (sourceTextureId == 0 || vertexArrayId == 0) return
60
61         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
62         GLES30.glEnable(GLES30.GL_BLEND)
63         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
64         currentProgram.use()
65         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
66         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, sourceTextureId)
67         GLES30.glUniform1i(textureUniform, 0)
68         GLES30.glBindVertexArray(vertexArrayId)
69         GLES30.glDrawArrays(GLES30.GL_TRIANGLE_STRIP, 0, 4)
70         GLES30.glBindVertexArray(0)
71         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
72         GLES30.glDisable(GLES30.GL_BLEND)
73     }
74
75     fun release() {
76         if (vertexBufferId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vertexBufferId), 0)
77         if (vertexArrayId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vertexArrayId), 0)
78         vertexBufferId = 0
79         vertexArrayId = 0
80         program?.release()
81         program = null
82         textureUniform = -1
83     }
84 }

```

44. app/src/main/java/com/wetinknext/engine/canvas/NonBuildupStrokeRenderer.kt

```

1  package com.wetinknext.engine.canvas
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.CanvasGeometry
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.RenderTarget
7  import com.wetinknext.engine.gl.ShaderLib
8
9  /** Applies a fixed linear stroke colour once, using a union coverage mask. */
10 class NonBuildupStrokeRenderer {
11     private var program: GlProgram? = null
12     private var uCanvasToClip = -1
13     private var uCanvasSize = -1
14     private var uCoverageTex = -1
15     private var uColorLinear = -1
16     private var uOpacity = -1
17
18     fun create() {
19         release()
20         program = GlProgram(ShaderLib.compositorVertex, ShaderLib.nonBuildupStrokeFragment)
21         val p = checkNotNull(program)
22         p.use()
23         uCanvasToClip = GLES30.glGetUniformLocation(p.id, "uCanvasToClip")
24         uCanvasSize = GLES30.glGetUniformLocation(p.id, "uCanvasSize")
25         uCoverageTex = GLES30.glGetUniformLocation(p.id, "uCoverageTex")
26         uColorLinear = GLES30.glGetUniformLocation(p.id, "uColorLinear")
27         uOpacity = GLES30.glGetUniformLocation(p.id, "uOpacity")
28         check((uCanvasToClip >= 0 && uCanvasSize >= 0 && uCoverageTex >= 0 && uColorLinear >= 0 && uOpacity >= 0) {
29             "NonBuildupStrokeRenderer uniforms missing"
30         })
31     }
32
33     fun blit(
34         layer: RenderTarget,
35         geometry: CanvasGeometry,
36         coverageTextureId: Int,
37         colorLinear: FloatArray,
38         canvasToFbo: FloatArray,
39         width: Int,
40         height: Int,
41         opacity: Float,
42         erase: Boolean = false,
43     ) {
44         val p = program ?: return
45         if (coverageTextureId == 0 || colorLinear.size < 3) return
46         layer.bind()
47         GLES30.glViewport(0, 0, width, height)
48         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
49         GLES30.glEnable(GLES30.GL_BLEND)
50         GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
51         if (erase) {
52             GLES30.glBlendFunc(GLES30.GL_ZERO, GLES30.GL_ONE_MINUS_SRC_ALPHA)
53         } else {
54             GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
55         }
56         p.use()
57         GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToFbo, 0)
58         GLES30.glUniform2f(uCanvasSize, width.toFloat(), height.toFloat())
59         GLES30.glActiveTexture(GLES30.GL_TEXTURE1)
60         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, coverageTextureId)
61         GLES30.glUniform1i(uCoverageTex, 1)
62         GLES30.glUniform3f(uColorLinear, colorLinear[0], colorLinear[1], colorLinear[2])
63         GLES30.glUniform1f(uOpacity, opacity.coerceIn(0f, 1f))
64         geometry.draw()
65         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
66         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
67         GLES30.glDisable(GLES30.GL_BLEND)
68         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
69     }
70
71     fun release() {
72         program?.release()
73         program = null
74         uCanvasToClip = -1
75         uCanvasSize = -1
76         uCoverageTex = -1
77         uColorLinear = -1
78         uOpacity = -1
79     }
80 }

```

45. app/src/main/java/com/wetinknext/engine/canvas/PaintLayer.kt

```
1 package com.wetinknext.engine.canvas
2
3 import com.wetinknext.engine.gl.RenderTarget
4
5 /** One paintable document layer and its private GPU target. */
6 class PaintLayer(
7     val id: Long,
8     var name: String,
9 ) {
10     val target = RenderTarget()
11
12     var isVisible = true
13     var isLocked = false
14     var opacity = 1f
15     var blendMode = BlendMode.NORMAL
16     var version = 0L
17
18     var created = false
19     private set
20
21     fun create(width: Int, height: Int, preferHalfFloat: Boolean) {
22         if (created && target.width == width && target.height == height) return
23         target.create(width, height, preferHalfFloat)
24         target.clear(0f, 0f, 0f, 0f)
25         created = true
26     }
27
28     fun clear() {
29         if (created) target.clear(0f, 0f, 0f, 0f)
30     }
31
32     fun release() {
33         target.release()
34         created = false
35     }
36 }
```

46. app/src/main/java/com/wetinknext/engine/canvas/ScreenPresentRenderer.kt

```

1 package com.wetinknext.engine.canvas
2
3 import android.opengl.GLES30
4 import com.wetinknext.engine.gl.GlCheck
5 import com.wetinknext.engine.gl.GlProgram
6 import com.wetinknext.engine.gl.ShaderLib
7
8 /** Presents a linear RGBA frame target to the default sRGB framebuffer. */
9 class ScreenPresentRenderer {
10     private var program: GlProgram? = null
11     private var vertexArrayId = 0
12     private var vertexBufferId = 0
13     private var textureUniform = -1
14
15     fun create() {
16         GlCheck.checkOnGThread()
17         release()
18         program = GlProgram(ShaderLib.fullscreenVertex, ShaderLib.linearPresentFragment)
19         val currentProgram = checkNotNull(program)
20         currentProgram.use()
21         textureUniform = GLES30.glGetUniformLocation(currentProgram.id, "uTexture")
22         check(textureUniform >= 0) { "Screen present uniform uTexture is missing" }
23
24         val vertices = floatArrayOf(-1f, -1f, 1f, -1f, 1f, 1f, 1f, 1f)
25         val data = java.nio.ByteBuffer
26             .allocateDirect(vertices.size * Float.SIZE_BYTES)
27             .order(java.nio.ByteOrder.nativeOrder())
28             .asFloatBuffer()
29             .apply { put(vertices); position(0) }
30         val ids = IntArray(1)
31         GLES30.glGenVertexArrays(1, ids, 0)
32         vertexArrayId = ids[0]
33         GLES30.glGenBuffers(1, ids, 0)
34         vertexBufferId = ids[0]
35         GLES30.glBindVertexArray(vertexArrayId)
36         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vertexBufferId)
37         GLES30.glBufferData(
38             GLES30.GL_ARRAY_BUFFER,
39             vertices.size * Float.SIZE_BYTES,
40             data,
41             GLES30.GL_STATIC_DRAW,
42         )
43         GLES30.glEnableVertexAttribArray(0)
44         GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
45         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0)
46         GLES30.glBindVertexArray(0)
47     }
48
49     fun render(sourceTextureId: Int, viewportWidth: Int, viewportHeight: Int) {
50         val currentProgram = program ?: return
51         if (sourceTextureId == 0 || vertexArrayId == 0) return
52         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
53         GLES30.glViewport(0, 0, viewportWidth, viewportHeight)
54         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
55         GLES30.glEnable(GLES30.GL_BLEND)
56         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
57         currentProgram.use()
58         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
59         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, sourceTextureId)
60         GLES30.glUniform1i(textureUniform, 0)
61         GLES30.glBindVertexArray(vertexArrayId)
62         GLES30.glDrawArrays(GLES30.GL_TRIANGLE_STRIP, 0, 4)
63         GLES30.glBindVertexArray(0)
64         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
65         GLES30.glDisable(GLES30.GL_BLEND)
66     }
67
68     fun release() {
69         if (vertexBufferId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vertexBufferId), 0)
70         if (vertexArrayId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vertexArrayId), 0)
71         vertexBufferId = 0
72         vertexArrayId = 0
73         program?.release()
74         program = null
75         textureUniform = -1
76     }
77 }

```

47. app/src/main/java/com/wetinknext/engine/canvas/StrokeBlitter.kt

```

1  package com.wetinknext.engine.canvas
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.CanvasGeometry
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.RenderTarget
7  import com.wetinknext.engine.gl.ShaderLib
8
9  /** Переносит готовый штрих из strokeTarget в слой: premultiplied source-over * opacity. */
10 class StrokeBlitter {
11     private var program: GlProgram? = null
12     private var uCanvasToClip = -1
13     private var uCanvasSize = -1
14     private var uStrokeTex = -1
15     private var uOpacity = -1
16
17     fun create() {
18         release()
19         program = GlProgram(ShaderLib.compositorVertex, ShaderLib.strokeBlitFragment)
20         val p = checkNotNull(program)
21         p.use()
22         uCanvasToClip = GLES30.glGetUniformLocation(p.id, "uCanvasToClip")
23         uCanvasSize = GLES30.glGetUniformLocation(p.id, "uCanvasSize")
24         uStrokeTex = GLES30.glGetUniformLocation(p.id, "uStrokeTex")
25         uOpacity = GLES30.glGetUniformLocation(p.id, "uOpacity")
26         check(uCanvasToClip >= 0 && uCanvasSize >= 0 && uStrokeTex >= 0 && uOpacity >= 0) {
27             "StrokeBlitter uniforms missing"
28         }
29     }
30
31     fun blit(
32         layer: RenderTarget,
33         geometry: CanvasGeometry,
34         strokeTextureId: Int,
35         canvasToFbo: FloatArray,
36         width: Int,
37         height: Int,
38         opacity: Float,
39         erase: Boolean = false,
40     ) {
41         val p = program ?: return
42         if (strokeTextureId == 0) return
43         layer.bind()
44         GLES30.glViewport(0, 0, width, height)
45         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
46         GLES30.glEnable(GLES30.GL_BLEND)
47         GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
48         if (erase) {
49             // Erase premultiplied RGB and alpha by the rendered stroke coverage.
50             GLES30.glBlendFunc(GLES30.GL_ZERO, GLES30.GL_ONE_MINUS_SRC_ALPHA)
51         } else {
52             GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
53         }
54         p.use()
55         GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToFbo, 0)
56         GLES30.glUniform2f(uCanvasSize, width.toFloat(), height.toFloat())
57         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
58         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, strokeTextureId)
59         GLES30.glUniform1i(uStrokeTex, 0)
60         GLES30.glUniform1f(uOpacity, opacity.coerceIn(0f, 1f))
61         geometry.draw()
62         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
63         GLES30.glDisable(GLES30.GL_BLEND)
64         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
65         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
66     }
67
68     fun release() {
69         program?.release()
70         program = null
71         uCanvasToClip = -1; uCanvasSize = -1; uStrokeTex = -1; uOpacity = -1
72     }
73 }

```

48. app/src/main/java/com/wetinknext/engine/core/Camera.kt

```
1 package com.wetinknext.engine.core
2
3 import java.util.concurrent.atomic.AtomicReference
4 import kotlin.math.min
5
6 /** Thread-safe camera state shared by the UI and GL threads. */
7 class Camera(initial: ViewTransform = ViewTransform()) {
8     private val state = AtomicReference(initial)
9
10    fun snapshot(): ViewTransform = state.get()
11
12    fun set(transform: ViewTransform) {
13        state.set(transform)
14    }
15
16    fun update(transform: (ViewTransform) -> ViewTransform) {
17        while (true) {
18            val current = state.get()
19            if (state.compareAndSet(current, transform(current))) return
20        }
21    }
22
23    fun reset() {
24        state.set(ViewTransform())
25    }
26
27    fun fitCanvas(
28        canvasWidth: Int,
29        canvasHeight: Int,
30        viewWidth: Int,
31        viewHeight: Int,
32        marginRatio: Float = 0.04f,
33    ) {
34        if (canvasWidth <= 0 || canvasHeight <= 0 || viewWidth <= 0 || viewHeight <= 0) return
35        val usableWidth = viewWidth * (1f - marginRatio * 2f)
36        val usableHeight = viewHeight * (1f - marginRatio * 2f)
37        val scale = min(usableWidth / canvasWidth, usableHeight / canvasHeight)
38        .coerceIn(ViewTransform.MIN_SCALE, ViewTransform.MAX_SCALE)
39        state.set(
40            ViewTransform(
41                scale = scale,
42                translateX = (viewWidth - canvasWidth * scale) * 0.5f,
43                translateY = (viewHeight - canvasHeight * scale) * 0.5f,
44            ),
45        )
46    }
47 }
```

49. app/src/main/java/com/wetinknext/engine/core/CanvasBackdropMode.kt

```
1 package com.wetinknext.engine.core
2
3 /** Visual treatment of the workspace surrounding the document canvas. */
4 enum class CanvasBackdropMode {
5     GRID,
6     CHECKERBOARD,
7     SOLID,
8 }
```

50. app/src/main/java/com/wetinknext/engine/core/DirtyRect.kt

```

1  package com.wetinknext.engine.core
2
3  import kotlin.math.ceil
4  import kotlin.math.floor
5  import kotlin.math.max
6  import kotlin.math.min
7
8  /** Reusable mutable dirty rectangle in canvas coordinates. */
9  class DirtyRect {
10     var left = 0f; private set
11     var top = 0f; private set
12     var right = 0f; private set
13     var bottom = 0f; private set
14     var isEmpty = true; private set
15
16     fun clear() {
17         left = 0f; top = 0f; right = 0f; bottom = 0f; isEmpty = true
18     }
19
20     fun include(x: Float, y: Float, radius: Float) = include(x - radius, y - radius, x + radius, y + radius)
21
22     fun include(left: Float, top: Float, right: Float, bottom: Float) {
23         if (right < left || bottom < top) return
24         if (isEmpty) {
25             this.left = left; this.top = top; this.right = right; this.bottom = bottom; isEmpty = false
26             return
27         }
28         this.left = min(this.left, left); this.top = min(this.top, top)
29         this.right = max(this.right, right); this.bottom = max(this.bottom, bottom)
30     }
31
32     fun include(other: DirtyRect) {
33         if (!other.isEmpty) include(other.left, other.top, other.right, other.bottom)
34     }
35
36     fun expand(pixels: Float) {
37         if (!isEmpty && pixels > 0f) {
38             left -= pixels; top -= pixels; right += pixels; bottom += pixels
39         }
40     }
41
42     fun clamp(canvasWidth: Float, canvasHeight: Float) {
43         if (isEmpty) return
44         left = left.coerceIn(0f, canvasWidth); top = top.coerceIn(0f, canvasHeight)
45         right = right.coerceIn(0f, canvasWidth); bottom = bottom.coerceIn(0f, canvasHeight)
46         if (right <= left || bottom <= top) clear()
47     }
48
49     fun copyTo(out: DirtyRect) {
50         out.left = left; out.top = top; out.right = right; out.bottom = bottom; out.isEmpty = isEmpty
51     }
52
53     fun toPixelBounds(out: IntArray) {
54         require(out.size >= 4)
55         if (isEmpty) {
56             out[0] = 0; out[1] = 0; out[2] = 0; out[3] = 0
57             return
58         }
59         out[0] = floor(left).toInt(); out[1] = floor(top).toInt()
60         out[2] = ceil(right).toInt(); out[3] = ceil(bottom).toInt()
61     }
62 }

```


51. app/src/main/java/com/wetinknext/engine/core/DocumentSession.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.domain.document.ProjectDocument
4 import com.wetinknext.engine.canvas.LayerStack
5 import com.wetinknext.engine.gl.GlCaps
6 import com.wetinknext.engine.undo.UndoManager
7
8 /** Render-thread lifetime of an opened document and its GPU layer resources. */
9 class DocumentSession(
10     val document: ProjectDocument,
11     val layerStack: LayerStack,
12     val undoManager: UndoManager,
13     private val layerTiles: Map<Long, ByteArray>,
14 ) {
15     /** Layers whose pixels must be persisted by the next autosave. GL-thread owned. */
16     val dirtyLayerIds = mutableSetOf<Long>()
17
18     /** Metadata or pixels changed since the last successful autosave. GL-thread owned. */
19     var projectDirty: Boolean = false
20     private set
21
22     fun markLayerDirty(layerId: Long) {
23         dirtyLayerIds += layerId
24         projectDirty = true
25     }
26
27     fun markProjectDirty() {
28         projectDirty = true
29     }
30
31     fun markSaved(savedLayerIds: Set<Long>) {
32         dirtyLayerIds.removeAll(savedLayerIds)
33         projectDirty = dirtyLayerIds.isNotEmpty()
34     }
35
36     fun loadIntoGpu(caps: GlCaps) {
37         layerStack.create(caps, document)
38         document.layers.forEach { layer ->
39             LayerTileCodec.uploadPersistent(
40                 target = checkNotNull(layerStack.findLayerById(layer.id)).target,
41                 payload = layerTiles[layer.id] ?: ByteArray(0),
42             )
43         }
44         check(layerStack.activeLayerId == document.activeLayerId) { "Active layer was not restored" }
45     }
46 }
```

52. app/src/main/java/com/wetinknext/engine/core/EditorState.kt

```
1 package com.wetinknext.engine.core
2
3 /** Immutable state published by the GL thread for Compose to render. */
4 data class LayerUiModel(
5     val id: Long,
6     val name: String,
7     val isVisible: Boolean,
8     val isLocked: Boolean,
9     val opacity: Float,
10    val active: Boolean,
11    val canDelete: Boolean,
12    val thumbnailPath: String? = null,
13    val thumbnailVersion: Long = 0L,
14 )
15
16 data class UndoDiagnostics(
17     val pendingJobs: Int = 0,
18     val staleResults: Int = 0,
19     val compressionFailures: Int = 0,
20     val restoreFailures: Int = 0,
21     val memoryBytes: Long = 0L,
22 )
23
24 data class EditorUiState(
25     val layers: List<LayerUiModel>,
26     val canUndo: Boolean,
27     val canRedo: Boolean,
28     val brushSizePx: Float,
29     val brushOpacity: Float,
30     val activeLayerId: Long,
31     val ready: Boolean,
32     val undoDiagnostics: UndoDiagnostics = UndoDiagnostics(),
33 ) {
34     companion object {
35         val empty = EditorUiState(
36             layers = emptyList(),
37             canUndo = false,
38             canRedo = false,
39             brushSizePx = 16f,
40             brushOpacity = 1f,
41             activeLayerId = -1L,
42             ready = false,
43         )
44     }
45 }
```

53. app/src/main/java/com/wetinknext/engine/core/EngineRenderer.kt

```

1 package com.wetinknext.engine.core
2
3 import android.opengl.GLES30
4 import android.opengl.GLSurfaceView
5 import android.util.Log
6 import android.view.MotionEvent
7 import androidx.compose.ui.graphics.toArgb
8 import com.wetinknext.BuildConfig
9 import com.wetinknext.domain.document.LayerDocument
10 import com.wetinknext.domain.document.ProjectDocument
11 import com.wetinknext.engine.brush.BrushSettings
12 import com.wetinknext.engine.brush.BrushTexture
13 import com.wetinknext.engine.brush.LoadedBrushTexture
14 import com.wetinknext.engine.brush.BrushRenderMode
15 import com.wetinknext.engine.brush.BlendPolicy
16 import com.wetinknext.engine.brush.StrokeRenderMode
17 import com.wetinknext.engine.brush.CapsuleEmitter
18 import com.wetinknext.engine.brush.CapsuleStrokeRenderer
19 import com.wetinknext.engine.brush.RibbonMeshRenderer
20 import com.wetinknext.engine.brush.ColorSpaces
21 import com.wetinknext.engine.brush.DabBuffer
22 import com.wetinknext.engine.brush.DabRenderer
23 import com.wetinknext.engine.brush.StampEmitter
24 import com.wetinknext.engine.canvas.CanvasBackdropRenderer
25 import com.wetinknext.engine.canvas.Compositor
26 import com.wetinknext.engine.canvas.LayerStack
27 import com.wetinknext.engine.canvas.LinearTextureBlitter
28 import com.wetinknext.engine.canvas.PaintLayer
29 import com.wetinknext.engine.canvas.StrokeBlitter
30 import com.wetinknext.engine.canvas.NonBuildupStrokeRenderer
31 import com.wetinknext.engine.canvas.ScreenPresentRenderer
32 import com.wetinknext.engine.gl.CanvasGeometry
33 import com.wetinknext.engine.gl.GlCaps
34 import com.wetinknext.engine.gl.GlCheck
35 import com.wetinknext.engine.gl.RenderTarget
36 import com.wetinknext.engine.input.InputAction
37 import com.wetinknext.engine.input.InputBatch
38 import com.wetinknext.engine.input.InputBatchPool
39 import com.wetinknext.engine.input.GestureRouter
40 import com.wetinknext.engine.input.StrokeInputCapturer
41 import com.wetinknext.engine.persistence.LayerTileStore
42 import com.wetinknext.engine.thumbnail.ThumbnailBuildResult
43 import com.wetinknext.engine.thumbnail.ThumbnailBuildScheduler
44 import com.wetinknext.engine.thumbnail.ThumbnailRenderer
45 import com.wetinknext.engine.undo.TileSnapshotRestore
46 import com.wetinknext.engine.undo.UndoManager
47 import com.wetinknext.engine.undo.UndoOperationType
48 import java.io.File
49 import java.util.concurrent.ArrayBlockingQueue
50 import java.util.concurrent.ExecutorService
51 import java.util.concurrent.Executors
52 import java.util.concurrent.RejectedExecutionException
53 import java.util.concurrent.atomic.AtomicBoolean
54 import javax.microedition.khronos.egl.EGLConfig
55 import javax.microedition.khronos.opengles.GL10
56
57 /**
58  * The P6 render-thread owner. Active dabs are accumulated only in strokeTarget;
59  * UP snapshots and merges them into the selected PaintLayer atomically.
60  */
61 class EngineRenderer(
62     private var projectDocument: ProjectDocument = ProjectDocument.newUntitled(),
63     private val initialLayerTiles: Map<Long, ByteArray> = emptyMap(),
64     private val thumbnailOutputDirectory: File? = null,
65 ) : GLSurfaceView.Renderer {
66     private var caps: GlCaps? = null
67     private val layerStack = LayerStack()
68     private val undoManager = UndoManager()
69     private val layerTileStore = LayerTileStore(initialLayerTiles)
70     private var documentSession: DocumentSession? = null
71     private val undoPipeline = UndoCompressionPipeline(
72         requestRender = { onInputRenderRequested?.invoke() },
73     )
74     /** Serializes persistent tile payloads off the GL thread after a commit. */
75     private var tilePayloadExecutor: ExecutorService = Executors.newSingleThreadExecutor()
76     private var undoRestoreFailures = 0
77     private var glThread: Thread? = null
78     /**
79      * Preview files may finish decoding before GLSurfaceView calls
80      * [onSurfaceCreated]. Keep them as plain JVM data until the document's
81      * LayerStack has been created on the GL thread.
82      */
83     private var deferredRestoredLayerThumbnails: Map<Long, File> = emptyMap()
84
85     private var geometry: CanvasGeometry? = null
86     private var compositor: Compositor? = null
87     private var backdropRenderer: CanvasBackdropRenderer? = null
88     private var screenPresentRenderer: ScreenPresentRenderer? = null
89     private var linearTextureBlitter: LinearTextureBlitter? = null
90     private val thumbnailCapture = ThumbnailCapture()

```

```

91 private val layerThumbnailPaths = mutableMapOf<Long, String>()
92 /** Last layer version for which a WebP preview was successfully published. */
93 private val layerThumbnailVersions = mutableMapOf<Long, Long>()
94 private val thumbnailScheduler = thumbnailOutputDirectory?.let { outputDirectory ->
95     ThumbnailBuildScheduler(
96         outputDirectory = outputDirectory,
97         renderer = ThumbnailRenderer(),
98         requestGlfFrame = { onInputRenderRequested?.invoke() },
99         onSave = { result -> onThumbnailBuildSaved?.invoke(result) },
100         onFailure = { error ->
101             if (BuildConfig.DEBUG) Log.e(THUMBNAIL_TAG, "Thumbnail build failed", error)
102         }
103     )
104 }
105 private var dabRenderer: DabRenderer? = null
106 private var capsuleRenderer: CapsuleStrokeRenderer? = null
107 private var ribbonMeshRenderer: RibbonMeshRenderer? = null
108 private var grainTexture: BrushTexture? = null
109 private var grainPath: String? = null
110 private var shapeTexture: BrushTexture? = null
111 private var shapePath: String? = null
112 private val strokeTarget = RenderTarget()
113 /** RGBA8 screen-space preview; never used for final document pixels. */
114 private val strokePreviewTarget = RenderTarget()
115 /** Screen-sized union mask for realtime STAMP NON_BUILDUP preview. */
116 private val strokeCoveragePreviewTarget = RenderTarget()
117 /** RGBA8 coverage accumulator used by the NON_BUILDUP stroke path. */
118 private val strokeCoverageTarget = RenderTarget()
119 /** Screen-sized linear frame composition. Source layers remain document-sized. */
120 private val compositeTarget = RenderTarget()
121 /** Confirmed layers below the active one, cached only for a live stroke preview. */
122 private val lowerCompositeTarget = RenderTarget()
123 /** Confirmed layers above the active one, cached only for a live stroke preview. */
124 private val upperCompositeTarget = RenderTarget()
125 private var compositeCacheDirty = true
126 private var cachedPreviewActiveLayerIndex = -1
127 private val cachedPreviewCanvasToClipMatrix = FloatArray(16)
128 private val canvasToFboMatrix = FloatArray(16)
129 private val canvasToPreviewMatrix = FloatArray(16)
130 private val strokeCommitter = StrokeCommitter(
131     strokeTarget = strokeTarget,
132     canvasToFbo = canvasToFboMatrix,
133     undoPipeline = undoPipeline,
134     undoManager = undoManager,
135     onTilesCommitted = { layer, tiles ->
136         invalidateCompositeCache()
137         layerTileStore.markDirty(layer.id, tiles)
138         documentSession?.markLayerDirty(layer.id)
139         publishDirtyTiles()
140         markLayerThumbnailDirty(layer.id)
141     },
142     onLayerCleared = { layer ->
143         invalidateCompositeCache()
144         layerTileStore.markLayerCleared(layer.id)
145         documentSession?.markLayerDirty(layer.id)
146         publishDirtyTiles()
147         markLayerThumbnailDirty(layer.id)
148     },
149 )
150 private val canvasToClipMatrix = FloatArray(16)
151
152 private var screenWidth = 1
153 private var screenHeight = 1
154 private val canvasBackdropColor = floatArrayOf(0.08f, 0.09f, 0.12f)
155 private val canvasGridColor = floatArrayOf(0.16f, 0.18f, 0.22f)
156
157 private val inputPool = InputBatchPool(batchCount = 64, maxSamplesPerBatch = 256)
158 private val inputQueue = ArrayBlockingQueue<InputBatch>(64)
159 private val inputCapturer = StrokeInputCapturer(layerStack.camera, inputPool, inputQueue)
160 private var maxInputQueueDepth = 0
161 private var droppedBatchCountAtLastLog = 0L
162 private var frameStartNanos = 0L
163 private var lastFrameLogNanos = 0L
164 private var frameCounter = 0
165 private var inputBatchesSinceLastFrameLog = 0
166 private val undoGestureRequested = AtomicBoolean(false)
167 private val redoGestureRequested = AtomicBoolean(false)
168 private val resetCameraGestureRequested = AtomicBoolean(false)
169 private val gestureRouter = GestureRouter(
170     camera = layerStack.camera,
171     drawingInput = inputCapturer,
172     callbacks = object : GestureRouter.Callbacks {
173         override fun cancelDrawing() {
174             requestCancelFromInput()
175         }
176
177         override fun undoGesture() {
178             undoGestureRequested.set(true)
179         }
180
181         override fun redoGesture() {
182             redoGestureRequested.set(true)
183         }
184     }
185 )

```

```

183         }
184
185         override fun resetCamera() {
186             resetCameraGestureRequested.set(true)
187         }
188
189         override fun requestRender() {
190             onInputRenderRequested?.invoke()
191         }
192     },
193 )
194
195 private var brushSettings = BrushSettings(
196     name = "G-Pen",
197     renderMode = BrushRenderMode.RIBBON,
198     baseRadiusPx = 9f,
199     spacing = 0.12f,
200     colorArgb = 0xFF1B1F24L,
201     smoothing = 0.35f,
202     streamline = 0.22f,
203     pressureToOpacity = false,
204     pressureGamma = 1.7f,
205     minSizeRatio = 0.06f,
206     ribbon = com.wetinknext.engine.brush.RibbonSettings(
207         cap = com.wetinknext.engine.brush.RibbonCap.ROUND,
208         join = com.wetinknext.engine.brush.RibbonJoin.ROUND,
209         miterLimit = 2.5f,
210         minPointDistancePx = 1.25f,
211         aaWidthPx = 1f,
212         taperStartPx = 16f,
213         taperEndPx = 64f,
214     ),
215 )
216 private val stampEmitter = StampEmitter(brushSettings)
217 private val capsuleEmitter = CapsuleEmitter(brushSettings)
218 private val dabBuffer = DabBuffer()
219 private val strokeColorLinear = FloatArray(3)
220 private val activeStrokeColorLinear = FloatArray(3)
221 private val strokeDirtyRect = DirtyRect()
222 private val dirtyBounds = IntArray(4)
223
224 private var strokeActive = false
225 /** Tool state, deliberately independent from the selected brush preset. */
226 private var eraserEnabled = false
227 /** Tool snapshot: toggling tools never changes an already-active stroke. */
228 private var activeStrokeErase = false
229 private var activeStrokeBrush: BrushSettings? = null
230 private var strokeBlitter: StrokeBlitter? = null
231 private var nonBuildupStrokeRenderer: NonBuildupStrokeRenderer? = null
232 private var capsulePreviewInitialized = false
233 private var renderedRibbonMeshVersion = -1L
234 private var stampPreviewInitialized = false
235 private val cancelRequested = AtomicBoolean(false)
236
237 /** Assigned by PaintSurfaceView; invoked on the GL thread. */
238 var onStateChange: ((EditorUiState) -> Unit)? = null
239
240 /** Metadata-only snapshot for persistence; invoked on the GL thread. */
241 var onProjectDocumentChange: ((ProjectDocument) -> Unit)? = null
242
243 /** Called only after document metadata and all stored layer payloads reached the GPU. */
244 var onDocumentSessionLoaded: (() -> Unit)? = null
245 var onDirtyLayerTiles: ((ProjectDocument, Map<Long, ByteArray>, Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>) ->
Unit)? = null
246 var onThumbnailCaptured: ((ThumbnailCapture.Rgba) -> Unit)? = null
247 /** Posted on the UI thread after the scheduler published the current previews. */
248 var onThumbnailBuildSaved: ((ThumbnailBuildResult) -> Unit)? = null
249
250 /** Assigned by PaintSurfaceView; invoked from the UI touch thread. */
251 var onInputRenderRequested: (() -> Unit)? = null
252
253 fun setOnSecondaryPointerDown(listener: () -> Unit) {
254     inputCapturer.onSecondaryPointerDown = listener
255 }
256
257 fun onTouchEvent(event: MotionEvent): Boolean = gestureRouter.onTouchEvent(event)
258
259 fun requestCancelFromInput() {
260     cancelRequested.set(true)
261 }
262
263 /** Fits the document into the current GL surface. Must run on the GL thread. */
264 fun resetCamera() {
265     layerStack.camera.fitCanvas(
266         canvasWidth = layerStack.canvasWidth,
267         canvasHeight = layerStack.canvasHeight,
268         viewWidth = screenWidth,
269         viewHeight = screenHeight,
270     )
271 }
272
273 /** Applies theme colors to the area that surrounds the document. */

```

```

274 private var canvasBackdropMode = CanvasBackdropMode.GRID
275
276 fun setCanvasBackdrop(backdropArgb: Int, gridArgb: Int, mode: CanvasBackdropMode) {
277     argbToRgb(backdropArgb, canvasBackdropColor)
278     argbToRgb(gridArgb, canvasGridColor)
279     canvasBackdropMode = mode
280 }
281
282 /** These methods are called from GLSurfaceView.queueEvent by the UI layer. */
283 fun undo() {
284     if (undoPipeline.pendingCount > 0) return
285     resetStroke()
286     while (true) {
287         val entry = undoManager.peekUndo() ?: return
288         val layer = layerStack.findLayerById(entry.layerId)
289         if (layer == null) {
290             undoManager.dropUndo(entry)
291             continue
292         }
293         val restored = TileSnapshotRestore.restore(layer.target, entry.beforeTiles)
294         if (!restored) {
295             undoRestoreFailures++
296             publishState()
297             return
298         }
299         if (!undoManager.commitUndo(entry)) {
300             publishState()
301             return
302         }
303         layer.version++
304         invalidateCompositeCache()
305         persistRestoredTiles(layer, entry.beforeTiles)
306         updateProjectDocument { it }
307         publishState()
308         return
309     }
310 }
311
312 fun redo() {
313     if (undoPipeline.pendingCount > 0) return
314     resetStroke()
315     while (true) {
316         val entry = undoManager.peekRedo() ?: return
317         val layer = layerStack.findLayerById(entry.layerId)
318         if (layer == null) {
319             undoManager.dropRedo(entry)
320             continue
321         }
322         val restored = if (entry.operation == UndoOperationType.CLEAR_LAYER) {
323             layer.clear()
324             true
325         } else {
326             TileSnapshotRestore.restore(layer.target, entry.afterTiles)
327         }
328         if (!restored) {
329             undoRestoreFailures++
330             publishState()
331             return
332         }
333         if (!undoManager.commitRedo(entry)) {
334             publishState()
335             return
336         }
337         layer.version++
338         invalidateCompositeCache()
339         if (entry.operation == UndoOperationType.CLEAR_LAYER) {
340             layerTileStore.markLayerCleared(layer.id)
341             documentSession?.markLayerDirty(layer.id)
342             publishDirtyTiles()
343             markLayerThumbnailDirty(layer.id)
344         } else {
345             persistRestoredTiles(layer, entry.afterTiles)
346         }
347         updateProjectDocument { it }
348         publishState()
349         return
350     }
351 }
352
353 fun setActiveLayer(id: Long): Boolean {
354     resetStroke()
355     return layerStack.setActive(id).also { changed ->
356         if (changed) {
357             invalidateCompositeCache()
358             updateProjectDocument { it.copy(activeLayerId = id) }
359             publishState()
360         }
361     }
362 }
363
364 fun addLayer(): Long = addLayer(nextLayerName())
365

```

```

366 fun addLayer(name: String): Long {
367     resetStroke()
368     val layer = layerStack.addLayer(name)
369     invalidateCompositeCache()
370     updateProjectDocument { document ->
371         document.copy(
372             layers = document.layers + layerDocumentFrom(layer),
373             activeLayerId = layer.id,
374         )
375     }
376     captureThumbnail(listOf(layer.id))
377     publishState()
378     return layer.id
379 }
380
381 /** Creates an editable pixel-identical copy directly above [id]. */
382 fun duplicateLayer(id: Long): Long? {
383     checkOnGLThread()
384     resetStroke()
385     val source = layerStack.findLayerById(id) ?: return null
386     val sourceIndex = layerStack.indexOfLayer(id)
387     if (sourceIndex < 0 || !source.created) return null
388
389     val duplicate = layerStack.addLayer(
390         name = nextDuplicateLayerName(source.name),
391         insertAt = sourceIndex + 1,
392     ).also {
393         it.isVisible = source.isVisible
394         it.isLocked = false
395         it.opacity = source.opacity
396         it.blendMode = source.blendMode
397         it.version = source.version
398     }
399
400     // Same-sized RenderTargets have the same format, so a framebuffer
401     // blit preserves premultiplied pixels exactly without a CPU readback.
402     GLES30.glBindFramebuffer(GLES30.GL_READ_FRAMEBUFFER, source.target.framebufferId)
403     GLES30.glBindFramebuffer(GLES30.GL_DRAW_FRAMEBUFFER, duplicate.target.framebufferId)
404     GLES30.glBlitFramebuffer(
405         0, 0, layerStack.canvasWidth, layerStack.canvasHeight,
406         0, 0, layerStack.canvasWidth, layerStack.canvasHeight,
407         GLES30.GL_COLOR_BUFFER_BIT,
408         GLES30.GL_NEAREST,
409     )
410     GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
411     GlCheck.noError("Duplicate layer")
412     invalidateCompositeCache()
413
414     val raw = com.wetinknext.engine.undo.TileSnapshotCapture.capture(
415         target = duplicate.target,
416         bounds = intArrayOf(0, 0, layerStack.canvasWidth, layerStack.canvasHeight),
417     )
418     layerTileStore.markDirty(duplicate.id, raw)
419     documentSession?.markLayerDirty(duplicate.id)
420     updateProjectDocument { it.copy(activeLayerId = duplicate.id, layers = layerStack.allLayers().map(::
layerDocumentFrom)) }
421     publishDirtyTiles()
422     captureThumbnail(listOf(duplicate.id))
423     publishState()
424     return duplicate.id
425 }
426
427 fun removeLayer(id: Long): Boolean {
428     checkOnGLThread()
429     resetStroke()
430     invalidatePendingUndoHistory()
431     val removed = layerStack.removeLayer(id) != null
432     if (removed) {
433         invalidateCompositeCache()
434         undoManager.removeEntriesForLayer(id)
435         layerThumbnailPaths.remove(id)
436         layerThumbnailVersions.remove(id)
437         thumbnailScheduler?.removeLayer(id)
438         updateProjectDocument { document ->
439             document.copy(
440                 layers = document.layers.filterNot { it.id == id },
441                 activeLayerId = layerStack.activeLayerId,
442             )
443         }
444         captureThumbnail()
445         publishState()
446     }
447     return removed
448 }
449
450 /** Moves one layer in the document order and rebuilds only the project preview. */
451 fun moveLayer(id: Long, delta: Int): Boolean {
452     checkOnGLThread()
453     resetStroke()
454     val moved = layerStack.moveLayer(id, delta)
455     if (moved) {
456         invalidateCompositeCache()

```

```

457         updateProjectDocument { it }
458         captureThumbnail()
459         publishState()
460     }
461     return moved
462 }
463
464 fun setLayerVisible(id: Long, visible: Boolean) {
465     resetStroke()
466     val layer = layerStack.findLayerById(id) ?: return
467     layer.isVisible = visible
468     invalidateCompositeCache()
469     syncLayerDocument(layer)
470     captureThumbnail()
471     publishState()
472 }
473
474 fun setLayerOpacity(id: Long, opacity: Float) {
475     val layer = layerStack.findLayerById(id) ?: return
476     layer.opacity = opacity.coerceIn(0f, 1f)
477     invalidateCompositeCache()
478     syncLayerDocument(layer)
479     captureThumbnail(listOf(layer.id))
480     publishState()
481 }
482
483 /** Clears only the editable active layer. The locked background is protected. */
484 fun clearActiveLayer(): Boolean {
485     checkOnMainThread()
486     resetStroke()
487     val layer = layerStack.activeLayer() ?: return false
488     val cleared = strokeCommitter.clearLayer(
489         layer = layer,
490         canvasWidth = layerStack.canvasWidth,
491         canvasHeight = layerStack.canvasHeight,
492     )
493     if (cleared) {
494         updateProjectDocument { it }
495         publishState()
496     }
497     return cleared
498 }
499
500 /** Mirrors only the camera view; document pixels and undo history stay unchanged. */
501 fun toggleCanvasMirror(): Boolean {
502     resetStroke()
503     val width = layerStack.canvasWidth
504     val height = layerStack.canvasHeight
505     if (width <= 0 || height <= 0) return false
506
507     // Flip around the document centre in screen space. Toggling the matrix
508     // sign alone flips around the camera origin and makes the canvas jump.
509     val camera = layerStack.camera
510     val current = camera.snapshot()
511     val canvasCenterX = width * .5f
512     val canvasCenterY = height * .5f
513     val before = FloatArray(2)
514     val after = FloatArray(2)
515     current.canvasToScreen(canvasCenterX, canvasCenterY, before)
516     val flipped = current.copy(flipX = !current.flipX)
517     flipped.canvasToScreen(canvasCenterX, canvasCenterY, after)
518     camera.set(
519         flipped.copy(
520             translateX = flipped.translateX + before[0] - after[0],
521             translateY = flipped.translateY + before[1] - after[1],
522         ),
523     )
524     return true
525 }
526
527 /** px = ДИАМЕТР кисти в canvas-пикселях, ровно то, что показывает UI. */
528 fun setBrushSize(px: Float) =
529     updateBrush(brushSettings.copy(baseRadiusPx = (px * .5f).coerceIn(.5f, 200f)))
530
531 fun setBrushOpacity(opacity: Float) =
532     updateBrush(brushSettings.copy(opacity = opacity.coerceIn(0f, 1f)))
533
534 fun setEraserEnabled(enabled: Boolean) {
535     eraserEnabled = enabled
536 }
537
538 fun setBrushColor(color: androidx.compose.ui.graphics.Color) =
539     updateBrush(brushSettings.copy(colorArgb = color.toArgb().toLong() and 0xFFFFFFFFFL))
540
541 fun applyBrush(settings: BrushSettings) {
542     val nextSettings = settings
543         .copy(colorArgb = brushSettings.colorArgb)
544         .resolved()
545     updateBrush(nextSettings)
546 }
547
548 fun setBrushSettings(settings: BrushSettings) {

```



```

549         updateBrush(settings)
550     }
551
552     /** Правка кисти во время штриха больше не отменяет штрих: применяем на следующем DOWN. */
553     private fun updateBrush(next: BrushSettings) {
554         brushSettings = next.resolved()
555         if (!strokeActive) {
556             applyBrushToEmitters(brushSettings)
557             ColorSpaces.srgb8ToLinear(brushSettings.colorArgb, strokeColorLinear)
558         }
559         publishState()
560     }
561
562     private fun applyBrushToEmitters(settings: BrushSettings) {
563         stampEmitter.updateSettings(settings)
564         capsuleEmitter.updateSettings(settings)
565     }
566
567     fun requestState() {
568         publishState()
569     }
570
571     fun checkOnGltThread() {
572         val current = Thread.currentThread()
573         check(current === glThread) {
574             "GL resource accessed outside GLSurfaceView render thread: current=$current, expected=$glThread"
575         }
576     }
577
578     override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
579         val currentThread = Thread.currentThread()
580         glThread = currentThread
581         GLCheck.setGLThread(currentThread)
582         releaseGLObjects()
583         undoPipeline.ensureRunning()
584         ensureTilePayloadExecutor()
585         thumbnailScheduler?.ensureRunning()
586         val nextCaps = GLCaps.query()
587         caps = nextCaps
588         documentSession = DocumentSession(
589             document = projectDocument,
590             layerStack = layerStack,
591             undoManager = undoManager,
592             layerTiles = initialLayerTiles,
593         ).also { it.loadIntoGpu(nextCaps) }
594         if (deferredRestoredLayerThumbnails.isNotEmpty()) {
595             val previews = deferredRestoredLayerThumbnails
596             deferredRestoredLayerThumbnails = emptyMap()
597             installRestoredLayerThumbnails(previews)
598         }
599         strokeTarget.create(
600             projectDocument.width,
601             projectDocument.height,
602             nextCaps.supportsHalfFloatColorBuffer,
603         )
604         strokeTarget.clear(0f, 0f, 0f, 0f)
605         strokeCoverageTarget.create(
606             projectDocument.width,
607             projectDocument.height,
608             preferHalfFloat = false,
609         )
610         strokeCoverageTarget.clear(0f, 0f, 0f, 0f)
611         ViewTransform.buildCanvasToFbo(
612             projectDocument.width.toFloat(),
613             projectDocument.height.toFloat(),
614             canvasToFboMatrix,
615         )
616         geometry = CanvasGeometry().also {
617             it.create(projectDocument.width, projectDocument.height)
618         }
619         compositor = Compositor().also { it.create() }
620         backdropRenderer = CanvasBackdropRenderer().also { it.create() }
621         screenPresentRenderer = ScreenPresentRenderer().also { it.create() }
622         linearTextureBlitter = LinearTextureBlitter().also { it.create() }
623         invalidateCompositeCache()
624         dabRenderer = DabRenderer(dabBuffer.capacity).also { it.create() }
625         capsuleRenderer = CapsuleStrokeRenderer().also {
626             it.create()
627         }
628         ribbonMeshRenderer = RibbonMeshRenderer().also { it.create() }
629         strokeBlitter = StrokeBlitter().also { it.create() }
630         nonBuildupStrokeRenderer = NonBuildupStrokeRenderer().also { it.create() }
631         // Build both the gallery preview and the individual layer previews
632         // once the complete GPU session is available.
633         captureThumbnail(layerStack.allLayers().map(PaintLayer::id))
634         onDocumentSessionLoaded?.invoke()
635         ColorSpaces.srgb8ToLinear(brushSettings.colorArgb, strokeColorLinear)
636         publishState()
637         GLCheck.noError("P6 surface creation")
638     }
639
640     override fun onSurfaceChanged(gl: GL10?, width: Int, height: Int) {

```

```

641     screenWidth = width.coerceAtLeast(1)
642     screenHeight = height.coerceAtLeast(1)
643     compositeTarget.create(
644         screenWidth,
645         screenHeight,
646         preferHalfFloat = false,
647     )
648     lowerCompositeTarget.create(
649         screenWidth,
650         screenHeight,
651         preferHalfFloat = false,
652     )
653     upperCompositeTarget.create(
654         screenWidth,
655         screenHeight,
656         preferHalfFloat = false,
657     )
658     strokePreviewTarget.create(
659         screenWidth,
660         screenHeight,
661         preferHalfFloat = false,
662     )
663     strokeCoveragePreviewTarget.create(
664         screenWidth,
665         screenHeight,
666         preferHalfFloat = false,
667     )
668     compositeTarget.clear(0f, 0f, 0f, 0f)
669     lowerCompositeTarget.clear(0f, 0f, 0f, 0f)
670     upperCompositeTarget.clear(0f, 0f, 0f, 0f)
671     strokePreviewTarget.clear(0f, 0f, 0f, 0f)
672     strokeCoveragePreviewTarget.clear(0f, 0f, 0f, 0f)
673     invalidateCompositeCache()
674     GLES30.glViewport(0, 0, screenWidth, screenHeight)
675     layerStack.camera.fitCanvas(layerStack.canvasWidth, layerStack.canvasHeight, screenWidth, screenHeight)
676     GlCheck.noError("P6 surface changed")
677 }
678
679 override fun onDrawFrame(gl: GL10?) {
680     frameStartNanos = System.nanoTime()
681     if (cancelRequested.compareAndSet(true, false)) resetStroke()
682     if (undoGestureRequested.compareAndSet(true, false)) undo()
683     if (redoGestureRequested.compareAndSet(true, false)) redo()
684     if (resetCameraGestureRequested.compareAndSet(true, false)) resetCamera()
685     inputBatchesSinceLastFrameLog += drainInput()
686     if (undoPipeline.process(undoManager)) publishState()
687     thumbnailScheduler?.processCompleted()
688     // Thumbnail capture calls glReadPixels. A pending gallery preview must
689     // never steal a frame from an active brush stroke.
690     if (!strokeActive) thumbnailScheduler?.buildIfNeeded(layerStack)
691
692     val frameStrokeBrush = activeStrokeBrush
693     if (strokeActive && frameStrokeBrush?.renderMode == BrushRenderMode.RIBBON) {
694         renderCapsulePreview(frameStrokeBrush)
695     } else if (strokeActive && frameStrokeBrush?.renderMode == BrushRenderMode.STAMP && dabBuffer.count > 0) {
696         // No longer clearing and drawing everything here, handled incrementally in drainInput
697     }
698
699     val currentGeometry = geometry ?: run {
700         logFrameIfNeeded()
701         return
702     }
703     val currentCompositor = compositor ?: run {
704         logFrameIfNeeded()
705         return
706     }
707     val currentScreenPresentRenderer = screenPresentRenderer ?: run {
708         logFrameIfNeeded()
709         return
710     }
711
712     layerStack.camera.snapshot().buildCanvasToClip(
713         screenWidth.toFloat(),
714         screenHeight.toFloat(),
715         canvasToClipMatrix,
716     )
717
718     val hasNonBuildupPreview = strokeActive &&
719         frameStrokeBrush?.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP &&
720         (dabBuffer.count > 0 || capsuleEmitter.hasStroke)
721     val previewTextureId = if (
722         strokeActive &&
723         !hasNonBuildupPreview &&
724         (dabBuffer.count > 0 ||
725             (frameStrokeBrush?.renderMode == BrushRenderMode.RIBBON && capsuleEmitter.hasStroke))
726     ) {
727         strokePreviewTarget.textureId
728     } else {
729         0
730     }
731     renderDocumentComposite(
732         compositor = currentCompositor,

```

```

733         geometry = currentGeometry,
734         previewTextureId = previewTextureId,
735         previewCoverageTextureId = if (hasNonBuildupPreview) {
736             strokeCoveragePreviewTarget.textureId
737         } else {
738             0
739         },
740         previewMode = if (hasNonBuildupPreview) {
741             StrokeRenderMode.NON_BUILDUP
742         } else {
743             StrokeRenderMode.NORMAL_BUILDUP
744         },
745         strokeOpacity = frameStrokeBrush?.opacity?.coerceIn(0f, 1f) ?: 1f,
746     )
747
748     GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
749     GLES30.glViewport(0, 0, screenWidth, screenHeight)
750     GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
751     GLES30.glDisable(GLES30.GL_BLEND)
752     GLES30.glClearColor(canvasBackdropColor[0], canvasBackdropColor[1], canvasBackdropColor[2], 1f)
753     GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
754     drawBackdropPattern()
755     currentScreenPresentRenderer.render(
756         sourceTextureId = compositeTarget.textureId,
757         viewportWidth = screenWidth,
758         viewportHeight = screenHeight,
759     )
760     logFrameIfNeeded()
761 }
762
763 /**
764  * During a live stroke, the confirmed layers above and below the active
765  * layer do not change. Cache those two screen-space composites and redraw
766  * only the active layer plus its preview on MOVE. This preserves layer
767  * ordering for both paint and eraser previews.
768  */
769 private fun renderDocumentComposite(
770     compositor: Compositor,
771     geometry: CanvasGeometry,
772     previewTextureId: Int,
773     previewCoverageTextureId: Int,
774     previewMode: StrokeRenderMode,
775     strokeOpacity: Float,
776 ) {
777     val activeLayerIndex = layerStack.indexOfLayer(layerStack.activeLayerId)
778     val textureBlitter = linearTextureBlitter
779     val canUseStrokeCache = strokeActive &&
780         textureBlitter != null &&
781         activeLayerIndex in 0 until layerStack.count &&
782         lowerCompositeTarget.textureId != 0 &&
783         upperCompositeTarget.textureId != 0
784
785     if (!canUseStrokeCache) {
786         compositeTarget.clear(0f, 0f, 0f, 0f)
787         GLES30.glViewport(0, 0, screenWidth, screenHeight)
788         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
789         GLES30.glDisable(GLES30.GL_BLEND)
790         compositor.render(
791             destination = compositeTarget,
792             geometry = geometry,
793             layers = layerStack,
794             activeLayerId = layerStack.activeLayerId,
795             strokeTextureId = previewTextureId,
796             strokeCoverageTextureId = previewCoverageTextureId,
797             strokeIsScreenSpace = previewTextureId != 0 || previewCoverageTextureId != 0,
798             strokeMode = previewMode,
799             strokeErase = activeStrokeErase,
800             strokeColorLinear = activeStrokeColorLinear,
801             strokeOpacity = strokeOpacity,
802             canvasToClip = canvasToClipMatrix,
803         )
804         return
805     }
806
807     rebuildPreviewCompositeCachesIfNeeded(
808         compositor = compositor,
809         geometry = geometry,
810         activeLayerIndex = activeLayerIndex,
811     )
812
813     compositeTarget.clear(0f, 0f, 0f, 0f)
814     compositeTarget.bind()
815     GLES30.glViewport(0, 0, screenWidth, screenHeight)
816     GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
817     GLES30.glDisable(GLES30.GL_BLEND)
818     if (activeLayerIndex > 0) {
819         textureBlitter.blit(lowerCompositeTarget.textureId)
820     }
821     compositor.render(
822         destination = compositeTarget,
823         geometry = geometry,
824         layers = layerStack,

```

```

825         activeLayerId = layerStack.activeLayerId,
826         strokeTextureId = previewTextureId,
827         strokeCoverageTextureId = previewCoverageTextureId,
828         strokeIsScreenSpace = previewTextureId != 0 || previewCoverageTextureId != 0,
829         strokeMode = previewMode,
830         strokeErase = activeStrokeErase,
831         strokeColorLinear = activeStrokeColorLinear,
832         strokeOpacity = strokeOpacity,
833         canvasToClip = canvasToClipMatrix,
834         firstLayerIndex = activeLayerIndex,
835         lastLayerExclusive = activeLayerIndex + 1,
836     )
837     if (activeLayerIndex < layerStack.count - 1) {
838         textureBlitter.blit(upperCompositeTarget.textureId)
839     }
840     GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
841 }
842
843 private fun rebuildPreviewCompositeCachesIfNeeded(
844     compositor: Compositor,
845     geometry: CanvasGeometry,
846     activeLayerIndex: Int,
847 ) {
848     val cameraChanged = !canvasToClipMatrix.contentEquals(cachedPreviewCanvasToClipMatrix)
849     if (!compositeCacheDirty &&
850         cachedPreviewActiveLayerIndex == activeLayerIndex &&
851         !cameraChanged
852     ) {
853         return
854     }
855
856     lowerCompositeTarget.clear(0f, 0f, 0f, 0f)
857     GLES30.glViewport(0, 0, screenWidth, screenHeight)
858     compositor.render(
859         destination = lowerCompositeTarget,
860         geometry = geometry,
861         layers = layerStack,
862         activeLayerId = -1L,
863         strokeTextureId = 0,
864         strokeErase = false,
865         strokeOpacity = 1f,
866         canvasToClip = canvasToClipMatrix,
867         firstLayerIndex = 0,
868         lastLayerExclusive = activeLayerIndex,
869     )
870
871     upperCompositeTarget.clear(0f, 0f, 0f, 0f)
872     GLES30.glViewport(0, 0, screenWidth, screenHeight)
873     compositor.render(
874         destination = upperCompositeTarget,
875         geometry = geometry,
876         layers = layerStack,
877         activeLayerId = -1L,
878         strokeTextureId = 0,
879         strokeErase = false,
880         strokeOpacity = 1f,
881         canvasToClip = canvasToClipMatrix,
882         firstLayerIndex = activeLayerIndex + 1,
883         lastLayerExclusive = layerStack.count,
884     )
885
886     canvasToClipMatrix.copyInto(cachedPreviewCanvasToClipMatrix)
887     cachedPreviewActiveLayerIndex = activeLayerIndex
888     compositeCacheDirty = false
889 }
890
891 private fun invalidateCompositeCache() {
892     compositeCacheDirty = true
893     cachedPreviewActiveLayerIndex = -1
894 }
895
896 private fun drawBackdropPattern() {
897     val mode = when (canvasBackdropMode) {
898         CanvasBackdropMode.SOLID -> return
899         CanvasBackdropMode.GRID -> CanvasBackdropRendererer.MODE_GRID
900         CanvasBackdropMode.CHECKERBOARD -> CanvasBackdropRendererer.MODE_CHECKERBOARD
901     }
902     backdropRenderer?.render(
903         viewportWidth = screenWidth,
904         viewportHeight = screenHeight,
905         backgroundColor = canvasBackdropColor,
906         gridColor = canvasGridColor,
907         mode = mode,
908     )
909 }
910
911 private fun argbToRgb(Argb: Int, out: FloatArray) {
912     out[0] = ((Argb ushr 16) and 0xFF) / 255f
913     out[1] = ((Argb ushr 8) and 0xFF) / 255f
914     out[2] = (Argb and 0xFF) / 255f
915 }
916

```

```

917  /** Invalidates worker results created before a destructive document change. */
918  private fun invalidatePendingUndoHistory() {
919      checkOnGLThread()
920      undoPipeline.invalidate()
921  }
922
923  private fun shutdownUndoExecutor() {
924      undoPipeline.shutdown()
925  }
926
927  private fun ensureTilePayloadExecutor() {
928      if (tilePayloadExecutor.isShutdown) {
929          tilePayloadExecutor = Executors.newSingleThreadExecutor()
930      }
931  }
932
933  private fun shutdownTilePayloadExecutor() {
934      tilePayloadExecutor.shutdownNow()
935  }
936
937  /** Safe to call from the UI thread; cancellation is executed on the next GL frame. */
938  fun cancelActiveStroke() {
939      cancelRequested.set(true)
940  }
941
942  /** Must run on the GL thread while the context is still current. */
943  fun releaseGLObjects() {
944      checkOnGLThread()
945      invalidatePendingUndoHistory()
946      discardPendingInput()
947      resetStroke()
948      undoManager.clear()
949      shutdownUndoExecutor()
950      shutdownTilePayloadExecutor()
951      dabRenderer?.release()
952      dabRenderer = null
953      grainTexture?.release()
954      grainTexture = null
955      grainPath = null
956      shapeTexture?.release()
957      shapeTexture = null
958      shapePath = null
959      capsuleRenderer?.release()
960      capsuleRenderer = null
961      ribbonMeshRenderer?.release()
962      ribbonMeshRenderer = null
963      strokeBlitter?.release()
964      strokeBlitter = null
965      nonBuildupStrokeRenderer?.release()
966      nonBuildupStrokeRenderer = null
967      compositor?.release()
968      compositor = null
969      backdropRenderer?.release()
970      backdropRenderer = null
971      screenPresentRenderer?.release()
972      screenPresentRenderer = null
973      linearTextureBlitter?.release()
974      linearTextureBlitter = null
975      thumbnailScheduler?.shutdown()
976      thumbnailCapture.release()
977      documentSession = null
978      geometry?.release()
979      geometry = null
980      strokeTarget.release()
981      strokePreviewTarget.release()
982      strokeCoveragePreviewTarget.release()
983      strokeCoverageTarget.release()
984      compositeTarget.release()
985      lowerCompositeTarget.release()
986      upperCompositeTarget.release()
987      layerStack.release()
988      caps = null
989  }
990
991  private fun drainInput(): Int {
992      var processedBatches = 0
993      val startedAtNanos = System.nanoTime()
994      while (
995          processedBatches < MAX_INPUT_BATCHES_PER_FRAME &&
996          System.nanoTime() - startedAtNanos < MAX_INPUT_PROCESS_NANOS
997      ) {
998          val batch = inputQueue.poll() ?: break
999          try {
1000              when (batch.action) {
1001                  InputAction.DOWN -> {
1002                      if (!batch.isEmpty()) {
1003                          resetStroke()
1004                          val strokeBrush = brushSettings.resolved()
1005                          activeStrokeBrush = strokeBrush
1006                          ColorSpaces.srgb8ToLinear(strokeBrush.colorArgb, activeStrokeColorLinear)
1007                          activeStrokeColorLinear.copyInto(strokeColorLinear)
1008                          activeStrokeErase = eraserEnabled

```

```

1009         strokeActive = true
1010
1011         if (strokeBrush.renderMode == BrushRenderMode.RIBBON) {
1012             if (strokeBrush.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP) {
1013                 strokeCoveragePreviewTarget.clear(0f, 0f, 0f, 0f)
1014             } else {
1015                 strokePreviewTarget.clear(0f, 0f, 0f, 0f)
1016             }
1017             capsuleEmitter.begin(
1018                 batch = batch,
1019                 out = checkNotNull(capsuleRenderer),
1020                 strokeSettings = strokeBrush,
1021             )
1022         } else {
1023             dabRenderer?.apply {
1024                 beginStroke()
1025             }
1026             stampEmitter.begin(batch, dabBuffer, strokeBrush)
1027             if (strokeBrush.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP) {
1028                 strokeCoveragePreviewTarget.clear(0f, 0f, 0f, 0f)
1029             } else {
1030                 strokePreviewTarget.clear(0f, 0f, 0f, 0f)
1031             }
1032             stampPreviewInitialized = true
1033
1034             drawPendingStampPreview("DOWN")
1035         }
1036     }
1037 }
1038
1039 InputAction.MOVE -> if (strokeActive) {
1040     val strokeBrush = activeStrokeBrush
1041     ?: error("Missing active stroke snapshot")
1042     if (strokeBrush.renderMode == BrushRenderMode.RIBBON) {
1043         capsuleEmitter.append(
1044             batch = batch,
1045             out = checkNotNull(capsuleRenderer),
1046         )
1047     } else {
1048         stampEmitter.append(batch, dabBuffer)
1049         drawPendingStampPreview("MOVE")
1050     }
1051 }
1052
1053 InputAction.UP -> if (strokeActive) {
1054     val strokeBrush = activeStrokeBrush
1055     ?: error("Missing active stroke snapshot")
1056     if (strokeBrush.renderMode == BrushRenderMode.RIBBON) {
1057         val renderer = checkNotNull(capsuleRenderer)
1058
1059         // UP должен сначала обработать historical samples,
1060         // затем текущую точку, которую уже положил capturer.
1061         capsuleEmitter.append(
1062             batch = batch,
1063             out = renderer,
1064         )
1065         capsuleEmitter.finish(
1066             out = renderer,
1067             cancel = false,
1068         )
1069         commitCapsuleStroke(strokeBrush)
1070     } else {
1071         stampEmitter.append(batch, dabBuffer)
1072         stampEmitter.finish(dabBuffer, cancel = false)
1073         commitStroke(strokeBrush)
1074     }
1075
1076     resetStroke()
1077 }
1078
1079 InputAction.CANCEL -> resetStroke()
1080 }
1081 } finally {
1082     inputPool.release(batch)
1083 }
1084 processedBatches++
1085 }
1086
1087 val queueDepth = inputQueue.size
1088 if (queueDepth > maxInputQueueDepth) maxInputQueueDepth = queueDepth
1089 if (BuildConfig.DEBUG && inputCapturer.droppedBatches != droppedBatchCountAtLastLog) {
1090     droppedBatchCountAtLastLog = inputCapturer.droppedBatches
1091     Log.w(
1092         INPUT_LATENCY_TAG,
1093         "queue=$queueDepth max=$maxInputQueueDepth dropped=${inputCapturer.droppedBatches}",
1094     )
1095 }
1096
1097 // Keep a continuation frame scheduled, but never let a delayed input
1098 // backlog monopolise the GL thread and visibly freeze the canvas.
1099 if (inputQueue.isNotEmpty()) {
1100     onInputRenderRequested?.invoke()

```

```

1101     }
1102     return processedBatches
1103 }
1104
1105 /** Debug-only aggregate; never writes Logcat from the MOVE hot path. */
1106 private fun logFrameIfNeeded() {
1107     frameCounter++
1108     if (!BuildConfig.DEBUG ||
1109         frameStartNanos - lastFrameLogNanos < FRAME_LOG_INTERVAL_NANOS
1110     ) {
1111         return
1112     }
1113
1114     val frameMs = (System.nanoTime() - frameStartNanos) / 1_000_000f
1115     Log.d(
1116         FRAME_DIAG_TAG,
1117         "frameMs=$frameMs queue=${inputQueue.size} frames=$frameCounter " +
1118         "inputBatches=$inputBatchesSinceLastFrameLog dabs=${dabBuffer.count} " +
1119         "undoPending=${undoPipeline.pendingCount}",
1120     )
1121     lastFrameLogNanos = frameStartNanos
1122     frameCounter = 0
1123     inputBatchesSinceLastFrameLog = 0
1124 }
1125
1126 private fun commitStroke(strokeBrush: BrushSettings) {
1127     val layer = layerStack.activeLayer() ?: return
1128     if (layer.isLocked || dabBuffer.count == 0) return
1129     if (!computeDirtyPixelBounds(dirtyBounds)) return
1130     val blitter = strokeBlitter ?: return
1131     val strokeGeometry = geometry ?: return
1132
1133     if (strokeBrush.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP) {
1134         val renderer = dabRenderer ?: return
1135         val nonBuildupBlitter = nonBuildupStrokeRenderer ?: return
1136         strokeCoverageTarget.clear(0f, 0f, 0f, 0f)
1137         renderer.drawCoverageInto(
1138             target = strokeCoverageTarget,
1139             width = layerStack.canvasWidth,
1140             height = layerStack.canvasHeight,
1141             canvasToFbo = canvasToFboMatrix,
1142             dabs = dabBuffer,
1143         )
1144         if (strokeCommitter.commitNonBuildup(
1145             layer = layer,
1146             geometry = strokeGeometry,
1147             blitter = nonBuildupBlitter,
1148             coverageTarget = strokeCoverageTarget,
1149             colorLinear = activeStrokeColorLinear,
1150             dirtyBounds = dirtyBounds,
1151             canvasWidth = layerStack.canvasWidth,
1152             canvasHeight = layerStack.canvasHeight,
1153             opacity = strokeBrush.opacity,
1154             erase = activeStrokeErase,
1155         )) {
1156             publishState()
1157         }
1158         return
1159     }
1160
1161     // Preview is screen-sized. Rebuild the document-sized target once on
1162     // UP, then commit those exact pixels to the active layer.
1163     strokeTarget.clear(0f, 0f, 0f, 0f)
1164     val renderer = dabRenderer ?: return
1165     renderer.drawInto(
1166         target = strokeTarget,
1167         width = layerStack.canvasWidth,
1168         height = layerStack.canvasHeight,
1169         canvasToFbo = canvasToFboMatrix,
1170         dabs = dabBuffer,
1171         colorLinear = activeStrokeColorLinear,
1172         blendPolicy = strokeBrush.blendPolicy,
1173         strokeOpacity = 1f,
1174     )
1175
1176     if (strokeCommitter.commit(
1177         layer = layer,
1178         geometry = strokeGeometry,
1179         blitter = blitter,
1180         dirtyBounds = dirtyBounds,
1181         canvasWidth = layerStack.canvasWidth,
1182         canvasHeight = layerStack.canvasHeight,
1183         opacity = strokeBrush.opacity,
1184         erase = activeStrokeErase,
1185     )) {
1186         publishState()
1187     }
1188 }
1189
1190 /** Returns a clamped pixel region covering the exact emitted dabs and their AA fringe. */
1191 private fun computeDirtyPixelBounds(out: IntArray): Boolean {
1192     if (dabBuffer.count == 0) return false

```

```

1193     strokeDirtyRect.clear()
1194     dabBuffer.includeDirtyRect(
1195         rect = strokeDirtyRect,
1196         extraMargin = STAMP_DIRTY_MARGIN_PX,
1197     )
1198     strokeDirtyRect.clamp(layerStack.canvasWidth.toFloat(), layerStack.canvasHeight.toFloat())
1199     strokeDirtyRect.toPixelBounds(out)
1200     return out[2] > out[0] && out[3] > out[1]
1201 }
1202
1203 private fun drawPendingStampPreview(phase: String) {
1204     val renderer = dabRenderer ?: return
1205     val strokeBrush = activeStrokeBrush
1206     ?: error("Missing active stroke snapshot")
1207     check(renderer.uploadedCount <= dabBuffer.count) {
1208         "Stamp preview rewound: uploaded=${renderer.uploadedCount}, dabs=${dabBuffer.count}"
1209     }
1210     if (LOG_STAMP_PREVIEW && BuildConfig.DEBUG) {
1211         Log.d(
1212             BRUSH_DIAG_TAG,
1213             "phase=$phase dabs=${dabBuffer.count} pending=${dabBuffer.count - renderer.uploadedCount}",
1214         )
1215     }
1216     layerStack.camera.snapshot().buildCanvasToClip(
1217         screenWidth.toFloat(),
1218         screenHeight.toFloat(),
1219         canvasToPreviewMatrix,
1220     )
1221     if (strokeBrush.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP) {
1222         renderer.drawPendingCoveragePreviewInto(
1223             target = strokeCoveragePreviewTarget,
1224             previewWidth = screenWidth,
1225             previewHeight = screenHeight,
1226             documentWidth = layerStack.canvasWidth,
1227             documentHeight = layerStack.canvasHeight,
1228             canvasToClip = canvasToPreviewMatrix,
1229             dabs = dabBuffer,
1230         )
1231     } else {
1232         renderer.drawPendingPreviewInto(
1233             target = strokePreviewTarget,
1234             previewWidth = screenWidth,
1235             previewHeight = screenHeight,
1236             documentWidth = layerStack.canvasWidth,
1237             documentHeight = layerStack.canvasHeight,
1238             canvasToClip = canvasToPreviewMatrix,
1239             dabs = dabBuffer,
1240             colorLinear = activeStrokeColorLinear,
1241             blendPolicy = BlendPolicy.NORMAL_BUILDUP,
1242             strokeOpacity = 1f,
1243         )
1244     }
1245     check(renderer.uploadedCount == dabBuffer.count) {
1246         "Stamp preview did not consume all dabs: uploaded=${renderer.uploadedCount}, dabs=${dabBuffer.count}"
1247     }
1248 }
1249
1250 private fun resetStroke() {
1251     strokeActive = false
1252     activeStrokeErase = false
1253     activeStrokeBrush = null
1254     stampEmitter.reset()
1255     dabBuffer.clear()
1256     capsuleEmitter.reset()
1257     capsuleRenderer?.clearStrokeData()
1258     capsulePreviewInitialized = false
1259     renderedRibbonMeshVersion = -1L
1260     stampPreviewInitialized = false
1261     dabRenderer?.clearStrokeData()
1262     if (strokeTarget.framebufferId != 0) {
1263         strokeTarget.clear(0f, 0f, 0f, 0f)
1264     }
1265     if (strokePreviewTarget.framebufferId != 0) {
1266         strokePreviewTarget.clear(0f, 0f, 0f, 0f)
1267     }
1268     if (strokeCoveragePreviewTarget.framebufferId != 0) {
1269         strokeCoveragePreviewTarget.clear(0f, 0f, 0f, 0f)
1270     }
1271     if (strokeCoverageTarget.framebufferId != 0) {
1272         strokeCoverageTarget.clear(0f, 0f, 0f, 0f)
1273     }
1274 }
1275
1276 /** Releases pooled batches without interpreting them during shutdown/context recreation. */
1277 private fun discardPendingInput() {
1278     while (true) {
1279         val batch = inputQueue.poll() ?: return
1280         inputPool.release(batch)
1281     }
1282 }
1283
1284 /**

```



```

1285     * Builds the persisted metadata from the GL-owned runtime state. UI state
1286     * is never treated as the document source of truth.
1287     */
1288     fun buildDocumentSnapshot(): ProjectDocument {
1289         checkOnGLThread()
1290         return projectDocument.copy(
1291             updatedAt = System.currentTimeMillis(),
1292             activeLayerId = layerStack.activeLayerId,
1293             layers = layerStack.allLayers().map(::layerDocumentFrom),
1294         )
1295     }
1296
1297     private fun updateProjectDocument(
1298         transform: (ProjectDocument) -> ProjectDocument,
1299     ) {
1300         projectDocument = transform(projectDocument)
1301         projectDocument = buildDocumentSnapshot()
1302         documentSession?.markProjectDirty()
1303         onProjectDocumentChange?.invoke(projectDocument)
1304     }
1305
1306     fun acknowledgeSavedTiles(saved: Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>) {
1307         layerTileStore.acknowledge(saved)
1308         documentSession?.markSaved(saved.keys)
1309     }
1310
1311     private fun publishDirtyTiles() {
1312         val dirty = layerTileStore.takeDirty()
1313         if (dirty.isEmpty()) return
1314         val documentSnapshot = buildDocumentSnapshot()
1315         projectDocument = documentSnapshot
1316         documentSession?.markProjectDirty()
1317         // PersistentLayerTiles.encode() can copy tens of MB for a mature 4K
1318         // layer. Never do that allocation while the GL thread is responsible
1319         // for accepting stylus input and drawing the next preview frame.
1320         try {
1321             tilePayloadExecutor.execute {
1322                 val payloads = runCatching {
1323                     layerTileStore.payloadsForDirty(dirty)
1324                 }.getOrElse { error ->
1325                     if (BuildConfig.DEBUG) Log.e("TilePersistence", "Tile payload encoding failed", error)
1326                     return@execute
1327                 }
1328                 onDirtyLayerTiles?.invoke(documentSnapshot, payloads, dirty)
1329             }
1330         } catch (_: RejectedExecutionException) {
1331             // Context release cancelled a stale save. The next active session
1332             // will retain its dirty tiles and schedule a fresh publication.
1333         }
1334     }
1335
1336     /** Re-captures restored GPU tiles so Undo/Redo is persisted like a stroke. */
1337     private fun persistRestoredTiles(
1338         layer: PaintLayer,
1339         tiles: List<com.wetinknext.engine.undo.TileSnapshot>,
1340     ) {
1341         if (tiles.isEmpty()) return
1342         val left = tiles.minOf { it.pixelLeft }
1343         val top = tiles.minOf { it.pixelTop }
1344         val right = tiles.maxOf { it.pixelLeft + it.pixelWidth }
1345         val bottom = tiles.maxOf { it.pixelTop + it.pixelHeight }
1346         val raw = com.wetinknext.engine.undo.TileSnapshotCapture.capture(
1347             target = layer.target,
1348             bounds = intArrayOf(left, top, right, bottom),
1349         )
1350         layerTileStore.markDirty(layer.id, raw)
1351         publishDirtyTiles()
1352         markLayerThumbnailDirty(layer.id)
1353     }
1354
1355     private fun captureThumbnail(dirtyLayerIds: Collection<Long> = emptyList()) {
1356         // Thumbnail capture performs glReadPixels. Preview must never queue it
1357         // while a stylus stroke owns the GL frame budget.
1358         if (strokeActive) return
1359         if (caps == null || geometry == null || compositor == null) return
1360         val scheduler = thumbnailScheduler
1361         if (scheduler != null) {
1362             scheduler.markDirty(projectDirty = true, dirtyLayerIds = dirtyLayerIds)
1363             onInputRenderRequested?.invoke()
1364         } else {
1365             onThumbnailCaptured?.invoke(thumbnailCapture.capture(layerStack))
1366         }
1367     }
1368
1369     /** Called only after a successful GPU edit of [layerId]. */
1370     private fun markLayerThumbnailDirty(layerId: Long) {
1371         val scheduler = thumbnailScheduler
1372         if (scheduler != null) {
1373             scheduler.markLayerDirty(layerId)
1374             scheduler.markProjectDirty()
1375             onInputRenderRequested?.invoke()
1376         } else {

```

```

1377         captureThumbnail()
1378     }
1379 }
1380
1381 /**
1382  * Records published preview files on the GL thread. Scheduler generations
1383  * ensure every listed layer still has the pixels used to create its image.
1384  */
1385 fun applyThumbnailBuild(result: ThumbnailBuildResult) {
1386     checkOnGLThread()
1387     result.layerPreviews.forEach { (layerId, file) ->
1388         layerStack.findLayerById(layerId)?.let { layer ->
1389             layerThumbnailPaths[layerId] = file.absolutePath
1390             layerThumbnailVersions[layerId] = layer.version
1391         }
1392     }
1393     layerThumbnailPaths.keys.retainAll(layerStack.allLayers().map(PaintLayer::id).toSet())
1394     layerThumbnailVersions.keys.retainAll(layerStack.allLayers().map(PaintLayer::id).toSet())
1395     // layerDocumentFrom copies the current PaintLayer.version into
1396     // thumbnailVersion, and updateProjectDocument atomically publishes the
1397     // resulting document snapshot to the autosave bridge.
1398     updateProjectDocument { it }
1399     publishState()
1400 }
1401
1402 /**
1403  * Makes WebP previews restored from the persisted project immediately
1404  * available to Compose. An older persisted preview may never replace a
1405  * newer runtime image after a stroke has incremented the layer version.
1406  */
1407 fun installRestoredLayerThumbnails(previews: Map<Long, File>) {
1408     // queueEvent() is allowed to run before Renderer.onSurfaceCreated().
1409     // At that time no GL owner or LayerStack exists yet, so checking the
1410     // thread here used to crash application startup.
1411     if (glThread == null) {
1412         deferredRestoredLayerThumbnails = previews
1413         return
1414     }
1415     checkOnGLThread()
1416     var changed = false
1417     previews.forEach { (layerId, file) ->
1418         val layer = layerStack.findLayerById(layerId) ?: return@forEach
1419         val storedVersion = projectDocument.layers
1420             .firstOrNull { it.id == layerId }
1421             ?.thumbnailVersion
1422         ?: return@forEach
1423         if (file.isFile && layer.version == storedVersion && layerThumbnailVersions[layerId] == null) {
1424             layerThumbnailPaths[layerId] = file.absolutePath
1425             layerThumbnailVersions[layerId] = storedVersion
1426             changed = true
1427         }
1428     }
1429     if (changed) publishState()
1430 }
1431
1432 private fun syncLayerDocument(layer: PaintLayer) {
1433     updateProjectDocument { document ->
1434         document.copy(
1435             layers = document.layers.map { stored ->
1436                 if (stored.id == layer.id) layerDocumentFrom(layer, stored.pixelFile) else stored
1437             },
1438             activeLayerId = layerStack.activeLayerId,
1439         )
1440     }
1441 }
1442
1443 private fun layerDocumentFrom(
1444     layer: PaintLayer,
1445     pixelFile: String = ProjectDocument.pixelFileFor(layer.id),
1446 ): LayerDocument = LayerDocument(
1447     id = layer.id,
1448     name = layer.name,
1449     visible = layer.isVisible,
1450     locked = layer.isLocked,
1451     opacity = layer.opacity,
1452     blendMode = layer.blendMode,
1453     pixelFile = pixelFile,
1454     thumbnailVersion = layerThumbnailVersions[layer.id]
1455         ?: projectDocument.layers.firstOrNull { it.id == layer.id }?.thumbnailVersion
1456     ?: 0L,
1457 )
1458
1459 private fun publishState() {
1460     val listener = onStateChange ?: return
1461     val layers = layerStack.allLayers()
1462     val activeLayerId = layerStack.activeLayerId
1463     val canDeleteLayers = layers.size > 1
1464     listener(
1465         EditorUiState(
1466             layers = layers.map { layer ->
1467                 LayerUiModel(
1468                     id = layer.id,

```

```

1469         name = layer.name,
1470         isVisible = layer.isVisible,
1471         isLocked = layer.isLocked,
1472         opacity = layer.opacity,
1473         active = layer.id == activeLayerId,
1474         canDelete = !layer.isLocked && canDeleteLayers,
1475         thumbnailPath = layer.ThumbnailPaths[layer.id],
1476         thumbnailVersion = layer.ThumbnailVersions[layer.id]
1477         ?: projectDocument.layers.firstOrNull { it.id == layer.id }?.thumbnailVersion
1478         ?: 0L,
1479     )
1480 },
1481 canUndo = undoManager.canUndo && undoPipeline.pendingCount == 0,
1482 canRedo = undoManager.canRedo && undoPipeline.pendingCount == 0,
1483 brushSizePx = brushSettings.baseRadiusPx * 2f,
1484 brushOpacity = brushSettings.opacity,
1485 activeLayerId = activeLayerId,
1486 ready = layerStack.canvasWidth > 0 && layerStack.canvasHeight > 0,
1487 undoDiagnostics = UndoDiagnostics(
1488     pendingJobs = undoPipeline.pendingCount,
1489     staleResults = undoPipeline.staleResultCount,
1490     compressionFailures = undoPipeline.compressionFailureCount,
1491     restoreFailures = undoRestoreFailures,
1492     memoryBytes = undoManager.memoryBytes,
1493 ),
1494 ),
1495 )
1496 }
1497
1498 /**
1499  * Вызывается только через GLSurfaceView.queueEvent,
1500  * то есть на GL-потоке с current EGL context.
1501  */
1502 fun applyLoadedGrain(
1503     loaded: LoadedBrushTexture,
1504     scale: Float,
1505     canvasLocked: Boolean,
1506     depth: Float,
1507     contrast: Float,
1508 ) {
1509     val oldTexture = grainTexture
1510     val newTexture = BrushTexture()
1511
1512     try {
1513         newTexture.createFromRgba(
1514             width = loaded.width,
1515             height = loaded.height,
1516             rgba = loaded.rgba,
1517         )
1518
1519         capsuleRenderer?.setGrainTexture(
1520             textureId = newTexture.textureId,
1521             scale = scale,
1522             canvasLocked = canvasLocked,
1523             depth = depth,
1524             contrast = contrast,
1525         )
1526
1527         dabRenderer?.setGrainTexture(
1528             textureId = newTexture.textureId,
1529             scale = scale,
1530             canvasLocked = canvasLocked,
1531             depth = depth,
1532             contrast = contrast,
1533         )
1534
1535         grainTexture = newTexture
1536         grainPath = loaded.path
1537
1538         oldTexture?.release()
1539     } catch (error: Throwable) {
1540         newTexture.release()
1541         throw error
1542     }
1543 }
1544
1545 fun clearGrainTexture() {
1546     capsuleRenderer?.clearGrainTexture()
1547     dabRenderer?.clearGrainTexture()
1548
1549     grainTexture?.release()
1550     grainTexture = null
1551     grainPath = null
1552 }
1553
1554 /** Must run on the GL thread. Shape masks affect STAMP dabs only. */
1555 fun applyLoadedShape(
1556     loaded: LoadedBrushTexture,
1557     reverse: Boolean,
1558     rgbToAlpha: Boolean,
1559 ) {
1560     val oldTexture = shapeTexture

```

```

1561     val newTexture = BrushTexture()
1562
1563     try {
1564         newTexture.createFromRgba(loaded.width, loaded.height, loaded.rgba)
1565         dabRenderer?.setShapeTexture(newTexture.textureId, reverse, rgbToAlpha)
1566         shapeTexture = newTexture
1567         shapePath = loaded.path
1568         oldTexture?.release()
1569     } catch (error: Throwable) {
1570         newTexture.release()
1571         throw error
1572     }
1573 }
1574
1575 fun clearShapeTexture() {
1576     dabRenderer?.clearShapeTexture()
1577     shapeTexture?.release()
1578     shapeTexture = null
1579     shapePath = null
1580 }
1581
1582 private fun nextLayerName(): String = "Слой ${layerStack.count + 1}"
1583
1584 private fun nextDuplicateLayerName(sourceName: String): String {
1585     val base = "$sourceName копия"
1586     val names = layerStack.allLayers().map(PaintLayer::name).toSet()
1587     if (base !in names) return base
1588     var number = 2
1589     while ("$base $number" in names) number++
1590     return "$base $number"
1591 }
1592
1593 private fun renderCapsulePreview(strokeBrush: BrushSettings) {
1594     val renderer = ribbonMeshRenderer ?: return
1595     if (renderedRibbonMeshVersion == capsuleEmitter.ribbonMeshVersion) return
1596     val mesh = capsuleEmitter.buildRibbonMesh() ?: return
1597
1598     // A geometric mesh is redrawn as a whole. Clearing avoids building up
1599     // the same triangles on every display frame.
1600     val previewTarget = if (strokeBrush.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP) {
1601         strokeCoveragePreviewTarget
1602     } else strokePreviewTarget
1603     previewTarget.clear(0f, 0f, 0f, 0f)
1604     capsulePreviewInitialized = true
1605     renderedRibbonMeshVersion = capsuleEmitter.ribbonMeshVersion
1606
1607     layerStack.camera.snapshot().buildCanvasToClip(
1608         screenWidth.toFloat(),
1609         screenHeight.toFloat(),
1610         canvasToPreviewMatrix,
1611     )
1612     if (strokeBrush.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP) {
1613         renderer.draw(
1614             target = strokeCoveragePreviewTarget,
1615             width = screenWidth,
1616             height = screenHeight,
1617             matrix = canvasToPreviewMatrix,
1618             mesh = mesh,
1619             color = activeStrokeColorLinear,
1620             flow = strokeBrush.flow,
1621             coverageOnly = true,
1622         )
1623     } else {
1624         renderer.draw(
1625             target = strokePreviewTarget,
1626             width = screenWidth,
1627             height = screenHeight,
1628             matrix = canvasToPreviewMatrix,
1629             mesh = mesh,
1630             color = activeStrokeColorLinear,
1631             flow = strokeBrush.flow,
1632             coverageOnly = false,
1633         )
1634     }
1635 }
1636
1637 private fun commitCapsuleStroke(strokeBrush: BrushSettings) {
1638     val layer = layerStack.activeLayer() ?: return
1639     val renderer = ribbonMeshRenderer ?: return
1640     val mesh = capsuleEmitter.buildRibbonMesh() ?: return
1641
1642     if (layer.isLocked) return
1643     if (!capsuleEmitter.hasStroke) return
1644     if (!computeCapsuleDirtyBounds(dirtyBounds)) return
1645     val blitter = strokeBlitter ?: return
1646     val strokeGeometry = geometry ?: return
1647
1648     if (strokeBrush.effectiveStrokeRenderMode == StrokeRenderMode.NON_BUILDUP) {
1649         val nonBuildupBlitter = nonBuildupStrokeRenderer ?: return
1650         strokeCoverageTarget.clear(0f, 0f, 0f, 0f)
1651         renderer.draw(
1652             target = strokeCoverageTarget,

```

```

1653         width = layerStack.canvasWidth,
1654         height = layerStack.canvasHeight,
1655         matrix = canvasToFboMatrix,
1656         mesh = mesh,
1657         color = activeStrokeColorLinear,
1658         flow = strokeBrush.flow,
1659         coverageOnly = true,
1660     )
1661     if (strokeCommitter.commitNonBuildup(
1662         layer = layer,
1663         geometry = strokeGeometry,
1664         blitter = nonBuildupBlitter,
1665         coverageTarget = strokeCoverageTarget,
1666         colorLinear = activeStrokeColorLinear,
1667         dirtyBounds = dirtyBounds,
1668         canvasWidth = layerStack.canvasWidth,
1669         canvasHeight = layerStack.canvasHeight,
1670         opacity = strokeBrush.opacity,
1671         erase = activeStrokeErase,
1672     )) {
1673         publishState()
1674     }
1675     return
1676 }
1677
1678 /*
1679  * Сначала строим stroke в отдельном offscreen target.
1680  * Старый слой и новый renderer не используют один FBO для рисования.
1681  */
1682 strokeTarget.clear(
1683     red = 0f,
1684     green = 0f,
1685     blue = 0f,
1686     alpha = 0f,
1687 )
1688
1689 renderer.draw(
1690     target = strokeTarget,
1691     width = layerStack.canvasWidth,
1692     height = layerStack.canvasHeight,
1693     matrix = canvasToFboMatrix,
1694     mesh = mesh,
1695     color = activeStrokeColorLinear,
1696     flow = strokeBrush.flow,
1697     coverageOnly = false,
1698 )
1699
1700 if (strokeCommitter.commit(
1701     layer = layer,
1702     geometry = strokeGeometry,
1703     blitter = blitter,
1704     dirtyBounds = dirtyBounds,
1705     canvasWidth = layerStack.canvasWidth,
1706     canvasHeight = layerStack.canvasHeight,
1707     opacity = strokeBrush.opacity,
1708     erase = activeStrokeErase,
1709 )) {
1710     publishState()
1711 }
1712 }
1713
1714 private fun computeCapsuleDirtyBounds(
1715     out: IntArray,
1716 ): Boolean {
1717     if (!capsuleEmitter.hasBounds) return false
1718
1719     strokeDirtyRect.clear()
1720     strokeDirtyRect.include(
1721         left = capsuleEmitter.minX,
1722         top = capsuleEmitter.minY,
1723         right = capsuleEmitter.maxX,
1724         bottom = capsuleEmitter.maxY,
1725     )
1726     strokeDirtyRect.expand(AA_MARGIN_PX)
1727     strokeDirtyRect.clamp(
1728         layerStack.canvasWidth.toFloat(),
1729         layerStack.canvasHeight.toFloat(),
1730     )
1731     strokeDirtyRect.toPixelBounds(out)
1732
1733     return out[2] > out[0] && out[3] > out[1]
1734 }
1735
1736 companion object {
1737     /** Bounded GL-thread work per frame; ordered input is never dropped. */
1738     private const val MAX_INPUT_BATCHES_PER_FRAME = 6
1739     /** Protects the render budget even when one backlog batch is expensive. */
1740     private const val MAX_INPUT_PROCESS_NANOS = 2_000_000L
1741     /** Emit a single aggregate timing record per second in debug builds. */
1742     private const val FRAME_LOG_INTERVAL_NANOS = 1_000_000_000L
1743     /** Per-MOVE Logcat I/O is expensive on debug tablet builds. */
1744     private const val LOG_STAMP_PREVIEW = false

```

```
1745     private const val BRUSH_DIAG_TAG = "BrushDiag"
1746     private const val INPUT_LATENCY_TAG = "InputLatency"
1747     private const val FRAME_DIAG_TAG = "WetInkFrame"
1748     private const val THUMBNAIL_TAG = "ThumbnailBuild"
1749     private const val AA_MARGIN_PX = 4f
1750     private const val STAMP_DIRTY_MARGIN_PX = 4f
1751 }
1752 }
```

54. app/src/main/java/com/wetinknext/engine/core/LatestValueDelivery.kt

```
1 package com.wetinknext.engine.core
2
3 import java.util.concurrent.atomic.AtomicReference
4
5 /**
6  * Retains the newest cross-thread value until a consumer is ready.
7  *
8  * GLSurfaceView can create EGL before Compose has installed its listener. A
9  * thumbnail is a replaceable snapshot, so retaining only the newest frame is
10  * both sufficient and bounded.
11  */
12 internal class LatestValueDelivery<T> {
13     private val pending = AtomicReference<T?>(null)
14
15     fun offer(value: T) {
16         pending.set(value)
17     }
18
19     fun dispatchTo(listener: ((T) -> Unit)? ) {
20         val callback = listener ?: return
21         pending.getAndSet(null)?.let(callback)
22     }
23
24     fun clear() {
25         pending.set(null)
26     }
27 }
```

55. app/src/main/java/com/wetinknext/engine/core/LayerTileCodec.kt

```

1 package com.wetinknext.engine.core
2
3 import android.opengl.GLES30
4 import com.wetinknext.engine.gl.RenderTarget
5 import com.wetinknext.engine.persistence.PersistentLayerTiles
6 import java.nio.ByteBuffer
7 import java.nio.ByteOrder
8
9 /**
10  * Stable payload for a complete layer texture. Empty data means a newly-created
11  * layer; later persistence can emit this format without changing the loader.
12  */
13 object LayerTileCodec {
14     private const val MAGIC = 0x5754494C // WTL
15     private const val VERSION = 1
16     private const val HEADER_BYTES = 20
17
18     data class Decoded(val width: Int, val height: Int, val bytesPerPixel: Int, val pixels: ByteArray)
19
20     fun encode(width: Int, height: Int, bytesPerPixel: Int, pixels: ByteArray): ByteArray {
21         require(width > 0 && height > 0 && bytesPerPixel in setOf(4, 8)) { "Invalid layer tile header" }
22         val expected = width.toLong() * height * bytesPerPixel
23         require(expected <= Int.MAX_VALUE && pixels.size == expected.toInt()) { "Invalid layer tile size" }
24         return ByteBuffer.allocate(HEADER_BYTES + pixels.size)
25             .order(ByteOrder.BIG_ENDIAN)
26             .putInt(MAGIC)
27             .putInt(VERSION)
28             .putInt(width)
29             .putInt(height)
30             .putInt(bytesPerPixel)
31             .put(pixels)
32             .array()
33     }
34
35     fun decode(bytes: ByteArray): Decoded? {
36         if (bytes.isEmpty()) return null
37         require(bytes.size >= HEADER_BYTES) { "Layer tile payload is truncated" }
38         val buffer = ByteBuffer.wrap(bytes).order(ByteOrder.BIG_ENDIAN)
39         require(buffer.int == MAGIC) { "Unknown layer tile payload" }
40         require(buffer.int == VERSION) { "Unsupported layer tile version" }
41         val width = buffer.int
42         val height = buffer.int
43         val bytesPerPixel = buffer.int
44         require(width > 0 && height > 0 && bytesPerPixel in setOf(4, 8)) { "Invalid layer tile header" }
45         val expected = width.toLong() * height * bytesPerPixel
46         require(expected <= Int.MAX_VALUE && bytes.size - HEADER_BYTES == expected.toInt()) { "Invalid layer tile size" }
47         return Decoded(width, height, bytesPerPixel, bytes.copyOfRange(HEADER_BYTES, bytes.size))
48     }
49
50     fun upload(target: RenderTarget, bytes: ByteArray) {
51         val decoded = decode(bytes) ?: return
52         require(decoded.width == target.width && decoded.height == target.height) { "Layer tile dimensions do not match
document" }
53         val expectedBpp = if (target.usesHalfFloat) 8 else 4
54         require(decoded.bytesPerPixel == expectedBpp) { "Layer tile pixel format does not match GPU target" }
55
56         val data = ByteBuffer.allocateDirect(decoded.pixels.size).order(ByteOrder.nativeOrder())
57         data.put(decoded.pixels).flip()
58         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
59         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, target.textureId)
60         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 1)
61         try {
62             GLES30.glTexSubImage2D(
63                 GLES30.GL_TEXTURE_2D, 0, 0, 0, decoded.width, decoded.height,
64                 GLES30.GL_RGBA,
65                 if (decoded.bytesPerPixel == 8) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE,
66                 data,
67             )
68             check(GLES30.glGetError() == GLES30.GL_NO_ERROR) { "Layer texture upload failed" }
69         } finally {
70             GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 4)
71             GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
72         }
73     }
74
75     fun uploadPersistent(target: RenderTarget, payload: ByteArray) {
76         if (payload.isEmpty()) return
77         val expectedBpp = if (target.usesHalfFloat) 8 else 4
78         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
79         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, target.textureId)
80         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 1)
81         try {
82             PersistentLayerTiles.decode(payload).forEach { tile ->
83                 require(tile.bytesPerPixel == expectedBpp && tile.pixelsLeft + tile.pixelsWidth <= target.width && tile.
pixelsTop + tile.pixelsHeight <= target.height) { "Invalid persistent tile" }
84                 val data = ByteBuffer.allocateDirect(tile.bytes.size).order(ByteOrder.nativeOrder())
85                 data.put(tile.bytes).flip()
86                 GLES30.glTexSubImage2D(GLES30.GL_TEXTURE_2D, 0, tile.pixelsLeft, tile.pixelsTop, tile.pixelsWidth, tile.
pixelsHeight, GLES30.GL_RGBA, if (expectedBpp == 8) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE, data)
87             }
88         }
89     }
90 }

```



```
88         check(GLES30.glGetError() == GLES30.GL_NO_ERROR) { "Persistent tile upload failed" }
89     } finally {
90         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 4)
91         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
92     }
93 }
94 }
```

56. app/src/main/java/com/wetinknext/engine/core/PaintEngine.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.canvas.LayerStack
4 import com.wetinknext.engine.gl.GlCaps
5 import com.wetinknext.engine.gl.RenderTarget
6
7 /** Lightweight document facade retained for engine callers outside the renderer. */
8 class PaintEngine {
9     val layerStack = LayerStack()
10    val camera: Camera get() = layerStack.camera
11    val canvasTarget: RenderTarget get() = checkNotNull(layerStack.activeLayer()).target
12
13    val canvasWidth: Int get() = layerStack.canvasWidth
14    val canvasHeight: Int get() = layerStack.canvasHeight
15    var initialized = false
16        private set
17
18    fun create(caps: GlCaps, width: Int, height: Int) {
19        layerStack.create(caps, width, height)
20        initialized = true
21    }
22
23    fun addLayer(name: String): Long = layerStack.addLayer(name).id
24
25    fun removeLayer(id: Long): Boolean = layerStack.removeLayer(id) != null
26
27    fun setActiveLayer(id: Long): Boolean = layerStack.setActive(id)
28
29    fun release() {
30        layerStack.release()
31        initialized = false
32    }
33 }
```

57. app/src/main/java/com/wetinknext/engine/core/PaintSurfaceView.kt

```

1  package com.wetinknext.engine.core
2
3  import android.content.Context
4  import android.opengl.GLSurfaceView
5  import android.util.AttributeSet
6  import android.view.MotionEvent
7  import com.wetinknext.domain.document.ProjectDocument
8  import com.wetinknext.engine.brush.BrushSettings
9  import com.wetinknext.engine.brush.TextureLoader
10 import com.wetinknext.engine.brush.LoadedBrushTexture
11 import com.wetinknext.engine.thumbnail.ThumbnailBuildResult
12 import com.wetinknext.engine.thumbnail.ThumbnailPreviewFiles
13 import java.io.File
14 import java.util.concurrent.ExecutorService
15 import java.util.concurrent.Executors
16
17 /** Thread boundary between Compose commands and the GL-owned paint engine. */
18 class PaintSurfaceView @JvmOverloads constructor(
19     context: Context,
20     attrs: AttributeSet? = null,
21     document: ProjectDocument = ProjectDocument.newUntitled(),
22     layerTiles: Map<Long, ByteArray> = emptyMap(),
23     layerPreviews: Map<Long, ByteArray> = emptyMap(),
24 ) : GLSurfaceView(context, attrs) {
25     private val thumbnailOutputDirectory = File(context.cacheDir, "wetink-thumbnails/${document.id}")
26     private val engineRenderer = EngineRenderer(
27         projectDocument = document,
28         initialLayerTiles = layerTiles,
29         thumbnailOutputDirectory = thumbnailOutputDirectory,
30     )
31
32     private val textureLoader = TextureLoader(context)
33     private val thumbnailRestoreExecutor: ExecutorService = Executors.newSingleThreadExecutor()
34     private var grainGeneration = 0L
35     private var shapeGeneration = 0L
36
37     var onTextureError: ((String, Throwable) -> Unit)? = null
38
39     var onEditorStateChange: ((EditorUiState) -> Unit)? = null
40     var onProjectDocumentChange: ((ProjectDocument) -> Unit)? = null
41     private var documentSessionLoaded = false
42     var onDocumentSessionLoaded: (() -> Unit)? = null
43     set(value) {
44         field = value
45         if (documentSessionLoaded && value != null) post(value)
46     }
47     var onDirtyLayerTiles: ((ProjectDocument, Map<Long, ByteArray>, Map<Long, Set<com.wetinknext.engine.undo.TileCoord>)) ->
Unit)? = null
48     private val pendingThumbnailCapture = LatestValueDelivery<ThumbnailCapture.Rgba>()
49     var onThumbnailCaptured: ((ThumbnailCapture.Rgba) -> Unit)? = null
50     set(value) {
51         field = value
52         if (value != null) {
53             // setRenderer() may initialize EGL before AndroidView's caller
54             // has installed its listener. Replay the latest capture then.
55             post(::dispatchPendingThumbnail)
56         }
57     }
58     /** File previews that were encoded on the worker and published to cache. */
59     var onThumbnailBuildSaved: ((ThumbnailBuildResult) -> Unit)? = null
60
61     init {
62         setEGLContextClientVersion(3)
63         setRenderer(engineRenderer)
64         renderMode = RENDERMODE_WHEN_DIRTY
65         engineRenderer.onStateChange = { state ->
66             post { onEditorStateChange?.invoke(state) }
67             requestRender()
68         }
69         engineRenderer.onProjectDocumentChange = { document ->
70             post { onProjectDocumentChange?.invoke(document) }
71         }
72         engineRenderer.onDocumentSessionLoaded = {
73             documentSessionLoaded = true
74             post { onDocumentSessionLoaded?.invoke() }
75         }
76         engineRenderer.onDirtyLayerTiles = { document, payloads, dirty -> post { onDirtyLayerTiles?.invoke(document, payloads,
dirty) } }
77         engineRenderer.onThumbnailCaptured = { image ->
78             pendingThumbnailCapture.offer(image)
79             post(::dispatchPendingThumbnail)
80         }
81         engineRenderer.onThumbnailBuildSaved = { result ->
82             // The worker has published files, but document metadata and layer
83             // UI state remain GL-owned. Update those first, then notify the
84             // app-level persistence bridge on the main thread.
85             post {
86                 queueEvent {
87                     engineRenderer.applyThumbnailBuild(result)
88                     requestRender()

```

```

89         post { onThumbnailBuildSaved?.invoke(result) }
90     }
91 }
92 }
93 engineRenderer.onInputRenderRequested = { requestRender() }
94 restoreStoredLayerPreviews(layerPreviews)
95 }
96
97 override fun onTouchEvent(event: MotionEvent): Boolean {
98     val handled = engineRenderer.onTouchEvent(event)
99     if (handled) requestRender()
100     return handled || super.onTouchEvent(event)
101 }
102
103 fun requestState() = queueEvent {
104     engineRenderer.requestState()
105     requestRender()
106 }
107
108 fun undo() = queueEvent {
109     engineRenderer.undo()
110     requestRender()
111 }
112
113 fun redo() = queueEvent {
114     engineRenderer.redo()
115     requestRender()
116 }
117
118 fun acknowledgeSavedTiles(saved: Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>) = queueEvent {
119     engineRenderer.acknowledgeSavedTiles(saved)
120 }
121
122 fun setActiveLayer(id: Long) = queueEvent {
123     engineRenderer.setActiveLayer(id)
124     requestRender()
125 }
126
127 fun addLayer() = queueEvent {
128     engineRenderer.addLayer()
129     requestRender()
130 }
131
132 fun duplicateLayer(id: Long) = queueEvent {
133     engineRenderer.duplicateLayer(id)
134     requestRender()
135 }
136
137 fun removeLayer(id: Long) = queueEvent {
138     engineRenderer.removeLayer(id)
139     requestRender()
140 }
141
142 fun moveLayer(id: Long, delta: Int) = queueEvent {
143     engineRenderer.moveLayer(id, delta)
144     requestRender()
145 }
146
147 fun setLayerVisible(id: Long, visible: Boolean) =
148     queueEvent {
149         engineRenderer.setLayerVisible(id, visible)
150         requestRender()
151     }
152
153 fun setLayerOpacity(id: Long, opacity: Float) =
154     queueEvent {
155         engineRenderer.setLayerOpacity(id, opacity)
156         requestRender()
157     }
158
159 fun clearActiveLayer() = queueEvent {
160     engineRenderer.clearActiveLayer()
161     requestRender()
162 }
163
164 fun toggleCanvasMirror() = queueEvent {
165     engineRenderer.toggleCanvasMirror()
166     requestRender()
167 }
168
169 fun setCanvasBackdrop(backdropArgb: Int, gridArgb: Int, mode: CanvasBackdropMode) = queueEvent {
170     engineRenderer.setCanvasBackdrop(backdropArgb, gridArgb, mode)
171     requestRender()
172 }
173
174 fun setBrushSize(px: Float) = queueEvent {
175     engineRenderer.setBrushSize(px)
176     requestRender()
177 }
178
179 fun setBrushOpacity(opacity: Float) = queueEvent {
180     engineRenderer.setBrushOpacity(opacity)

```

```

181         requestRender()
182     }
183
184     fun setEraserEnabled(enabled: Boolean) = queueEvent {
185         engineRenderer.setEraserEnabled(enabled)
186         requestRender()
187     }
188
189     fun applyBrush(settings: BrushSettings) = queueEvent {
190         engineRenderer.applyBrush(settings)
191         requestRender()
192     }
193
194     fun setBrushSettings(settings: BrushSettings) = queueEvent {
195         engineRenderer.setBrushSettings(settings)
196         requestRender()
197     }
198
199     fun setBrushColor(color: androidx.compose.ui.graphics.Color) = queueEvent {
200         engineRenderer.setBrushColor(color)
201         requestRender()
202     }
203
204     fun loadGrainTexture(
205         path: String,
206         scale: Float = 1f,
207         canvasLocked: Boolean = true,
208         depth: Float = 1f,
209         contrast: Float = 1f,
210     ) {
211         val generation = ++grainGeneration
212
213         textureLoader.loadAsync(
214             path = path,
215             onLoad = { loaded ->
216                 queueEvent {
217                     if (generation != grainGeneration) {
218                         return@queueEvent
219                     }
220
221                     engineRenderer.applyLoadedGrain(
222                         loaded = loaded,
223                         scale = scale,
224                         canvasLocked = canvasLocked,
225                         depth = depth,
226                         contrast = contrast,
227                     )
228                     requestRender()
229                 }
230             },
231             onError = { error ->
232                 post {
233                     onTextureError?.invoke(path, error)
234                 }
235             },
236         )
237     }
238
239     fun clearGrainTexture() {
240         grainGeneration++
241
242         queueEvent {
243             engineRenderer.clearGrainTexture()
244             requestRender()
245         }
246     }
247
248     fun loadShapeTexture(
249         path: String,
250         reverse: Boolean = false,
251         rgbToAlpha: Boolean = false,
252     ) {
253         val generation = ++shapeGeneration
254         textureLoader.loadAsync(
255             path = path,
256             onLoad = { loaded ->
257                 queueEvent {
258                     if (generation != shapeGeneration) return@queueEvent
259                     engineRenderer.applyLoadedShape(loaded, reverse, rgbToAlpha)
260                     requestRender()
261                 }
262             },
263             onError = { error -> post { onTextureError?.invoke(path, error) } },
264         )
265     }
266
267     fun clearShapeTexture() {
268         shapeGeneration++
269         queueEvent {
270             engineRenderer.clearShapeTexture()
271             requestRender()
272         }
273     }

```

```

273     }
274
275     private fun dispatchPendingThumbnail() {
276         pendingThumbnailCapture.dispatchTo(onThumbnailCaptured)
277     }
278
279     /** Restores persisted WebP bytes off the main and GL threads. */
280     private fun restoreStoredLayerPreviews(layerPreviews: Map<Long, ByteArray>) {
281         if (layerPreviews.isEmpty()) return
282         thumbnailRestoreExecutor.execute {
283             val files = runCatching {
284                 ThumbnailPreviewFiles.restoreLayerPreviews(
285                     outputDirectory = thumbnailOutputDirectory,
286                     previews = layerPreviews,
287                 )
288             }.getOrElse { error ->
289                 post { onTextureError?.invoke("layer previews", error) }
290                 return@execute
291             }
292             queueEvent {
293                 engineRendererer.installRestoredLayerThumbnails(files)
294                 requestRender()
295             }
296         }
297     }
298
299     override fun onPause() {
300         engineRendererer.cancelActiveStroke()
301         super.onPause()
302     }
303
304     override fun onDetachedFromWindow() {
305         grainGeneration++
306         shapeGeneration++
307         textureLoader.shutdown()
308         thumbnailRestoreExecutor.shutdownNow()
309
310         // Очищаем listeners, чтобы избежать утечек или вызовов в пустоту
311         onTextureError = null
312         onEditorStateChange = null
313         onProjectDocumentChange = null
314         onDocumentSessionLoaded = null
315         onDirtyLayerTiles = null
316         onThumbnailCaptured = null
317         onThumbnailBuildSaved = null
318         pendingThumbnailCapture.clear()
319         engineRendererer.onStateChange = null
320         engineRendererer.onProjectDocumentChange = null
321         engineRendererer.onDocumentSessionLoaded = null
322         engineRendererer.onDirtyLayerTiles = null
323         engineRendererer.onThumbnailCaptured = null
324         engineRendererer.onThumbnailBuildSaved = null
325         engineRendererer.onInputRenderRequested = null
326         engineRendererer.setOnSecondaryPointerDown { }
327
328         // Пытаемся освободить GL-объекты, но не рассчитываем на 100% успех здесь
329         queueEvent {
330             // Отменяем активный штрих перед освобождением
331             engineRendererer.cancelActiveStroke()
332             engineRendererer.releaseGLObjects()
333         }
334         super.onDetachedFromWindow()
335     }
336 }

```

58. app/src/main/java/com/wetinknext/engine/core/StrokeCommitter.kt

```

1  package com.wetinknext.engine.core
2
3  import android.opengl.GLES30
4  import android.util.Log
5  import com.wetinknext.BuildConfig
6  import com.wetinknext.engine.canvas.PaintLayer
7  import com.wetinknext.engine.canvas.NonBuildupStrokeRenderer
8  import com.wetinknext.engine.canvas.StrokeBlitter
9  import com.wetinknext.engine.gl.CanvasGeometry
10 import com.wetinknext.engine.gl.GlCheck
11 import com.wetinknext.engine.gl.RenderTarget
12 import com.wetinknext.engine.undo.TileSnapshot
13 import com.wetinknext.engine.undo.TileSnapshotCapture
14 import com.wetinknext.engine.undo.UndoManager
15 import com.wetinknext.engine.undo.UndoOperationType
16 import com.wetinknext.engine.undo.RawTileSnapshot
17
18 /**
19  * GL-thread transaction boundary for a prepared stroke.
20  *
21  * The caller renders a normal stroke into [strokeTarget]. This class then
22  * snapshots the affected layer tiles, performs the single premultiplied blit,
23  * captures the after-state, and sends the reversible transaction to history.
24  */
25 class StrokeCommitter(
26     private val strokeTarget: RenderTarget,
27     private val canvasToFbo: FloatArray,
28     private val undoPipeline: UndoCompressionPipeline,
29     private val undoManager: UndoManager,
30     private val onTilesCommitted: (PaintLayer, List<RawTileSnapshot>) -> Unit = { _, _ -> },
31     private val onLayerCleared: (PaintLayer) -> Unit = {},
32 ) {
33     /**
34      * Commits the already-rendered stroke target into [layer]. [dirtyBounds]
35      * is in canvas coordinates and is expanded to a deterministic tile set.
36      */
37     fun commit(
38         layer: PaintLayer,
39         geometry: CanvasGeometry,
40         blitter: StrokeBlitter,
41         dirtyBounds: IntArray,
42         canvasWidth: Int,
43         canvasHeight: Int,
44         opacity: Float,
45         erase: Boolean = false,
46         operation: UndoOperationType = UndoOperationType.TILE_EDIT,
47         tag: String = "stroke",
48     ): Boolean {
49         GlCheck.checkOnGUIThread()
50         if (!layer.created || layer.isLocked || strokeTarget.textureId == 0) return false
51         if (dirtyBounds.size < 4) return false
52
53         expandBoundsToTiles(dirtyBounds, canvasWidth, canvasHeight)
54         if (dirtyBounds[2] <= dirtyBounds[0] || dirtyBounds[3] <= dirtyBounds[1]) return false
55         logLargeUndoReadback(layer, dirtyBounds)
56
57         val beforeRaw = TileSnapshotCapture.capture(layer.target, dirtyBounds)
58         blitter.blit(
59             layer = layer.target,
60             geometry = geometry,
61             strokeTextureId = strokeTarget.textureId,
62             canvasToFbo = canvasToFbo,
63             width = canvasWidth,
64             height = canvasHeight,
65             opacity = opacity.coerceIn(0f, 1f),
66             erase = erase,
67         )
68         layer.version++
69         val afterRaw = TileSnapshotCapture.capture(layer.target, dirtyBounds)
70         undoPipeline.enqueue(
71             undoManager = undoManager,
72             layerId = layer.id,
73             beforeRaw = beforeRaw,
74             afterRaw = afterRaw,
75             operation = operation,
76             tag = tag,
77         )
78         onTilesCommitted(layer, afterRaw)
79         return true
80     }
81
82     /** Commits a STAMP_NON_BUILDUP stroke from its union coverage target. */
83     fun commitNonBuildup(
84         layer: PaintLayer,
85         geometry: CanvasGeometry,
86         blitter: NonBuildupStrokeRenderer,
87         coverageTarget: RenderTarget,
88         colorLinear: FloatArray,
89         dirtyBounds: IntArray,
90         canvasWidth: Int,

```

```

91         canvasHeight: Int,
92         opacity: Float,
93         erase: Boolean = false,
94     ): Boolean {
95         GLCheck.checkOnGLThread()
96         if (!layer.created || layer.isLocked || coverageTarget.textureId == 0 || colorLinear.size < 3) return false
97         if (dirtyBounds.size < 4) return false
98
99         expandBoundsToTiles(dirtyBounds, canvasWidth, canvasHeight)
100         if (dirtyBounds[2] <= dirtyBounds[0] || dirtyBounds[3] <= dirtyBounds[1]) return false
101         logLargeUndoReadback(layer, dirtyBounds)
102
103         val beforeRaw = TileSnapshotCapture.capture(layer.target, dirtyBounds)
104         blitter.blit(
105             layer = layer.target,
106             geometry = geometry,
107             coverageTextureId = coverageTarget.textureId,
108             colorLinear = colorLinear,
109             canvasToFbo = canvasToFbo,
110             width = canvasWidth,
111             height = canvasHeight,
112             opacity = opacity,
113             erase = erase,
114         )
115         layer.version++
116         val afterRaw = TileSnapshotCapture.capture(layer.target, dirtyBounds)
117         undoPipeline.enqueue(
118             undoManager = undoManager,
119             layerId = layer.id,
120             beforeRaw = beforeRaw,
121             afterRaw = afterRaw,
122         )
123         onTilesCommitted(layer, afterRaw)
124         return true
125     }
126
127     /** Clears one editable layer and records the full canvas as one undo transaction. */
128     fun clearLayer(
129         layer: PaintLayer,
130         canvasWidth: Int,
131         canvasHeight: Int,
132     ): Boolean {
133         GLCheck.checkOnGLThread()
134         if (!layer.created || layer.isLocked) return false
135
136         val fullBounds = intArrayOf(0, 0, canvasWidth, canvasHeight)
137         logLargeUndoReadback(layer, fullBounds)
138         val beforeRaw = TileSnapshotCapture.capture(layer.target, fullBounds)
139         layer.clear()
140         layer.version++
141         undoPipeline.enqueue(
142             undoManager = undoManager,
143             layerId = layer.id,
144             beforeRaw = beforeRaw,
145             // Redo clears the target directly. Keeping a full transparent copy would
146             // double memory for a large document and can exhaust the Java heap.
147             afterRaw = emptyList(),
148             operation = UndoOperationType.CLEAR_LAYER,
149             tag = "clear_layer",
150         )
151         onLayerCleared(layer)
152         return true
153     }
154
155     private fun expandBoundsToTiles(
156         bounds: IntArray,
157         canvasWidth: Int,
158         canvasHeight: Int,
159     ) {
160         val tileSize = TileSnapshot.TILE_SIZE
161         bounds[0] = (bounds[0] / tileSize) * tileSize
162         bounds[1] = (bounds[1] / tileSize) * tileSize
163         bounds[2] = ((bounds[2] + tileSize - 1) / tileSize) * tileSize
164         bounds[3] = ((bounds[3] + tileSize - 1) / tileSize) * tileSize
165         bounds[0] = bounds[0].coerceIn(0, canvasWidth)
166         bounds[1] = bounds[1].coerceIn(0, canvasHeight)
167         bounds[2] = bounds[2].coerceIn(0, canvasWidth)
168         bounds[3] = bounds[3].coerceIn(0, canvasHeight)
169     }
170
171     private fun logLargeUndoReadback(
172         layer: PaintLayer,
173         bounds: IntArray,
174     ) {
175         if (!BuildConfig.DEBUG) return
176
177         val tileCount = TileSnapshotCapture.countTiles(layer.target, bounds)
178         if (tileCount > MAX_UNDO_TILES_PER_OPERATION) {
179             Log.w(
180                 TAG,
181                 "Large commit: tiles=$tileCount layer=${layer.id} " +
182                 "bounds=${bounds.contentToString()}",

```



```
183         )
184     }
185 }
186
187 private companion object {
188     const val TAG = "StrokeCommitter"
189     const val MAX_UNDO_TILES_PER_OPERATION = 64
190 }
191 }
```

59. app/src/main/java/com/wetinknext/engine/core/ThumbnailCapture.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.canvas.LayerStack
4 import com.wetinknext.engine.gl.GlCheck
5 import com.wetinknext.engine.thumbnail.ThumbnailRenderer
6
7 /** Captures only the composited canvas; Compose panels and system UI are excluded. */
8 class ThumbnailCapture {
9     data class Rgba(val width: Int, val height: Int, val pixels: ByteArray)
10     private val renderer = ThumbnailRenderer()
11
12     fun capture(layers: LayerStack): Rgba {
13         GlCheck.checkOnGhread()
14         require(layers.canvasWidth > 0 && layers.canvasHeight > 0) { "Cannot capture an empty canvas" }
15         val longestEdge = 384
16         val scale = longestEdge.toFloat() / maxOf(layers.canvasWidth, layers.canvasHeight).toFloat()
17         val captureWidth = (layers.canvasWidth * scale).toInt().coerceAtLeast(1)
18         val captureHeight = (layers.canvasHeight * scale).toInt().coerceAtLeast(1)
19         return Rgba(
20             captureWidth,
21             captureHeight,
22             renderer.renderProject(layers, captureWidth, captureHeight),
23         )
24     }
25
26     fun release() = renderer.release()
27 }
```

60. app/src/main/java/com/wetinknext/engine/core/UndoCompressionPipeline.kt

```

1  package com.wetinknext.engine.core
2
3  import android.util.Log
4  import com.wetinknext.BuildConfig
5  import com.wetinknext.engine.undo.DeflateTileCompressor
6  import com.wetinknext.engine.undo.RawTileSnapshot
7  import com.wetinknext.engine.undo.TileCompressor
8  import com.wetinknext.engine.undo.UndoEntry
9  import com.wetinknext.engine.undo.UndoManager
10 import com.wetinknext.engine.undo.UndoOperationType
11 import java.util.HashMap
12 import java.util.concurrent.ConcurrentLinkedQueue
13 import java.util.concurrent.ExecutorService
14 import java.util.concurrent.Executors
15 import java.util.concurrent.RejectedExecutionException
16 import java.util.concurrent.TimeUnit
17
18 /**
19  * Orders tile-compression work without ever touching GL from a worker thread.
20  *
21  * Call [enqueue], [process], [invalidate], and [shutdown] on the GL thread.
22  * The worker only converts [RawTileSnapshot] objects into compressed [UndoEntry]
23  * instances and requests another frame once a result is ready.
24  */
25 class UndoCompressionPipeline(
26     private val requestRender: () -> Unit,
27     private val compressor: TileCompressor = DeflateTileCompressor(),
28     private val maxPendingJobs: Int = DEFAULT_MAX_PENDING_JOBS,
29     private val executorFactory: () -> ExecutorService = { Executors.newSingleThreadExecutor() },
30 ) {
31     private var executor = executorFactory()
32     private val pendingResults = ConcurrentLinkedQueue<UndoJobResult>()
33     private val completedResults = HashMap<Long, UndoJobResult>()
34
35     private var nextSequence = 0L
36     private var nextSequenceToApply = 0L
37     private var epoch = 0L
38
39     var pendingCount: Int = 0
40     private set
41     var compressionFailureCount: Int = 0
42     private set
43     var staleResultCount: Int = 0
44     private set
45
46     /**
47      * Schedules one ordered undo transaction. The synchronous fallback still
48      * travels through the same result queue, so it cannot overtake older work.
49      */
50     fun enqueue(
51         undoManager: UndoManager,
52         layerId: Long,
53         beforeRaw: List<RawTileSnapshot>,
54         afterRaw: List<RawTileSnapshot>,
55         operation: UndoOperationType = UndoOperationType.TILE_EDIT,
56         tag: String = "stroke",
57     ) {
58         val jobEpoch = epoch
59         val sequence = allocateSequence()
60         pendingCount++
61
62         if (pendingCount >= maxPendingJobs) {
63             compressSynchronously(
64                 epoch = jobEpoch,
65                 sequence = sequence,
66                 layerId = layerId,
67                 beforeRaw = beforeRaw,
68                 afterRaw = afterRaw,
69                 operation = operation,
70                 tag = tag,
71             )
72             process(undoManager)
73             return
74         }
75
76         submit(
77             epoch = jobEpoch,
78             sequence = sequence,
79             layerId = layerId,
80             beforeRaw = beforeRaw,
81             afterRaw = afterRaw,
82             operation = operation,
83             tag = tag,
84         )
85     }
86
87     /** Applies completed work in commit order. Must be called on the GL thread. */
88     fun process(undoManager: UndoManager): Boolean {
89         var changed = false
90

```

```

91         while (true) {
92             val result = pendingResults.poll() ?: break
93             if (result.sequence < nextSequenceToApply) {
94                 discard(result)
95                 decrementPending()
96                 changed = true
97             } else {
98                 completedResults.put(result.sequence, result)?.let(::disposeReady)
99             }
100         }
101
102         while (true) {
103             val result = completedResults.remove(nextSequenceToApply) ?: break
104             apply(result, undoManager)
105             decrementPending()
106             nextSequenceToApply++
107             changed = true
108         }
109
110         return changed
111     }
112
113     /** Rejects compression results that belong to a document state that no longer exists. */
114     fun invalidate() {
115         epoch++
116         nextSequenceToApply = nextSequence
117         val discardedCount = completedResults.size
118         completedResults.values.forEach(::disposeReady)
119         completedResults.clear()
120         pendingCount = (pendingCount - discardedCount).coerceAtLeast(0)
121         staleResultCount += discardedCount
122     }
123
124     /** Reopens the worker after an EGL/context recreation. */
125     fun ensureRunning() {
126         if (executor.isShutdown) executor = executorFactory()
127     }
128
129     /** Releases worker-owned results without changing the caller's [UndoManager]. */
130     fun shutdown() {
131         executor.shutdownNow()
132         try {
133             executor.awaitTermination(SHUTDOWN_TIMEOUT_MS, TimeUnit.MILLISECONDS)
134         } catch (error: InterruptedException) {
135             Thread.currentThread().interrupt()
136         }
137
138         while (true) {
139             val result = pendingResults.poll() ?: break
140             disposeReady(result)
141         }
142         completedResults.values.forEach(::disposeReady)
143         completedResults.clear()
144         pendingCount = 0
145         nextSequenceToApply = nextSequence
146     }
147
148     private fun allocateSequence(): Long = nextSequence++
149
150     private fun submit(
151         epoch: Long,
152         sequence: Long,
153         layerId: Long,
154         beforeRaw: List<RawTileSnapshot>,
155         afterRaw: List<RawTileSnapshot>,
156         operation: UndoOperationType,
157         tag: String,
158     ) {
159         try {
160             executor.execute {
161                 val result = runCatching {
162                     readyResult(
163                         epoch = epoch,
164                         sequence = sequence,
165                         layerId = layerId,
166                         beforeRaw = beforeRaw,
167                         afterRaw = afterRaw,
168                         operation = operation,
169                         tag = tag,
170                     )
171                 }.getOrElse { error ->
172                     UndoJobResult.Failed(epoch = epoch, sequence = sequence, error = error)
173                 }
174                 pendingResults.add(result)
175                 requestRender()
176             }
177         } catch (error: RejectedExecutionException) {
178             pendingResults.add(
179                 UndoJobResult.Failed(epoch = epoch, sequence = sequence, error = error),
180             )
181             requestRender()
182         }

```

```

183     }
184
185     private fun compressSynchronously(
186         epoch: Long,
187         sequence: Long,
188         layerId: Long,
189         beforeRaw: List<RawTileSnapshot>,
190         afterRaw: List<RawTileSnapshot>,
191         operation: UndoOperationType,
192         tag: String,
193     ) {
194         val result = runCatching {
195             readyResult(
196                 epoch = epoch,
197                 sequence = sequence,
198                 layerId = layerId,
199                 beforeRaw = beforeRaw,
200                 afterRaw = afterRaw,
201                 operation = operation,
202                 tag = tag,
203             )
204         }.getOrElse { error ->
205             UndoJobResult.Failed(epoch = epoch, sequence = sequence, error = error)
206         }
207         pendingResults.add(result)
208     }
209
210     private fun readyResult(
211         epoch: Long,
212         sequence: Long,
213         layerId: Long,
214         beforeRaw: List<RawTileSnapshot>,
215         afterRaw: List<RawTileSnapshot>,
216         operation: UndoOperationType,
217         tag: String,
218     ): UndoJobResult.Ready = UndoJobResult.Ready(
219         epoch = epoch,
220         sequence = sequence,
221         entry = UndoEntry(
222             layerId = layerId,
223             beforeTiles = beforeRaw.map { it.compress(compressor) },
224             afterTiles = afterRaw.map { it.compress(compressor) },
225             operation = operation,
226             tag = tag,
227         ),
228     )
229
230     private fun apply(result: UndoJobResult, undoManager: UndoManager) {
231         when (result) {
232             is UndoJobResult.Ready -> {
233                 if (isUndoResultCurrent(result.epoch, epoch)) {
234                     undoManager.push(result.entry)
235                 } else {
236                     result.entry.dispose()
237                     staleResultCount++
238                 }
239             }
240             is UndoJobResult.Failed -> {
241                 compressionFailureCount++
242                 if (BuildConfig.DEBUG) {
243                     Log.e(TAG, "Undo compression failed", result.error)
244                 }
245             }
246         }
247     }
248 }
249
250 private fun discard(result: UndoJobResult) {
251     disposeReady(result)
252     staleResultCount++
253 }
254
255 private fun disposeReady(result: UndoJobResult) {
256     if (result is UndoJobResult.Ready) result.entry.dispose()
257 }
258
259 private fun decrementPending() {
260     pendingCount = (pendingCount - 1).coerceAtLeast(0)
261 }
262
263 private companion object {
264     const val DEFAULT_MAX_PENDING_JOBS = 8
265     const val SHUTDOWN_TIMEOUT_MS = 100L
266     const val TAG = "TileUndo"
267 }
268 }

```

61. app/src/main/java/com/wetinknext/engine/core/UndoHistory.kt

```
1 package com.wetinknext.engine.core
2
3 /** Pure epoch contract used to reject undo results from a previous document state. */
4 fun isUndoResultCurrent(
5     resultEpoch: Long,
6     currentEpoch: Long,
7 ): Boolean = resultEpoch == currentEpoch
```

62. app/src/main/java/com/wetinknext/engine/core/UndoJobResult.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.undo.UndoEntry
4
5 /** Terminal outcome of one tile-compression task. */
6 sealed interface UndoJobResult {
7     val epoch: Long
8     val sequence: Long
9
10     data class Ready(
11         override val epoch: Long,
12         override val sequence: Long,
13         val entry: UndoEntry,
14     ) : UndoJobResult
15
16     data class Failed(
17         override val epoch: Long,
18         override val sequence: Long,
19         val error: Throwable,
20     ) : UndoJobResult
21 }
```

63. app/src/main/java/com/wetinknext/engine/core/ViewTransform.kt

```

1 package com.wetinknext.engine.core
2
3 import kotlin.math.abs
4 import kotlin.math.cos
5 import kotlin.math.sin
6
7 /** Canonical canvas-to-view-local-screen transformation. */
8 data class ViewTransform(
9     val translateX: Float = 0f,
10    val translateY: Float = 0f,
11    val scale: Float = 1f,
12    val rotationRad: Float = 0f,
13    val flipX: Boolean = false,
14 ) {
15     private val flipSign: Float
16     get() = if (flipX) -1f else 1f
17
18     val m00: Float get() = cos(rotationRad) * scale * flipSign
19     val m01: Float get() = -sin(rotationRad) * scale
20     val m10: Float get() = sin(rotationRad) * scale * flipSign
21     val m11: Float get() = cos(rotationRad) * scale
22
23     fun canvasToScreen(canvasX: Float, canvasY: Float, out: FloatArray) {
24         require(out.size >= 2)
25         out[0] = m00 * canvasX + m01 * canvasY + translateX
26         out[1] = m10 * canvasX + m11 * canvasY + translateY
27     }
28
29     fun screenToCanvas(screenX: Float, screenY: Float, out: FloatArray) {
30         require(out.size >= 2)
31         val determinant = m00 * m11 - m01 * m10
32         if (abs(determinant) < 1e-8f) {
33             out[0] = 0f
34             out[1] = 0f
35             return
36         }
37         val inverseDeterminant = 1f / determinant
38         val dx = screenX - translateX
39         val dy = screenY - translateY
40         out[0] = (m11 * dx - m01 * dy) * inverseDeterminant
41         out[1] = (-m10 * dx + m00 * dy) * inverseDeterminant
42     }
43
44     fun zoomAround(anchorX: Float, anchorY: Float, newScale: Float): ViewTransform {
45         val safeScale = newScale.coerceIn(MIN_SCALE, MAX_SCALE)
46         val factor = safeScale / scale
47         return copy(
48             scale = safeScale,
49             translateX = anchorX - (anchorX - translateX) * factor,
50             translateY = anchorY - (anchorY - translateY) * factor,
51         )
52     }
53
54     fun rotateAround(anchorX: Float, anchorY: Float, deltaRad: Float): ViewTransform {
55         val c = cos(deltaRad)
56         val s = sin(deltaRad)
57         val dx = translateX - anchorX
58         val dy = translateY - anchorY
59         return copy(
60             rotationRad = rotationRad + deltaRad,
61             translateX = anchorX + dx * c - dy * s,
62             translateY = anchorY + dx * s + dy * c,
63         )
64     }
65
66     /** Applies one screen-space navigation update around a shared gesture anchor. */
67     fun applyGestureDelta(
68         panX: Float,
69         panY: Float,
70         anchorX: Float,
71         anchorY: Float,
72         zoomFactor: Float,
73         rotationDelta: Float,
74     ): ViewTransform {
75         val translated = copy(
76             translateX = translateX + panX,
77             translateY = translateY + panY,
78         )
79         val zoomed = translated.zoomAround(
80             anchorX = anchorX,
81             anchorY = anchorY,
82             newScale = scale * zoomFactor,
83         )
84         return zoomed.rotateAround(anchorX, anchorY, rotationDelta)
85     }
86
87     /** Builds a column-major matrix for canvas to OpenGL clip coordinates. */
88     fun buildCanvasToClip(viewWidth: Float, viewHeight: Float, out: FloatArray) {
89         require(out.size >= 16)
90         require(viewWidth > 0f)

```



```

91         require(viewHeight > 0f)
92         val sx = 2f / viewWidth
93         val sy = -2f / viewHeight
94         out[0] = m00 * sx; out[1] = m10 * sy; out[2] = 0f; out[3] = 0f
95         out[4] = m01 * sx; out[5] = m11 * sy; out[6] = 0f; out[7] = 0f
96         out[8] = 0f; out[9] = 0f; out[10] = 1f; out[11] = 0f
97         out[12] = translateX * sx - 1f; out[13] = translateY * sy + 1f
98         out[14] = 0f; out[15] = 1f
99     }
100
101     companion object {
102         const val MIN_SCALE = 0.02f
103         const val MAX_SCALE = 64f
104
105         fun buildCanvasToFbo(canvasWidth: Float, canvasHeight: Float, out: FloatArray) {
106             require(out.size >= 16); require(canvasWidth > 0f); require(canvasHeight > 0f)
107             out[0]=2f/canvasWidth;out[1]=0f;out[2]=0f;out[3]=0f;out[4]=0f;out[5]=2f/canvasHeight;out[6]=0f;out[7]=0f;out[
8]=0f;out[9]=0f;out[10]=1f;out[11]=0f;out[12]=-1f;out[13]=-1f;out[14]=0f;out[15]=1f
108         }
109     }
110 }

```

64. app/src/main/java/com/wetinknext/engine/gl/CanvasGeometry.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4 import java.nio.ByteBuffer
5 import java.nio.ByteOrder
6
7 /** A document-sized quad; GL resources are owned by the render thread. */
8 class CanvasGeometry {
9     var vaoId = 0
10     private set
11     private var vboId = 0
12
13     fun create(width: Int, height: Int) {
14         release()
15         val vertices = floatArrayOf(0f, 0f, width.toFloat(), 0f, 0f, height.toFloat(), width.toFloat(), height.toFloat())
16         val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
17         data.put(vertices).position(0)
18         val ids = IntArray(1)
19         GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
20         GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
21         GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
22         GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
23         GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
24         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
25     }
26
27     fun draw() {
28         check(vaoId != 0)
29         GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.GL_TRIANGLE_STRIP, 0, 4); GLES30.glBindVertexArray(0)
30     }
31
32     fun release() {
33         if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0)
34         if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0)
35         vaoId = 0; vboId = 0
36     }
37 }
```

65. app/src/main/java/com/wetinknext/engine/gl/GeometryCache.kt

```

1  package com.wetinknext.engine.gl
2
3  import android.opengl.GLES30
4  import java.nio.ByteBuffer
5  import java.nio.ByteOrder
6
7  /** A single cached fullscreen triangle for future present passes. */
8  class GeometryCache {
9      private var vaoId = 0
10     private var vboId = 0
11     fun create() {
12         if (vaoId != 0) return
13         val vertices = floatArrayOf(-1f, -1f, 3f, -1f, -1f, 3f)
14         val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
15         data.put(vertices).position(0)
16         val ids = IntArray(1); GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
17         GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
18         GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
19         GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
20         GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
21         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
22     }
23     fun drawFullscreenTriangle() { check(vaoId != 0); GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.
GL_TRIANGLES, 0, 3); GLES30.glBindVertexArray(0) }
24     fun release() { if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0); if (vaoId != 0) GLES30.
glDeleteVertexArrays(1, intArrayOf(vaoId), 0); vboId = 0; vaoId = 0 }
25 }

```

66. app/src/main/java/com/wetinknext/engine/gl/GlCaps.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4
5 /** Capabilities queried only after a GLES 3 context is current. */
6 data class GlCaps(
7     val renderer: String,
8     val version: String,
9     val supportsHalfFloatColorBuffer: Boolean,
10 ) {
11     companion object {
12         fun query(): GlCaps {
13             val extensions = GLES30.glGetString(GLES30.GL_EXTENSIONS).orEmpty()
14             val declared = extensions.contains("EXT_color_buffer_half_float") ||
15                 extensions.contains("EXT_color_buffer_float")
16             // Extension string сам по себе не доказательство: проверяем реальный FBO 4x4.
17             val supported = declared && RenderTarget().probeHalfFloatColorBuffer()
18             return GlCaps(
19                 renderer = GLES30.glGetString(GLES30.GL_RENDERER).orEmpty(),
20                 version = GLES30.glGetString(GLES30.GL_VERSION).orEmpty(),
21                 supportsHalfFloatColorBuffer = supported,
22             )
23         }
24     }
25 }
```

67. app/src/main/java/com/wetinknext/engine/gl/GLCheck.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4
5 /** GL error checks intended for setup boundaries, never the render hot path. */
6 object GLCheck {
7     @Volatile
8     private var glThread: Thread? = null
9
10    fun setGLThread(thread: Thread) {
11        glThread = thread
12    }
13
14    fun checkOnGLThread() {
15        val current = Thread.currentThread()
16        val expected = glThread
17        check(current === expected) {
18            "GL resource accessed outside GLSurfaceView render thread: current=$current, expected=$expected"
19        }
20    }
21
22    fun noError(label: String) {
23        val error = GLES30.glGetError()
24        check(error == GLES30.GL_NO_ERROR) { "$label: GL error 0x${error.toString(16)}" }
25    }
26
27    fun framebufferComplete(label: String) {
28        val status = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER)
29        check(status == GLES30.GL_FRAMEBUFFER_COMPLETE) {
30            "$label: framebuffer incomplete (0x${status.toString(16)})"
31        }
32    }
33 }
```

68. app/src/main/java/com/wetinknext/engine/gl/GLProgram.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4
5 /** Linked GLSL program. Construction and release must run on the GL thread. */
6 class GLProgram(vertexSource: String, fragmentSource: String) {
7     val id: Int = GLES30.glCreateProgram().also { program ->
8         check(program != 0) { "Unable to create GL program" }
9         val vertex = compile(GLES30.GL_VERTEX_SHADER, vertexSource)
10        val fragment = compile(GLES30.GL_FRAGMENT_SHADER, fragmentSource)
11        GLES30.glAttachShader(program, vertex); GLES30.glAttachShader(program, fragment); GLES30.glLinkProgram(program)
12        val linked = IntArray(1); GLES30.glGetProgramiv(program, GLES30.GL_LINK_STATUS, linked, 0)
13        GLES30.glDeleteShader(vertex); GLES30.glDeleteShader(fragment)
14        check(linked[0] == GLES30.GL_TRUE) { "Program link failed: ${GLES30.glGetProgramInfoLog(program)}" }
15    }
16    fun use() = GLES30.glUseProgram(id)
17    fun release() = GLES30.glDeleteProgram(id)
18
19    private fun compile(type: Int, source: String): Int = GLES30.glCreateShader(type).also { shader ->
20        check(shader != 0) { "Unable to create shader" }
21        GLES30.glShaderSource(shader, source); GLES30.glCompileShader(shader)
22        val compiled = IntArray(1); GLES30.glGetShaderiv(shader, GLES30.GL_COMPILE_STATUS, compiled, 0)
23        check(compiled[0] == GLES30.GL_TRUE) { "Shader compile failed: ${GLES30.glGetShaderInfoLog(shader)}" }
24    }
25 }
```

69. app/src/main/java/com/wetinknext/engine/gl/RenderTarget.kt

```

1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4
5 /** Texture-backed FBO with RGBA16F probing and a guaranteed RGBA8 fallback. */
6 class RenderTarget {
7     var framebufferId: Int = 0
8     private set
9     var textureId: Int = 0
10    private set
11    var width: Int = 0
12    private set
13    var height: Int = 0
14    private set
15    var usesHalfFloat: Boolean = false
16    private set
17
18    fun create(width: Int, height: Int, preferHalfFloat: Boolean) {
19        GLCheck.checkOnGLThread()
20        require(width > 0 && height > 0)
21        release()
22        if (preferHalfFloat && createInternal(width, height, true)) return
23        check(createInternal(width, height, false)) { "RGBA8 render target creation failed" }
24    }
25
26    /** A 4x4 completeness probe used before allocating a document-sized target. */
27    fun probeHalfFloatColorBuffer(): Boolean {
28        release()
29        val result = createInternal(4, 4, true)
30        release()
31        return result
32    }
33
34    fun bind() {
35        check(framebufferId != 0) { "RenderTarget is not created" }
36        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebufferId)
37    }
38
39    fun clear(red: Float, green: Float, blue: Float, alpha: Float) {
40        bind()
41        // Clearing is constrained by the current viewport. Always restore the
42        // offscreen target's extent, not the last screen-sized present viewport.
43        GLES30.glViewport(0, 0, width, height)
44        GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
45        GLES30.glDisable(GLES30.GL_BLEND)
46        GLES30.glColorMask(true, true, true, true)
47        GLES30.glClearColor(red, green, blue, alpha)
48        GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
49        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
50    }
51
52    fun release() {
53        GLCheck.checkOnGLThread()
54        if (framebufferId != 0) GLES30.glDeleteFramebuffers(1, intArrayOf(framebufferId), 0)
55        if (textureId != 0) GLES30.glDeleteTextures(1, intArrayOf(textureId), 0)
56        framebufferId = 0; textureId = 0; width = 0; height = 0; usesHalfFloat = false
57    }
58
59    private fun createInternal(targetWidth: Int, targetHeight: Int, halfFloat: Boolean): Boolean {
60        val textures = IntArray(1)
61        val framebuffers = IntArray(1)
62        GLES30.glGenTextures(1, textures, 0)
63        GLES30.glGenFramebuffers(1, framebuffers, 0)
64        val texture = textures[0]
65        val framebuffer = framebuffers[0]
66        if (texture == 0 || framebuffer == 0) return false
67        GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, texture)
68        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MIN_FILTER, GLES30.GL_LINEAR)
69        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MAG_FILTER, GLES30.GL_LINEAR)
70        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_S, GLES30.GL_CLAMP_TO_EDGE)
71        GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_T, GLES30.GL_CLAMP_TO_EDGE)
72        GLES30.glTexImage2D(GLES30.GL_TEXTURE_2D, 0, if (halfFloat) GLES30.GL_RGBA16F else GLES30.GL_RGBA8,
73            targetWidth, targetHeight, 0, GLES30.GL_RGBA, if (halfFloat) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE,
74            null)
75        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebuffer)
76        GLES30.glFramebufferTexture2D(GLES30.GL_FRAMEBUFFER, GLES30.GL_COLOR_ATTACHMENT0, GLES30.GL_TEXTURE_2D, texture, 0)
77        val complete = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER) == GLES30.GL_FRAMEBUFFER_COMPLETE
78        GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
79        GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
80        if (!complete) {
81            GLES30.glDeleteFramebuffers(1, framebuffers, 0); GLES30.glDeleteTextures(1, textures, 0)
82            return false
83        }
84        framebufferId = framebuffer; textureId = texture; width = targetWidth; height = targetHeight; usesHalfFloat =
85        halfFloat
86    }
87    }
88 }

```

70. app/src/main/java/com/wetinknext/engine/gl/ShaderLib.kt

```

1  package com.wetinknext.engine.gl
2
3  object ShaderLib {
4      const val compositorVertex = ""#version 300 es
5          layout(location=0) in vec2 aPosition;uniform mat4 uCanvasToClip;uniform vec2 uCanvasSize;out vec2 vUv;out vec2
vScreenUv;void main(){vUv=aPosition/uCanvasSize;gl_Position=uCanvasToClip*vec4(aPosition,0.,1.);vScreenUv=gl_Position.xy*.5+.5;}""
6      const val compositorFragment = ""#version 300 es
7          precision highp float;
8
9          in vec2 vUv;
10         in vec2 vScreenUv;
11         uniform sampler2D uLayerTex;
12         uniform sampler2D uStrokeTex;
13         uniform sampler2D uScreenStrokeTex;
14         uniform sampler2D uStrokeCoverageTex;
15         uniform int uStrokeActive;
16         uniform int uStrokeIsScreenSpace;
17         uniform int uStrokeMode;
18         uniform int uStrokeErase;
19         uniform float uOpacity;
20         uniform float uStrokeOpacity;
21         uniform vec3 uStrokeColorLinear;
22
23         out vec4 fragColor;
24
25         void main() {
26             vec4 layer = texture(uLayerTex, vUv);
27
28             if (uStrokeActive == 1) {
29                 vec4 stroke;
30                 if (uStrokeMode == 1) {
31                     float coverage = (
32                         uStrokeIsScreenSpace == 1
33                         ? texture(uStrokeCoverageTex, vScreenUv).a
34                         : texture(uStrokeCoverageTex, vUv).a
35                     );
36                     float alpha = coverage * uStrokeOpacity;
37                     stroke = vec4(uStrokeColorLinear * alpha, alpha);
38                 } else {
39                     stroke = (
40                         uStrokeIsScreenSpace == 1
41                         ? texture(uScreenStrokeTex, vScreenUv)
42                         : texture(uStrokeTex, vUv)
43                     ) * uStrokeOpacity;
44                 }
45
46                 // Both textures contain premultiplied linear RGBA.
47                 if (uStrokeErase == 1) {
48                     layer *= (1.0 - stroke.a);
49                 } else {
50                     layer = stroke + layer * (1.0 - stroke.a);
51                 }
52             }
53
54             float alpha = layer.a * uOpacity;
55
56             // Layer opacity is applied to premultiplied linear RGB as well.
57             fragColor = vec4(layer.rgb * uOpacity, alpha);
58         }
59     ""
60     const val linearPresentFragment = ""#version 300 es
61         precision highp float;
62
63         in vec2 vUv;
64         uniform sampler2D uTexture;
65         out vec4 fragColor;
66
67         vec3 linearToSrgb(vec3 value) {
68             vec3 c = clamp(value, 0.0, 1.0);
69             vec3 low = c * 12.92;
70             vec3 high = 1.055 * pow(c, vec3(1.0 / 2.4)) - 0.055;
71             return mix(low, high, step(vec3(0.0031308), c));
72         }
73
74         vec3 unpremultiply(vec4 value) {
75             return value.a <= 0.0001 ? vec3(0.0) : value.rgb / value.a;
76         }
77
78         void main() {
79             vec4 color = texture(uTexture, vUv);
80             vec3 srgb = linearToSrgb(unpremultiply(color));
81             fragColor = vec4(srgb * color.a, color.a);
82         }
83     ""
84     const val linearCopyFragment = ""#version 300 es
85         precision highp float;
86
87         in vec2 vUv;
88         uniform sampler2D uTexture;
89         out vec4 fragColor;

```



```

90
91     void main() {
92         fragColor = texture(uTexture, vUv);
93     }
94     ""
95     const val cachedStrokePresentFragment = ""#version 300 es
96         precision highp float;
97
98         in vec2 vUv;
99         uniform sampler2D uLowerTex;
100        uniform sampler2D uActiveLayerTex;
101        uniform sampler2D uStrokeTex;
102        uniform int uStrokeActive;
103        uniform int uStrokeErase;
104        uniform float uLayerOpacity;
105        uniform float uStrokeOpacity;
106        out vec4 fragColor;
107
108        vec3 linearToSrgb(vec3 value) {
109            vec3 c = clamp(value, 0.0, 1.0);
110            vec3 low = c * 12.92;
111            vec3 high = 1.055 * pow(c, vec3(1.0 / 2.4)) - 0.055;
112            return mix(low, high, step(vec3(0.0031308), c));
113        }
114
115        void main() {
116            vec4 lower = texture(uLowerTex, vUv);
117            vec4 activeLayer = texture(uActiveLayerTex, vUv);
118
119            if (uStrokeActive == 1) {
120                vec4 stroke = texture(uStrokeTex, vUv) * uStrokeOpacity;
121                activeLayer = uStrokeErase == 1
122                    ? activeLayer * (1.0 - stroke.a)
123                    : stroke + activeLayer * (1.0 - stroke.a);
124            }
125
126            activeLayer *= uLayerOpacity;
127            vec4 document = activeLayer + lower * (1.0 - activeLayer.a);
128            vec3 straightLinear = document.a <= 0.0001
129                ? vec3(0.0)
130                : document.rgb / document.a;
131            vec3 srgb = linearToSrgb(straightLinear);
132            fragColor = vec4(srgb * document.a, document.a);
133        }
134        ""
135        const val dabVertex = ""#version 300 es
136            layout(location = 0) in vec2 aCorner;
137            layout(location = 1) in vec4 iDab0;
138            layout(location = 2) in vec3 iDab1;
139            uniform mat4 uCanvasToClip;
140            uniform vec2 uCanvasSize;
141            out vec2 vLocal;
142            out vec2 vCanvasUv;
143            flat out float vCoverage;
144            flat out float vFlow;
145            flat out float vHardness;
146            void main(){
147                float c=cos(iDab0.w), s=sin(iDab0.w);
148                vec2 r=vec2(c*aCorner.x-s*aCorner.y,s*aCorner.x+c*aCorner.y);
149                vLocal=aCorner;
150                vCoverage=iDab1.x;
151                vFlow=iDab1.y;
152                vHardness=iDab1.z;
153                vec2 p = iDab0.xy+r*iDab0.z;
154                vCanvasUv = p / uCanvasSize;
155                gl_Position=uCanvasToClip*vec4(p,0.,1.);
156            }
157        ""
158        const val dabFragment = ""#version 300 es
159            precision highp float;
160            in vec2 vLocal;
161            in vec2 vCanvasUv;
162            flat in float vCoverage;
163            flat in float vFlow;
164            flat in float vHardness;
165            uniform vec3 uColorLinear;
166            uniform sampler2D uGrainTex;
167            uniform int uGrainActive;
168            uniform float uGrainScale;
169            uniform int uGrainCanvasLocked;
170            uniform float uTextureDepth;
171            uniform float uTextureContrast;
172            uniform float uStrokeOpacity;
173            uniform int uCoverageOnly;
174            uniform sampler2D uShapeTex;
175            uniform int uShapeActive;
176            uniform int uReverseShape;
177            uniform int uRgbToAlpha;
178            out vec4 fragColor;
179            float luminance(vec3 color) {
180                return dot(color, vec3(0.299, 0.587, 0.114));
181            }

```

```

182     float applyTextureLevels(float value, float contrast, float depth) {
183         float adjusted = clamp((value - 0.5) * contrast + 0.5, 0.0, 1.0);
184         return mix(1.0 - depth, 1.0, adjusted);
185     }
186     float dabCoverage(float distanceFromCenter, float hardness) {
187         float aa=max(fwidth(distanceFromCenter),.001);
188         float edge=mix(
189             1.0-aa,
190             1.0-aa*.15,
191             clamp(hardness,0.0,1.0)
192         );
193         return 1.0-smoothstep(edge,1.0+aa,distanceFromCenter);
194     }
195     void main(){
196         float r=length(vLocal);
197         float cov=dabCoverage(r,vHardness);
198         if (cov <= 0.0) discard;
199
200         float shapeMask = 1.0;
201         if (uShapeActive == 1) {
202             vec2 shapeUv = vLocal * 0.5 + 0.5;
203             vec4 shapeColor = texture(uShapeTex, shapeUv);
204             shapeMask = uRgbToAlpha == 1
205                 ? dot(shapeColor.rgb, vec3(0.299, 0.587, 0.114))
206                 : shapeColor.a;
207             if (uReverseShape == 1) shapeMask = 1.0 - shapeMask;
208         }
209
210         float grainFactor = 1.0;
211         if (uGrainActive == 1) {
212             vec2 uv = uGrainCanvasLocked == 1
213                 ? vCanvasUv * max(uGrainScale, 0.0001)
214                 : (vLocal * 0.5 + 0.5) * max(uGrainScale, 0.0001);
215             vec4 grainColor = texture(uGrainTex, uv);
216             grainFactor = applyTextureLevels(
217                 luminance(grainColor.rgb), uTextureContrast, uTextureDepth
218             );
219         }
220
221         float coverage = vCoverage * vFlow * cov * shapeMask * grainFactor;
222         if (uCoverageOnly == 1) {
223             fragColor = vec4(0.0, 0.0, 0.0, coverage);
224             return;
225         }
226         float a = coverage * uStrokeOpacity;
227         fragColor = vec4(uColorLinear * a, a);
228     }
229     ""
230     const val nonBuildupStrokeFragment = ""#version 300 es
231     precision highp float;
232
233     in vec2 vUv;
234     uniform sampler2D uCoverageTex;
235     uniform vec3 uColorLinear;
236     uniform float uOpacity;
237     out vec4 fragColor;
238
239     void main() {
240         float coverage = texture(uCoverageTex, vUv).a;
241         float alpha = coverage * uOpacity;
242         fragColor = vec4(uColorLinear * alpha, alpha);
243     }
244     ""
245     const val strokeCompositeFragment = ""#version 300 es
246     precision highp float; in vec2 vUv; uniform sampler2D uCanvasTex; uniform sampler2D uStrokeTex; uniform int
247     uStrokeActive; out vec4 fragColor;
248     vec3 toSrgb(vec3 c){c=clamp(c,0.,1.);return mix(c*12.92,1.055*pow(c,vec3(1./2.4))-.055,step(vec3(.0031308),c));}
249     vec3 unsp(vec4 c){return c.a<=.0001?vec3(0.):c.rgb/c.a;}
250     void main(){vec4 c=texture(uCanvasTex,vUv);vec4 s=texture(uStrokeTex,vUv);float a=uStrokeActive==1?s.a+c.a*(1.-s.a):c
251     .a;vec3 rgb=uStrokeActive==1&&a>.0001?(unp(s)*s.a+unp(c)*c.a*(1.-s.a))/a:unp(c);fragColor=vec4(toSrgb(rgb),1.);}
252     ""
253     const val canvasPresentVertex = ""#version 300 es
254     layout(location = 0) in vec2 aPosition;
255     uniform mat4 uCanvasToClip;
256     uniform vec2 uCanvasSize;
257     out vec2 vUv;
258     void main() {
259         vUv = aPosition / uCanvasSize;
260         gl_Position = uCanvasToClip * vec4(aPosition, 0.0, 1.0);
261     }
262     ""
263     const val presentFragment = ""#version 300 es
264     precision highp float;
265     in vec2 vUv;
266     uniform sampler2D uTexture;
267     out vec4 fragColor;
268     void main() { fragColor = texture(uTexture, vUv); }
269     ""
270     const val fullscreenVertex = ""#version 300 es
271     layout(location = 0) in vec2 aPosition;
272     out vec2 vUv;
273     void main() { vUv = aPosition * 0.5 + 0.5; gl_Position = vec4(aPosition, 0.0, 1.0); }

```

```

272     ""
273     const val canvasBackdropFragment = ""#version 300 es
274         precision highp float;
275
276         uniform vec3 uBackgroundColor;
277         uniform vec3 uGridColor;
278         uniform int uMode;
279         out vec4 fragColor;
280
281         void main() {
282             const float cellPx = 28.0;
283             vec2 pixel = gl_FragCoord.xy;
284             vec3 color = uBackgroundColor;
285
286             if (uMode == 1) {
287                 vec2 insideCell = mod(pixel, cellPx);
288                 bool isGridLine = insideCell.x < 1.0 || insideCell.y < 1.0;
289                 color = isGridLine ? uGridColor : uBackgroundColor;
290             } else {
291                 vec2 cell = floor(pixel / cellPx);
292                 float parity = mod(cell.x + cell.y, 2.0);
293                 color = parity < 1.0 ? uGridColor : uBackgroundColor;
294             }
295
296             fragColor = vec4(color, 1.0);
297         }
298     ""
299     const val solidFragment = ""#version 300 es
300         precision highp float;
301         in vec2 vUv;
302         out vec4 fragColor;
303         void main() { fragColor = vec4(vUv, 0.0, 1.0); }
304     ""
305     const val geometricRibbonVertex = ""#version 300 es
306         layout(location=0) in vec2 aPos; layout(location=1) in float aCoverage; layout(location=2) in float aAlpha;
307         uniform mat4 uCanvasToClip; out float vCoverage; out float vAlpha;
308         void main(){vCoverage=aCoverage;vAlpha=aAlpha;gl_Position=uCanvasToClip*vec4(aPos,0.,1.);}
309     ""
310     const val geometricRibbonFragment = ""#version 300 es
311         precision highp float; in float vCoverage; in float vAlpha; uniform vec3 uColorLinear; uniform float uFlow;
312         uniform int uAntiAliasLevel; uniform int uNoAntialias; out vec4 fragColor;
313         void main(){float cov=vCoverage;if(uNoAntialias==1||uAntiAliasLevel==0)cov=step(.5,cov);else{float
e=uAntiAliasLevel==1?.35:uAntiAliasLevel==2?.5:.7;cov=smoothstep(.5-e,.5+e,cov);}float a=vAlpha*cov*uFlow;fragColor=vec4(
uColorLinear*a,a);}
314     ""
315
316     /**
317     * Один сегмент штриха = капсула (round cone): два круга с линейной интерполяцией радиуса.
318     * Вершинный шейдер строит только AABB сегмента, форма считается во фрагменте.
319     */
320     const val capsuleVertex = ""#version 300 es
321         layout(location = 0) in vec2 aCorner; // -1..1, TRIANGLE_STRIP из 4 вершин
322         layout(location = 1) in vec4 iA; // x, y, radius, coverage (начало сегмента)
323         layout(location = 2) in vec4 iB; // x, y, radius, coverage (конец сегмента)
324         uniform mat4 uCanvasToClip;
325         uniform vec2 uCanvasSize;
326         out vec2 vPos;
327         out vec2 vCanvasUv;
328         flat out vec4 vA;
329         flat out vec4 vB;
330         void main() {
331             // Запас 2 px на AA-переход, иначе край обрежется по границе квада.
332             vec2 lo = min(iA.xy - iA.z, iB.xy - iB.z) - 2.0;
333             vec2 hi = max(iA.xy + iA.z, iB.xy + iB.z) + 2.0;
334             vec2 p = mix(lo, hi, aCorner * 0.5 + 0.5);
335             vPos = p;
336             vCanvasUv = p / uCanvasSize;
337             vA = iA;
338             vB = iB;
339             gl_Position = uCanvasToClip * vec4(p, 0.0, 1.0);
340         }
341     ""
342
343     /**
344     * Аналитическое покрытие: нет каппов, джойнов, miter и AA-юбки как отдельных сущностей.
345     * Выход строго premultiplied linear: rgb = color * a, a = cov.
346     * Рисовать premultiplied source-over blending:
347     * GL_FUNC_ADD + GL_ONE, GL_ONE_MINUS_SRC_ALPHA.
348     */
349     const val capsuleFragment = ""#version 300 es
350         precision highp float;
351         in vec2 vPos;
352         in vec2 vCanvasUv;
353         flat in vec4 vA;
354         flat in vec4 vB;
355         uniform vec3 uColorLinear;
356         uniform sampler2D uGrainTex;
357         uniform int uGrainActive;
358         uniform float uGrainScale;
359         uniform int uGrainCanvasLocked;
360         uniform float uTextureDepth;
361         uniform float uTextureContrast;

```

```

362     uniform float uFlow;
363     uniform int uCoverageOnly;
364     out vec4 fragColor;
365
366     float sdRoundCone(vec2 p, vec2 a, vec2 b, float r1, float r2) {
367         vec2 ba = b - a;
368         float l2 = dot(ba, ba);
369         float rr = r1 - r2;
370
371         // Guard 1: вырожденный сегмент (тап, дубль сэмпла) -> обычный круг.
372         if (l2 < 1e-6) return length(p - a) - max(r1, r2);
373         // Guard 2: один круг поглощает другой (скачок давления на коротком шаге).
374         // Без этого a2 < 0 -> sqrt(отрицательного) -> NaN -> чёрные клинья.
375         if (rr * rr >= l2) return (r1 > r2) ? (length(p - a) - r1) : (length(p - b) - r2);
376
377         float a2 = l2 - rr * rr;
378         float il2 = 1.0 / l2;
379         vec2 pa = p - a;
380         float y = dot(pa, ba);
381         float z = y - l2;
382         vec2 xv = pa * l2 - ba * y;
383         float x2 = dot(xv, xv);
384         float y2 = y * y * l2;
385         float z2 = z * z * l2;
386         float k = sign(rr) * rr * rr * x2;
387
388         if (sign(z) * a2 * z2 > k) return sqrt(x2 + z2) * il2 - r2; // круглый конец у В
389         if (sign(y) * a2 * y2 < k) return sqrt(x2 + y2) * il2 - r1; // круглый конец у А
390         return (sqrt(x2 * a2 * il2) + y * rr) * il2 - r1; // касательная боковина
391     }
392
393     void main() {
394         float d = sdRoundCone(vPos, vA.xy, vB.xy, vA.z, vB.z);
395         float w = max(fwidth(d), 1e-4);
396         float cov = 1.0 - smoothstep(-w, w, d);
397         if (cov <= 0.0) discard;
398
399         float grainFactor = 1.0;
400         if (uGrainActive == 1) {
401             vec2 uv = vCanvasUv * max(uGrainScale, 0.0001);
402             float val = texture(uGrainTex, uv).r;
403             val = clamp((val - 0.5) * uTextureContrast + 0.5, 0.0, 1.0);
404             grainFactor = mix(1.0 - uTextureDepth, 1.0, val);
405         }
406
407         vec2 ab = vB.xy - vA.xy;
408         float abLengthSquared = max(dot(ab, ab), 0.000001);
409         float segmentT = clamp(
410             dot(vPos - vA.xy, ab) / abLengthSquared,
411             0.0,
412             1.0
413         );
414         float segmentCoverage = mix(vA.w, vB.w, segmentT);
415         float coverage = cov * segmentCoverage * grainFactor * uFlow;
416         if (uCoverageOnly == 1) {
417             fragColor = vec4(0.0, 0.0, 0.0, coverage);
418         } else {
419             fragColor = vec4(uColorLinear * coverage, coverage);
420         }
421     }
422     ""
423
424     const val strokeBlitFragment = ""#version 300 es
425     precision highp float;
426     in vec2 vUv;
427     uniform sampler2D uStrokeTex;
428     uniform float uOpacity;
429     out vec4 fragColor;
430     void main() { fragColor = texture(uStrokeTex, vUv) * uOpacity; }
431     ""
432
433     const val ribbonVertex = ""#version 300 es
434     layout(location=0) in vec2 aPosition;
435     layout(location=1) in float aCoverage;
436     uniform mat4 uCanvasToClip;
437     flat out float vCoverage;
438     void main(){ vCoverage=aCoverage; gl_Position=uCanvasToClip*vec4(aPosition,0.0,1.0); }
439     ""
440     const val ribbonFragment = ""#version 300 es
441     precision highp float;
442     flat in float vCoverage;
443     uniform vec3 uColorLinear;
444     uniform int uCoverageOnly;
445     uniform float uFlow;
446     out vec4 fragColor;
447     void main(){
448         float coverage = vCoverage * uFlow;
449         if(uCoverageOnly==1) fragColor=vec4(0.0,0.0,0.0,coverage);
450         else fragColor=vec4(uColorLinear*coverage,coverage);
451     }
452     ""
453 }

```

71. app/src/main/java/com/wetinknext/engine/input/GestureRouter.kt

```

1 package com.wetinknext.engine.input
2
3 import android.view.MotionEvent
4 import com.wetinknext.engine.core.Camera
5 import kotlin.math.atan2
6 import kotlin.math.hypot
7
8 /**
9  * Routes view touch input before it reaches the brush pipeline.
10  *
11  * One pointer draws. A second pointer cancels that stroke immediately and
12  * takes ownership of the interaction for canvas navigation. Two- and
13  * three-pointer taps trigger undo and redo respectively.
14  */
15 class GestureRouter(
16     private val camera: Camera,
17     private val drawingInput: StrokeInputCapturer,
18     private val callbacks: Callbacks,
19 ) {
20     interface Callbacks {
21         fun cancelDrawing()
22         fun undoGesture()
23         fun redoGesture()
24         fun resetCamera()
25         fun requestRender()
26     }
27
28     private var mode = InputMode.IDLE
29     private var firstX = 0f
30     private var firstY = 0f
31     private var secondX = 0f
32     private var secondY = 0f
33     private var previousDistance = 0f
34     private var previousAngle = 0f
35     private var gestureStartTime = 0L
36     private var gestureMoved = false
37     private var navigationPointerCount = 0
38     private var drawingStartTime = 0L
39     private var drawingStartX = 0f
40     private var drawingStartY = 0f
41     private var drawingMoved = false
42     private var lastTapTime = 0L
43     private var lastTapX = 0f
44     private var lastTapY = 0f
45
46     fun onTouchEvent(event: MotionEvent): Boolean = when (event.actionMasked) {
47         MotionEvent.ACTION_DOWN -> {
48             mode = InputMode.DRAWING
49             drawingStartTime = event.eventTime
50             drawingStartX = event.x
51             drawingStartY = event.y
52             drawingMoved = false
53             drawingInput.onTouchEvent(event)
54             true
55         }
56
57         MotionEvent.ACTION_POINTER_DOWN -> {
58             if (event.pointerCount >= 2) {
59                 beginNavigation(event)
60                 true
61             } else {
62                 false
63             }
64         }
65
66         MotionEvent.ACTION_MOVE -> when (mode) {
67             InputMode.DRAWING -> {
68                 if (hypot(event.x - drawingStartX, event.y - drawingStartY) > TAP_MOVE_THRESHOLD_PX) {
69                     drawingMoved = true
70                     lastTapTime = 0L
71                 }
72                 drawingInput.onTouchEvent(event)
73                 true
74             }
75
76             InputMode.NAVIGATING -> {
77                 updateNavigation(event)
78                 true
79             }
80
81             else -> false
82         }
83
84         MotionEvent.ACTION_POINTER_UP -> {
85             if (mode == InputMode.NAVIGATING) {
86                 handleNavigationPointerUp(event)
87                 true
88             } else {
89                 drawingInput.onTouchEvent(event)
90                 true

```

```

91     }
92 }
93
94 MotionEvent.ACTION_UP -> {
95     when (mode) {
96         InputMode.DRAWING -> {
97             drawingInput.onTouchEvent(event)
98             val duration = event.eventTime - drawingStartTime
99             if (!drawingMoved && duration <= TAP_MAX_DURATION_MS) {
100                 handleSingleFingerTap(event)
101             }
102         }
103         InputMode.NAVIGATING -> finishNavigation(event)
104         else -> Unit
105     }
106     mode = InputMode.IDLE
107     true
108 }
109
110 MotionEvent.ACTION_CANCEL -> {
111     drawingInput.cancelActiveStroke()
112     callbacks.cancelDrawing()
113     mode = InputMode.CANCELLED
114     callbacks.requestRender()
115     true
116 }
117
118 else -> false
119 }
120
121 private fun beginNavigation(event: MotionEvent) {
122     lastTapTime = 0L
123     if (mode == InputMode.DRAWING) {
124         // The queued CANCEL keeps the brush pipeline coherent; the callback
125         // clears an already-rendered preview on the GL thread.
126         drawingInput.cancelActiveStroke()
127         callbacks.cancelDrawing()
128     }
129
130     mode = InputMode.NAVIGATING
131     navigationPointerCount = event.pointerCount
132     gestureStartTime = event.eventTime
133     gestureMoved = false
134     captureNavigationPointers(event)
135     callbacks.requestRender()
136 }
137
138 private fun updateNavigation(event: MotionEvent) {
139     if (event.pointerCount < 2) return
140
141     val newFirstX = event.getX(0)
142     val newFirstY = event.getY(0)
143     val newSecondX = event.getX(1)
144     val newSecondY = event.getY(1)
145     val oldCenterX = (firstX + secondX) * .5f
146     val oldCenterY = (firstY + secondY) * .5f
147     val newCenterX = (newFirstX + newSecondX) * .5f
148     val newCenterY = (newFirstY + newSecondY) * .5f
149     val newDistance = distance(newFirstX, newFirstY, newSecondX, newSecondY)
150     val zoomFactor = if (previousDistance > MIN_DISTANCE) newDistance / previousDistance else 1f
151     val newAngle = angle(newFirstX, newFirstY, newSecondX, newSecondY)
152     val rotationDelta = normalizeAngle(newAngle - previousAngle)
153     val panX = newCenterX - oldCenterX
154     val panY = newCenterY - oldCenterY
155
156     camera.set(
157         camera.snapshot().applyGestureDelta(
158             panX = panX,
159             panY = panY,
160             anchorX = newCenterX,
161             anchorY = newCenterY,
162             zoomFactor = zoomFactor,
163             rotationDelta = rotationDelta,
164         ),
165     )
166
167     if (hypot(panX, panY) > TAP_MOVE_THRESHOLD_PX ||
168         kotlin.math.abs(zoomFactor - 1f) > TAP_ZOOM_THRESHOLD ||
169         kotlin.math.abs(rotationDelta) > TAP_ROTATION_THRESHOLD_RAD
170     ) {
171         gestureMoved = true
172     }
173
174     firstX = newFirstX
175     firstY = newFirstY
176     secondX = newSecondX
177     secondY = newSecondY
178     previousDistance = newDistance
179     previousAngle = newAngle
180     callbacks.requestRender()
181 }
182

```

```

183 private fun handleNavigationPointerUp(event: MotionEvent) {
184     // Once fewer than two pointers remain, navigation stays active until
185     // ACTION_UP so a two/three-finger tap can be recognised reliably.
186     if (event.pointerCount - 1 < 2) return
187     captureNavigationPointers(event, event.actionIndex)
188 }
189
190 private fun finishNavigation(event: MotionEvent) {
191     val duration = event.eventTime - gestureStartTime
192     if (!gestureMoved && duration <= TAP_MAX_DURATION_MS) {
193         when (navigationPointerCount) {
194             2 -> callbacks.undoGesture()
195             3 -> callbacks.redoGesture()
196         }
197     }
198     callbacks.requestRender()
199 }
200
201 private fun handleSingleFingerTap(event: MotionEvent) {
202     val now = event.eventTime
203     val dx = event.x - lastTapX
204     val dy = event.y - lastTapY
205     val isDoubleTap =
206         now - lastTapTime <= DOUBLE_TAP_TIMEOUT_MS &&
207         hypot(dx, dy) <= DOUBLE_TAP_DISTANCE_PX
208     if (isDoubleTap) {
209         callbacks.resetCamera()
210         lastTapTime = 0L
211     } else {
212         lastTapTime = now
213         lastTapX = event.x
214         lastTapY = event.y
215     }
216 }
217
218 private fun captureNavigationPointers(event: MotionEvent, excludedIndex: Int = -1) {
219     var firstIndex = -1
220     var secondIndex = -1
221     for (index in 0 until event.pointerCount) {
222         if (index == excludedIndex) continue
223         if (firstIndex < 0) firstIndex = index else {
224             secondIndex = index
225             break
226         }
227     }
228     if (firstIndex < 0 || secondIndex < 0) return
229     firstX = event.getX(firstIndex)
230     firstY = event.getY(firstIndex)
231     secondX = event.getX(secondIndex)
232     secondY = event.getY(secondIndex)
233     previousDistance = distance(firstX, firstY, secondX, secondY)
234     previousAngle = angle(firstX, firstY, secondX, secondY)
235 }
236
237 private fun distance(x0: Float, y0: Float, x1: Float, y1: Float): Float =
238     hypot(x1 - x0, y1 - y0)
239
240 private fun angle(x0: Float, y0: Float, x1: Float, y1: Float): Float =
241     atan2(y1 - y0, x1 - x0)
242
243 private fun normalizeAngle(value: Float): Float {
244     var result = value
245     while (result > Math.PI) result -= (Math.PI * 2.0).toFloat()
246     while (result < -Math.PI) result += (Math.PI * 2.0).toFloat()
247     return result
248 }
249
250 private companion object {
251     const val TAP_MAX_DURATION_MS = 220L
252     const val MIN_DISTANCE = .001f
253     const val TAP_MOVE_THRESHOLD_PX = 4f
254     const val TAP_ZOOM_THRESHOLD = .02f
255     const val TAP_ROTATION_THRESHOLD_RAD = .035f
256     const val DOUBLE_TAP_TIMEOUT_MS = 280L
257     const val DOUBLE_TAP_DISTANCE_PX = 48f
258 }
259 }

```

72. app/src/main/java/com/wetinknext/engine/input/InputAction.kt

```
1 package com.wetinknext.engine.input
2
3 enum class InputAction {
4     DOWN,
5     MOVE,
6     UP,
7     CANCEL,
8 }
```


73. app/src/main/java/com/wetinknext/engine/input/InputBatch.kt

```
1 package com.wetinknext.engine.input
2
3 /** Fixed-capacity container for passing input from the UI thread to the GL thread. */
4 class InputBatch(val maxSamples: Int) {
5     val samples = Array(maxSamples) { InputSample() }
6
7     var action: InputAction = InputAction.MOVE
8     private set
9     var sampleCount: Int = 0
10    private set
11    var prediction: Boolean = false
12    private set
13
14    fun begin(action: InputAction, prediction: Boolean = false) {
15        this.action = action
16        this.prediction = prediction
17        sampleCount = 0
18    }
19
20    fun addSample(
21        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
22        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
23        historical: Boolean,
24    ): Boolean {
25        if (sampleCount >= maxSamples) return false
26        samples[sampleCount].set(
27            canvasX, canvasY, pressure.coerceIn(0f, 1f), tiltX.coerceIn(-1f, 1f),
28            tiltY.coerceIn(-1f, 1f), orientationRad, timestampNanos, pointerId, tool, historical,
29        )
30        sampleCount += 1
31        return true
32    }
33
34    fun isEmpty(): Boolean = sampleCount == 0
35
36    fun clear() {
37        for (index in 0 until sampleCount) samples[index].clear()
38        sampleCount = 0
39        prediction = false
40        action = InputAction.MOVE
41    }
42 }
```

74. app/src/main/java/com/wetinknext/engine/input/InputBatchPool.kt

```
1 package com.wetinknext.engine.input
2
3 import java.util.concurrent.atomic.AtomicInteger
4 import java.util.concurrent.atomic.AtomicReferenceArray
5
6 /** Fixed-size pool that returns null instead of allocating when exhausted. */
7 class InputBatchPool(
8     batchCount: Int = DEFAULT_BATCH_COUNT,
9     maxSamplesPerBatch: Int = DEFAULT_MAX_SAMPLES,
10 ) {
11     private val batches = AtomicReferenceArray<InputBatch>(batchCount)
12     private val available = AtomicInteger(batchCount)
13
14     init {
15         require(batchCount > 0)
16         require(maxSamplesPerBatch > 0)
17         for (index in 0 until batchCount) batches.set(index, InputBatch(maxSamplesPerBatch))
18     }
19
20     fun acquire(): InputBatch? {
21         while (true) {
22             val current = available.get()
23             if (current <= 0) return null
24             if (available.compareAndSet(current, current - 1)) {
25                 for (index in 0 until batches.length()) {
26                     val batch = batches.getAndSet(index, null)
27                     if (batch != null) return batch
28                 }
29                 available.incrementAndGet()
30                 return null
31             }
32         }
33     }
34
35     fun release(batch: InputBatch) {
36         batch.clear()
37         for (index in 0 until batches.length()) {
38             if (batches.compareAndSet(index, null, batch)) {
39                 available.incrementAndGet()
40                 return
41             }
42         }
43         error("InputBatchPool overflow: released batch does not belong to pool")
44     }
45
46     val freeCount: Int get() = available.get()
47
48     companion object {
49         const val DEFAULT_BATCH_COUNT = 96
50         const val DEFAULT_MAX_SAMPLES = 64
51     }
52 }
```

75. app/src/main/java/com/wetinknext/engine/input/InputMode.kt

```
1 package com.wetinknext.engine.input
2
3 /** Current ownership of touch input. Only DRAWING reaches the brush pipeline. */
4 enum class InputMode {
5     IDLE,
6     DRAWING,
7     NAVIGATING,
8     CANCELLED,
9 }
```

76. app/src/main/java/com/wetinknext/engine/input/InputSample.kt

```
1 package com.wetinknext.engine.input
2
3 /** Mutable input sample allocated only while a batch pool is initialized. */
4 class InputSample {
5     var canvasX = 0f
6     var canvasY = 0f
7     var pressure = 0f
8     var tiltX = 0f
9     var tiltY = 0f
10    var orientationRad = 0f
11    var timestampNanos = 0L
12    var pointerId = -1
13    var tool = PointerTool.UNKNOWN
14    var historical = false
15
16    fun set(
17        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
18        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
19        historical: Boolean,
20    ) {
21        this.canvasX = canvasX; this.canvasY = canvasY; this.pressure = pressure
22        this.tiltX = tiltX; this.tiltY = tiltY; this.orientationRad = orientationRad
23        this.timestampNanos = timestampNanos; this.pointerId = pointerId; this.tool = tool
24        this.historical = historical
25    }
26
27    fun clear() {
28        canvasX = 0f; canvasY = 0f; pressure = 0f; tiltX = 0f; tiltY = 0f
29        orientationRad = 0f; timestampNanos = 0L; pointerId = -1
30        tool = PointerTool.UNKNOWN; historical = false
31    }
32 }
```

77. app/src/main/java/com/wetinknext/engine/input/PointerTool.kt

```
1 package com.wetinknext.engine.input
2
3 enum class PointerTool {
4     FINGER,
5     STYLUS,
6     ERASER,
7     UNKNOWN,
8 }
```

78. app/src/main/java/com/wetinknext/engine/input/StrokeInputCapturer.kt

```

1 package com.wetinknext.engine.input
2
3 import android.view.MotionEvent
4 import com.wetinknext.engine.core.Camera
5 import java.util.concurrent.ArrayBlockingQueue
6
7 /** Converts View-local MotionEvent samples to canvas coordinates on the UI thread. */
8 class StrokeInputCapturer(private val camera: Camera, private val pool: InputBatchPool, private val queue:
ArrayBlockingQueue<InputBatch>) {
9     private var activePointerId = -1
10    private val canvasPoint = FloatArray(2)
11    private val stylusAxes = StylusAxes()
12    private val tiltOut = FloatArray(2)
13
14    var onSecondaryPointerDown: (() -> Unit)? = null
15
16    /** Не скрываем потери: пул кончился => сэмплы утрачены. */
17    var droppedBatches = 0L
18    private set
19
20    /** Cancels the one pointer currently owned by the drawing pipeline. */
21    fun cancelActiveStroke(): Boolean {
22        if (activePointerId < 0) return false
23        activePointerId = -1
24        return enqueueEmpty(InputAction.CANCEL)
25    }
26
27    fun onTouchEvent(event: MotionEvent): Boolean = when (event.actionMasked) {
28        MotionEvent.ACTION_DOWN -> {
29            activePointerId = event.getPointerId(0)
30            enqueue(event, InputAction.DOWN, 0, false)
31        }
32        // Multi-touch is owned by GestureRouter. Drawing input only receives
33        // the one-pointer draw path and explicit cancellation from that router.
34        MotionEvent.ACTION_POINTER_DOWN -> false
35        MotionEvent.ACTION_MOVE -> {
36            val i = event.findPointerIndex(activePointerId)
37            i >= 0 && enqueue(event, InputAction.MOVE, i, true)
38        }
39        MotionEvent.ACTION_UP, MotionEvent.ACTION_POINTER_UP -> {
40            val i = if (event.actionMasked == MotionEvent.ACTION_UP) 0 else event.actionIndex
41            if (event.getPointerId(i) != activePointerId) true else {
42                activePointerId = -1
43                // historical = true: хвост быстрого штриха больше не теряется.
44                enqueue(event, InputAction.UP, i, true)
45            }
46        }
47        MotionEvent.ACTION_CANCEL -> { activePointerId = -1; enqueueEmpty(InputAction.CANCEL) }
48        else -> false
49    }
50
51    private fun enqueue(event: MotionEvent, action: InputAction, index: Int, historical: Boolean): Boolean {
52        val batch = pool.acquire() ?: run { droppedBatches++; return false }
53        batch.begin(action); val t = camera.snapshot(); val id = event.getPointerId(index); val tool = tool(event, index)
54        if (historical) {
55            for (h in 0 until event.historySize) {
56                t.screenToCanvas(event.getHistoricalX(index, h), event.getHistoricalY(index, h), canvasPoint)
57                stylusAxes.readTilt(event, index, h, tool, tiltOut)
58                val orientation = stylusAxes.readOrientation(event, index, h, tool)
59                if (!batch.addSample(
60                    canvasPoint[0], canvasPoint[1], pressure(event, tool, index, h),
61                    tiltOut[0], tiltOut[1], orientation,
62                    event.getHistoricalEventTime(h) * 1_000_000L, id, tool, true
63                )) break
64            }
65        }
66        t.screenToCanvas(event.getX(index), event.getY(index), canvasPoint)
67        stylusAxes.readTilt(event, index, -1, tool, tiltOut)
68        val orientation = stylusAxes.readOrientation(event, index, -1, tool)
69        batch.addSample(
70            canvasPoint[0], canvasPoint[1], pressure(event, tool, index, -1),
71            tiltOut[0], tiltOut[1], orientation,
72            event.eventTime * 1_000_000L, id, tool, false
73        )
74        if (!queue.offer(batch)) {
75            pool.release(batch)
76            droppedBatches++
77            if (action == InputAction.UP) {
78                enqueueEmpty(InputAction.CANCEL)
79            }
80            return false
81        }
82        return true
83    }
84
85    private fun enqueueEmpty(action: InputAction): Boolean {
86        val batch = pool.acquire() ?: run { droppedBatches++; return false }
87        batch.begin(action); if (!queue.offer(batch)) { pool.release(batch); droppedBatches++; return false }; return true }
88    private fun tool(e: MotionEvent, i: Int) = when (e.getToolType(i)) { MotionEvent.TOOL_TYPE_STYLUS -> PointerTool.STYLUS; MotionEvent.
TOOL_TYPE_ERASER -> PointerTool.ERASER; MotionEvent.TOOL_TYPE_FINGER -> PointerTool.FINGER; else -> PointerTool.UNKNOWN }

```

```
89     private fun pressure(e:MotionEvent,t:PointerTool,i:Int,h:Int):Float=if(t==PointerTool.FINGER||t==PointerTool.UNKNOWN)
SYNTHETIC_FINGER_PRESSURE else (if(h>=0)e.getHistoricalPressure(i,h) else e.getPressure(i)).coerceIn(0f,1f)
90
91     companion object {
92         private const val SYNTHETIC_FINGER_PRESSURE = 0.62f
93     }
94 }
```

79. app/src/main/java/com/wetinknext/engine/input/StylusAxes.kt

```

1 package com.wetinknext.engine.input
2
3 import android.view.MotionEvent
4 import kotlin.math.cos
5 import kotlin.math.sin
6
7 class StylusAxes {
8
9     fun readTilt(
10         event: MotionEvent,
11         pointerIndex: Int,
12         historicalIndex: Int,
13         tool: PointerTool,
14         out: FloatArray,
15     ) {
16         if (tool == PointerTool.FINGER ||
17             tool == PointerTool.UNKNOWN) {
18             {
19                 out[0] = 0f
20                 out[1] = 0f
21                 return
22             }
23
24             val tilt = if (historicalIndex >= 0) {
25                 event.getHistoricalAxisValue(
26                     MotionEvent.AXIS_TILT,
27                     pointerIndex,
28                     historicalIndex,
29                 )
30             } else {
31                 event.getAxisValue(
32                     MotionEvent.AXIS_TILT,
33                     pointerIndex,
34                 )
35             }
36
37             val orientation = if (historicalIndex >= 0) {
38                 event.getHistoricalAxisValue(
39                     MotionEvent.AXIS_ORIENTATION,
40                     pointerIndex,
41                     historicalIndex,
42                 )
43             } else {
44                 event.getAxisValue(
45                     MotionEvent.AXIS_ORIENTATION,
46                     pointerIndex,
47                 )
48             }
49
50             val normalizedTilt =
51                 (tilt / (Math.PI.toFloat() * 0.5f))
52                 .coerceIn(0f, 1f)
53
54             out[0] =
55                 (cos(orientation) * normalizedTilt)
56                 .coerceIn(-1f, 1f)
57
58             out[1] =
59                 (sin(orientation) * normalizedTilt)
60                 .coerceIn(-1f, 1f)
61         }
62
63     fun readOrientation(
64         event: MotionEvent,
65         pointerIndex: Int,
66         historicalIndex: Int,
67         tool: PointerTool,
68     ): Float {
69         if (tool == PointerTool.FINGER || tool == PointerTool.UNKNOWN) return 0f
70         return if (historicalIndex >= 0) {
71             event.getHistoricalAxisValue(MotionEvent.AXIS_ORIENTATION, pointerIndex, historicalIndex)
72         } else {
73             event.getAxisValue(MotionEvent.AXIS_ORIENTATION, pointerIndex)
74         }
75     }
76 }

```


80. app/src/main/java/com/wetinknext/engine/persistence/LayerTileStore.kt

```

1 package com.wetinknext.engine.persistence
2
3 import com.wetinknext.engine.canvas.PaintLayer
4 import com.wetinknext.engine.undo.RawTileSnapshot
5 import com.wetinknext.engine.undo.TileCoord
6 import java.util.concurrent.ConcurrentHashMap
7
8 /** CPU cache of tiles captured after completed edits. No dab triggers persistence. */
9 class LayerTileStore(initialPayloads: Map<Long, ByteArray> = emptyMap()) {
10     private val tiles = ConcurrentHashMap<Long, ConcurrentHashMap<TileCoord, PersistentLayerTiles.Tile>>()
11     private val dirty = ConcurrentHashMap<Long, MutableSet<TileCoord>>()
12     /** A cleared layer is persisted as a tiny empty tile collection, not as a full transparent canvas. */
13     private val clearedLayers = ConcurrentHashMap.newKeySet<Long>()
14
15     suspend fun writeTile(layerId: Long, tile: RawTileSnapshot) {
16         tiles.getOrPut(layerId) { ConcurrentHashMap() }[tile.coord] = PersistentLayerTiles.fromRaw(tile)
17         dirty.getOrPut(layerId) { ConcurrentHashMap.newKeySet() }.add(tile.coord)
18     }
19
20     suspend fun readTile(layerId: Long, coord: TileCoord): ByteArray =
21         tiles[layerId]?.get(coord)?.bytes?.copyOf() ?: ByteArray(0)
22
23     suspend fun writeLayer(layer: PaintLayer, changedTiles: List<RawTileSnapshot>) {
24         require(layer.created)
25         changedTiles.forEach { writeTile(layer.id, it) }
26     }
27
28     /**
29      * GL-thread hand-off from a completed commit. Raw snapshots are immutable
30      * after capture, and Undo only reads them, so retain their arrays rather
31      * than allocating another full tile copy before persistence begins.
32      */
33     fun markDirty(layerId: Long, changedTiles: List<RawTileSnapshot>) {
34         clearedLayers.remove(layerId)
35         val target = tiles.getOrPut(layerId) { ConcurrentHashMap() }
36         val dirtyCoords = dirty.getOrPut(layerId) { ConcurrentHashMap.newKeySet() }
37         changedTiles.forEach { tile ->
38             target[tile.coord] = PersistentLayerTiles.fromRawOwned(tile)
39             dirtyCoords.add(tile.coord)
40         }
41     }
42
43     /** Drops the previous cached pixels after a layer clear without allocating transparent tile data. */
44     fun markLayerCleared(layerId: Long) {
45         tiles.remove(layerId)
46         dirty.remove(layerId)
47         clearedLayers.add(layerId)
48     }
49
50     fun takeDirty(): Map<Long, Set<TileCoord>> = buildMap {
51         dirty.forEach { (layerId, coords) -> put(layerId, coords.toSet()) }
52         clearedLayers.forEach { layerId -> putIfAbsent(layerId, emptySet()) }
53     }
54
55     fun acknowledge(saved: Map<Long, Set<TileCoord>>) {
56         saved.forEach { (layerId, coords) ->
57             dirty[layerId]?.removeAll(coords)
58             clearedLayers.remove(layerId)
59         }
60     }
61
62     /** Full layer payloads only for layers that have dirty tiles. */
63     fun payloadsForDirty(dirtySnapshot: Map<Long, Set<TileCoord>> = takeDirty()): Map<Long, ByteArray> =
64         dirtySnapshot.keys.associateWith { layerId ->
65             PersistentLayerTiles.encode(tiles[layerId]?.values.orEmpty())
66         }
67
68     init {
69         initialPayloads.forEach { (layerId, payload) ->
70             if (payload.isNotEmpty()) {
71                 val layer = tiles.getOrPut(layerId) { ConcurrentHashMap() }
72                 PersistentLayerTiles.decode(payload).forEach { layer[it.coord] = it }
73             }
74         }
75     }
76 }

```

81. app/src/main/java/com/wetinknext/engine/persistence/PersistentLayerTiles.kt

```

1 package com.wetinknext.engine.persistence
2
3 import com.wetinknext.engine.undo.RawTileSnapshot
4 import com.wetinknext.engine.undo.TileCoord
5 import java.nio.ByteBuffer
6 import java.nio.ByteOrder
7 import java.util.zip.DataFormatException
8 import java.util.zip.Deflater
9 import java.util.zip.Inflater
10
11 /** Binary, versioned collection of 256px tiles for one persistent paint layer. */
12 object PersistentLayerTiles {
13     private const val MAGIC = 0x57544C53 // WTLS
14     private const val VERSION = 2
15     private const val HEADER_BYTES = 12
16     private const val TILE_HEADER_BYTES_V1 = 32
17     private const val TILE_HEADER_BYTES_V2 = 36
18
19     data class Tile(
20         val coord: TileCoord,
21         val pixelLeft: Int,
22         val pixelTop: Int,
23         val pixelWidth: Int,
24         val pixelHeight: Int,
25         val bytesPerPixel: Int,
26         val bytes: ByteArray,
27     )
28
29     fun decode(payload: ByteArray): List<Tile> {
30         if (payload.isEmpty()) return emptyList()
31         val input = ByteBuffer.wrap(payload).order(ByteOrder.BIG_ENDIAN)
32         require(input.remaining() >= HEADER_BYTES && input.int == MAGIC) { "Unknown persistent layer tile payload" }
33         val version = input.int
34         require(version in setOf(1, VERSION)) { "Unsupported persistent layer tile version" }
35         val count = input.int
36         require(count >= 0) { "Invalid tile count" }
37         return List(count) {
38             val headerBytes = if (version == 1) TILE_HEADER_BYTES_V1 else TILE_HEADER_BYTES_V2
39             require(input.remaining() >= headerBytes) { "Truncated tile header" }
40             val tx = input.int; val ty = input.int
41             val left = input.int; val top = input.int
42             val width = input.int; val height = input.int
43             val bpp = input.int
44             val rawSize = input.int
45             require(
46                 width > 0 && height > 0 && bpp in setOf(4, 8) &&
47                 rawSize == width * height * bpp
48             ) { "Invalid tile payload" }
49
50             val bytes = if (version == 1) {
51                 require(rawSize <= input.remaining()) { "Truncated tile payload" }
52                 ByteArray(rawSize).also(input::get)
53             } else {
54                 val storedSize = input.int
55                 require(storedSize > 0 && storedSize <= input.remaining()) { "Invalid compressed tile payload" }
56                 val compressed = ByteArray(storedSize).also(input::get)
57                 inflate(compressed, rawSize)
58             }
59             Tile(TileCoord(tx, ty), left, top, width, height, bpp, bytes)
60         }.also { require(!input.hasRemaining()) { "Trailing tile payload data" } }
61     }
62
63     fun encode(tiles: Collection<Tile>): ByteArray {
64         val encoded = tiles
65             .sortedWith(compareBy<Tile> { it.coord.ty }.thenBy { it.coord.tx })
66             .map { tile -> tile to deflate(tile.bytes) }
67         val size = HEADER_BYTES + encoded.sumOf { (tile, compressed) ->
68             TILE_HEADER_BYTES_V2 + compressed.size
69         }
70         val output = ByteBuffer.allocate(size).order(ByteOrder.BIG_ENDIAN)
71         output.putInt(MAGIC).putInt(VERSION).putInt(tiles.size)
72         encoded.forEach { (tile, compressed) ->
73             output.putInt(tile.coord.tx).putInt(tile.coord.ty)
74             output.putInt(tile.pixelLeft).putInt(tile.pixelTop)
75             output.putInt(tile.pixelWidth).putInt(tile.pixelHeight)
76             output.putInt(tile.bytesPerPixel).putInt(tile.bytes.size)
77             output.putInt(compressed.size).put(compressed)
78         }
79         return output.array()
80     }
81
82     private fun deflate(raw: ByteArray): ByteArray {
83         val compressor = Deflater(Deflater.BEST_SPEED)
84         return try {
85             compressor.setInput(raw)
86             compressor.finish()
87             // zlib's incompressible-data bound; a tile is at most 512 KiB.
88             val destination = ByteArray(raw.size + raw.size / 8 + 64)
89             val size = compressor.deflate(destination)
90             require(compressor.finished()) { "Tile compression did not finish" }

```

```

91         destination.copyOf(size)
92     } finally {
93         compressor.end()
94     }
95 }
96
97 private fun inflate(compressed: ByteArray, rawSize: Int): ByteArray {
98     val inflater = Inflater()
99     return try {
100         inflater.setInput(compressed)
101         val raw = ByteArray(rawSize)
102         val size = inflater.inflate(raw)
103         require(size == rawSize && inflater.finished()) { "Tile decompression failed" }
104         raw
105     } catch (error: DataFormatException) {
106         throw IllegalArgumentException("Invalid compressed tile data", error)
107     } finally {
108         inflater.end()
109     }
110 }
111
112 fun fromRaw(tile: RawTileSnapshot) = Tile(
113     tile.coord, tile.pixelLeft, tile.pixelTop, tile.pixelWidth, tile.pixelHeight,
114     tile.bytesPerPixel, tile.rawBytes.copyOf(),
115 )
116
117 /**
118  * Shares the immutable raw byte array captured for a completed commit.
119  * The Undo compression job only reads the same array, so no additional
120  * copy is needed on the GL thread. Callers must not mutate [tile] after
121  * passing it here.
122  */
123 fun fromRawOwned(tile: RawTileSnapshot) = Tile(
124     tile.coord,
125     tile.pixelLeft,
126     tile.pixelTop,
127     tile.pixelWidth,
128     tile.pixelHeight,
129     tile.bytesPerPixel,
130     tile.rawBytes,
131 )
132 }

```

82. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailBuildScheduler.kt

```

1 package com.wetinknext.engine.thumbnail
2
3 import android.os.Handler
4 import android.os.Looper
5 import com.wetinknext.engine.canvas.LayerStack
6 import com.wetinknext.engine.gl.GlCheck
7 import java.io.File
8 import java.io.IOException
9 import java.nio.file.AtomicMoveNotSupportedException
10 import java.nio.file.Files
11 import java.nio.file.StandardCopyOption
12 import java.util.concurrent.ConcurrentLinkedQueue
13 import java.util.concurrent.ExecutorService
14 import java.util.concurrent.Executors
15 import java.util.concurrent.RejectedExecutionException
16 import java.util.concurrent.atomic.AtomicLong
17
18 /** Result published to the UI only after the current generation reached disk. */
19 data class ThumbnailBuildResult(
20     val generation: Long,
21     val projectPreview: File?,
22     val layerPreviews: Map<Long, File>,
23 )
24
25 /**
26  * Coordinates thumbnail builds across the GL, worker and UI threads.
27  *
28  * Call [markDirty], [buildIfNeeded], [processCompleted] and [shutdown] on the
29  * GL thread. [buildIfNeeded] is the sole point that reads GPU textures. WebP
30  * encoding and file publication happen on [executor]; callbacks are dispatched
31  * to the Android main thread.
32  */
33 class ThumbnailBuildScheduler(
34     private val outputDirectory: File,
35     private val renderer: ThumbnailRenderer,
36     private val requestGlFrame: () -> Unit,
37     private val onSave: (ThumbnailBuildResult) -> Unit,
38     private val onFailure: (Throwable) -> Unit = {},
39     private val executorFactory: () -> ExecutorService = { Executors.newSingleThreadExecutor() },
40     private val uiHandler: Handler = Handler(Looper.getMainLooper()),
41     private val encoder: ThumbnailEncoder = ThumbnailEncoder(),
42     private val previewSizePx: Int = DEFAULT_PREVIEW_SIZE_PX,
43 ) {
44     private var executor = executorFactory()
45     private val pendingResults = ConcurrentLinkedQueue<WorkerResult>()
46     private val generation = AtomicLong(0L)
47     private val dirtyLayerIds = linkedSetOf<Long>()
48
49     private var projectDirty = false
50     private var buildRunning = false
51
52     /**
53      * Records the newest desired preview state. A newer mark makes an ongoing
54      * worker result stale before it is allowed to notify the UI.
55      */
56     fun markDirty(
57         projectDirty: Boolean,
58         dirtyLayerIds: Collection<Long> = emptyList(),
59     ) {
60         GlCheck.checkOnGlThread()
61         if (!projectDirty && dirtyLayerIds.isEmpty()) return
62         this.projectDirty = this.projectDirty || projectDirty
63         this.dirtyLayerIds += dirtyLayerIds
64         generation.incrementAndGet()
65     }
66
67     fun markProjectDirty() = markDirty(projectDirty = true)
68
69     fun markLayerDirty(layerId: Long) = markDirty(
70         projectDirty = false,
71         dirtyLayerIds = listOf(layerId),
72     )
73
74     /** Stops future work for a deleted layer without touching the GL context. */
75     fun removeLayer(layerId: Long) {
76         GlCheck.checkOnGlThread()
77         if (dirtyLayerIds.remove(layerId)) generation.incrementAndGet()
78     }
79
80     /** Recreates the worker after a GL context was deliberately released. */
81     fun ensureRunning() {
82         GlCheck.checkOnGlThread()
83         if (executor.isShutdown) executor = executorFactory()
84     }
85
86     /**
87      * Captures all currently dirty pixels on the GL thread and starts one
88      * worker job. Calling this while another job runs is intentionally a no-op.
89      */
90     fun buildIfNeeded(layers: LayerStack): Boolean {

```

```

91     GlCheck.checkOnGltThread()
92     if (buildRunning || (!projectDirty && dirtyLayerIds.isEmpty())) return false
93
94     val jobGeneration = generation.get()
95     val dimensions = previewDimensions(layers.canvasWidth, layers.canvasHeight)
96     val projectPixels = if (projectDirty) {
97         renderer.renderProject(layers, dimensions.width, dimensions.height)
98     } else {
99         null
100     }
101     val layerPixels = dirtyLayerIds
102         .mapNotNull { layerId ->
103             layers.findLayerById(layerId)?.let { layer ->
104                 layerId to renderer.renderLayer(layer, dimensions.width, dimensions.height)
105             }
106         }
107         .toMap()
108
109     val request = BuildRequest(
110         generation = jobGeneration,
111         width = dimensions.width,
112         height = dimensions.height,
113         projectPixels = projectPixels,
114         layerPixels = layerPixels,
115     )
116     buildRunning = true
117     try {
118         executor.execute {
119             pendingResults.add(encodeAndPublish(request))
120             requestGltFrame()
121         }
122     } catch (error: RejectedExecutionException) {
123         pendingResults.add(WorkerResult.Failed(jobGeneration, error))
124         requestGltFrame()
125     }
126     return true
127 }
128
129 /**
130  * Finishes worker jobs on the GL thread and starts no UI callback for stale
131  * generations. The next frame can immediately call [buildIfNeeded] again.
132  */
133 fun processCompleted(): Boolean {
134     GlCheck.checkOnGltThread()
135     var changed = false
136     while (true) {
137         val result = pendingResults.poll() ?: break
138         buildRunning = false
139         changed = true
140
141         if (result.generation != generation.get()) {
142             continue
143         }
144
145         when (result) {
146             is WorkerResult.Saved -> {
147                 if (result.projectPreview != null) projectDirty = false
148                 dirtyLayerIds.removeAll(result.layerPreviews.keys)
149                 uiHandler.post {
150                     onSave {
151                         ThumbnailBuildResult(
152                             generation = result.generation,
153                             projectPreview = result.projectPreview,
154                             layerPreviews = result.layerPreviews,
155                         ),
156                     }
157                 }
158             }
159             is WorkerResult.Failed -> uiHandler.post { onFailure(result.error) }
160             is WorkerResult.Stale -> Unit
161         }
162     }
163     return changed
164 }
165
166 /** Makes in-flight work obsolete and releases the background encoder. */
167 fun shutdown() {
168     GlCheck.checkOnGltThread()
169     generation.incrementAndGet()
170     projectDirty = false
171     dirtyLayerIds.clear()
172     buildRunning = false
173     executor.shutdownNow()
174     pendingResults.clear()
175     renderer.release()
176 }
177
178 private fun encodeAndPublish(request: BuildRequest): WorkerResult = try {
179     val temporaryProject = request.projectPixels?.let {
180         temporaryFileFor(projectPreviewFile, request.generation)

```

```

183     }
184     val temporaryLayers = request.layerPixels.keys.associateWith { layerId ->
185         temporaryFileFor(layerPreviewFile(layerId), request.generation)
186     }
187
188     try {
189         request.projectPixels?.let { rgba ->
190             // Project preview is small and opaque in the usual case: 88 is
191             // a good size/quality balance.
192             encoder.encodeTopDown(rgba, request.width, request.height, checkNotNull(temporaryProject), lossless = false)
193         }
194         request.layerPixels.forEach { (layerId, rgba) ->
195             // Layer previews must preserve alpha and thin marks.
196             encoder.encodeTopDown(rgba, request.width, request.height, checkNotNull(temporaryLayers[layerId]), lossless =
true)
197         }
198
199         if (request.generation != generation.get()) {
200             temporaryProject?.delete()
201             temporaryLayers.values.forEach(File::delete)
202             WorkerResult.Stale(request.generation)
203         } else {
204             val publishedProject = temporaryProject?.let {
205                 publish(it, projectPreviewFile)
206                 projectPreviewFile
207             }
208             val publishedLayers = temporaryLayers.mapNotNull { (layerId, temporary) ->
209                 val target = layerPreviewFile(layerId)
210                 publish(temporary, target)
211                 layerId to target
212             }.toMap()
213             WorkerResult.Saved(request.generation, publishedProject, publishedLayers)
214         }
215     } catch (error: Throwable) {
216         temporaryProject?.delete()
217         temporaryLayers.values.forEach(File::delete)
218         throw error
219     }
220 } catch (error: Throwable) {
221     WorkerResult.Failed(request.generation, error)
222 }
223
224 private fun publish(temporary: File, destination: File) {
225     destination.parentFile?.mkdirs()
226     try {
227         Files.move(
228             temporary.toPath(),
229             destination.toPath(),
230             StandardCopyOption.ATOMIC_MOVE,
231             StandardCopyOption.REPLACE_EXISTING,
232         )
233     } catch (_: AtomicMoveNotSupportedException) {
234         Files.move(
235             temporary.toPath(),
236             destination.toPath(),
237             StandardCopyOption.REPLACE_EXISTING,
238         )
239     } catch (error: IOException) {
240         throw IOException("Unable to publish thumbnail ${destination.path}", error)
241     }
242 }
243
244 private fun temporaryFileFor(destination: File, jobGeneration: Long): File {
245     val parent = destination.parentFile ?: error("Thumbnail output requires a parent directory")
246     if (!parent.exists() && !parent.mkdirs()) {
247         throw IOException("Cannot create thumbnail directory: ${parent.path}")
248     }
249     return File(parent, ".$${destination.name}.${jobGeneration.tmp}")
250 }
251
252 private val projectPreviewFile: File
253     get() = File(outputDirectory, PROJECT_PREVIEW_FILE_NAME)
254
255 private fun layerPreviewFile(layerId: Long): File =
256     File(File(outputDirectory, LAYER_PREVIEW_DIRECTORY), "$layerId.webp")
257
258 /** Generates a file with the same aspect ratio as its document. */
259 private fun previewDimensions(canvasWidth: Int, canvasHeight: Int): PreviewDimensions {
260     val width = canvasWidth.coerceAtLeast(1)
261     val height = canvasHeight.coerceAtLeast(1)
262     return if (width >= height) {
263         PreviewDimensions(previewSizePx, (previewSizePx.toLong() * height / width).toInt().coerceAtLeast(1))
264     } else {
265         PreviewDimensions((previewSizePx.toLong() * width / height).toInt().coerceAtLeast(1), previewSizePx)
266     }
267 }
268
269 private data class BuildRequest(
270     val generation: Long,
271     val width: Int,
272     val height: Int,
273     val projectPixels: ByteArray?,

```

```

274         val layerPixels: Map<Long, ByteArray>,
275     )
276
277     private data class PreviewDimensions(val width: Int, val height: Int)
278
279     private sealed interface WorkerResult {
280         val generation: Long
281
282         data class Saved(
283             override val generation: Long,
284             val projectPreview: File?,
285             val layerPreviews: Map<Long, File>,
286         ) : WorkerResult
287
288         data class Failed(
289             override val generation: Long,
290             val error: Throwable,
291         ) : WorkerResult
292
293         data class Stale(
294             override val generation: Long,
295         ) : WorkerResult
296     }
297
298
299     private companion object {
300         const val DEFAULT_PREVIEW_SIZE_PX = 256
301         const val PROJECT_PREVIEW_FILE_NAME = "thumbnail.webp"
302         const val LAYER_PREVIEW_DIRECTORY = "previews"
303     }
304 }

```

83. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailEncoder.kt

```

1 package com.wetinknext.engine.thumbnail
2
3 import android.graphics.Bitmap
4 import android.os.Build
5 import java.io.File
6 import kotlin.math.pow
7 import kotlin.math.roundToInt
8
9 /**
10  * Persists a raw OpenGL thumbnail as WebP.
11  *
12  * [rgba] follows [android.opengl.GLES30.glReadPixels] row ordering: the first
13  * row is the bottom row. The encoder converts it to Android Bitmap's top-down
14  * order while preserving RGBA alpha exactly before WebP compression.
15  */
16 class ThumbnailEncoder {
17     fun encode(
18         rgba: ByteArray,
19         width: Int,
20         height: Int,
21         output: File,
22         lossless: Boolean,
23     ) = encodeInternal(
24         rgba = rgba,
25         width = width,
26         height = height,
27         output = output,
28         lossless = lossless,
29         inputRowsBottomUp = true,
30     )
31
32     /**
33      * Writes an image returned by [ThumbnailRenderer], whose rows have already
34      * been normalised into Android's top-to-bottom order.
35      */
36     fun encodeTopDown(
37         rgba: ByteArray,
38         width: Int,
39         height: Int,
40         output: File,
41         lossless: Boolean,
42     ) = encodeInternal(
43         rgba = rgba,
44         width = width,
45         height = height,
46         output = output,
47         lossless = lossless,
48         inputRowsBottomUp = false,
49     )
50
51     private fun encodeInternal(
52         rgba: ByteArray,
53         width: Int,
54         height: Int,
55         output: File,
56         lossless: Boolean,
57         inputRowsBottomUp: Boolean,
58     ) {
59         require(width > 0 && height > 0) { "Thumbnail dimensions must be positive" }
60         val expectedByteCount = width * height * RGBA_BYTES_PER_PIXEL
61         require(rgba.size == expectedByteCount) {
62             "RGBA byte count ${rgba.size} does not match ${width}x$height"
63         }
64
65         val pixels = IntArray(width * height)
66         for (bitmapY in 0 until height) {
67             // Bitmap begins at the top; glReadPixels begins at the bottom.
68             val sourceY = if (inputRowsBottomUp) height - 1 - bitmapY else bitmapY
69             var source = sourceY * width * RGBA_BYTES_PER_PIXEL
70             val destination = bitmapY * width
71             for (x in 0 until width) {
72                 val redLinear = (rgba[source++].toInt() and 0xFF) / 255f
73                 val greenLinear = (rgba[source++].toInt() and 0xFF) / 255f
74                 val blueLinear = (rgba[source++].toInt() and 0xFF) / 255f
75                 val alpha = rgba[source++].toInt() and 0xFF
76                 val red = (linearToSrgb(redLinear) * 255f).roundToInt().coerceIn(0, 255)
77                 val green = (linearToSrgb(greenLinear) * 255f).roundToInt().coerceIn(0, 255)
78                 val blue = (linearToSrgb(blueLinear) * 255f).roundToInt().coerceIn(0, 255)
79                 pixels[destination + x] =
80                     (alpha shl 24) or (red shl 16) or (green shl 8) or blue
81             }
82         }
83
84         output.parentFile?.mkdirs()
85         val bitmap = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)
86         try {
87             bitmap.setPixels(pixels, 0, width, 0, width, height)
88             output.outputStream().use { stream ->
89                 check(bitmap.compress(webpFormat(lossless), quality(lossless), stream)) {
90                     "WebP encoding failed for ${output.absolutePath}"
91                 }
92             }
93         }
94     }
95 }

```



```

91         }
92     }
93 } finally {
94     bitmap.recycle()
95 }
96 }
97
98 private fun webpFormat(lossless: Boolean): Bitmap.CompressFormat = when {
99     lossless && Build.VERSION.SDK_INT >= Build.VERSION_CODES.R ->
100         Bitmap.CompressFormat.WEBP_LOSSLESS
101     !lossless && Build.VERSION.SDK_INT >= Build.VERSION_CODES.R ->
102         Bitmap.CompressFormat.WEBP_LOSSY
103     else -> Bitmap.CompressFormat.WEBP
104 }
105
106 private fun quality(lossless: Boolean): Int = if (lossless) 100 else PROJECT_PREVIEW_QUALITY
107
108 private fun linearToSrgb(value: Float): Float {
109     val linear = value.coerceIn(0f, 1f)
110     return if (linear <= 0.0031308f) linear * 12.92f
111         else (1.055 * linear.toDouble().pow(1.0 / 2.4) - 0.055).toFloat()
112 }
113
114 private companion object {
115     const val RGBA_BYTES_PER_PIXEL = 4
116     const val PROJECT_PREVIEW_QUALITY = 88
117 }
118 }

```

84. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailPreviewFiles.kt

```

1 package com.wetinknext.engine.thumbnail
2
3 import java.io.File
4 import java.io.IOException
5 import java.nio.file.AtomicMoveNotSupportedException
6 import java.nio.file.Files
7 import java.nio.file.StandardCopyOption
8
9 /** Cache-file contract shared by preview restoration and the GL build scheduler. */
10 object ThumbnailPreviewFiles {
11     fun layerPreviewFile(outputDirectory: File, layerId: Long): File =
12         File(File(outputDirectory, LAYER_PREVIEW_DIRECTORY), "$layerId.webp")
13
14     /**
15      * Restores previews unpacked from a .wetink container to cache atomically.
16      * It deliberately performs no bitmap decoding, so this work remains cheap
17      * and safe to execute before the GL session is ready.
18      */
19     fun restoreLayerPreviews(
20         outputDirectory: File,
21         previews: Map<Long, ByteArray>,
22     ): Map<Long, File> = previews.mapNotNull { (layerId, bytes) ->
23         if (bytes.isEmpty()) return@mapNotNull null
24         val target = layerPreviewFile(outputDirectory, layerId)
25         val parent = target.parentFile ?: return@mapNotNull null
26         if (!parent.exists() && !parent.mkdirs()) {
27             throw IOException("Cannot create preview directory: ${parent.path}")
28         }
29         val temporary = File(parent, ".$${target.name}.restore.tmp")
30         try {
31             temporary.outputStream().use { it.write(bytes) }
32             publish(temporary, target)
33             layerId to target
34         } finally {
35             if (temporary.exists()) temporary.delete()
36         }
37     }.toMap()
38
39     private fun publish(temporary: File, destination: File) {
40         try {
41             Files.move(
42                 temporary.toPath(),
43                 destination.toPath(),
44                 StandardCopyOption.ATOMIC_MOVE,
45                 StandardCopyOption.REPLACE_EXISTING,
46             )
47         } catch (_: AtomicMoveNotSupportedException) {
48             Files.move(
49                 temporary.toPath(),
50                 destination.toPath(),
51                 StandardCopyOption.REPLACE_EXISTING,
52             )
53         }
54     }
55
56     private const val LAYER_PREVIEW_DIRECTORY = "previews"
57 }

```

85. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailRenderer.kt

```

1 package com.wetinknext.engine.thumbnail
2
3 import android.opengl.GLES30
4 import com.wetinknext.engine.canvas.Compositor
5 import com.wetinknext.engine.canvas.LayerStack
6 import com.wetinknext.engine.canvas.PaintLayer
7 import com.wetinknext.engine.gl.CanvasGeometry
8 import com.wetinknext.engine.gl.GlCheck
9 import com.wetinknext.engine.gl.RenderTarget
10 import java.nio.ByteBuffer
11 import java.nio.ByteOrder
12
13 /**
14  * GL-thread renderer for document thumbnails.
15  *
16  * It renders directly from document render targets, never from the surface
17  * framebuffer. Camera position, zoom, editor chrome and Android system UI can
18  * therefore never leak into a preview.
19  */
20 class ThumbnailRenderer {
21     private val target = RenderTarget()
22     private val compositor = Compositor()
23     private var geometry: CanvasGeometry? = null
24     private var geometryWidth = 0
25     private var geometryHeight = 0
26     private val canvasToClip = FloatArray(16)
27     private var readBuffer: ByteBuffer? = null
28     private var created = false
29
30     /**
31      * Returns top-to-bottom RGBA8 bytes for one layer.
32      *
33      * A hidden layer still receives a preview of its own pixels; the visibility
34      * flag belongs to project composition, whereas a layer thumbnail is used to
35      * identify that layer in the layers panel. Layer opacity is preserved.
36      */
37     fun renderLayer(
38         layer: PaintLayer,
39         outputWidth: Int,
40         outputHeight: Int,
41     ): ByteArray {
42         GlCheck.checkOnGLThread()
43         require(outputWidth > 0 && outputHeight > 0) { "Thumbnail size must be positive" }
44         if (!layer.created || layer.target.width <= 0 || layer.target.height <= 0) {
45             return ByteArray(outputWidth * outputHeight * RGBA_BYTES_PER_PIXEL)
46         }
47
48         prepare(
49             canvasWidth = layer.target.width,
50             canvasHeight = layer.target.height,
51             outputWidth = outputWidth,
52             outputHeight = outputHeight,
53         )
54         target.clear(0f, 0f, 0f, 0f)
55         target.bind()
56         GLES30.glViewport(0, 0, outputWidth, outputHeight)
57         try {
58             compositor.renderLayer(
59                 geometry = checkNotNull(geometry),
60                 layer = layer,
61                 canvasWidth = layer.target.width,
62                 canvasHeight = layer.target.height,
63                 canvasToClip = canvasToClip,
64             )
65             return readTopDownRgba(outputWidth, outputHeight)
66         } finally {
67             GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
68         }
69     }
70
71     /**
72      * Returns a top-to-bottom RGBA8 image of all visible layers, from the
73      * bottom of [layers] to the top. The compositor applies every layer's
74      * opacity and follows the same blend-policy path as the on-canvas view.
75      */
76     fun renderProject(
77         layers: LayerStack,
78         outputWidth: Int,
79         outputHeight: Int,
80     ): ByteArray {
81         GlCheck.checkOnGLThread()
82         require(outputWidth > 0 && outputHeight > 0) { "Thumbnail size must be positive" }
83         require(layers.canvasWidth > 0 && layers.canvasHeight > 0) {
84             "Cannot render a thumbnail for an empty canvas"
85         }
86
87         prepare(
88             canvasWidth = layers.canvasWidth,
89             canvasHeight = layers.canvasHeight,
90             outputWidth = outputWidth,

```

```

91         outputHeight = outputHeight,
92     )
93     target.clear(0f, 0f, 0f, 0f)
94     target.bind()
95     GLES30.glViewport(0, 0, outputWidth, outputHeight)
96     try {
97         compositor.render(
98             geometry = checkNotNull(geometry),
99             layers = layers,
100             activeLayerId = NO_ACTIVE_LAYER,
101             strokeTextureId = 0,
102             strokeErase = false,
103             canvasToClip = canvasToClip,
104             strokeOpacity = 1f,
105         )
106         return readTopDownRgba(outputWidth, outputHeight)
107     } finally {
108         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
109     }
110 }
111
112 /** Releases GL objects. Call while the context is current. */
113 fun release() {
114     GlCheck.checkOnGThread()
115     readBuffer = null
116     target.release()
117     geometry?.release()
118     geometry = null
119     geometryWidth = 0
120     geometryHeight = 0
121     if (created) compositor.release()
122     created = false
123 }
124
125 private fun prepare(
126     canvasWidth: Int,
127     canvasHeight: Int,
128     outputWidth: Int,
129     outputHeight: Int,
130 ) {
131     if (!created) {
132         compositor.create()
133         created = true
134     }
135     if (geometry == null || geometryWidth != canvasWidth || geometryHeight != canvasHeight) {
136         geometry?.release()
137         geometry = CanvasGeometry().also { it.create(canvasWidth, canvasHeight) }
138         geometryWidth = canvasWidth
139         geometryHeight = canvasHeight
140     }
141     if (target.width != outputWidth || target.height != outputHeight) {
142         target.create(outputWidth, outputHeight, preferHalfFloat = false)
143     }
144     buildAspectFitMatrix(canvasWidth, canvasHeight, outputWidth, outputHeight, canvasToClip)
145 }
146
147 private fun readTopDownRgba(width: Int, height: Int): ByteArray {
148     val rowBytes = width * RGBA_BYTES_PER_PIXEL
149     val byteCount = rowBytes * height
150     val buffer = obtainReadBuffer(byteCount)
151     buffer.clear()
152     GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 1)
153     try {
154         GLES30.glReadPixels(
155             0,
156             0,
157             width,
158             height,
159             GLES30.GL_RGBA,
160             GLES30.GL_UNSIGNED_BYTE,
161             buffer,
162         )
163         GlCheck.noError("ThumbnailRenderer glReadPixels")
164
165         val pixels = ByteArray(byteCount)
166         for (destinationRow in 0 until height) {
167             val sourceOffset = (height - 1 - destinationRow) * rowBytes
168             buffer.position(sourceOffset)
169             buffer.get(pixels, destinationRow * rowBytes, rowBytes)
170         }
171         return pixels
172     } finally {
173         GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 4)
174     }
175 }
176
177 private fun obtainReadBuffer(size: Int): ByteBuffer {
178     val current = readBuffer
179     if (current != null && current.capacity() >= size) return current
180     return ByteBuffer.allocateDirect(size)
181         .order(ByteOrder.nativeOrder())
182         .also { readBuffer = it }

```

```

183     }
184
185     private fun buildAspectFitMatrix(
186         canvasWidth: Int,
187         canvasHeight: Int,
188         outputWidth: Int,
189         outputHeight: Int,
190         out: FloatArray,
191     ) {
192         val canvasAspect = canvasWidth.toFloat() / canvasHeight.toFloat()
193         val outputAspect = outputWidth.toFloat() / outputHeight.toFloat()
194         val scaleX = if (canvasAspect < outputAspect) canvasAspect / outputAspect else 1f
195         val scaleY = if (canvasAspect > outputAspect) outputAspect / canvasAspect else 1f
196
197         out.fill(0f)
198         out[0] = 2f * scaleX / canvasWidth.toFloat()
199         out[5] = 2f * scaleY / canvasHeight.toFloat()
200         out[10] = 1f
201         out[12] = -scaleX
202         out[13] = -scaleY
203         out[15] = 1f
204     }
205
206     private companion object {
207         const val RGBA_BYTES_PER_PIXEL = 4
208         const val NO_ACTIVE_LAYER = -1L
209     }
210 }

```

86. app/src/main/java/com/wetinknext/engine/undo/DeflateTileCompressor.kt

```

1  package com.wetinknext.engine.undo
2
3  import java.io.ByteArrayOutputStream
4  import java.util.zip.Deflater
5  import java.util.zip.Inflater
6
7  class DeflateTileCompressor(
8      private val level: Int = Deflater.BEST_SPEED,
9  ) : TileCompressor {
10
11      override fun compress(data: ByteArray): ByteArray {
12          val deflater = Deflater(level)
13          return try {
14              deflater.setInput(data)
15              deflater.finish()
16
17              val output = ByteArrayOutputStream(data.size)
18              val buffer = ByteArray(16 * 1024)
19
20              while (!deflater.finished()) {
21                  val count = deflater.deflate(buffer)
22                  output.write(buffer, 0, count)
23              }
24
25              output.toByteArray()
26          } finally {
27              deflater.end()
28          }
29      }
30
31      override fun decompress(
32          data: ByteArray,
33          expectedSize: Int,
34      ): ByteArray {
35          val inflater = Inflater()
36          return try {
37              inflater.setInput(data)
38
39              val output = ByteArrayOutputStream(expectedSize)
40              val buffer = ByteArray(16 * 1024)
41
42              while (!inflater.finished()) {
43                  val count = inflater.inflate(buffer)
44                  if (count == 0 && inflater.finished()) break
45                  check(count > 0) { "Decompression stalled or data corrupted" }
46                  output.write(buffer, 0, count)
47              }
48
49              output.toByteArray()
50          } finally {
51              inflater.end()
52          }
53      }
54  }

```

87. app/src/main/java/com/wetinknext/engine/undo/RawTileSnapshot.kt

```
1 package com.wetinknext.engine.undo
2
3 /** Uncompressed tile data captured on the GL thread. */
4 class RawTileSnapshot(
5     val coord: TileCoord,
6     val pixelLeft: Int,
7     val pixelTop: Int,
8     val pixelWidth: Int,
9     val pixelHeight: Int,
10    val bytesPerPixel: Int,
11    val rawBytes: ByteArray,
12 ) {
13     val rawSize: Int get() = rawBytes.size
14
15     fun compress(compressor: TileCompressor): TileSnapshot {
16         return TileSnapshot(
17             coord = coord,
18             pixelLeft = pixelLeft,
19             pixelTop = pixelTop,
20             pixelWidth = pixelWidth,
21             pixelHeight = pixelHeight,
22             bytesPerPixel = bytesPerPixel,
23             storedData = compressor.compress(rawBytes),
24             compressor = compressor,
25         )
26     }
27 }
```

88. app/src/main/java/com/wetinknext/engine/undo/TileCompressor.kt

```
1 package com.wetinknext.engine.undo
2
3 /**
4  * Extension point for P6.5 compression. P6 uses the identity implementation,
5  * so the 96 MB budget reflects the exact bytes that are currently retained.
6  */
7 interface TileCompressor {
8     fun compress(data: ByteArray): ByteArray
9
10    fun decompress(data: ByteArray, expectedSize: Int): ByteArray
11 }
12
13 /** P6 storage policy: no compression and no extra byte-array allocation. */
14 object IdentityTileCompressor : TileCompressor {
15     override fun compress(data: ByteArray): ByteArray = data
16
17     override fun decompress(data: ByteArray, expectedSize: Int): ByteArray {
18         check(data.size == expectedSize) {
19             "Raw tile has ${data.size} bytes, expected $expectedSize"
20         }
21         return data
22     }
23 }
```


89. app/src/main/java/com/wetinknext/engine/undo/TileSnapshot.kt

```
1 package com.wetinknext.engine.undo
2
3 data class TileCoord(val tx: Int, val ty: Int)
4
5 /** One full 256 px tile (or an edge tile) in the source layer's native format. */
6 class TileSnapshot(
7     val coord: TileCoord,
8     val pixelLeft: Int,
9     val pixelTop: Int,
10    val pixelWidth: Int,
11    val pixelHeight: Int,
12    val bytesPerPixel: Int,
13    private val storedData: ByteArray,
14    private val compressor: TileCompressor = IdentityTileCompressor,
15 ) {
16     val memorySize: Int get() = storedData.size
17     val rawSize: Int get() = pixelWidth * pixelHeight * bytesPerPixel
18
19     /** Returns raw bytes. */
20     fun decompress(): ByteArray = compressor.decompress(storedData, rawSize)
21
22     fun dispose() {
23         // P6 keeps heap ByteArrays; a later off-heap implementation can release here.
24     }
25
26     companion object {
27         const val TILE_SIZE = 256
28         const val BYTES_PER_PIXEL_RGBA16F = 8
29         const val BYTES_PER_PIXEL_RGBA8 = 4
30     }
31 }
```

90. app/src/main/java/com/wetinknext/engine/undo/TileSnapshotCapture.kt

```

1 package com.wetinknext.engine.undo
2 import android.opengl.GLES30
3 import android.util.Log
4 import com.wetinknext.BuildConfig
5 import com.wetinknext.engine.gl.GlCheck
6 import com.wetinknext.engine.gl.RenderTarget
7 import java.nio.ByteBuffer
8 import java.nio.ByteOrder
9 object TileSnapshotCapture {
10     private var reusable: ByteBuffer? = null
11
12     /**
13      * Returns the number of whole undo tiles intersecting [bounds].
14      *
15      * This is intentionally CPU-only: callers can use it to flag expensive
16      * readback before calling [capture], without changing undo semantics.
17      */
18     fun countTiles(target: RenderTarget, bounds: IntArray): Int {
19         require(bounds.size >= 4)
20
21         val left = bounds[0].coerceIn(0, target.width)
22         val top = bounds[1].coerceIn(0, target.height)
23         val right = bounds[2].coerceIn(0, target.width)
24         val bottom = bounds[3].coerceIn(0, target.height)
25         if (right <= left || bottom <= top) return 0
26
27         val firstTileX = left / TileSnapshot.TILE_SIZE
28         val lastTileX = (right - 1) / TileSnapshot.TILE_SIZE
29         val firstTileY = top / TileSnapshot.TILE_SIZE
30         val lastTileY = (bottom - 1) / TileSnapshot.TILE_SIZE
31         return (lastTileX - firstTileX + 1) *
32             (lastTileY - firstTileY + 1)
33     }
34
35     /** Captures every 256 px tile intersecting [bounds] = left, top, right, bottom. */
36     fun capture(target: RenderTarget, bounds: IntArray): List<RawTileSnapshot> {
37         GlCheck.checkOnGLThread()
38         require(target.framebufferId != 0)
39         require(target.textureId != 0)
40         require(bounds.size >= 4)
41         val left = bounds[0].coerceIn(0, target.width)
42         val top = bounds[1].coerceIn(0, target.height)
43         val right = bounds[2].coerceIn(0, target.width)
44         val bottom = bounds[3].coerceIn(0, target.height)
45         if (right <= left || bottom <= top) return emptyList()
46
47         if (BuildConfig.DEBUG) {
48             Log.d(
49                 TAG,
50                 "capture target=${target.textureId} fbo=${target.framebufferId} " +
51                 "canvasBounds=[$left,$top,$right,$bottom] " +
52                 "target=${target.width}x${target.height}",
53             )
54         }
55
56         val bytesPerPixel = if (target.usesHalfFloat) {
57             TileSnapshot.BYTES_PER_PIXEL_RGBA16F
58         } else {
59             TileSnapshot.BYTES_PER_PIXEL_RGBA8
60         }
61         val glType = if (target.usesHalfFloat) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE
62         val result = ArrayList<RawTileSnapshot>()
63         target.bind()
64         GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 1)
65         try {
66             for (tileY in top / TileSnapshot.TILE_SIZE..(bottom - 1) / TileSnapshot.TILE_SIZE) {
67                 for (tileX in left / TileSnapshot.TILE_SIZE..(right - 1) / TileSnapshot.TILE_SIZE) {
68                     val pixelLeft = tileX * TileSnapshot.TILE_SIZE
69                     // Tile coordinates are document/canvas coordinates (Y grows down).
70                     val canvasTop = tileY * TileSnapshot.TILE_SIZE
71                     val pixelWidth = minOf(TileSnapshot.TILE_SIZE, target.width - pixelLeft)
72                     val pixelHeight = minOf(TileSnapshot.TILE_SIZE, target.height - canvasTop)
73                     // buildCanvasToFbo maps canvas y=0 to the FBO's bottom row.
74                     // Canvas tile coordinates therefore already equal GL's bottom Y.
75                     val glBottom = canvasTop
76                     if (BuildConfig.DEBUG) {
77                         Log.d(
78                             TAG,
79                             "capture tile=($tileX,$tileY) " +
80                             "canvas=[$pixelLeft,$canvasTop,$pixelWidth,$pixelHeight] " +
81                             "gl=[$pixelLeft,$glBottom,$pixelWidth,$pixelHeight]",
82                         )
83                     }
84                     val byteCount = pixelWidth * pixelHeight * bytesPerPixel
85                     val buffer = obtainBuffer(byteCount)
86                     buffer.clear()
87                     GLES30.glReadPixels(
88                         pixelLeft, glBottom, pixelWidth, pixelHeight,
89                         GLES30.GL_RGBA, glType, buffer,
90                     )

```

```

91         if (BuildConfig.DEBUG) {
92             val error = GLES30.glGetError()
93             check(error == GLES30.GL_NO_ERROR) {
94                 "TileSnapshotCapture.glReadPixels failed: 0x${error.toString(16)}"
95             }
96         }
97         buffer.rewind()
98         val rawBytes = ByteArray(byteCount)
99         buffer.get(rawBytes)
100         result += RawTileSnapshot(
101             coord = TileCoord(tileX, tileY),
102             pixelLeft = pixelLeft,
103             pixelTop = canvasTop,
104             pixelWidth = pixelWidth,
105             pixelHeight = pixelHeight,
106             bytesPerPixel = bytesPerPixel,
107             rawBytes = rawBytes,
108         )
109     }
110 }
111     return result
112 } finally {
113     GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 4)
114     GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
115 }
116 }
117
118 private fun obtainBuffer(size: Int): ByteBuffer {
119     val current = reusable
120     if (current != null && current.capacity() >= size) return current
121     return ByteBuffer.allocateDirect(size).order(ByteOrder.nativeOrder()).also { reusable = it }
122 }
123
124 private const val TAG = "TileUndo"
125 }

```

91. app/src/main/java/com/wetinknext/engine/undo/TileSnapshotRestore.kt

```

1  package com.wetinknext.engine.undo
2
3  import android.opengl.GLES30
4  import android.util.Log
5  import com.wetinknext.BuildConfig
6  import com.wetinknext.engine.gl.GlCheck
7  import com.wetinknext.engine.gl.RenderTarget
8  import java.nio.ByteBuffer
9  import java.nio.ByteOrder
10
11 /** Restores snapshot tiles into an already-created layer texture on the GL thread. */
12 object TileSnapshotRestore {
13     private var reusableBuffer: ByteBuffer? = null
14
15     fun restore(target: RenderTarget, tiles: List<TileSnapshot>): Boolean {
16         GlCheck.checkOnGLThread()
17         if (target.textureId == 0) return false
18         if (tiles.isEmpty()) return true
19
20         // Discard an unrelated previous error, then verify every upload below.
21         GLES30.glGetError()
22         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
23         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, target.textureId)
24         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 1)
25         try {
26             for (tile in tiles) {
27                 if (!isValidTile(target, tile)) return false
28                 val data = tile.decompress()
29                 if (data.size != tile.rawSize) return false
30
31                 val glType = when (tile.bytesPerPixel) {
32                     TileSnapshot.BYTES_PER_PIXEL_RGBA16F -> GLES30.GL_HALF_FLOAT
33                     TileSnapshot.BYTES_PER_PIXEL_RGBA8 -> GLES30.GL_UNSIGNED_BYTE
34                     else -> return false
35                 }
36                 val buffer = obtainBuffer(data.size)
37                 buffer.clear()
38                 buffer.put(data)
39                 buffer.position(0)
40
41                 // buildCanvasToFbo maps canvas y=0 to the FBO's bottom row.
42                 val glBottom = tile.pixelTop
43                 if (BuildConfig.DEBUG) {
44                     Log.d(TAG, "restore tex=${target.textureId} tile=(${tile.coord.tx},${tile.coord.ty}) " +
45                         "canvas=(${tile.pixelLeft},${tile.pixelTop},${tile.pixelWidth},${tile.pixelHeight})")
46                 }
47                 GLES30.glTexSubImage2D(
48                     GLES30.GL_TEXTURE_2D, 0, tile.pixelLeft, glBottom,
49                     tile.pixelWidth, tile.pixelHeight, GLES30.GL_RGBA, glType, buffer,
50                 )
51                 if (GLES30.glGetError() != GLES30.GL_NO_ERROR) return false
52             }
53             return true
54         } catch (error: Throwable) {
55             if (BuildConfig.DEBUG) Log.e(TAG, "Restore failed", error)
56             return false
57         } finally {
58             GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 4)
59             GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
60             GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
61             GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
62         }
63     }
64
65     private fun isValidTile(target: RenderTarget, tile: TileSnapshot): Boolean {
66         if (tile.pixelWidth <= 0 || tile.pixelHeight <= 0) return false
67         if (tile.pixelLeft < 0 || tile.pixelTop < 0) return false
68         if (tile.pixelLeft + tile.pixelWidth > target.width ||
69             tile.pixelTop + tile.pixelHeight > target.height) return false
70         val expectedBytesPerPixel = if (target.usesHalfFloat) {
71             TileSnapshot.BYTES_PER_PIXEL_RGBA16F
72         } else {
73             TileSnapshot.BYTES_PER_PIXEL_RGBA8
74         }
75         return tile.bytesPerPixel == expectedBytesPerPixel
76     }
77
78     private fun obtainBuffer(size: Int): ByteBuffer {
79         val current = reusableBuffer
80         if (current != null && current.capacity() >= size) return current
81         return ByteBuffer.allocateDirect(size).order(ByteOrder.nativeOrder()).also { reusableBuffer = it }
82     }
83
84     private const val TAG = "TileUndo"
85 }

```

92. app/src/main/java/com/wetinknext/engine/undo/UndoEntry.kt

```
1 package com.wetinknext.engine.undo
2
3 /** One reversible history operation, retaining exact states from before and after its merge. */
4 data class UndoEntry(
5     val layerId: Long,
6     val beforeTiles: List<TileSnapshot>,
7     val afterTiles: List<TileSnapshot>,
8     val operation: UndoOperationType = UndoOperationType.TILE_EDIT,
9     val tag: String = "stroke",
10 ) {
11     val memorySize: Int
12     get() = beforeTiles.sumOf(TileSnapshot::memorySize) + afterTiles.sumOf(TileSnapshot::memorySize)
13
14     fun dispose() {
15         beforeTiles.forEach(TileSnapshot::dispose)
16         afterTiles.forEach(TileSnapshot::dispose)
17     }
18 }
```

93. app/src/main/java/com/wetinknext/engine/undo/UndoManager.kt

```

1 package com.wetinknext.engine.undo
2
3 /**
4  * Pure memory-bounded history. GPU reads and texture restores deliberately live
5  * in EngineRenderer, keeping this class deterministic and unit-testable.
6  */
7 class UndoManager(
8     private val maxMemoryBytes: Long = DEFAULT_MAX_MEMORY_BYTES,
9     private val maxSteps: Int = DEFAULT_MAX_STEPS,
10 ) {
11     private val undoStack = ArrayDeque<UndoEntry>()
12     private val redoStack = ArrayDeque<UndoEntry>()
13     private var undoMemoryBytes = 0L
14     private var redoMemoryBytes = 0L
15
16     val canUndo: Boolean get() = undoStack.isNotEmpty()
17     val canRedo: Boolean get() = redoStack.isNotEmpty()
18     val memoryBytes: Long get() = undoMemoryBytes + redoMemoryBytes
19     val undoCount: Int get() = undoStack.size
20     val redoCount: Int get() = redoStack.size
21
22     /** Records a new operation and invalidates the alternate redo history. */
23     fun push(entry: UndoEntry) {
24         undoStack.addLast(entry)
25         undoMemoryBytes += entry.memorySize
26         clearRedo()
27         trimToLimits()
28     }
29
30     /** Returns the candidate without changing history; restore it before committing. */
31     fun peekUndo(): UndoEntry? = undoStack.lastOrNull()
32
33     /** Moves the exact current undo candidate to redo after a successful restore. */
34     fun commitUndo(entry: UndoEntry): Boolean {
35         if (undoStack.lastOrNull() != entry) return false
36         undoStack.removeLast()
37         undoMemoryBytes -= entry.memorySize
38         redoStack.addLast(entry)
39         redoMemoryBytes += entry.memorySize
40         trimToLimits()
41         return true
42     }
43
44     /** Discards a no-longer-restorable undo record without moving it to redo. */
45     fun dropUndo(entry: UndoEntry): Boolean {
46         if (undoStack.lastOrNull() != entry) return false
47         undoStack.removeLast()
48         undoMemoryBytes -= entry.memorySize
49         entry.dispose()
50         return true
51     }
52
53     /** Returns the candidate without changing history; restore it before committing. */
54     fun peekRedo(): UndoEntry? = redoStack.lastOrNull()
55
56     /** Moves the exact current redo candidate to undo after a successful restore. */
57     fun commitRedo(entry: UndoEntry): Boolean {
58         if (redoStack.lastOrNull() != entry) return false
59         redoStack.removeLast()
60         redoMemoryBytes -= entry.memorySize
61         undoStack.addLast(entry)
62         undoMemoryBytes += entry.memorySize
63         trimToLimits()
64         return true
65     }
66
67     /** Discards a no-longer-restorable redo record without moving it to undo. */
68     fun dropRedo(entry: UndoEntry): Boolean {
69         if (redoStack.lastOrNull() != entry) return false
70         redoStack.removeLast()
71         redoMemoryBytes -= entry.memorySize
72         entry.dispose()
73         return true
74     }
75
76     fun clearRedo() {
77         while (redoStack.isNotEmpty()) {
78             val entry = redoStack.removeLast()
79             redoMemoryBytes -= entry.memorySize
80             entry.dispose()
81         }
82         redoMemoryBytes = 0L
83     }
84
85     fun clear() {
86         while (undoStack.isNotEmpty()) undoStack.removeLast().dispose()
87         undoMemoryBytes = 0L
88         clearRedo()
89     }
90

```

```

91  /** Removes history that can no longer be restored after deleting a layer. */
92  fun removeEntriesForLayer(layerId: Long) {
93      removeEntriesForLayer(undoStack, layerId) { entry -> undoMemoryBytes -= entry.memorySize }
94      removeEntriesForLayer(redoStack, layerId) { entry -> redoMemoryBytes -= entry.memorySize }
95      undoMemoryBytes = undoMemoryBytes.coerceAtLeast(0L)
96      redoMemoryBytes = redoMemoryBytes.coerceAtLeast(0L)
97  }
98
99  private fun removeEntriesForLayer(
100      stack: ArrayDeque<UndoEntry>,
101      layerId: Long,
102      subtractMemory: (UndoEntry) -> Unit,
103  ) {
104      if (stack.isEmpty()) return
105      val retained = ArrayDeque<UndoEntry>(stack.size)
106      while (stack.isNotEmpty()) {
107          val entry = stack.removeFirst()
108          if (entry.layerId == layerId) {
109              subtractMemory(entry)
110              entry.dispose()
111          } else {
112              retained.addLast(entry)
113          }
114      }
115      stack.addAll(retained)
116  }
117
118  private fun trimToLimits() {
119      while (undoStack.size > maxSteps) evictOldestUndo()
120      while (redoStack.size > maxSteps) evictOldestRedo()
121      while (memoryBytes > maxMemoryBytes && undoStack.size + redoStack.size > 1) {
122          when {
123              redoStack.size > 1 && redoMemoryBytes >= undoMemoryBytes -> evictOldestRedo()
124              undoStack.size > 1 -> evictOldestUndo()
125              redoStack.size > 1 -> evictOldestRedo()
126              else -> return
127          }
128      }
129  }
130
131  private fun evictOldestUndo() {
132      val entry = undoStack.removeFirstOrNull() ?: return
133      undoMemoryBytes -= entry.memorySize
134      entry.dispose()
135  }
136
137  private fun evictOldestRedo() {
138      val entry = redoStack.removeFirstOrNull() ?: return
139      redoMemoryBytes -= entry.memorySize
140      entry.dispose()
141  }
142
143  companion object {
144      const val DEFAULT_MAX_MEMORY_BYTES = 96L * 1024L * 1024L
145      const val DEFAULT_MAX_STEPS = 50
146  }
147 }

```

94. app/src/main/java/com/wetinknext/engine/undo/UndoOperationType.kt

```
1 package com.wetinknext.engine.undo
2
3 /**
4  * A lightweight operation discriminator for the current tile-snapshot history.
5  * Future history entries can replace this with full typed operations without
6  * changing existing stored tile edits.
7  */
8 enum class UndoOperationType {
9     TILE_EDIT,
10    CLEAR_LAYER,
11    LAYER_PROPERTIES,
12 }
```


95. app/src/main/java/com/wetinknext/ui/animation/AnimationSettingsMenu.kt

```

1  package com.wetinknext.ui.animation
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.rememberScrollState
8  import androidx.compose.foundation.shape.CircleShape
9  import androidx.compose.foundation.verticalScroll
10 import androidx.compose.foundation.shape.RoundedCornerShape
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.*
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.graphics.Color
17 import androidx.compose.ui.graphics.toArgb
18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.unit.dp
20 import androidx.compose.ui.unit.sp
21 import com.wetinknext.domain.animation.AnimationDocument
22 import com.wetinknext.domain.animation.AnimationPlaybackMode
23 import com.wetinknext.domain.animation.OnionSkinSettings
24 import com.wetinknext.ui.theme.AppTheme
25
26 @Composable
27 fun AnimationSettingsMenu(
28     theme: AppTheme,
29     document: AnimationDocument,
30     onDocumentChange: (AnimationDocument) -> Unit,
31     onDismiss: () -> Unit,
32     modifier: Modifier = Modifier
33 ) {
34     Box(
35         modifier = modifier
36             .width(280.dp)
37             .heightIn(max = 360.dp)
38             .clip(RoundedCornerShape(24.dp))
39             .background(theme.panelBg.copy(alpha = 0.98f))
40             .border(1.dp, theme.panelStroke, RoundedCornerShape(24.dp))
41             .padding(16.dp)
42     ) {
43         Column(
44             modifier = Modifier.verticalScroll(rememberScrollState()),
45             verticalArrangement = Arrangement.spacedBy(10.dp),
46         ) {
47             Text("Настройки анимации", color = theme.textPrimary, fontSize = 16.sp, fontWeight = FontWeight.Bold)
48
49             // Playback Mode
50             Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
51                 Text("Режим", color = theme.textSecondary, fontSize = 12.sp)
52                 Row(
53                     modifier = Modifier.fillMaxWidth(),
54                     horizontalArrangement = Arrangement.spacedBy(8.dp)
55                 ) {
56                     PlaybackModeButton("Цикл", AnimationPlaybackMode.LOOP, document.playbackMode, theme) {
57                         onDocumentChange(document.copy(playbackMode = it))
58                     }
59                     PlaybackModeButton("Туда-сюда", AnimationPlaybackMode.PING_PONG, document.playbackMode, theme) {
60                         onDocumentChange(document.copy(playbackMode = it))
61                     }
62                     PlaybackModeButton("Один раз", AnimationPlaybackMode.ONE_SHOT, document.playbackMode, theme) {
63                         onDocumentChange(document.copy(playbackMode = it))
64                     }
65                 }
66             }
67
68             // FPS
69             Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
70                 Row(horizontalArrangement = Arrangement.SpaceBetween, modifier = Modifier.fillMaxWidth()) {
71                     Text("Скорость (FPS)", color = theme.textSecondary, fontSize = 12.sp)
72                     Text("${document.framesPerSecond}", color = theme.accent, fontSize = 12.sp, fontWeight = FontWeight.Bold)
73                 }
74                 Slider(
75                     value = document.framesPerSecond.toFloat(),
76                     onValueChange = { onDocumentChange(document.copy(framesPerSecond = it.toInt())) },
77                     valueRange = 1f..60f,
78                     colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
79                 )
80             }
81
82             HorizontalDivider(color = theme.panelStroke)
83
84             // Onion Skin Settings
85             Row(
86                 modifier = Modifier.fillMaxWidth(),
87                 horizontalArrangement = Arrangement.SpaceBetween,
88                 verticalAlignment = Alignment.CenterVertically
89             ) {
90                 Text("Калька (Onion Skin)", color = theme.textPrimary, fontSize = 14.sp)

```

```

91         Switch(
92             checked = document.onionSkin.enabled,
93             onCheckedChange = { onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(enabled = it))) },
94             colors = SwitchDefaults.colors(checkedThumbColor = theme.accent, checkedTrackColor = theme.accent.copy(
alpha = 0.5f))
95         )
96     }
97
98     if (document.onionSkin.enabled) {
99         OnionCountSlider("Предыдущие", document.onionSkin.previousFrames, theme) {
100             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(previousFrames = it)))
101         }
102         OnionColorPicker(
103             label = "Цвет предыдущих",
104             selectedArgb = document.onionSkin.previousColorArgb,
105             theme = theme,
106             ) { color ->
107             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(previousColorArgb = color.toArgb())))
108         }
109         OnionCountSlider("Следующие", document.onionSkin.nextFrames, theme) {
110             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(nextFrames = it)))
111         }
112         OnionColorPicker(
113             label = "Цвет следующих",
114             selectedArgb = document.onionSkin.nextColorArgb,
115             theme = theme,
116             ) { color ->
117             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(nextColorArgb = color.toArgb())))
118         }
119
120         Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
121             Text("Прозрачность", color = theme.textSecondary, fontSize = 12.sp)
122             Slider(
123                 value = document.onionSkin.opacity,
124                 onValueChange = { onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(opacity = it)))
},
125                 valueRange = 0f..1f,
126                 colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
127             )
128         }
129     }
130 }
131 }
132 }
133
134 @Composable
135 private fun RowScope.PlaybackModeButton(
136     label: String,
137     mode: AnimationPlaybackMode,
138     current: AnimationPlaybackMode,
139     theme: AppTheme,
140     onClick: (AnimationPlaybackMode) -> Unit
141 ) {
142     val selected = mode == current
143     Box(
144         modifier = Modifier
145             .weight(1f)
146             .height(32.dp)
147             .clip(RoundedCornerShape(8.dp))
148             .background(if (selected) theme.accent else theme.panelInsetSoft)
149             .clickable { onClick(mode) },
150         contentAlignment = Alignment.Center
151     ) {
152         Text(label, color = if (selected) Color.White else theme.textSecondary, fontSize = 10.sp)
153     }
154 }
155
156 @Composable
157 private fun OnionCountSlider(
158     label: String,
159     value: Int,
160     theme: AppTheme,
161     onValueChange: (Int) -> Unit
162 ) {
163     Column(verticalArrangement = Arrangement.spacedBy(2.dp)) {
164         Row(horizontalArrangement = Arrangement.SpaceBetween, modifier = Modifier.fillMaxWidth()) {
165             Text(label, color = theme.textSecondary, fontSize = 11.sp)
166             Text("$value", color = theme.textPrimary, fontSize = 11.sp)
167         }
168         Slider(
169             value = value.toFloat(),
170             onValueChange = { onValueChange(it.toInt()) },
171             valueRange = 0f..12f,
172             colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
173         )
174     }
175 }
176
177 @Composable
178 private fun OnionColorPicker(
179     label: String,
180     selectedArgb: Int,

```

```

181     theme: AppTheme,
182     onColorChange: (Color) -> Unit,
183 ) {
184     val palette = listOf(
185         Color(0xFFFFF4FA3),
186         Color(0xFFFFF7A59),
187         Color(0xFFFFFC857),
188         Color(0xFF57D68D),
189         Color(0xFF4A8DFF),
190         Color(0xFFB079FF),
191     )
192     Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
193         Text(label, color = theme.textSecondary, fontSize = 12.sp)
194         Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
195             palette.forEach { color ->
196                 val selected = color.toArgb() == selectedArgb
197                 Box(
198                     modifier = Modifier
199                         .size(24.dp)
200                         .clip(CircleShape)
201                         .background(color)
202                         .border(
203                             width = if (selected) 2.dp else 1.dp,
204                             color = if (selected) Color.White else theme.panelStroke,
205                             shape = CircleShape,
206                         )
207                     .clickable { onColorChange(color) }
208                 )
209             }
210         }
211     }
212 }

```

96. app/src/main/java/com/wetinknext/ui/animation/AnimationTimelineToolbar.kt

```

1 package com.wetinknext.ui.animation
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.border
6 import androidx.compose.foundation.clickable
7 import androidx.compose.foundation.gestures.detectDragGesturesAfterLongPress
8 import androidx.compose.foundation.layout.*
9 import androidx.compose.foundation.lazy.LazyRow
10 import androidx.compose.foundation.lazy.itemsIndexed
11 import androidx.compose.foundation.lazy.rememberLazyListState
12 import androidx.compose.foundation.shape.CircleShape
13 import androidx.compose.foundation.shape.RoundedCornerShape
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.filled.*
16 import androidx.compose.material3.*
17 import androidx.compose.runtime.*
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.draw.clip
21 import androidx.compose.ui.graphics.Color
22 import androidx.compose.ui.graphics.ImageBitmap
23 import androidx.compose.ui.graphics.graphicsLayer
24 import androidx.compose.ui.graphics.vector.ImageVector
25 import androidx.compose.ui.input.pointer.pointerInput
26 import androidx.compose.ui.layout.ContentScale
27 import androidx.compose.ui.platform.LocalDensity
28 import androidx.compose.ui.text.font.FontWeight
29 import androidx.compose.ui.unit.IntOffset
30 import androidx.compose.ui.unit.dp
31 import androidx.compose.ui.unit.sp
32 import androidx.compose.ui.window.Popup
33 import com.wetinknext.domain.animation.AnimationDocument
34 import com.wetinknext.domain.animation.AnimationFrame
35 import com.wetinknext.domain.animation.AnimationFrameRole
36 import com.wetinknext.ui.theme.AppTheme
37 import kotlin.math.roundToInt
38
39 data class FrameThumbnailLayer(
40     val image: ImageBitmap,
41     val opacity: Float,
42 )
43
44 @Composable
45 fun AnimationTimelineToolbar(
46     theme: AppTheme,
47     document: AnimationDocument,
48     currentFrameId: Long,
49     frameThumbnails: Map<Long, List<FrameThumbnailLayer>> = emptyMap(),
50     onFrameSelect: (Long) -> Unit,
51     onFrameMove: (frameId: Long, direction: Int) -> Unit,
52     onAddFrame: () -> Unit,
53     onDuplicateFrame: (Long) -> Unit,
54     onDeleteFrame: (Long) -> Unit,
55     onHoldChange: (Long, Int) -> Unit,
56     onRoleToggle: (Long, AnimationFrameRole) -> Unit,
57     onAssembleLayers: () -> Unit,
58     onLayersClick: () -> Unit,
59     onClose: () -> Unit,
60     onSettingsClick: () -> Unit = {},
61     isPlaying: Boolean = false,
62     onPlayToggle: () -> Unit = {},
63     modifier: Modifier = Modifier
64 ) {
65     val listState = rememberLazyListState()
66     val frames = document.frames
67     var showContextMenuForId by remember { mutableStateOf<Long?>(null) }
68
69     // Auto-scroll to current frame
70     LaunchedEffect(currentFrameId) {
71         val index = frames.indexOfFirst { it.id == currentFrameId }
72         if (index >= 0) {
73             listState.animateScrollToItem(index)
74         }
75     }
76
77     Column(
78         modifier = modifier
79             .fillMaxWidth()
80             .padding(bottom = 16.dp),
81         horizontalAlignment = Alignment.CenterHorizontally
82     ) {
83         Row(
84             modifier = Modifier
85                 .widthIn(min = 400.dp, max = 820.dp)
86                 .height(86.dp)
87                 .clip(RoundedCornerShape(28.dp))
88                 .background(theme.panelBg.copy(alpha = 0.94f))
89                 .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
90                 .padding(horizontal = 14.dp, vertical = 8.dp),

```

```

91         verticalAlignment = Alignment.CenterVertically
92     ) {
93         // Play toggle
94         Column(
95             horizontalAlignment = Alignment.CenterHorizontally,
96             modifier = Modifier.padding(end = 4.dp)
97         ) {
98             Box(
99                 modifier = Modifier
100                     .size(44.dp)
101                     .clip(CircleShape)
102                     .background(if (isPlaying) theme.accent.copy(alpha = 0.4f) else theme.accent)
103                     .clickable { onPlayToggle() },
104                 contentAlignment = Alignment.Center
105             ) {
106                 Icon(
107                     if (isPlaying) Icons.Default.Pause else Icons.Default.PlayArrow,
108                     null,
109                     tint = Color.White
110                 )
111             }
112             Text(if (isPlaying) "Пауза" else "Пуск", color = theme.textSecondary, fontSize = 9.sp)
113         }
114
115         Spacer(Modifier.width(10.dp))
116         Box(Modifier.width(1.dp).fillMaxHeight(0.6f).background(theme.panelStroke))
117         Spacer(Modifier.width(10.dp))
118
119         // Frames timeline
120         LazyRow(
121             state = listState,
122             modifier = Modifier.weight(1f),
123             horizontalArrangement = Arrangement.spacedBy(8.dp),
124             verticalAlignment = Alignment.CenterVertically
125         ) {
126             itemsIndexed(frames, key = { _, f -> f.id }) { _, frame ->
127                 TimelineFrameItem(
128                     frame = frame,
129                     isSelected = frame.id == currentFrameId,
130                     thumbnailLayers = frameThumbnails[frame.id].orEmpty(),
131                     theme = theme,
132                     canMoveLeft = frames.indexOf(frame) > 0,
133                     canMoveRight = frames.indexOf(frame) < frames.lastIndex,
134                     onMove = { direction -> onFrameMove(frame.id, direction) },
135                     onClick = {
136                         if (currentFrameId == frame.id) {
137                             showContextMenuForId = frame.id
138                         } else {
139                             onFrameSelect(frame.id)
140                         }
141                     }
142                 )
143             }
144         }
145
146         Spacer(Modifier.width(10.dp))
147         Box(Modifier.width(1.dp).fillMaxHeight(0.6f).background(theme.panelStroke))
148         Spacer(Modifier.width(10.dp))
149
150         // Actions
151         Row(
152             verticalAlignment = Alignment.CenterVertically,
153             horizontalArrangement = Arrangement.spacedBy(12.dp)
154         ) {
155             TimelineActionIcon(Icons.Default.Add, "Кадр", theme, onClick = onAddFrame)
156             TimelineActionIcon(Icons.Default.Layers, "Собрать", theme, onClick = onAssembleLayers)
157             TimelineActionIcon(Icons.Default.ContentCopy, "Копия", theme, onClick = { onDuplicateFrame(currentFrameId) })
158             TimelineActionIcon(Icons.Default.Delete, "Удалить", theme, enabled = frames.size > 1, onClick = {
onDeleteFrame(currentFrameId) })
159             TimelineActionIcon(Icons.Default.Settings, "Настр.", theme, onClick = onSettingsClick)
160             TimelineActionIcon(Icons.Default.Layers, "Слой", theme, onClick = onLayersClick)
161             TimelineActionIcon(Icons.Default.Close, "Закрыть", theme, onClick = onClose)
162         }
163     }
164
165     // Context Menu
166     showContextMenuForId?.let { frameId ->
167         val frame = frames.find { it.id == frameId }
168         if (frame != null) {
169             Popup(
170                 alignment = Alignment.TopCenter,
171                 offset = IntOffset(0, -220),
172                 onDismissRequest = { showContextMenuForId = null }
173             ) {
174                 FrameContextMenu(
175                     frame = frame,
176                     theme = theme,
177                     onDuplicate = { onDuplicateFrame(frameId); showContextMenuForId = null },
178                     onDelete = { onDeleteFrame(frameId); showContextMenuForId = null },
179                     onHoldChange = { onHoldChange(frameId, it) },
180                     onRoleToggle = { onRoleToggle(frameId, it); showContextMenuForId = null },
181                     canDelete = frames.size > 1

```

```

182     )
183     }
184     }
185     }
186     }
187 }
188
189 @Composable
190 private fun FrameContextMenu(
191     frame: AnimationFrame,
192     theme: AppTheme,
193     onDuplicate: () -> Unit,
194     onDelete: () -> Unit,
195     onHoldChange: (Int) -> Unit,
196     onRoleToggle: (AnimationFrameRole) -> Unit,
197     canDelete: Boolean
198 ) {
199     Surface(
200         modifier = Modifier
201             .width(220.dp)
202             .clip(RoundedCornerShape(16.dp))
203             .border(1.dp, theme.panelStroke, RoundedCornerShape(16.dp)),
204         color = theme.panelBg,
205         tonalElevation = 8.dp
206     ) {
207         Column(modifier = Modifier.padding(12.dp)) {
208             ContextMenuItem("Дублировать", Icons.Default.ContentCopy, theme, onClick = onDuplicate)
209             ContextMenuItem("Удалить", Icons.Default.Delete, theme, enabled = canDelete, onClick = onDelete)
210
211             HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = theme.panelStroke)
212
213             Text("Задержка: ${frame.holdFrames}", color = theme.textSecondary, fontSize = 11.sp, modifier = Modifier.padding(
horizontal = 8.dp))
214             Slider(
215                 value = frame.holdFrames.toFloat(),
216                 onValueChange = { onHoldChange(it.toInt()) },
217                 valueRange = 1f..120f,
218                 colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
219             )
220
221             HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = theme.panelStroke)
222
223             val isBg = frame.role == AnimationFrameRole.BACKGROUND
224             val isFg = frame.role == AnimationFrameRole.FOREGROUND
225
226             Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth().clickable { onRoleToggle(
if (isBg) AnimationFrameRole.NORMAL else AnimationFrameRole.BACKGROUND) }.padding(8.dp)) {
227                 Checkbox(checked = isBg, onCheckedChange = null, colors = CheckboxDefaults.colors(checkedColor = theme.
accent))
228                 Spacer(Modifier.width(8.dp))
229                 Text("Фоновый слой", color = theme.textPrimary, fontSize = 13.sp)
230             }
231             Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth().clickable { onRoleToggle(
if (isFg) AnimationFrameRole.NORMAL else AnimationFrameRole.FOREGROUND) }.padding(8.dp)) {
232                 Checkbox(checked = isFg, onCheckedChange = null, colors = CheckboxDefaults.colors(checkedColor = theme.
accent))
233                 Spacer(Modifier.width(8.dp))
234                 Text("Передний план", color = theme.textPrimary, fontSize = 13.sp)
235             }
236         }
237     }
238 }
239
240 @Composable
241 private fun ContextMenuItem(
242     label: String,
243     icon: ImageVector,
244     theme: AppTheme,
245     enabled: Boolean = true,
246     onClick: () -> Unit
247 ) {
248     Row(
249         modifier = Modifier
250             .fillMaxWidth()
251             .clickable(enabled = enabled) { onClick() }
252             .padding(8.dp)
253             .alphaIfDisabled(!enabled),
254         verticalAlignment = Alignment.CenterVertically
255     ) {
256         Icon(icon, null, tint = if (enabled) theme.textPrimary else theme.textSecondary.copy(alpha = 0.5f), modifier =
Modifier.size(18.dp))
257         Spacer(Modifier.width(12.dp))
258         Text(label, color = if (enabled) theme.textPrimary else theme.textSecondary.copy(alpha = 0.5f), fontSize = 14.sp)
259     }
260 }
261
262 @Composable
263 private fun TimelineFrameItem(
264     frame: AnimationFrame,
265     isSelected: Boolean,
266     thumbnailLayers: List<FrameThumbnailLayer>,
267     theme: AppTheme,

```

```

268     canMoveLeft: Boolean,
269     canMoveRight: Boolean,
270     onMove: (Int) -> Unit,
271     onClick: () -> Unit
272 ) {
273     val moveThresholdPx = with(LocalDensity.current) { 44.dp.toPx() }
274     var accumulatedDragX by remember(frame.id) { mutableFloatStateOf(0f) }
275     Column(
276         horizontalAlignment = Alignment.CenterHorizontally,
277         modifier = Modifier
278             .pointerInput(frame.id, canMoveLeft, canMoveRight) {
279                 detectDragGesturesAfterLongPress(
280                     onDragStart = { accumulatedDragX = 0f },
281                     onDrag = { change, dragAmount ->
282                         accumulatedDragX += dragAmount.x
283                         val direction = when {
284                             accumulatedDragX <= -moveThresholdPx && canMoveLeft -> -1
285                             accumulatedDragX >= moveThresholdPx && canMoveRight -> 1
286                             else -> 0
287                         }
288                         if (direction != 0) {
289                             change.consume()
290                             onMove(direction)
291                             accumulatedDragX = 0f
292                         }
293                     }
294             )
295     )
296     .clickable { onClick() }
297 ) {
298     Box(
299         modifier = Modifier
300             .size(54.dp, 40.dp)
301             .clip(RoundedCornerShape(4.dp))
302             .background(if (isSelected) theme.accent.copy(alpha = 0.2f) else theme.panelInsetSoft)
303             .border(
304                 width = if (isSelected) 2.dp else 1.dp,
305                 color = if (isSelected) theme.accent else theme.panelStroke,
306                 shape = RoundedCornerShape(4.dp)
307             ),
308         contentAlignment = Alignment.Center
309     ) {
310         if (thumbnailLayers.isNotEmpty()) {
311             thumbnailLayers.forEach { layer ->
312                 Image(
313                     bitmap = layer.image,
314                     contentDescription = null,
315                     contentScale = ContentScale.Fit,
316                     modifier = Modifier
317                         .fillMaxSize()
318                         .padding(2.dp)
319                         .graphicsLayer(alpha = layer.opacity.coerceIn(0f, 1f)),
320                 )
321             }
322         } else {
323             Icon(Icons.Default.Image, null, tint = theme.textSecondary.copy(alpha = 0.3f), modifier = Modifier.size(20.
324 dp))
325         }
326         if (frame.holdFrames > 1) {
327             Text("+${frame.holdFrames - 1}", color = theme.accent, fontSize = 9.sp, fontWeight = FontWeight.Bold)
328         } else {
329             Spacer(Modifier.height(11.dp))
330         }
331     }
332 }
333
334 @Composable
335 private fun TimelineActionIcon(
336     icon: ImageVector,
337     label: String,
338     theme: AppTheme,
339     enabled: Boolean = true,
340     onClick: () -> Unit
341 ) {
342     Column(
343         horizontalAlignment = Alignment.CenterHorizontally,
344         modifier = Modifier
345             .alphaIfDisabled(!enabled)
346             .clickable(enabled = enabled) { onClick() }
347     ) {
348         Icon(icon, null, tint = if (enabled) theme.textPrimary else theme.textSecondary.copy(alpha = 0.5f), modifier =
349 Modifier.size(22.dp))
350         Text(label, color = theme.textSecondary, fontSize = 9.sp)
351     }
352
353     private fun Modifier.alphaIfDisabled(disabled: Boolean): Modifier = if (disabled) this.then(Modifier.graphicsLayer { alpha =
354 0.5f }) else this

```

97. app/src/main/java/com/wetinknext/ui/color/ColorCollection.kt

```

1  package com.wetinknext.ui.color
2
3  import androidx.compose.ui.graphics.Color
4
5  data class ColorCollection(
6      val id: Long,
7      val name: String,
8      val colors: List<Int>,
9      val builtin: Boolean = false,
10 )
11
12 fun ColorCollection.composeColors(): List<Color> = colors.map { Color(it) }
13
14 private fun hex(s: String): Int {
15     val v = s.removePrefix("#").trim()
16     val r = v.substring(0, 2).toInt(16)
17     val g = v.substring(2, 4).toInt(16)
18     val b = v.substring(4, 6).toInt(16)
19     return (0xFF shl 24) or (r shl 16) or (g shl 8) or b
20 }
21
22 private fun palette(vararg hex: String): List<Int> = hex.map { hex(it) }
23
24 object BuiltinCollections {
25     val all: List<ColorCollection> = listOf(
26         ColorCollection(
27             id = -1L,
28             name = "Стандарт",
29             colors = palette(
30                 "#FFF3B0", "#FFD6A5", "#FFC2B0", "#FFCCD5", "#F1C6F2",
31                 "#D8C7F5", "#C5D7FF", "#B8E5FF", "#B6F0EE", "#C9F2D8",
32                 "#FFE066", "#FFB87A", "#FF9F8B", "#FF9CB0", "#E59FE6",
33                 "#B89BE8", "#9CB6F5", "#86CFF2", "#7FD9D5", "#8AD9A8",
34                 "#F7C548", "#F09870", "#E07A6E", "#D9748E", "#C383C6",
35                 "#9678D0", "#7798E0", "#5DB1E6", "#5CB8B5", "#65BD8A",
36             ),
37             builtin = true,
38         ),
39         ColorCollection(
40             id = -2L,
41             name = "Пастель",
42             colors = palette(
43                 "#F9E79F", "#F7D08A", "#F5B187", "#F0998A", "#EC9090",
44                 "#E68F9C", "#D88BAE", "#C58CC0", "#B391CD", "#9C95D6",
45                 "#A8D5B6", "#9FD2C9", "#9CCFD8", "#A0C9E0", "#A8C0E3",
46                 "#B5BBE3", "#C0B8DE", "#CCB5D3", "#D6B3C5", "#D8B3B6",
47                 "#C7E1A1", "#B7DAA3", "#A8D2AE", "#9CCABD", "#94C0CB",
48                 "#94B5D2", "#9DA9D1", "#B0A0CB", "#C19BBE", "#CC9AAB",
49             ),
50             builtin = true,
51         ),
52         ColorCollection(
53             id = -3L,
54             name = "Персонажи",
55             colors = palette(
56                 "#F4C9B5", "#F0D4BC", "#E8B89A", "#D89C7E", "#C0805F",
57                 "#9C5F3F", "#74442B", "#4F2C1C", "#311A11", "#1B0F0A",
58                 "#FBE3D4", "#F8DCC9", "#F2C5A5", "#E2A77F", "#C58557",
59                 "#A36739", "#7A4824", "#552E14", "#321A0A", "#FFFAF0",
60                 "#E8C7C1", "#D49C9C", "#B97070", "#8C4F4F", "#5C3030",
61                 "#C7BF81", "#9C9484", "#6E6757", "#A0B0C4", "#5A6878",
62             ),
63             builtin = true,
64         ),
65         ColorCollection(
66             id = -4L,
67             name = "Пейзаж",
68             colors = palette(
69                 "#AEDCF0", "#7FBFE1", "#56A0CB", "#3D80B0", "#2E608E",
70                 "#244670", "#1B335A", "#142545", "#0D1830", "#070D1E",
71                 "#CFE3B0", "#A8CF8C", "#7FB36A", "#5C9651", "#3F7A3F",
72                 "#2E5E33", "#214728", "#16321C", "#5C7B4A", "#3F5732",
73                 "#F4D58D", "#E3B26E", "#D08A4A", "#B5642E", "#923F1C",
74                 "#6E2A14", "#8B5A2B", "#5C3A1A", "#A38560", "#6F5B3D",
75             ),
76             builtin = true,
77         ),
78         ColorCollection(
79             id = -5L,
80             name = "Анимация",
81             colors = palette(
82                 "#111318", "#3B414A", "#7B8491", "#B8C0CC", "#FFFFFF",
83                 "#4A90E2", "#35B9D5", "#42B883", "#A5D646", "#F4D44D",
84                 "#F5A23D", "#F26B4F", "#E6536F", "#E66FAD", "#A368DB",
85                 "#6D8CFF", "#64D4E8", "#76D7A6", "#D8EA72", "#FFE58A",
86                 "#245EA8", "#167E98", "#20794F", "#6B8F20", "#B58B08",
87                 "#B65A12", "#B53832", "#A62E58", "#A23E86", "#633B9B",
88             ),
89             builtin = true,
90         ),
91     ),
92 }

```


91)
92 }

98. app/src/main/java/com/wetinknext/ui/color/ColorMath.kt

```

1 package com.wetinknext.ui.color
2
3 import androidx.compose.ui.graphics.Color
4 import kotlin.math.abs
5
6 data class Hsl(val h: Float, val s: Float, val l: Float)
7
8 fun rgbToHsl(c: Color): Hsl {
9     val r = c.red
10    val g = c.green
11    val b = c.blue
12    val max = maxOf(r, g, b)
13    val min = minOf(r, g, b)
14    val l = (max + min) / 2f
15    val d = max - min
16    if (d == 0f) return Hsl(0f, 0f, l)
17    val s = if (l > 0.5f) d / (2f - max - min) else d / (max + min)
18    val h = when (max) {
19        r -> 60f * (((g - b) / d) % 6f)
20        g -> 60f * ((b - r) / d + 2f)
21        else -> 60f * ((r - g) / d + 4f)
22    }
23    return Hsl(((h % 360f) + 360f) % 360f, s, l)
24 }
25
26 fun hslToColor(h: Float, s: Float, l: Float): Color {
27     val hh = ((h % 360f) + 360f) % 360f
28     val c = (1f - abs(2f * l - 1f)) * s
29     val x = c * (1f - abs((hh / 60f) % 2f - 1f))
30     val m = l - c / 2f
31     val (r1, g1, b1) = when ((hh / 60f).toInt()) {
32         0 -> Triple(c, x, 0f)
33         1 -> Triple(x, c, 0f)
34         2 -> Triple(0f, c, x)
35         3 -> Triple(0f, x, c)
36         4 -> Triple(x, 0f, c)
37         else -> Triple(c, 0f, x)
38     }
39     return Color(
40         (r1 + m).coerceIn(0f, 1f),
41         (g1 + m).coerceIn(0f, 1f),
42         (b1 + m).coerceIn(0f, 1f),
43         1f,
44     )
45 }
46
47 fun hslToColor(hsl: Hsl): Color = hslToColor(hsl.h, hsl.s, hsl.l)
48
49 fun Color.toHex6(): String {
50     val r = (red * 255f).toInt().coerceIn(0, 255)
51     val g = (green * 255f).toInt().coerceIn(0, 255)
52     val b = (blue * 255f).toInt().coerceIn(0, 255)
53     return "%02X%02X%02X".format(r, g, b)
54 }
55
56 enum class HarmonyType(
57     val displayName: String,
58     val offsets: FloatArray,
59 ) {
60     Complementary("Комплементарная", floatArrayOf(0f, 180f)),
61     Analogous("Аналоговая", floatArrayOf(-30f, 0f, 30f)),
62     Triadic("Триадная", floatArrayOf(0f, 120f, 240f)),
63     SplitComplementary(
64         "Сплит-комплементарная",
65         floatArrayOf(0f, 150f, 210f),
66     ),
67     Tetradic("Тетрадная", floatArrayOf(0f, 60f, 180f, 240f)),
68     Square("Квадратная", floatArrayOf(0f, 90f, 180f, 270f)),
69     ;
70
71     fun derive(base: Hsl): List<Hsl> {
72         val h = base.h
73         return offsets.map { off ->
74             Hsl(((h + off) % 360f + 360f) % 360f, base.s, base.l)
75         }
76     }
77 }

```

99. app/src/main/java/com/wetinknext/ui/color/ColorPanel.kt

```

1  package com.wetinknext.ui.color
2
3  import androidx.compose.animation.core.animateFloatAsState
4  import androidx.compose.animation.core.tween
5  import androidx.compose.foundation.Canvas
6  import androidx.compose.foundation.ExperimentalFoundationApi
7  import androidx.compose.foundation.background
8  import androidx.compose.foundation.border
9  import androidx.compose.foundation.clickable
10 import androidx.compose.foundation.combinedClickable
11 import androidx.compose.foundation.gestures.detectDragGestures
12 import androidx.compose.foundation.gestures.detectHorizontalDragGestures
13 import androidx.compose.foundation.gestures.detectTapGestures
14 import androidx.compose.foundation.layout.*
15 import androidx.compose.foundation.rememberScrollState
16 import androidx.compose.foundation.shape.CircleShape
17 import androidx.compose.foundation.shape.RoundedCornerShape
18 import androidx.compose.foundation.text.BasicTextField
19 import androidx.compose.foundation.verticalScroll
20 import androidx.compose.material3.*
21 import androidx.compose.runtime.*
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.draw.clip
25 import androidx.compose.ui.draw.shadow
26 import androidx.compose.ui.geometry.Offset
27 import androidx.compose.ui.geometry.Size
28 import androidx.compose.ui.graphics.Brush
29 import androidx.compose.ui.graphics.Color
30 import androidx.compose.ui.graphics.SolidColor
31 import androidx.compose.ui.graphics.drawscope.DrawScope
32 import androidx.compose.ui.graphics.drawscope.Stroke
33 import androidx.compose.ui.input.pointer.pointerInput
34 import androidx.compose.ui.platform.LocalDensity
35 import androidx.compose.ui.text.TextStyle
36 import androidx.compose.ui.text.font.FontWeight
37 import androidx.compose.ui.text.style.TextAlign
38 import androidx.compose.ui.unit.dp
39 import androidx.compose.ui.unit.IntOffset
40 import androidx.compose.ui.unit.sp
41 import androidx.compose.ui.window.Dialog
42 import com.wetinknext.ui.components.PanelSurface
43 import com.wetinknext.ui.theme.AppTheme
44 import kotlinx.coroutines.delay
45 import kotlin.math.atan2
46 import kotlin.math.cos
47 import kotlin.math.hypot
48 import kotlin.math.sin
49 import kotlin.math.roundToInt
50
51 private enum class ColorTab { Wheel, Map }
52
53 @Composable
54 fun ColorPanel(
55     state: GlesColorState,
56     color: Color,
57     theme: AppTheme,
58     onColorChange: (Color) -> Unit,
59     onAddFromPhoto: () -> Unit,
60     title: String = "Ғбер",
61     recordRecent: Boolean = true,
62     allowCollectionEditing: Boolean = true,
63     onPinCollection: (ColorCollection) -> Unit = {},
64     modifier: Modifier = Modifier,
65     onDismiss: () -> Unit,
66 ) {
67     var tab by remember { mutableStateOf(ColorTab.Wheel) }
68     var hsl by remember { mutableStateOf(rgbToHsl(color)) }
69     var harmonyType by remember { mutableStateOf(HarmonyType.Triadic) }
70     val latestHsl by rememberUpdatedState(hsl)
71
72     val wheelScrollState = rememberScrollState()
73     val mapScrollState = rememberScrollState()
74     val tabScrollState = when (tab) {
75         ColorTab.Wheel -> wheelScrollState
76         ColorTab.Map -> mapScrollState
77     }
78
79     LaunchedEffect(hsl, recordRecent) {
80         val selectedColor = hslToColor(hsl)
81         onColorChange(selectedColor)
82         if (recordRecent) {
83             delay(350)
84             state.pushRecentColor(selectedColor)
85         }
86     }
87
88     val dismissAndSave = {
89         if (recordRecent) state.pushRecentColor(hslToColor(hsl))
90         onDismiss()
91     }

```

```

91     }
92
93     DisposableEffect(Unit) {
94         onDispose {
95             if (recordRecent) state.pushRecentColor(hslToColor(latestHsl))
96         }
97     }
98
99     PanelSurface(
100         modifier = modifier.width(320.dp),
101     ) {
102         Column(
103             modifier = Modifier.padding(14.dp),
104         ) {
105             ColorHeader(
106                 preview = hslToColor(hsl),
107                 title = title,
108                 theme = theme,
109                 onConfirm = dismissAndSave,
110             )
111             Spacer(Modifier.height(10.dp))
112             TabBar(tab, theme) { tab = it }
113             Spacer(Modifier.height(12.dp))
114             Box(
115                 modifier = Modifier
116                     .height(TabContentHeight)
117                     .fillMaxWidth(),
118             ) {
119                 Column(
120                     modifier = Modifier
121                         .fillMaxSize()
122                         .verticalScroll(tabScrollState),
123                 ) {
124                     when (tab) {
125                         ColorTab.Wheel -> WheelTab(
126                             hsl = hsl,
127                             theme = theme,
128                             onHslChange = { hsl = it },
129                             harmonyType = harmonyType,
130                             onHarmonyTypeChange = { harmonyType = it },
131                             recent = state.recentColors.toList(),
132                             onPickRecent = { picked ->
133                                 val p = rgbToHsl(picked)
134                                 hsl = hsl.copy(h = p.h, s = p.s)
135                                 if (recordRecent) state.pushRecentColor(hslToColor(hsl))
136                             },
137                         )
138                         ColorTab.Map -> MapTab(
139                             userCollections = state.userCollections.toList(),
140                             currentColor = hslToColor(hsl),
141                             theme = theme,
142                             onPick = { picked ->
143                                 hsl = rgbToHsl(picked)
144                                 if (recordRecent) state.pushRecentColor(picked)
145                             },
146                             onDeleteUserCollection = { id ->
147                                 state.deleteUserCollection(id)
148                             },
149                             onAddFromPhoto = onAddFromPhoto,
150                             allowCollectionEditing = allowCollectionEditing,
151                             onCreatePalette = { name ->
152                                 state.addUserCollection(
153                                     ColorCollection(
154                                         id = System.currentTimeMillis(),
155                                         name = name.ifBlank { "Новая палитра" },
156                                         colors = emptyList(),
157                                         builtin = false,
158                                     ),
159                                 )
160                             },
161                             onAddColorToCollection = { id, color ->
162                                 state.addColorToCollection(id, color)
163                             },
164                             onReplaceColorInCollection = { id, slot, color ->
165                                 state.replaceColorInCollection(id, slot, color)
166                             },
167                             onRemoveColorFromCollection = { id, slot ->
168                                 state.removeColorFromCollection(id, slot)
169                             },
170                             onPinCollection = onPinCollection,
171                         )
172                     }
173                 }
174             }
175         }
176     }
177 }
178
179 private val TabContentHeight = 440.dp
180
181 @Composable
182 private fun ColorHeader(preview: Color, onConfirm: () -> Unit, title: String, theme: AppTheme) {

```

```

183     Row(verticalAlignment = Alignment.CenterVertically) {
184         Text(
185             title,
186             color = theme.textPrimary,
187             fontWeight = FontWeight.Bold,
188             fontSize = 18.sp,
189         )
190         Spacer(Modifier.weight(1f))
191         Box(
192             modifier = Modifier
193                 .size(28.dp)
194                 .clip(CircleShape)
195                 .background(preview)
196                 .border(1.dp, theme.panelStroke, CircleShape)
197                 .clickable(onClick = onConfirm),
198         )
199     }
200 }
201
202 @Composable
203 private fun TabBar(current: ColorTab, theme: AppTheme, onChange: (ColorTab) -> Unit) {
204     Row(
205         modifier = Modifier
206             .fillMaxWidth()
207             .clip(RoundedCornerShape(10.dp))
208             .background(theme.panelInset)
209             .padding(3.dp),
210         horizontalArrangement = Arrangement.spacedBy(3.dp),
211     ) {
212         TabChip(label = "Кольцо", selected = current == ColorTab.Wheel, theme = theme) { onChange(ColorTab.Wheel) }
213         TabChip(label = "Карта", selected = current == ColorTab.Map, theme = theme) { onChange(ColorTab.Map) }
214     }
215 }
216
217 @Composable
218 private fun RowScope.TabChip(
219     label: String,
220     selected: Boolean,
221     theme: AppTheme,
222     onClick: () -> Unit,
223 ) {
224     Box(
225         modifier = Modifier
226             .weight(1f)
227             .clip(RoundedCornerShape(8.dp))
228             .background(if (selected) theme.accent.copy(alpha = 0.25f) else Color.Transparent)
229             .clickable(onClick = onClick)
230             .padding(vertical = 7.dp),
231         contentAlignment = Alignment.Center,
232     ) {
233         Text(
234             label,
235             color = if (selected) theme.textPrimary else theme.textSecondary,
236             fontSize = 13.sp,
237             fontWeight = if (selected) FontWeight.SemiBold else FontWeight.Normal,
238         )
239     }
240 }
241
242 @Composable
243 private fun WheelTab(
244     hsl: Hsl,
245     theme: AppTheme,
246     onHslChange: (Hsl) -> Unit,
247     harmonyType: HarmonyType,
248     onHarmonyTypeChange: (HarmonyType) -> Unit,
249     recent: List<Color>,
250     onPickRecent: (Color) -> Unit,
251 ) {
252     HueSatWheel(
253         hsl = hsl,
254         harmonyType = harmonyType,
255         onHsChange = { newH, newS -> onHslChange(hsl.copy(h = newH, s = newS)) },
256         modifier = Modifier
257             .fillMaxWidth()
258             .height(160.dp),
259     )
260     Spacer(Modifier.height(8.dp))
261     LightnessSlider(
262         hsl = hsl,
263         theme = theme,
264         onChange = { newL -> onHslChange(hsl.copy(l = newL)) },
265     )
266     Spacer(Modifier.height(8.dp))
267     HexRow(color = hslToColor(hsl), theme = theme)
268     Spacer(Modifier.height(10.dp))
269     HarmonySection(
270         hsl = hsl,
271         theme = theme,
272         harmonyType = harmonyType,
273         onHarmonyTypeChange = onHarmonyTypeChange,
274         onPick = onHslChange,

```

```

275     )
276     Spacer(Modifier.height(10.dp))
277     SectionTitle("Недавние", theme)
278     Spacer(Modifier.height(6.dp))
279     RecentRow(colors = recent.take(GRID_COLUMNS), theme = theme, onPick = onPickRecent)
280 }
281
282 @Composable
283 private fun HueSatWheel(
284     hsl: Hsl,
285     harmonyType: HarmonyType,
286     onHsChange: (Float, Float) -> Unit,
287     modifier: Modifier = Modifier,
288 ) {
289     val latestOnHsChange by rememberUpdatedState(onHsChange)
290     Canvas(
291         modifier = modifier
292             .clip(RoundedCornerShape(16.dp))
293             .pointerInput(Unit) {
294                 detectDragGestures { change, _ ->
295                     val (nh, ns) = pickHueSat(change.position, size.width.toFloat(), size.height.toFloat())
296                     latestOnHsChange(nh, ns)
297                 }
298             }
299             .pointerInput(Unit) {
300                 detectTapGestures(onTap = { offset ->
301                     val (nh, ns) = pickHueSat(offset, size.width.toFloat(), size.height.toFloat())
302                     latestOnHsChange(nh, ns)
303                 })
304             },
305     ) {
306         val w = size.width
307         val hgt = size.height
308         val cx = w / 2f
309         val cy = hgt / 2f
310         val outer = minOf(w, hgt) / 2f - 4f
311         if (outer <= 0f) return@Canvas
312
313         val steps = 120
314         for (i in 0 until steps) {
315             val a0 = i * 360f / steps
316             val a1 = (i + 1) * 360f / steps
317             val midA = (a0 + a1) / 2f
318             val hueColor = hslToColor(midA, 1f, 0.5f)
319             drawArc(
320                 color = hueColor,
321                 startAngle = a0 - 90f,
322                 sweepAngle = (a1 - a0) + 0.6f,
323                 useCenter = true,
324                 topLeft = Offset(cx - outer, cy - outer),
325                 size = Size(outer * 2, outer * 2),
326             )
327         }
328         drawCircle(
329             brush = Brush.radialGradient(
330                 colors = listOf(Color.White, Color.White.copy(alpha = 0f)),
331                 center = Offset(cx, cy),
332                 radius = outer,
333             ),
334             radius = outer,
335             center = Offset(cx, cy),
336         )
337
338         val derived = harmonyType.derive(hsl)
339         val verts = derived.map { d -> hslToWheelPoint(d, cx, cy, outer) }
340         drawHarmonyShape(harmonyType, verts)
341
342         val current = hslToWheelPoint(hsl, cx, cy, outer)
343         drawCircle(color = Color.White, radius = 7f, center = current, style = Stroke(width = 2f))
344         drawCircle(color = Color.Black, radius = 7f, center = current, style = Stroke(width = 1f))
345     }
346 }
347
348 private fun hslToWheelPoint(h: Hsl, cx: Float, cy: Float, outer: Float): Offset {
349     val a = Math.toRadians((h.h - 90.0))
350     val r = h.s.coerceIn(0f, 1f) * outer
351     return Offset(cx + (r * cos(a)).toFloat(), cy + (r * sin(a)).toFloat())
352 }
353
354 private fun DrawScope.drawHarmonyShape(type: HarmonyType, verts: List<Offset>) {
355     if (verts.size < 2) return
356     val edges: List<Pair<Int, Int>> = when (type) {
357         HarmonyType.Complementary -> listOf(0 to 1)
358         HarmonyType.Analogous -> listOf(0 to 1, 1 to 2)
359         HarmonyType.Triadic -> listOf(0 to 1, 1 to 2, 2 to 0)
360         HarmonyType.SplitComplementary -> listOf(0 to 1, 0 to 2, 1 to 2)
361         HarmonyType.Tetradic,
362         HarmonyType.Square -> listOf(0 to 1, 1 to 2, 2 to 3, 3 to 0)
363     }
364     edges.forEach { (a, b) ->
365         drawLine(color = Color.White.copy(alpha = 0.85f), start = verts[a], end = verts[b], strokeWidth = 2.5f)
366         drawLine(color = Color.Black.copy(alpha = 0.55f), start = verts[a], end = verts[b], strokeWidth = 1f)
367     }
368 }

```

```

367     }
368     verts.forEach { v ->
369         drawCircle(color = Color.White, radius = 4f, center = v)
370         drawCircle(color = Color.Black, radius = 4f, center = v, style = Stroke(width = 1f))
371     }
372 }
373
374 private fun pickHueSat(p: Offset, w: Float, h: Float): Pair<Float, Float> {
375     val cx = w / 2f
376     val cy = h / 2f
377     val outer = minOf(w, h) / 2f - 4f
378     val dx = p.x - cx
379     val dy = p.y - cy
380     val r = hypot(dx, dy)
381     val ang = Math.toDegrees(atan2(dy, dx).toDouble()) + 90.0
382     val nh = ((ang + 360.0) % 360.0).toFloat()
383     val ns = (r / outer).coerceIn(0f, 1f)
384     return nh to ns
385 }
386
387 @Composable
388 private fun LightnessSlider(hsl: Hsl, theme: AppTheme, onChange: (Float) -> Unit) {
389     val hue = hsl.h
390     val sat = hsl.s
391     val gradient = remember(hue, sat) {
392         Brush.horizontalGradient(
393             colors = listOf(
394                 hslToColor(hue, sat, 0f),
395                 hslToColor(hue, sat, 0.5f),
396                 hslToColor(hue, sat, 1f),
397             ),
398         )
399     }
400     val latestOnChange by rememberUpdatedState(onChange)
401     Row(verticalAlignment = Alignment.CenterVertically) {
402         Text("L", color = theme.textSecondary, fontSize = 12.sp, modifier = Modifier.width(16.dp))
403         Spacer(Modifier.width(8.dp))
404         Box(
405             modifier = Modifier
406                 .height(20.dp)
407                 .fillMaxWidth()
408                 .clip(RoundedCornerShape(10.dp))
409                 .background(gradient)
410                 .border(1.5.dp, theme.panelStroke, RoundedCornerShape(10.dp))
411                 .pointerInput(Unit) {
412                     detectDragGestures { change, _ ->
413                         val f = (change.position.x / size.width).coerceIn(0f, 1f)
414                         latestOnChange(f)
415                     }
416                 }
417                 .pointerInput(Unit) {
418                     detectTapGestures(onTap = { offset ->
419                         val f = (offset.x / size.width).coerceIn(0f, 1f)
420                         latestOnChange(f)
421                     })
422                 },
423         ) {
424             Canvas(modifier = Modifier.fillMaxSize()) {
425                 val knobR = (size.height / 2f - 2f).coerceAtMost(size.width / 2f).coerceAtLeast(0f)
426                 if (knobR <= 0f) return@Canvas
427                 val knobX = (hsl.l.coerceIn(0f, 1f) * size.width).coerceIn(knobR + 2f, size.width - knobR - 2f)
428
429                 drawCircle(color = Color.White, radius = knobR, center = Offset(knobX, size.height / 2f), style = Stroke(
width = 2.5f))
430                 drawCircle(color = theme.panelStroke, radius = knobR, center = Offset(knobX, size.height / 2f), style =
Stroke(width = 1f))
431             }
432         }
433     }
434 }
435
436 @Composable
437 private fun HexRow(color: Color, theme: AppTheme) {
438     Row(verticalAlignment = Alignment.CenterVertically) {
439         Text("Hex", color = theme.textSecondary, fontSize = 11.sp)
440         Spacer(Modifier.weight(1f))
441         Box(
442             modifier = Modifier
443                 .clip(RoundedCornerShape(8.dp))
444                 .background(theme.panelInset)
445                 .border(1.dp, theme.panelStroke, RoundedCornerShape(8.dp))
446                 .padding(horizontal = 10.dp, vertical = 4.dp),
447         ) {
448             Text(color.toHexString(), color = theme.textPrimary, fontSize = 12.sp, fontWeight = FontWeight.Medium)
449         }
450     }
451 }
452
453 @Composable
454 private fun HarmonySection(
455     hsl: Hsl,
456     theme: AppTheme,

```

```

457     harmonyType: HarmonyType,
458     onHarmonyTypeChange: (HarmonyType) -> Unit,
459     onPick: (Hsl) -> Unit,
460 ) {
461     SectionTitle("Тип гармонии", theme)
462     Spacer(Modifier.height(6.dp))
463     Row(
464         horizontalArrangement = Arrangement.spacedBy(6.dp),
465         modifier = Modifier.fillMaxWidth(),
466     ) {
467         HarmonyType.entries.forEach { type ->
468             HarmonyIcon(
469                 type = type,
470                 theme = theme,
471                 selected = type == harmonyType,
472                 modifier = Modifier.weight(1f),
473                 ) { onHarmonyTypeChange(type) }
474         }
475     }
476     Spacer(Modifier.height(6.dp))
477     Text(harmonyType.displayName, color = theme.textSecondary, fontSize = 11.sp, textAlign = TextAlign.Center, modifier =
478     Modifier.fillMaxWidth())
479     Spacer(Modifier.height(8.dp))
480     val derived = remember(harmonyType, hsl) { harmonyType.derive(hsl) }
481     Row(
482         horizontalArrangement = Arrangement.spacedBy(6.dp),
483         verticalAlignment = Alignment.CenterVertically,
484         modifier = Modifier.fillMaxWidth(),
485     ) {
486         derived.forEach { d ->
487             Box(
488                 modifier = Modifier
489                     .weight(1f)
490                     .height(32.dp)
491                     .clip(RoundedCornerShape(8.dp))
492                     .background(hslToColor(d))
493                     .border(1.dp, theme.panelInset, RoundedCornerShape(8.dp))
494                     .clickable { onPick(d) },
495             )
496         }
497         repeat(4 - derived.size) { Box(modifier = Modifier.weight(1f)) }
498     }
499 }
500 @Composable
501 private fun HarmonyIcon(
502     type: HarmonyType,
503     theme: AppTheme,
504     selected: Boolean,
505     modifier: Modifier = Modifier,
506     onClick: () -> Unit,
507 ) {
508     Box(
509         modifier = modifier
510             .aspectRatio(1f)
511             .clip(RoundedCornerShape(8.dp))
512             .background(if (selected) theme.accent.copy(alpha = 0.25f) else theme.panelInset)
513             .border(1.dp, if (selected) theme.accent else theme.panelInset, RoundedCornerShape(8.dp))
514             .clickable(onClick = onClick)
515             .padding(6.dp),
516         contentAlignment = Alignment.Center,
517     ) {
518         Canvas(modifier = Modifier.fillMaxSize()) {
519             val cx = size.width / 2f
520             val cy = size.height / 2f
521             val r = minOf(size.width, size.height) / 2f - 1f
522             val ringColor = if (selected) Color.White else theme.textSecondary.copy(alpha = 0.6f)
523             val shapeColor = if (selected) Color.White else theme.textSecondary
524             drawCircle(color = ringColor, radius = r, center = Offset(cx, cy), style = Stroke(width = 1.2f))
525             val verts = type.offsets.map { off ->
526                 val a = Math.toRadians((off - 90.0).toDouble())
527                 Offset(cx + (r * 0.78f * cos(a)).toFloat(), cy + (r * 0.78f * sin(a)).toFloat())
528             }
529             val edges: List<Pair<Int, Int>> = when (type) {
530                 HarmonyType.Complementary -> listOf(0 to 1)
531                 HarmonyType.Analogous -> listOf(0 to 1, 1 to 2)
532                 HarmonyType.Triadic -> listOf(0 to 1, 1 to 2, 2 to 0)
533                 HarmonyType.SplitComplementary -> listOf(0 to 1, 0 to 2, 1 to 2)
534                 HarmonyType.Tetradic,
535                 HarmonyType.Square -> listOf(0 to 1, 1 to 2, 2 to 3, 3 to 0)
536             }
537             edges.forEach { (a, b) -> drawLine(color = shapeColor, start = verts[a], end = verts[b], strokeWidth = 1.8f) }
538             verts.forEach { v -> drawCircle(color = shapeColor, radius = 2.2f, center = v) }
539         }
540     }
541 }
542
543 private const val GRID_COLUMNS = 10
544
545 @Composable
546 private fun MapTab(
547     userCollections: List<ColorCollection>,

```



```

548     currentColor: Color,
549     theme: AppTheme,
550     onPick: (Color) -> Unit,
551     onDeleteUserCollection: (Long) -> Unit,
552     onAddFromPhoto: () -> Unit,
553     allowCollectionEditing: Boolean,
554     onCreatePalette: (String) -> Unit,
555     onAddColorToCollection: (Long, Color) -> Unit,
556     onReplaceColorInCollection: (Long, Int, Color) -> Unit,
557     onRemoveColorFromCollection: (Long, Int) -> Unit,
558     onPinCollection: (ColorCollection) -> Unit,
559 ) {
560     var showCreate by remember { mutableStateOf(false) }
561     Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
562         if (allowCollectionEditing) {
563             Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
564                 MapActionButton(label = "+ Создать", theme = theme, modifier = Modifier.weight(1f), onClick = { showCreate =
true })
565                 MapActionButton(label = "+ Из фото", theme = theme, modifier = Modifier.weight(1f), onClick = onAddFromPhoto)
566             }
567         }
568         userCollections.forEach { collection ->
569             CollectionCard(collection = collection, theme = theme, onPick = onPick, onDelete = { onDeleteUserCollection(
collection.id) },
570                 onAddCurrentColor = { onAddColorToCollection(collection.id, currentColor) },
571                 onReplaceAt = { slot -> onReplaceColorInCollection(collection.id, slot, currentColor) },
572                 onRemoveAt = { slot -> onRemoveColorFromCollection(collection.id, slot) },
573                 onPin = { onPinCollection(collection) },
574             )
575         }
576         BuiltinCollections.all.forEach { collection ->
577             CollectionCard(collection = collection, theme = theme, onPick = onPick, onDelete = null, onAddCurrentColor = null,
onReplaceAt = null, onRemoveAt = null, onPin = { onPinCollection(collection) })
578         }
579     }
580     if (showCreate && allowCollectionEditing) {
581         CreatePaletteDialog(theme = theme, onConfirm = { name -> onCreatePalette(name); showCreate = false }, onDismiss = {
showCreate = false })
582     }
583 }
584
585 @Composable
586 private fun MapActionButton(label: String, theme: AppTheme, modifier: Modifier = Modifier, onClick: () -> Unit) {
587     Box(modifier = modifier.fillMaxWidth().clip(RoundedCornerShape(10.dp)).background(theme.accent.copy(alpha = 0.15f)).
border(1.2.dp, theme.accentSoft, RoundedCornerShape(10.dp)).clickable(onClick = onClick).padding(vertical = 10.dp, horizontal = 10.
dp), contentAlignment = Alignment.Center) {
588         Text(label, color = theme.textPrimary, fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
589     }
590 }
591
592 @Composable
593 private fun CollectionCard(collection: ColorCollection, theme: AppTheme, onPick: (Color) -> Unit, onDelete: (() -> Unit)?,
onAddCurrentColor: (() -> Unit)?, onReplaceAt: ((Int) -> Unit)?, onRemoveAt: ((Int) -> Unit)?, onPin: () -> Unit) {
594     var editingIdx by remember { mutableStateOf<Int?>(null) }
595     val revealPx = with(LocalDensity.current) { 96.dp.toPx() }
596     var rawOffset by remember(collection.id) { mutableFloatStateOf(0f) }
597     val animatedOffset by animateFloatAsState(rawOffset, animationSpec = tween(140), label = "paletteSwipe")
598     val cardShape = RoundedCornerShape(10.dp)
599     Box(modifier = Modifier.fillMaxWidth().clip(cardShape)) {
600         Box(
601             modifier = Modifier.matchParentSize().background(theme.accent.copy(alpha = 0.9f)),
602             contentAlignment = Alignment.CenterEnd,
603         ) {
604             Text(
605                 "Закрепить",
606                 color = Color.White,
607                 fontSize = 12.sp,
608                 fontWeight = FontWeight.SemiBold,
609                 modifier = Modifier.width(96.dp).fillMaxHeight().clickable { onPin(); rawOffset = 0f }.wrapContentHeight(
Alignment.CenterVertically),
610                 textAlign = TextAlign.Center,
611             )
612         }
613         Column(
614             modifier = Modifier
615                 .fillMaxWidth()
616                 .offset { IntOffset(animatedOffset.roundToInt(), 0) }
617                 .background(theme.panelInset)
618                 .border(1.dp, theme.panelStroke, cardShape)
619                 .pointerInput(collection.id, revealPx) {
620                     detectHorizontalDragGestures(
621                         onHorizontalDrag = { change, dx ->
622                             if (dx < 0f || rawOffset < 0f) {
623                                 change.consume()
624                                 rawOffset = (rawOffset + dx).coerceIn(-revealPx, 0f)
625                             }
626                         },
627                         onDragEnd = { rawOffset = if (rawOffset < -revealPx * 0.4f) -revealPx else 0f },
628                         onDragCancel = { rawOffset = 0f },
629                     )
630                 }
631             ).padding(horizontal = 8.dp, vertical = 8.dp),

```

```

632         verticalArrangement = Arrangement.spacedBy(6.dp),
633     ) {
634         Row(verticalAlignment = Alignment.CenterVertically) {
635             Text(collection.name, color = theme.textPrimary, fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
636             Spacer(Modifier.weight(1f))
637             if (onDelete != null) {
638                 Box(modifier = Modifier.size(22.dp).clip(CircleShape).background(theme.panelBg.copy(alpha = 0.7f)).border(1.
dp, theme.panelStroke, CircleShape).clickable(onClick = onDelete), contentAlignment = Alignment.Center) {
639                     Text("x", color = theme.textSecondary, fontSize = 14.sp, fontWeight = FontWeight.Bold)
640                 }
641             }
642         }
643         SwatchGrid(colors = collection.composeColors(), theme = theme, onPick = onPick, trailingPlusOnClick =
onAddCurrentColor, onLongPressIndex = if (onReplaceAt != null && onRemoveAt != null) { { idx -> editingIdx = idx } } else null)
644     }
645     }
646     val activeIdx = editingIdx
647     if (activeIdx != null && onReplaceAt != null && onRemoveAt != null) {
648         val swatchColor = collection.composeColors().getOrNull(activeIdx)
649         if (swatchColor != null) {
650             EditSwatchSheet(swatchColor = swatchColor, theme = theme, onReplace = { onReplaceAt(activeIdx); editingIdx = null
}, onRemove = { onRemoveAt(activeIdx); editingIdx = null }, onDismiss = { editingIdx = null })
651         } else editingIdx = null
652     }
653 }
654
655 @OptIn(ExperimentalFoundationApi::class)
656 @Composable
657 private fun SwatchGrid(colors: List<Color>, theme: AppTheme, onPick: (Color) -> Unit, trailingPlusOnClick: (() -> Unit)? =
null, onLongPressIndex: ((Int) -> Unit)? = null) {
658     val total = colors.size + if (trailingPlusOnClick != null) 1 else 0
659     if (total == 0) return
660     val itemRows = (0 until total).toList().chunked(GRID_COLUMNS)
661     Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
662         itemRows.forEach { row ->
663             Row(horizontalArrangement = Arrangement.spacedBy(4.dp), modifier = Modifier.fillMaxWidth()) {
664                 row.forEach { i ->
665                     if (i < colors.size) {
666                         val c = colors[i]
667                         val cellModifier = Modifier.weight(1f).aspectRatio(1f).clip(RoundedCornerShape(5.dp)).background(c).
border(1.dp, theme.panelInset, RoundedCornerShape(5.dp))
668                         if (onLongPressIndex != null) Box(modifier = cellModifier.combinedClickable(onClick = { onPick(c) },
onLongClick = { onLongPressIndex(i) }))
669                         else Box(modifier = cellModifier.clickable { onPick(c) })
670                     } else PlusTile(theme = theme, modifier = Modifier.weight(1f).aspectRatio(1f), onClick =
trailingPlusOnClick!!)
671                 }
672                 repeat(GRID_COLUMNS - row.size) { Box(modifier = Modifier.weight(1f)) }
673             }
674         }
675     }
676 }
677
678 @Composable
679 private fun EditSwatchSheet(swatchColor: Color, theme: AppTheme, onReplace: () -> Unit, onRemove: () -> Unit, onDismiss: () -
> Unit) {
680     Dialog(onDismissRequest = onDismiss) {
681         Column(modifier = Modifier.width(280.dp).clip(RoundedCornerShape(16.dp)).background(theme.panelBg).padding(16.dp),
verticalArrangement = Arrangement.spacedBy(12.dp)) {
682             Row(verticalAlignment = Alignment.CenterVertically) {
683                 Box(modifier = Modifier.size(28.dp).clip(RoundedCornerShape(6.dp)).background(swatchColor).border(1.dp, theme.
panelInset, RoundedCornerShape(6.dp)))
684                 Spacer(Modifier.width(10.dp)); Text("Цвет", color = theme.textPrimary, fontSize = 16.sp, fontWeight =
FontWeight.SemiBold)
685             }
686             DialogButton(label = "Заменить текущим", theme = theme, primary = true, modifier = Modifier.fillMaxWidth(),
onClick = onReplace)
687             DialogButton(label = "Удалить", theme = theme, primary = false, modifier = Modifier.fillMaxWidth(), onClick =
onRemove)
688             DialogButton(label = "Отмена", theme = theme, primary = false, modifier = Modifier.fillMaxWidth(), onClick =
onDismiss)
689         }
690     }
691 }
692
693 @Composable
694 private fun PlusTile(theme: AppTheme, modifier: Modifier = Modifier, onClick: () -> Unit) {
695     Box(modifier = modifier.clip(RoundedCornerShape(5.dp)).background(theme.panelBg.copy(alpha = 0.5f)).border(1.dp, theme.
accent.copy(alpha = 0.7f), RoundedCornerShape(5.dp)).clickable(onClick = onClick), contentAlignment = Alignment.Center) {
696         Text("+", color = theme.textPrimary, fontSize = 16.sp, fontWeight = FontWeight.Bold)
697     }
698 }
699
700 @Composable
701 private fun CreatePaletteDialog(theme: AppTheme, onConfirm: (String) -> Unit, onDismiss: () -> Unit) {
702     var name by remember { mutableStateOf("") }
703     Dialog(onDismissRequest = onDismiss) {
704         Column(modifier = Modifier.width(280.dp).clip(RoundedCornerShape(16.dp)).background(theme.panelBg).padding(16.dp),
verticalArrangement = Arrangement.spacedBy(12.dp)) {
705             Text("Новая палитра", color = theme.textPrimary, fontSize = 16.sp, fontWeight = FontWeight.SemiBold)
706             Box(modifier = Modifier.fillMaxWidth().clip(RoundedCornerShape(8.dp)).background(theme.panelInset).border(1.dp,
theme.panelInset, RoundedCornerShape(8.dp)).padding(horizontal = 10.dp, vertical = 8.dp)) {

```

```

707         BasicTextField(value = name, onValueChange = { name = it.take(40) }, singleLine = true, textStyle = TextStyle(
color = theme.textPrimary, fontSize = 14.sp), cursorBrush = SolidColor(theme.accent), decorationBox = { inner -> if (name.isEmpty(
)) Text("Название", color = theme.textSecondary, fontSize = 14.sp); inner() }, modifier = Modifier.fillMaxWidth())
708     }
709     Row(horizontalArrangement = Arrangement.spacedBy(8.dp), modifier = Modifier.fillMaxWidth()) {
710         DialogButton(label = "Отмена", theme = theme, primary = false, modifier = Modifier.weight(1f), onClick =
onDismiss)
711         DialogButton(label = "Создать", theme = theme, primary = true, modifier = Modifier.weight(1f), onClick = {
onConfirm(name) })
712     }
713 }
714 }
715 }
716 }
717 @Composable
718 private fun DialogButton(label: String, theme: AppTheme, primary: Boolean, modifier: Modifier = Modifier, onClick: () ->
Unit) {
719     Box(modifier = modifier.clip(RoundedCornerShape(8.dp)).background(if (primary) theme.accent.copy(alpha = 0.35f) else
theme.panelInset).border(1.dp, if (primary) theme.accent else theme.panelInset, RoundedCornerShape(8.dp)).clickable(onClick =
onClick).padding(vertical = 10.dp), contentAlignment = Alignment.Center) {
720         Text(label, color = theme.textPrimary, fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
721     }
722 }
723 }
724 @Composable
725 private fun SectionTitle(text: String, theme: AppTheme) {
726     Text(text, color = theme.textSecondary, fontSize = 11.sp, fontWeight = FontWeight.Medium)
727 }
728 }
729 @Composable
730 private fun RecentRow(colors: List<Color>, theme: AppTheme, onPick: (Color) -> Unit) {
731     Row(horizontalArrangement = Arrangement.spacedBy(4.dp), verticalAlignment = Alignment.CenterVertically, modifier =
Modifier.fillMaxWidth()) {
732         repeat(GRID_COLUMNS) { i ->
733             val c = colors.getOrNull(i)
734             if (c != null) Box(modifier = Modifier.weight(1f).aspectRatio(1f).clip(RoundedCornerShape(5.dp)).background(c).
border(1.dp, theme.panelInset, RoundedCornerShape(5.dp)).clickable { onPick(c) })
735             else Box(modifier = Modifier.weight(1f).aspectRatio(1f).clip(RoundedCornerShape(5.dp)).background(theme.
panelInset).border(1.dp, theme.panelInset, RoundedCornerShape(5.dp)))
736         }
737     }
738 }

```

100. app/src/main/java/com/wetinknext/ui/color/GlesColorState.kt

```

1  package com.wetinknext.ui.color
2
3  import android.content.Context
4  import androidx.compose.runtime.mutableStateListOf
5  import androidx.compose.runtime.mutableLongStateOf
6  import androidx.compose.runtime.mutableStateOf
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.setValue
9  import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.graphics.toArgb
11 import androidx.compose.ui.geometry.Offset
12 import org.json.JSONArray
13 import org.json.JSONObject
14
15 class GlesColorState(context: Context? = null) {
16     private val prefs = context?.applicationContext?.getSharedPreferences(PrefsName, Context.MODE_PRIVATE)
17
18     val recentColors = mutableStateListOf<Color>()
19     val userCollections = mutableStateListOf<ColorCollection>()
20     var pinnedCollectionId by mutableLongStateOf(prefs?.getLong(PinnedCollectionKey, NoPinnedCollection) ?:
NoPinnedCollection)
21     private set
22     var pinnedPaletteOffset by mutableStateOf(
23         Offset(
24             prefs?.getFloat(PinnedPaletteOffsetXKey, DefaultPinnedOffsetX) ?: DefaultPinnedOffsetX,
25             prefs?.getFloat(PinnedPaletteOffsetYKey, DefaultPinnedOffsetY) ?: DefaultPinnedOffsetY,
26         ),
27     )
28     private set
29
30     init {
31         loadRecentColors()
32         loadUserCollections()
33     }
34
35     fun pushRecentColor(color: Color) {
36         val argb = color.toArgb()
37         val existingIndex = recentColors.indexOfFirst { it.toArgb() == argb }
38         if (existingIndex == 0) return
39         if (existingIndex > 0) recentColors.removeAt(existingIndex)
40         recentColors.add(0, color)
41         while (recentColors.size > MaxRecentColors) {
42             recentColors.removeAt(recentColors.lastIndex)
43         }
44         saveRecentColors()
45     }
46
47     private fun loadRecentColors() {
48         val encoded = prefs?.getString(RecentColorsKey, null) ?: return
49         encoded.split(',')
50             .mapNotNull { it.toIntOrNull() }
51             .take(MaxRecentColors)
52             .forEach { recentColors.add(Color(it)) }
53     }
54
55     private fun saveRecentColors() {
56         prefs?.edit()
57             ?.putString(RecentColorsKey, recentColors.joinToString(",") { it.toArgb().toString() })
58             ?.apply()
59     }
60
61     fun addUserCollection(collection: ColorCollection) {
62         userCollections.add(0, collection.copy(builtin = false))
63         saveUserCollections()
64     }
65
66     fun deleteUserCollection(id: Long) {
67         userCollections.removeAll { it.id == id }
68         if (pinnedCollectionId == id) unpinCollection()
69         saveUserCollections()
70     }
71
72     fun pinCollection(id: Long) {
73         pinnedCollectionId = id
74         prefs?.edit()?.putLong(PinnedCollectionKey, id)?.apply()
75     }
76
77     fun unpinCollection() {
78         pinnedCollectionId = NoPinnedCollection
79         prefs?.edit()?.remove(PinnedCollectionKey)?.apply()
80     }
81
82     fun persistPinnedPaletteOffset(offset: Offset) {
83         pinnedPaletteOffset = offset
84         prefs?.edit()
85             ?.putFloat(PinnedPaletteOffsetXKey, offset.x)
86             ?.putFloat(PinnedPaletteOffsetYKey, offset.y)
87             ?.apply()
88     }
89

```

```

90 fun addColorToCollection(id: Long, color: Color) {
91     updateUserCollection(id) { collection ->
92         collection.copy(colors = collection.colors + color.toArgb())
93     }
94 }
95
96 fun replaceColorInCollection(id: Long, slot: Int, color: Color) {
97     updateUserCollection(id) { collection ->
98         if (slot !in collection.colors.indices) return@updateUserCollection collection
99         val colors = collection.colors.toMutableList()
100         colors[slot] = color.toArgb()
101         collection.copy(colors = colors)
102     }
103 }
104
105 fun removeColorFromCollection(id: Long, slot: Int) {
106     updateUserCollection(id) { collection ->
107         if (slot !in collection.colors.indices) return@updateUserCollection collection
108         val colors = collection.colors.toMutableList()
109         colors.removeAt(slot)
110         collection.copy(colors = colors)
111     }
112 }
113
114 private fun updateUserCollection(id: Long, transform: (ColorCollection) -> ColorCollection) {
115     val index = userCollections.indexOfFirst { it.id == id }
116     if (index < 0) return
117     userCollections[index] = transform(userCollections[index]).copy(builtin = false)
118     saveUserCollections()
119 }
120
121 private fun loadUserCollections() {
122     val encoded = prefs?.getString(UserCollectionsKey, null) ?: return
123     runCatching {
124         val array = JSONArray(encoded)
125         for (i in 0 until array.length()) {
126             val item = array.optJSONObject(i) ?: continue
127             val colorsJson = item.optJSONArray("colors") ?: JSONArray()
128             val colors = buildList {
129                 for (j in 0 until colorsJson.length()) {
130                     add(colorsJson.getInt(j))
131                 }
132             }
133             userCollections.add(
134                 ColorCollection(
135                     id = item.optLong("id", System.currentTimeMillis() + i),
136                     name = item.optString("name", "Новая палитра"),
137                     colors = colors,
138                     builtin = false,
139                 ),
140             )
141         }
142     }
143 }
144
145 private fun saveUserCollections() {
146     val array = JSONArray()
147     userCollections.filterNot { it.builtin }.forEach { collection ->
148         val colors = JSONArray()
149         collection.colors.forEach { colors.put(it) }
150         array.put(
151             JSONObject()
152                 .put("id", collection.id)
153                 .put("name", collection.name)
154                 .put("colors", colors),
155         )
156     }
157     prefs?.edit()
158         ?.putString(UserCollectionsKey, array.toString())
159         ?.apply()
160 }
161
162 private companion object {
163     const val PrefsName = "wetinknext_color_state"
164     const val RecentColorsKey = "recent_colors"
165     const val UserCollectionsKey = "user_collections"
166     const val PinnedCollectionKey = "pinned_collection"
167     const val PinnedPaletteOffsetXKey = "pinned_palette_offset_x"
168     const val PinnedPaletteOffsetYKey = "pinned_palette_offset_y"
169     const val NoPinnedCollection = Long.MIN_VALUE
170     const val DefaultPinnedOffsetX = 84f
171     const val DefaultPinnedOffsetY = 128f
172     const val MaxRecentColors = 14
173 }
174 }

```

101. app/src/main/java/com/wetinknext/ui/components/AdjustmentsPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.rememberScrollState
8  import androidx.compose.foundation.shape.RoundedCornerShape
9  import androidx.compose.foundation.verticalScroll
10 import androidx.compose.material3.HorizontalDivider
11 import androidx.compose.material3.Text
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.draw.shadow
17 import androidx.compose.ui.graphics.Color
18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.unit.dp
20 import androidx.compose.ui.unit.sp
21 import com.wetinknext.ui.theme.AppTheme
22
23 enum class AdjustmentAction(val title: String) {
24     BRIGHTNESS_CONTRAST("Яркость и контраст"),
25     THRESHOLD("Бинаризация"),
26     BRIGHTNESS_TO_OPACITY("Преобразовать яркость в непрозрачность"),
27 }
28
29 @Composable
30 fun AdjustmentsPanel(
31     theme: AppTheme,
32     onClose: () -> Unit,
33     onAction: (AdjustmentAction) -> Unit,
34     modifier: Modifier = Modifier
35 ) {
36     Column(
37         modifier = modifier
38         .width(280.dp)
39         .shadow(8.dp, RoundedCornerShape(16.dp))
40         .clip(RoundedCornerShape(16.dp))
41         .background(theme.panelBg.copy(alpha = 0.98f))
42         .border(1.dp, theme.panelStroke, RoundedCornerShape(16.dp))
43         .padding(vertical = 8.dp)
44     ) {
45         Text(
46             text = "Коррекция",
47             color = theme.textPrimary,
48             fontSize = 16.sp,
49             fontWeight = FontWeight.Bold,
50             modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp)
51         )
52
53         Column(
54             modifier = Modifier
55                 .fillMaxWidth()
56                 .verticalScroll(rememberScrollState())
57         ) {
58             AdjustmentAction.entries.forEachIndexed { index, action ->
59                 AdjustmentItem(
60                     title = action.title,
61                     theme = theme,
62                     onClick = {
63                         onClose()
64                         onAction(action)
65                     }
66                 )
67                 if (index < AdjustmentAction.entries.size - 1) {
68                     HorizontalDivider(
69                         modifier = Modifier.padding(horizontal = 16.dp),
70                         thickness = 0.5.dp,
71                         color = theme.panelStroke.copy(alpha = 0.5f)
72                     )
73                 }
74             }
75         }
76     }
77 }
78
79 @Composable
80 private fun AdjustmentItem(
81     title: String,
82     theme: AppTheme,
83     onClick: () -> Unit
84 ) {
85     Row(
86         modifier = Modifier
87             .fillMaxWidth()
88             .clickable { onClick() }
89             .padding(horizontal = 16.dp, vertical = 12.dp),
90         verticalAlignment = Alignment.CenterVertically,

```

```
91         horizontalArrangement = Arrangement.SpaceBetween
92     ) {
93         Text(
94             text = title,
95             color = theme.textPrimary,
96             fontSize = 14.sp,
97             modifier = Modifier.weight(1f)
98         )
99     }
100 }
```

102. app/src/main/java/com/wetinknext/ui/components/BrightnessContrastPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.shape.CircleShape
8  import androidx.compose.foundation.shape.RoundedCornerShape
9  import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.Check
11 import androidx.compose.material.icons.filled.Close
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Slider
14 import androidx.compose.material3.SliderDefaults
15 import androidx.compose.material3.Text
16 import androidx.compose.runtime.Composable
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.draw.clip
20 import androidx.compose.ui.graphics.Color
21 import androidx.compose.ui.text.font.FontWeight
22 import androidx.compose.ui.unit.dp
23 import androidx.compose.ui.unit.sp
24 import com.wetinknext.ui.theme.AppTheme
25 import kotlin.math.roundToInt
26
27 @Composable
28 fun BrightnessContrastPanel(
29     theme: AppTheme,
30     brightness: Float, // -1..1
31     contrast: Float,   // -1..1
32     onBrightnessChange: (Float) -> Unit,
33     onContrastChange: (Float) -> Unit,
34     onApply: () -> Unit,
35     onCancel: () -> Unit,
36     isApplyEnabled: Boolean = true,
37     modifier: Modifier = Modifier
38 ) {
39     Column(
40         modifier = modifier
41             .width(360.dp)
42             .clip(RoundedCornerShape(28.dp))
43             .background(theme.panelBg.copy(alpha = 0.95f))
44             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
45             .padding(horizontal = 20.dp, vertical = 16.dp),
46         verticalArrangement = Arrangement.spacedBy(16.dp)
47     ) {
48         Row(
49             modifier = Modifier.fillMaxWidth(),
50             horizontalArrangement = Arrangement.SpaceBetween,
51             verticalAlignment = Alignment.CenterVertically
52         ) {
53             Text(
54                 text = "Яркость и контраст",
55                 color = theme.textPrimary,
56                 fontSize = 15.sp,
57                 fontWeight = FontWeight.Bold
58             )
59
60             Row(
61                 verticalAlignment = Alignment.CenterVertically,
62                 horizontalArrangement = Arrangement.spacedBy(12.dp)
63             ) {
64                 Box(
65                     modifier = Modifier
66                         .size(36.dp)
67                         .clip(CircleShape)
68                         .background(theme.panelInset)
69                         .clickable { onCancel() },
70                     contentAlignment = Alignment.Center
71                 ) {
72                     Icon(Icons.Default.Close, null, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
73                 }
74
75                 Box(
76                     modifier = Modifier
77                         .size(36.dp)
78                         .clip(CircleShape)
79                         .background(if (isApplyEnabled) theme.accent else theme.panelInset)
80                         .clickable(enabled = isApplyEnabled) { onApply() },
81                     contentAlignment = Alignment.Center
82                 ) {
83                     Icon(Icons.Default.Check, null, tint = Color.White, modifier = Modifier.size(20.dp))
84                 }
85             }
86         }
87
88         // Brightness
89         Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
90             Row(

```



```

91         modifier = Modifier.fillMaxWidth(),
92         horizontalArrangement = Arrangement.SpaceBetween
93     ) {
94         Text("Яркость", color = theme.textSecondary, fontSize = 13.sp)
95         Text("${(brightness * 100).roundToInt()}", color = theme.accent, fontSize = 13.sp, fontWeight = FontWeight.
Bold)
96     }
97     Slider(
98         value = brightness,
99         onValueChange = onBrightnessChange,
100        valueRange = -1f..1f,
101        colors = SliderDefaults.colors(
102            thumbColor = theme.accent,
103            activeTrackColor = theme.accent,
104            inactiveTrackColor = theme.panelInset
105        )
106    )
107 }
108
109 // Contrast
110 Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
111     Row(
112         modifier = Modifier.fillMaxWidth(),
113         horizontalArrangement = Arrangement.SpaceBetween
114     ) {
115         Text("Контраст", color = theme.textSecondary, fontSize = 13.sp)
116         Text("${(contrast * 100).roundToInt()}", color = theme.accent, fontSize = 13.sp, fontWeight = FontWeight.Bold)
117     }
118     Slider(
119         value = contrast,
120         onValueChange = onContrastChange,
121         valueRange = -1f..1f,
122         colors = SliderDefaults.colors(
123             thumbColor = theme.accent,
124             activeTrackColor = theme.accent,
125             inactiveTrackColor = theme.panelInset
126         )
127     )
128 }
129 }
130 }

```

103. app/src/main/java/com/wetinknext/ui/components/BrightnessToOpacityPanel.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.shape.CircleShape
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.Check
11 import androidx.compose.material.icons.filled.Close
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Switch
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.draw.clip
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.text.font.FontWeight
21 import androidx.compose.ui.unit.dp
22 import androidx.compose.ui.unit.sp
23 import com.wetinknext.ui.theme.AppTheme
24
25 @Composable
26 fun BrightnessToOpacityPanel(
27     theme: AppTheme,
28     isInverted: Boolean,
29     onInvertChange: (Boolean) -> Unit,
30     onApply: () -> Unit,
31     onCancel: () -> Unit,
32     isApplyEnabled: Boolean = true,
33     modifier: Modifier = Modifier
34 ) {
35     Column(
36         modifier = modifier
37         .width(360.dp)
38         .clip(RoundedCornerShape(28.dp))
39         .background(theme.panelBg.copy(alpha = 0.95f))
40         .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
41         .padding(horizontal = 20.dp, vertical = 16.dp),
42         verticalArrangement = Arrangement.spacedBy(16.dp)
43     ) {
44         Row(
45             modifier = Modifier.fillMaxWidth(),
46             horizontalArrangement = Arrangement.SpaceBetween,
47             verticalAlignment = Alignment.CenterVertically
48         ) {
49             Text(
50                 text = "Яркость в непрозрачность",
51                 color = theme.textPrimary,
52                 fontSize = 15.sp,
53                 fontWeight = FontWeight.Bold
54             )
55
56             Row(
57                 verticalAlignment = Alignment.CenterVertically,
58                 horizontalArrangement = Arrangement.spacedBy(12.dp)
59             ) {
60                 Box(
61                     modifier = Modifier
62                     .size(36.dp)
63                     .clip(CircleShape)
64                     .background(theme.panelInset)
65                     .clickable { onCancel() },
66                     contentAlignment = Alignment.Center
67                 ) {
68                     Icon(Icons.Default.Close, null, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
69                 }
70
71                 Box(
72                     modifier = Modifier
73                     .size(36.dp)
74                     .clip(CircleShape)
75                     .background(if (isApplyEnabled) theme.accent else theme.panelInset)
76                     .clickable(enabled = isApplyEnabled) { onApply() },
77                     contentAlignment = Alignment.Center
78                 ) {
79                     Icon(Icons.Default.Check, null, tint = Color.White, modifier = Modifier.size(20.dp))
80                 }
81             }
82         }
83
84         Text(
85             text = "Светлые области станут прозрачными, тёмные – непрозрачными",
86             color = theme.textSecondary,
87             fontSize = 12.sp,
88             lineHeight = 16.sp
89         )
90     }

```

```

91         Row(
92             modifier = Modifier.fillMaxWidth(),
93             verticalAlignment = Alignment.CenterVertically,
94             horizontalArrangement = Arrangement.SpaceBetween
95         ) {
96             Text(
97                 text = "Инвертировать",
98                 color = theme.textPrimary,
99                 fontSize = 14.sp
100            )
101            Switch(
102                checked = isInverted,
103                onCheckedChange = onInvertChange
104            )
105        }
106    }
107 }

```

104. app/src/main/java/com/wetinknext/ui/components/BrushPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.foundation.Canvas
4  import androidx.compose.foundation.Image
5  import androidx.compose.foundation.background
6  import androidx.compose.foundation.border
7  import androidx.compose.foundation.clickable
8  import androidx.compose.foundation.gestures.detectTapGestures
9  import androidx.compose.foundation.layout.*
10 import androidx.compose.foundation.lazy.LazyColumn
11 import androidx.compose.foundation.lazy.items
12 import androidx.compose.foundation.shape.CircleShape
13 import androidx.compose.foundation.shape.RoundedCornerShape
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.filled.*
16 import androidx.compose.material3.Icon
17 import androidx.compose.material3.Text
18 import androidx.compose.runtime.*
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.draw.clip
22 import androidx.compose.ui.draw.drawBehind
23 import androidx.compose.ui.geometry.Offset
24 import androidx.compose.ui.geometry.Size
25 import androidx.compose.ui.graphics.Color
26 import androidx.compose.ui.graphics.ImageBitmap
27 import androidx.compose.ui.graphics.vector.ImageVector
28 import androidx.compose.ui.input.pointer.pointerInput
29 import androidx.compose.ui.layout.ContentScale
30 import androidx.compose.ui.text.font.FontWeight
31 import androidx.compose.ui.unit.dp
32 import androidx.compose.ui.unit.sp
33 import com.wetinknext.engine.brush.BrushLibrary
34 import com.wetinknext.engine.brush.BrushPreset
35 import com.wetinknext.engine.brush.BrushUiCategory
36 import com.wetinknext.ui.theme.AppTheme
37
38 private val BRUSH_SWIPE_REVEAL_WIDTH = 88.dp
39
40 @Composable
41 fun BrushPanel(
42     currentBrush: BrushPreset,
43     onBrushSelect: (BrushPreset) -> Unit,
44     onBrushStudioOpen: (BrushPreset) -> Unit,
45     theme: AppTheme,
46     onDismiss: () -> Unit,
47     onDuplicate: (BrushPreset) -> Unit,
48     onDelete: (BrushPreset) -> Unit,
49     modifier: Modifier = Modifier,
50     categories: List<BrushUiCategory> = BrushLibrary.allCategories,
51     previews: Map<String, ImageBitmap> = emptyMap(),
52     previewKey: (BrushPreset) -> String = { it.id },
53     onRequestPreview: (BrushPreset) -> Unit = {},
54     title: String = "Кисти",
55 ) {
56     fun categoryIndexForCurrentBrush(): Int {
57         return categories.indexOfFirst { category -> category.brushes.any { it.id == currentBrush.id } }
58     }
59     ?.takeIf { it >= 0 }
60     ?: 0
61
62     var selectedCategoryIndex by remember(categories, currentBrush.id) {
63         mutableIntStateOf(categoryIndexForCurrentBrush())
64     }
65     LaunchedEffect(categories, currentBrush.id) {
66         selectedCategoryIndex = categoryIndexForCurrentBrush()
67     }
68     val selectedCategory = categories.getOrNull(selectedCategoryIndex)
69
70     PanelSurface(
71         modifier = modifier
72             .width(420.dp)
73             .height(560.dp),
74     ) {
75         Row(Modifier.fillMaxSize()) {
76             Column(
77                 modifier = Modifier
78                     .width(70.dp)
79                     .fillMaxHeight()
80                     .clip(RoundedCornerShape(topStart = 22.dp, bottomStart = 22.dp))
81                     .background(theme.panelInset)
82                     .drawBehind {
83                         drawLine(
84                             color = theme.panelStroke,
85                             start = Offset(size.width, 0f),
86                             end = Offset(size.width, size.height),
87                             strokeWidth = 1.dp.toPx(),
88                         )
89                     }
90             )
91             .padding(vertical = 16.dp),
92             horizontalAlignment = Alignment.CenterHorizontally,

```

```

91         verticalArrangement = Arrangement.spacedBy(20.dp),
92     ) {
93         categories.forEachIndexed { index, category ->
94             val isSelected = index == selectedCategoryId
95             Box(
96                 modifier = Modifier
97                     .size(44.dp)
98                     .clip(CircleShape)
99                     .background(if (isSelected) theme.accent.copy(alpha = 0.3f) else Color.Transparent)
100                     .border(
101                         if (isSelected) 1.dp else 0.dp,
102                         if (isSelected) theme.accentSoft else Color.Transparent,
103                         CircleShape,
104                     )
105                 .clickable { selectedCategoryId = index },
106                 contentAlignment = Alignment.Center,
107             ) {
108                 Icon(
109                     category.icon,
110                     contentDescription = category.name,
111                     tint = if (isSelected) theme.accent else theme.iconInactive,
112                     modifier = Modifier.size(24.dp),
113                 )
114             }
115         }
116     }
117
118     Column(
119         modifier = Modifier
120             .weight(1f)
121             .fillMaxHeight()
122             .padding(14.dp),
123     ) {
124         Row(
125             verticalAlignment = Alignment.CenterVertically,
126             horizontalArrangement = Arrangement.spacedBy(12.dp)
127         ) {
128             Column(verticalArrangement = Arrangement.spacedBy(1.dp)) {
129                 Text(
130                     text = title,
131                     color = theme.textPrimary,
132                     fontSize = 18.sp,
133                     fontWeight = FontWeight.Bold,
134                 )
135                 selectedCategory?.let { category ->
136                     Text(
137                         text = category.name,
138                         color = theme.accent,
139                         fontSize = 11.sp,
140                         fontWeight = FontWeight.SemiBold,
141                     )
142                 }
143             }
144             Spacer(Modifier.weight(1f))
145
146             Icon(
147                 imageVector = Icons.Default.Close,
148                 contentDescription = "Закреть",
149                 tint = theme.textSecondary,
150                 modifier = Modifier.size(24.dp).clickable { onDismiss() }
151             )
152         }
153
154         Spacer(Modifier.height(14.dp))
155
156         LazyColumn(
157             verticalArrangement = Arrangement.spacedBy(8.dp),
158             modifier = Modifier.fillMaxSize(),
159         ) {
160             items(selectedCategory?.brushes.orEmpty(), key = { it.id }) { brush ->
161                 val isSelected = brush.id == currentBrush.id
162                 BrushItem(
163                     brush = brush,
164                     isSelected = isSelected,
165                     theme = theme,
166                     preview = previews[previewKey(brush)],
167                     onRequestPreview = { onRequestPreview(brush) },
168                     onStudioOpen = { onBrushStudioOpen(brush) },
169                     onDuplicate = { onDuplicate(brush) },
170                     onDelete = { onDelete(brush) },
171                     onClick = {
172                         if (isSelected) onBrushStudioOpen(brush)
173                         else onBrushSelect(brush)
174                     },
175                 )
176             }
177
178             if (selectedCategory?.brushes.isNullOrEmpty()) {
179                 item {
180                     Text(
181                         text = "Кисти отсутствуют",
182                         color = theme.textSecondary,

```

```

183                                     fontSize = 14.sp,
184                                     modifier = Modifier.padding(top = 20.dp),
185                                 )
186                             }
187                         }
188                     }
189                 }
190             }
191         }
192     }
193
194     @Composable
195     private fun BrushItem(
196         brush: BrushPreset,
197         isSelected: Boolean,
198         theme: AppTheme,
199         preview: ImageBitmap?,
200         onRequestPreview: () -> Unit,
201         onStudioOpen: () -> Unit,
202         onDuplicate: () -> Unit,
203         onDelete: () -> Unit,
204         onClick: () -> Unit,
205     ) {
206         val contentBg = if (isSelected) theme.panelBgSolid else theme.panelInset
207         LaunchedEffect(brush.id, brush.settings, preview) {
208             if (preview == null) {
209                 onRequestPreview()
210             }
211         }
212
213         SwipeRevealRow(
214             revealWidth = BRUSH_SWIPE_REVEAL_WIDTH,
215             onSwipeRight = onStudioOpen,
216             modifier = Modifier
217                 .fillMaxWidth()
218                 .clip(RoundedCornerShape(8.dp))
219                 .background(theme.panelInset),
220             actions = {
221                 BrushSwipeActions(
222                     theme = theme,
223                     onDuplicate = onDuplicate,
224                     onDelete = onDelete,
225                     canDelete = true // Simplified for now
226                 )
227             },
228         ) {
229             Box(
230                 modifier = Modifier
231                     .fillMaxWidth()
232                     .background(contentBg)
233                     .border(1.dp, if (isSelected) theme.accentSoft else theme.panelStroke, RoundedCornerShape(8.dp))
234                     .pointerInput(brush.id) {
235                         detectTapGestures(
236                             onTap = { onClick() },
237                         )
238                     }
239                     .padding(10.dp),
240             ) {
241                 Row(
242                     verticalAlignment = Alignment.CenterVertically,
243                     horizontalArrangement = Arrangement.spacedBy(12.dp),
244                 ) {
245                     BrushPreviewStrip(
246                         preview = preview,
247                         brush = brush,
248                         theme = theme,
249                         modifier = Modifier
250                             .width(142.dp)
251                             .height(46.dp),
252                     )
253                     Column(Modifier.weight(1f)) {
254                         Text(
255                             text = brush.settings.name,
256                             color = if (isSelected) theme.accent else theme.textPrimary,
257                             fontSize = 15.sp,
258                             fontWeight = if (isSelected) FontWeight.SemiBold else FontWeight.Medium,
259                             maxLines = 1,
260                         )
261                         Text(
262                             text = "Размер: ${((brush.settings.baseRadiusPx * 2f).toInt())} px",
263                             color = theme.textSecondary,
264                             fontSize = 12.sp,
265                             maxLines = 1,
266                         )
267                     }
268                     if (isSelected) {
269                         Box(
270                             Modifier
271                                 .size(8.dp)
272                                 .clip(CircleShape)
273                                 .background(theme.accent),
274                         )
275                     }
276                 }
277             }
278         }
279     }

```

```

275         }
276     }
277 }
278 }
279 }
280
281 @Composable
282 private fun BrushSwipeActions(
283     theme: AppTheme,
284     onDuplicate: () -> Unit,
285     onDelete: () -> Unit,
286     canDelete: Boolean
287 ) {
288     Row(
289         modifier = Modifier
290             .fillMaxWidth()
291             .fillMaxHeight(),
292         horizontalArrangement = Arrangement.End,
293         verticalAlignment = Alignment.CenterVertically,
294     ) {
295         BrushSwipeButton(
296             icon = Icons.Default.ContentCopy,
297             label = "Дубль",
298             color = theme.accent,
299             onClick = onDuplicate
300         )
301         BrushSwipeButton(
302             icon = Icons.Default.Delete,
303             label = "Удал.",
304             color = theme.danger,
305             enabled = canDelete,
306             onClick = onDelete
307         )
308     }
309 }
310
311 @Composable
312 private fun BrushSwipeButton(
313     icon: ImageVector,
314     label: String,
315     color: Color,
316     enabled: Boolean = true,
317     onClick: () -> Unit,
318 ) {
319     Column(
320         modifier = Modifier
321             .width(44.dp)
322             .fillMaxHeight()
323             .background(if (enabled) color else color.copy(alpha = 0.3f))
324             .clickable(enabled = enabled, onClick = onClick)
325             .padding(vertical = 4.dp),
326         horizontalAlignment = Alignment.CenterHorizontally,
327         verticalArrangement = Arrangement.Center,
328     ) {
329         Icon(icon, contentDescription = label, tint = Color.White, modifier = Modifier.size(18.dp))
330         Text(label, color = Color.White, fontSize = 8.sp, fontWeight = FontWeight.Medium)
331     }
332 }
333
334 @Composable
335 private fun BrushPreviewStrip(
336     preview: ImageBitmap?,
337     brush: BrushPreset,
338     theme: AppTheme,
339     modifier: Modifier = Modifier,
340 ) {
341     Box(
342         modifier = modifier
343             .clip(RoundedCornerShape(4.dp))
344             .background(Color.White)
345             .border(1.dp, theme.panelStroke, RoundedCornerShape(4.dp)),
346         contentAlignment = Alignment.Center,
347     ) {
348         PreviewCheckerboard(Modifier.fillMaxSize(), cellSize = 7f)
349         if (preview != null) {
350             Image(
351                 bitmap = preview,
352                 contentDescription = null,
353                 contentScale = ContentScale.Fit,
354                 modifier = Modifier.fillMaxSize(),
355             )
356         } else {
357             Canvas(Modifier.fillMaxSize()) {
358                 drawLine(
359                     color = theme.accent.copy(alpha = brush.settings.opacity.coerceIn(0.25f, 1f)),
360                     start = Offset(size.width * 0.12f, size.height * 0.5f),
361                     end = Offset(size.width * 0.88f, size.height * 0.5f),
362                     strokeWidth = (brush.settings.baseRadiusPx * 2f).coerceIn(2f, 12f),
363                 )
364             }
365         }
366     }

```

```

367 }
368
369 @Composable
370 private fun PreviewCheckerboard(
371     modifier: Modifier = Modifier,
372     cellSize: Float,
373 ) {
374     Canvas(modifier) {
375         val light = Color(0xFFE9E9E9)
376         val dark = Color(0xFFD0D0D0)
377         var y = 0f
378         var row = 0
379         while (y < size.height) {
380             var x = 0f
381             var col = 0
382             while (x < size.width) {
383                 drawRect(
384                     color = if ((row + col) % 2 == 0) light else dark,
385                     topLeft = Offset(x, y),
386                     size = Size(cellSize, cellSize),
387                 )
388                 x += cellSize
389                 col += 1
390             }
391             y += cellSize
392             row += 1
393         }
394     }
395 }

```


105. app/src/main/java/com/wetinknext/ui/components/LayersPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import android.graphics.BitmapFactory
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.border
6  import androidx.compose.foundation.clickable
7  import androidx.compose.foundation.layout.*
8  import androidx.compose.foundation.lazy.LazyColumn
9  import androidx.compose.foundation.lazy.items
10 import androidx.compose.foundation.shape.CircleShape
11 import androidx.compose.foundation.shape.RoundedCornerShape
12 import androidx.compose.material.icons.Icons
13 import androidx.compose.material.icons.filled.*
14 import androidx.compose.material3.*
15 import androidx.compose.runtime.*
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.draw.clip
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.graphics.asImageBitmap
21 import androidx.compose.ui.graphics.vector.ImageVector
22 import androidx.compose.ui.layout.ContentScale
23 import androidx.compose.ui.unit.Dp
24 import androidx.compose.ui.text.font.FontWeight
25 import androidx.compose.ui.text.style.TextOverflow
26 import androidx.compose.ui.unit.dp
27 import androidx.compose.ui.unit.sp
28 import com.wetinknext.ui.state.LayerItem
29 import com.wetinknext.ui.state.LayerState
30 import com.wetinknext.ui.theme.AppTheme
31 import com.wetinknext.ui.thumbnail.LayerThumbnail
32 import java.io.File
33 import kotlinx.coroutines.Dispatchers
34 import kotlinx.coroutines.withContext
35 import kotlin.math.roundToInt
36
37 private val PANEL_WIDTH = 320.dp
38
39 @Composable
40 fun LayersPanel(
41     state: LayerState,
42     theme: AppTheme,
43     modifier: Modifier = Modifier,
44     onDismiss: () -> Unit,
45     onAddLayer: () -> Unit,
46     onSelectLayer: (Long) -> Unit,
47     onVisibleChange: (Long, Boolean) -> Unit,
48     onOpacityChange: (Long, Float) -> Unit,
49     onDuplicateLayer: (Long) -> Unit,
50     onRemoveLayer: (Long) -> Unit,
51 ) {
52     PanelSurface(
53         modifier = modifier.width(PANEL_WIDTH),
54     ) {
55         Column(
56             modifier = Modifier.padding(10.dp),
57             verticalArrangement = Arrangement.spacedBy(8.dp),
58         ) {
59             Row(
60                 verticalAlignment = Alignment.CenterVertically,
61                 horizontalArrangement = Arrangement.spacedBy(6.dp),
62             ) {
63                 Text(
64                     "Слой",
65                     color = theme.textPrimary,
66                     fontWeight = FontWeight.Bold,
67                     fontSize = 18.sp,
68                 )
69                 Spacer(Modifier.weight(1f))
70                 HeaderIconButton(
71                     theme = theme,
72                     icon = Icons.Default.Add,
73                     description = "Добавить слой",
74                     onClick = onAddLayer,
75                 )
76                 HeaderIconButton(
77                     theme = theme,
78                     icon = Icons.Default.Close,
79                     description = "Закрыть",
80                     onClick = onDismiss,
81                 )
82             }
83
84             LazyColumn(
85                 modifier = Modifier
86                     .fillMaxWidth()
87                     .heightIn(max = 400.dp),
88                 verticalArrangement = Arrangement.spacedBy(4.dp),
89             ) {
90                 items(

```

```

91         items = state.layers.asReversed(),
92         key = { it.id },
93     ) { layerItem ->
94         LayerRow(
95             layer = layerItem,
96             theme = theme,
97             active = layerItem.id == state.activeLayerId,
98             onSelect = { onSelectLayer(layerItem.id) },
99             onVisibleChange = { onVisibleChange(layerItem.id, it) },
100             onOpacityChange = { onOpacityChange(layerItem.id, it) },
101             onDuplicate = { onDuplicateLayer(layerItem.id) },
102             onDelete = { onRemoveLayer(layerItem.id) },
103             canDelete = state.layers.size > 1 && !layerItem.locked
104         )
105     }
106 }
107 }
108 }
109 }
110
111 @Composable
112 private fun LayerRow(
113     layer: LayerItem,
114     theme: AppTheme,
115     active: Boolean,
116     onSelect: () -> Unit,
117     onVisibleChange: (Boolean) -> Unit,
118     onOpacityChange: (Float) -> Unit,
119     onDuplicate: () -> Unit,
120     onDelete: () -> Unit,
121     canDelete: Boolean,
122 ) {
123     val cardBg = if (active) theme.panelBgSolid else theme.panelBg
124
125     Column(
126         modifier = Modifier
127             .fillMaxWidth()
128             .clip(RoundedCornerShape(12.dp))
129             .background(cardBg)
130             .border(
131                 width = if (active) 1.2.dp else 0.dp,
132                 color = if (active) theme.accent else Color.Transparent,
133                 shape = RoundedCornerShape(12.dp),
134             )
135         .clickable { onSelect() }
136         .padding(8.dp),
137         verticalArrangement = Arrangement.spacedBy(4.dp)
138     ) {
139         Row(
140             modifier = Modifier.fillMaxWidth(),
141             verticalAlignment = Alignment.CenterVertically,
142         ) {
143             HeaderIconButton(
144                 theme = theme,
145                 icon = if (layer.visible) Icons.Default.Visibility else Icons.Default.VisibilityOff,
146                 description = "Видимость",
147                 onClick = { onVisibleChange(!layer.visible) },
148                 tint = if (layer.visible) (if (active) theme.accent else theme.textPrimary) else theme.iconInactive,
149             )
150
151             Spacer(Modifier.width(8.dp))
152
153             LayerThumbnail(
154                 layerId = layer.id,
155                 path = layer.thumbnailPath,
156                 thumbnailVersion = layer.thumbnailVersion,
157                 modifier = Modifier.height(36.dp),
158             )
159
160             Spacer(Modifier.width(8.dp))
161
162             Text(
163                 layer.name,
164                 modifier = Modifier.weight(1f),
165                 color = theme.textPrimary,
166                 fontSize = 14.sp,
167                 fontWeight = if (active) FontWeight.SemiBold else FontWeight.Normal,
168                 maxLines = 1,
169                 overflow = TextOverflow.Ellipsis,
170             )
171
172             if (canDelete) {
173                 HeaderIconButton(
174                     theme = theme,
175                     icon = Icons.Default.ContentCopy,
176                     description = "Дублировать слой",
177                     onClick = onDuplicate,
178                 )
179                 Spacer(Modifier.width(4.dp))
180                 HeaderIconButton(
181                     theme = theme,
182                     icon = Icons.Default.Delete,

```

```

183         description = "Удалить",
184         onClick = onDelete,
185         tint = theme.danger
186     )
187 }
188 }
189
190 if (active) {
191     Column(modifier = Modifier.fillMaxWidth().padding(horizontal = 4.dp)) {
192         Row(verticalAlignment = Alignment.CenterVertically) {
193             Text("Непрозрачность", color = theme.textSecondary, fontSize = 11.sp)
194             Spacer(Modifier.weight(1f))
195             Text(
196                 "${(layer.opacity * 100f).roundToInt()}%",
197                 color = theme.textSecondary,
198                 fontSize = 11.sp,
199             )
200         }
201         Slider(
202             value = layer.opacity.coerceIn(0f, 1f),
203             onValueChange = onOpacityChange,
204             valueRange = 0f..1f,
205             colors = SliderDefaults.colors(
206                 thumbColor = theme.accent,
207                 activeTrackColor = theme.accent,
208                 inactiveTrackColor = theme.accentMuted,
209             ),
210         )
211     }
212 }
213 }
214 }
215
216 /** Small, non-blocking thumbnail loader for the persisted layer WebP preview. */
217 @Composable
218 private fun LegacyLayerThumbnail(
219     layer: LayerItem,
220     theme: AppTheme,
221     size: Dp = 38.dp,
222 ) {
223     var preview by remember(layer.thumbnailPath, layer.thumbnailVersion) {
224         mutableStateOf<androidx.compose.ui.graphics.ImageBitmap?>(null)
225     }
226     LaunchedEffect(layer.thumbnailPath, layer.thumbnailVersion) {
227         preview = withContext(Dispatchers.IO) {
228             layer.thumbnailPath
229                 ?.let(::File)
230                 ?.takeIf(File::isFile)
231                 ?.let { BitmapFactory.decodeFile(it.absolutePath) }
232                 ?.asImageBitmap()
233         }
234     }
235
236     Box(
237         modifier = Modifier
238             .size(size)
239             .clip(RoundedCornerShape(7.dp))
240             .background(theme.panelInset)
241             .border(1.dp, theme.panelStroke, RoundedCornerShape(7.dp)),
242         contentAlignment = Alignment.Center,
243     ) {
244         if (preview != null) {
245             androidx.compose.foundation.Image(
246                 bitmap = checkNotNull(preview),
247                 contentDescription = "Превью слоя ${layer.name}",
248                 modifier = Modifier.fillMaxSize(),
249                 contentScale = ContentScale.Fit,
250             )
251         } else {
252             Text(
253                 text = layer.name.take(1).uppercase(),
254                 color = theme.iconInactive,
255                 fontSize = 12.sp,
256                 fontWeight = FontWeight.Bold,
257             )
258         }
259     }
260 }
261
262 @Composable
263 private fun HeaderIconButton(
264     theme: AppTheme,
265     icon: ImageVector,
266     description: String,
267     onClick: () -> Unit,
268     enabled: Boolean = true,
269     tint: Color = theme.textPrimary,
270 ) {
271     Box(
272         modifier = Modifier
273             .size(32.dp)
274             .clip(CircleShape)

```

```
275         .background(if (enabled) theme.panelInsetSoft else theme.panelInset)
276         .clickable(enabled = enabled, onClick = onClick),
277         contentAlignment = Alignment.Center,
278     ) {
279         Icon(
280             icon,
281             contentDescription = description,
282             tint = if (enabled) tint else theme.iconInactive,
283             modifier = Modifier.size(18.dp),
284         )
285     }
286 }
```

106. app/src/main/java/com/wetinknext/ui/components/PanelSurface.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.layout.Box
6 import androidx.compose.foundation.layout.BoxScope
7 import androidx.compose.foundation.shape.RoundedCornerShape
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.draw.clip
11 import androidx.compose.ui.draw.shadow
12 import androidx.compose.ui.input.pointer.PointerEventPass
13 import androidx.compose.ui.input.pointer.pointerInput
14 import androidx.compose.ui.unit.dp
15 import com.wetinknext.ui.theme.LocalAppTheme
16
17 @Composable
18 fun PanelSurface(
19     modifier: Modifier = Modifier,
20     content: @Composable BoxScope.() -> Unit,
21 ) {
22     val theme = LocalAppTheme.current
23     val shape = RoundedCornerShape(22.dp)
24     Box(
25         modifier = modifier
26             .shadow(12.dp, shape, ambientColor = theme.panelInset.copy(alpha = 0.5f), spotColor = theme.panelInset.copy(alpha
= 0.5f))
27             .clip(shape)
28             .background(theme.panelBg)
29             .border(1.2.dp, theme.panelStroke, shape)
30     ) {
31         Box(modifier = Modifier.matchParentSize().swallowChrome())
32         content()
33     }
34 }
35
36 private fun Modifier.swallowChrome(): Modifier = pointerInput(Unit) {
37     awaitPointerEventScope {
38         while (true) {
39             val event = awaitPointerEvent(PointerEventPass.Initial)
40             for (change in event.changes) {
41                 if (change.pressed && !change.previousPressed) {
42                     change.consume()
43                 }
44             }
45         }
46     }
47 }

```

107. app/src/main/java/com/wetinknext/ui/components/SelectionToolbar.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.BorderStroke
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.shape.CircleShape
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.automirrored.filled.CallMerge
11 import androidx.compose.material.icons.filled.*
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Surface
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.draw.clip
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.graphics.vector.ImageVector
21 import androidx.compose.ui.text.font.FontWeight
22 import androidx.compose.ui.unit.dp
23 import androidx.compose.ui.unit.sp
24 import com.wetinknext.ui.theme.AppTheme
25
26 enum class SelectionShapeUi {
27     FREEHAND,
28     RECTANGLE,
29     ELLIPSE,
30 }
31
32 @Composable
33 fun SelectionToolbar(
34     theme: AppTheme,
35     currentShape: SelectionShapeUi,
36     onShapeChange: (SelectionShapeUi) -> Unit,
37     onDone: () -> Unit,
38     modifier: Modifier = Modifier,
39 ) {
40     val panelBorder = BorderStroke(1.dp, theme.panelStroke)
41
42     Column(
43         modifier = modifier
44             .fillMaxWidth()
45             .navigationBarsPadding()
46             .padding(horizontal = 16.dp, vertical = 20.dp),
47         horizontalAlignment = Alignment.CenterHorizontally,
48         verticalArrangement = Arrangement.spacedBy(12.dp),
49     ) {
50         // Row 1 – shape modes
51         Surface(
52             color = theme.panelBg.copy(alpha = 0.94f),
53             shape = RoundedCornerShape(24.dp),
54             border = panelBorder,
55         ) {
56             Row(
57                 modifier = Modifier
58                     .width(340.dp)
59                     .height(32.dp)
60                     .padding(4.dp),
61                 horizontalArrangement = Arrangement.spacedBy(4.dp),
62                 verticalAlignment = Alignment.CenterVertically,
63             ) {
64                 val shapes = listOf(
65                     SelectionShapeUi.FREEHAND to "От руки",
66                     SelectionShapeUi.RECTANGLE to "Прямоугольник",
67                     SelectionShapeUi.ELLIPSE to "Эллипс",
68                 )
69                 shapes.forEach { (shape, label) ->
70                     val isSelected = currentShape == shape
71                     Box(
72                         modifier = Modifier
73                             .weight(1f)
74                             .fillMaxHeight()
75                             .clip(RoundedCornerShape(20.dp))
76                             .background(if (isSelected) theme.accent.copy(alpha = 0.4f) else Color.Transparent)
77                             .clickable { onShapeChange(shape) },
78                         contentAlignment = Alignment.Center,
79                     ) {
80                         Text(
81                             text = label,
82                             color = if (isSelected) Color.White.copy(alpha = 0.4f) else theme.textPrimary.copy(alpha = 0.4f),
83                             fontSize = 13.sp,
84                             fontWeight = if (isSelected) FontWeight.Bold else FontWeight.Normal,
85                             maxLines = 1,
86                         )
87                     }
88                 }
89             }
90         }
91     }
92 }

```

```

91
92 // Row 2 – Action icons (mostly disabled for now)
93 Surface(
94     color = theme.panelBg.copy(alpha = 0.94f),
95     shape = RoundedCornerShape(24.dp),
96     border = panelBorder,
97 ) {
98     Row(
99         modifier = Modifier.padding(horizontal = 4.dp, vertical = 4.dp),
100         horizontalArrangement = Arrangement.spacedBy(4.dp),
101         verticalAlignment = Alignment.CenterVertically,
102     ) {
103         SelectionIconButton(Icons.Default.Add, "Добавить", theme, isSelected = true, enabled = false)
104         SelectionIconButton(Icons.Default.Delete, "Удалить", theme, enabled = false)
105         SelectionIconButton(Icons.AutoMirrored.Filled.CallMerge, "Пересечь", theme, enabled = false)
106         SelectionIconButton(Icons.Default.Flip, "Инверсия", theme, enabled = false)
107         SelectionIconButton(Icons.Default.FormatColorFill, "Залить", theme, enabled = false)
108         SelectionIconButton(Icons.Default.ContentCopy, "Копировать", theme, enabled = false)
109         SelectionIconButton(Icons.Default.ContentCut, "Вырезать", theme, enabled = false)
110         SelectionIconButton(Icons.Default.AddToPhotos, "Вставить", theme, enabled = false)
111     }
112 }
113
114 // Row 3 – Reset / Delete / Done
115 Row(
116     horizontalArrangement = Arrangement.spacedBy(16.dp),
117     verticalAlignment = Alignment.CenterVertically,
118 ) {
119     SelectionRoundButton(Icons.Default.Refresh, "Сбросить", theme, enabled = false)
120     SelectionRoundButton(Icons.Default.Delete, "Удалить", theme, enabled = false)
121     Surface(
122         color = theme.accent,
123         shape = CircleShape,
124         modifier = Modifier.size(48.dp),
125     ) {
126         Box(
127             modifier = Modifier
128                 .fillMaxSize()
129                 .clickable { onDone() },
130             contentAlignment = Alignment.Center,
131         ) {
132             Icon(Icons.Default.Check, "Готово", tint = Color.White)
133         }
134     }
135 }
136 }
137 }
138
139 @Composable
140 private fun SelectionIconButton(
141     icon: ImageVector,
142     description: String,
143     theme: AppTheme,
144     isSelected: Boolean = false,
145     enabled: Boolean = true,
146     onClick: () -> Unit = {},
147 ) {
148     Box(
149         modifier = Modifier
150             .size(48.dp)
151             .clip(RoundedCornerShape(8.dp))
152             .background(if (isSelected && enabled) theme.accent.copy(alpha = 0.30f) else Color.Transparent)
153             .clickable(enabled = enabled) { onClick() },
154         contentAlignment = Alignment.Center,
155     ) {
156         Icon(
157             imageVector = icon,
158             contentDescription = description,
159             tint = if (!enabled) theme.iconInactive.copy(alpha = 0.5f) else if (isSelected) theme.accent else theme.
160 textPrimary,
161             modifier = Modifier.size(24.dp),
162         )
163     }
164 }
165
166 @Composable
167 private fun SelectionRoundButton(
168     icon: ImageVector,
169     description: String,
170     theme: AppTheme,
171     enabled: Boolean = true,
172     onClick: () -> Unit = {},
173 ) {
174     Surface(
175         color = if (enabled) theme.panelBg.copy(alpha = 0.9f) else theme.panelBg.copy(alpha = 0.4f),
176         shape = CircleShape,
177         border = BorderStroke(1.dp, if (enabled) theme.panelStroke else theme.panelStroke.copy(alpha = 0.4f)),
178         modifier = Modifier.size(48.dp),
179     ) {
180         Box(
181             modifier = Modifier
182                 .fillMaxSize()

```

```
182         .clickable(enabled = enabled) { onClick() },
183         contentAlignment = Alignment.Center,
184     ) {
185         Icon(
186             icon,
187             description,
188             tint = if (enabled) theme.textPrimary else theme.textPrimary.copy(alpha = 0.4f)
189         )
190     }
191 }
192 }
```


108. app/src/main/java/com/wetinknext/ui/components/SwipeRevealRow.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.animation.core.animateFloatAsState
4 import androidx.compose.foundation.gestures.detectHorizontalDragGestures
5 import androidx.compose.foundation.layout.Arrangement
6 import androidx.compose.foundation.layout.Box
7 import androidx.compose.foundation.layout.Row
8 import androidx.compose.foundation.layout.RowScope
9 import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.offset
11 import androidx.compose.runtime.Composable
12 import androidx.compose.runtime.getValue
13 import androidx.compose.runtime.mutableStateOf
14 import androidx.compose.runtime.remember
15 import androidx.compose.runtime.setValue
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.input.pointer.pointerInput
18 import androidx.compose.ui.platform.LocalDensity
19 import androidx.compose.ui.unit.Dp
20 import androidx.compose.ui.unit.IntOffset
21 import androidx.compose.ui.unit.dp
22 import kotlin.math.roundToInt
23
24 @Composable
25 fun SwipeRevealRow(
26     revealWidth: Dp,
27     actions: @Composable RowScope() -> Unit,
28     modifier: Modifier = Modifier,
29     rightSwipeThreshold: Dp = 56.dp,
30     onSwipeRight: (() -> Unit)? = null,
31     content: @Composable () -> Unit,
32 ) {
33     var rawOffset by remember { mutableStateOf(0f) }
34     var rightDragOffset by remember { mutableStateOf(0f) }
35     val revealPx = with(LocalDensity.current) { revealWidth.toPx() }
36     val rightSwipeThresholdPx = with(LocalDensity.current) { rightSwipeThreshold.toPx() }
37     val animatedOffset by animateFloatAsState(
38         targetValue = rawOffset.coerceIn(-revealPx, 0f),
39         label = "swipeRevealRow",
40     )
41
42     Box(modifier = modifier.fillMaxWidth()) {
43         Row(
44             modifier = Modifier.fillMaxWidth(),
45             horizontalArrangement = Arrangement.End,
46             content = actions,
47         )
48
49         Box(
50             modifier = Modifier
51                 .fillMaxWidth()
52                 .offset { IntOffset(animatedOffset.roundToInt(), 0) }
53                 .pointerInput(revealWidth) {
54                     detectHorizontalDragGestures(
55                         onHorizontalDrag = { change, dx ->
56                             if (dx < 0f || rawOffset < 0f) {
57                                 change.consume()
58                                 rawOffset += dx
59                                 rightDragOffset = 0f
60                             } else if (dx > 0f && rawOffset == 0f && onSwipeRight != null) {
61                                 change.consume()
62                                 rightDragOffset += dx
63                             }
64                         },
65                         onDragEnd = {
66                             if (rightDragOffset > rightSwipeThresholdPx) {
67                                 onSwipeRight?.invoke()
68                             }
69                             rawOffset = if (rawOffset < -revealPx * 0.4f) -revealPx else 0f
70                             rightDragOffset = 0f
71                         },
72                         onDragCancel = {
73                             rawOffset = 0f
74                             rightDragOffset = 0f
75                         },
76                     )
77                 },
78         ) {
79             content()
80         }
81     }
82 }

```

109. app/src/main/java/com/wetinknext/ui/components/ThresholdPanel.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.shape.CircleShape
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.Check
11 import androidx.compose.material.icons.filled.Close
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Slider
14 import androidx.compose.material3.SliderDefaults
15 import androidx.compose.material3.Switch
16 import androidx.compose.material3.Text
17 import androidx.compose.runtime.Composable
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.draw.clip
21 import androidx.compose.ui.graphics.Color
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.unit.dp
24 import androidx.compose.ui.unit.sp
25 import com.wetinknext.ui.theme.AppTheme
26 import kotlin.math.roundToInt
27
28 @Composable
29 fun ThresholdPanel(
30     theme: AppTheme,
31     threshold: Float, // 0..1
32     preserveTransparency: Boolean,
33     onThresholdChange: (Float) -> Unit,
34     onPreserveTransparencyChange: (Boolean) -> Unit,
35     onApply: () -> Unit,
36     onCancel: () -> Unit,
37     isApplyEnabled: Boolean = true,
38     modifier: Modifier = Modifier
39 ) {
40     Column(
41         modifier = modifier
42             .width(360.dp)
43             .clip(RoundedCornerShape(28.dp))
44             .background(theme.panelBg.copy(alpha = 0.95f))
45             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
46             .padding(horizontal = 20.dp, vertical = 16.dp),
47         verticalArrangement = Arrangement.spacedBy(16.dp)
48     ) {
49         Row(
50             modifier = Modifier.fillMaxWidth(),
51             horizontalArrangement = Arrangement.SpaceBetween,
52             verticalAlignment = Alignment.CenterVertically
53         ) {
54             Text(
55                 text = "Бинаризация",
56                 color = theme.textPrimary,
57                 fontSize = 15.sp,
58                 fontWeight = FontWeight.Bold
59             )
60
61             Row(
62                 verticalAlignment = Alignment.CenterVertically,
63                 horizontalArrangement = Arrangement.spacedBy(12.dp)
64             ) {
65                 Box(
66                     modifier = Modifier
67                         .size(36.dp)
68                         .clip(CircleShape)
69                         .background(theme.panelInset)
70                         .clickable { onCancel() },
71                     contentAlignment = Alignment.Center
72                 ) {
73                     Icon(Icons.Default.Close, null, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
74                 }
75
76                 Box(
77                     modifier = Modifier
78                         .size(36.dp)
79                         .clip(CircleShape)
80                         .background(if (isApplyEnabled) theme.accent else theme.panelInset)
81                         .clickable(enabled = isApplyEnabled) { onApply() },
82                     contentAlignment = Alignment.Center
83                 ) {
84                     Icon(Icons.Default.Check, null, tint = Color.White, modifier = Modifier.size(20.dp))
85                 }
86             }
87         }
88
89         Text(
90             text = "Пиксели светлее порога станут белыми, остальные – чёрными",

```

```

91         color = theme.textSecondary,
92         fontSize = 12.sp,
93         lineHeight = 16.sp
94     )
95
96     // Threshold
97     Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
98         Row(
99             modifier = Modifier.fillMaxWidth(),
100             horizontalArrangement = Arrangement.SpaceBetween
101         ) {
102             Text("Попор", color = theme.textSecondary, fontSize = 13.sp)
103             Text("${(threshold * 255).roundToInt()}", color = theme.accent, fontSize = 13.sp, fontWeight = FontWeight.
Bold)
104         }
105         Slider(
106             value = threshold,
107             onValueChange = onThresholdChange,
108             valueRange = 0f..1f,
109             colors = SliderDefaults.colors(
110                 thumbColor = theme.accent,
111                 activeTrackColor = theme.accent,
112                 inactiveTrackColor = theme.panelInset
113             )
114         )
115     }
116
117     Row(
118         modifier = Modifier.fillMaxWidth(),
119         verticalAlignment = Alignment.CenterVertically,
120         horizontalArrangement = Arrangement.SpaceBetween
121     ) {
122         Text(
123             text = "Сохранять прозрачность",
124             color = theme.textPrimary,
125             fontSize = 14.sp
126         )
127         Switch(
128             checked = preserveTransparency,
129             onCheckedChange = onPreserveTransparencyChange
130         )
131     }
132 }
133 }

```

110. app/src/main/java/com/wetinknext/ui/components/Toolbars.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.animation.core.Spring
4 import androidx.compose.animation.core.animateFloatAsState
5 import androidx.compose.animation.core.spring
6 import androidx.compose.foundation.background
7 import androidx.compose.foundation.border
8 import androidx.compose.foundation.clickable
9 import androidx.compose.foundation.gestures.detectDragGestures
10 import androidx.compose.foundation.gestures.detectTapGestures
11 import androidx.compose.foundation.layout.*
12 import androidx.compose.foundation.shape.CircleShape
13 import androidx.compose.foundation.shape.RoundedCornerShape
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.automirrored.filled.Redo
16 import androidx.compose.material.icons.automirrored.filled.Undo
17 import androidx.compose.material.icons.filled.*
18 import androidx.compose.material3.Icon
19 import androidx.compose.runtime.*
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.draw.clip
23 import androidx.compose.ui.graphics.Brush
24 import androidx.compose.ui.graphics.Color
25 import androidx.compose.ui.graphics.graphicsLayer
26 import androidx.compose.ui.graphics.vector.ImageVector
27 import androidx.compose.ui.input.pointer.pointerInput
28 import androidx.compose.ui.layout.onSizeChanged
29 import androidx.compose.ui.platform.LocalDensity
30 import androidx.compose.ui.res.vectorResource
31 import androidx.compose.ui.unit.IntOffset
32 import androidx.compose.ui.unit.dp
33 import com.wetinknext.R
34 import com.wetinknext.ui.theme.AppTheme
35
36 @Composable
37 fun TopToolbar(
38     theme: AppTheme,
39     currentColor: Color,
40     canUndo: Boolean,
41     canRedo: Boolean,
42     isSelectionActive: Boolean,
43     isTransformActive: Boolean,
44     isAnimationActive: Boolean,
45     isAdjustmentsActive: Boolean = false,
46     onUndoClick: () -> Unit,
47     onRedoClick: () -> Unit,
48     onSelectionClick: () -> Unit,
49     onTransformClick: () -> Unit,
50     onAnimationClick: () -> Unit,
51     onAdjustmentsClick: () -> Unit = {},
52     onSettingsClick: () -> Unit,
53     onLayersClick: () -> Unit,
54     onColorClick: () -> Unit,
55 ) {
56     Row(
57         modifier = Modifier
58             .padding(top = 12.dp, end = 12.dp)
59             .clip(RoundedCornerShape(25.dp))
60             .background(theme.panelBg.copy(alpha = 0.9f))
61             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(25.dp))
62             .padding(horizontal = 8.dp, vertical = 6.dp),
63         verticalAlignment = Alignment.CenterVertically,
64         horizontalArrangement = Arrangement.spacedBy(10.dp)
65     ) {
66         ToolbarIcon(Icons.AutoMirrored.Filled.Undo, theme, isSelected = false, enabled = canUndo, onClick = onUndoClick)
67         ToolbarIcon(Icons.AutoMirrored.Filled.Redo, theme, isSelected = false, enabled = canRedo, onClick = onRedoClick)
68
69         // SelectAll has a visually denser glyph than Fullscreen; its 18 dp drawing size
70         // matches Transform's perceived 20 dp footprint while both buttons stay 34 dp.
71         ToolbarIcon(Icons.Default.SelectAll, theme, isSelected = isSelectionActive, iconSize = 18.dp, onClick =
onSelectionClick)
72         ToolbarIcon(Icons.Default.Movie, theme, isSelected = isAnimationActive, onClick = onAnimationClick)
73         ToolbarIcon(Icons.Default.Fullscreen, theme, isSelected = isTransformActive, onClick = onTransformClick)
74         ToolbarIcon(Icons.Default.Tune, theme, isSelected = isAdjustmentsActive, onClick = onAdjustmentsClick)
75         ToolbarIcon(Icons.Default.Layers, theme, onClick = onLayersClick)
76         ToolbarIcon(Icons.Default.Settings, theme, contentDescription = "Настройки", onClick = onSettingsClick)
77
78         ColorPickerDot(
79             color = currentColor,
80             theme = theme,
81             onClick = onColorClick
82         )
83     }
84 }
85
86 /** Compact actions placed directly under the left brush toolbar. */
87 @Composable
88 fun LeftCanvasActions(
89     theme: AppTheme,

```

```

90     onClearLayerClick: () -> Unit,
91     onMirrorClick: () -> Unit,
92 ) {
93     Column(
94         modifier = Modifier.padding(start = 12.dp, top = 10.dp),
95         verticalArrangement = Arrangement.spacedBy(10.dp),
96         horizontalAlignment = Alignment.CenterHorizontally,
97     ) {
98         CanvasActionButton(
99             icon = Icons.Default.DeleteSweep,
100             description = "Очистить активный слой",
101             theme = theme,
102             onClick = onClearLayerClick,
103         )
104         CanvasActionButton(
105             icon = Icons.Default.Flip,
106             description = "Зеркальный просмотр холста",
107             theme = theme,
108             onClick = onMirrorClick,
109         )
110     }
111 }
112
113 @Composable
114 fun CanvasActionButton(
115     icon: ImageVector,
116     description: String,
117     theme: AppTheme,
118     onClick: () -> Unit,
119 ) {
120     Box(
121         modifier = Modifier
122             .size(44.dp)
123             .clip(CircleShape)
124             .background(theme.panelBg.copy(alpha = 0.9f))
125             .border(1.2.dp, theme.panelStroke, CircleShape)
126             .clickable(onClick = onClick),
127         contentAlignment = Alignment.Center,
128     ) {
129         Icon(icon, contentDescription = description, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
130     }
131 }
132
133 @Composable
134 fun ColorPickerDot(
135     color: Color,
136     theme: AppTheme,
137     onClick: () -> Unit
138 ) {
139     var isPressed by remember { mutableStateOf(false) }
140
141     val scale by animateFloatAsState(
142         targetValue = if (isPressed) 1.15f else 1f,
143         animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy, stiffness = Spring.StiffnessLow),
144         label = "scale"
145     )
146
147     Box(
148         modifier = Modifier
149             .graphicsLayer {
150                 scaleX = scale
151                 scaleY = scale
152             }
153             .size(30.dp)
154             .clip(CircleShape)
155             .background(color)
156             .border(1.5.dp, theme.panelStroke, CircleShape)
157             .clickable { onClick() }
158     )
159 }
160
161 @Composable
162 fun SideToolbar(
163     theme: AppTheme,
164     brushSize: Float,
165     brushOpacity: Float,
166     isEraser: Boolean,
167     onBrushSizeChange: (Float) -> Unit,
168     onBrushOpacityChange: (Float) -> Unit,
169     onBrushClick: () -> Unit,
170     onEraserClick: (Boolean) -> Unit,
171 ) {
172     Column(
173         modifier = Modifier
174             .padding(start = 12.dp)
175             .width(44.dp)
176             .clip(RoundedCornerShape(22.dp))
177             .background(theme.panelBg.copy(alpha = 0.9f))
178             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(22.dp))
179             .padding(vertical = 10.dp),
180         horizontalAlignment = Alignment.CenterHorizontally,
181         verticalArrangement = Arrangement.spacedBy(12.dp)

```

```

182     ) {
183         ToolbarIcon(Icons.Default.Brush, theme, isSelected = !isEraser, onClick = {
184             onEraserClick(false)
185             onBrushClick()
186         })
187         ToolbarIcon(ImageVector.vectorResource(R.drawable.ic_eraser), theme, isSelected = isEraser, onClick = { onEraserClick(
true) })
188
189         Spacer(Modifier.height(4.dp))
190
191         VerticalSlider(
192             value = brushSize,
193             onValueChange = onBrushSizeChange,
194             valueRange = 2f..400f,
195             theme = theme
196         )
197
198         VerticalSlider(
199             value = brushOpacity,
200             onValueChange = onBrushOpacityChange,
201             valueRange = 0f..1f,
202             theme = theme
203         )
204     }
205 }
206
207 @Composable
208 private fun VerticalSlider(
209     value: Float,
210     onValueChange: (Float) -> Unit,
211     valueRange: ClosedFloatingPointRange<Float> = 0f..1f,
212     theme: AppTheme,
213     modifier: Modifier = Modifier
214 ) {
215     val range = valueRange.endInclusive - valueRange.start
216     var trackHeightPx by remember { mutableFloatStateOf(1f) }
217     val density = LocalDensity.current
218
219     val norm = ((value - valueRange.start) / range).coerceIn(0f, 1f)
220
221     fun updateFromY(y: Float) {
222         val n = (1f - (y / trackHeightPx)).coerceIn(0f, 1f)
223         onValueChange(valueRange.start + n * range)
224     }
225
226     Box(
227         modifier = modifier
228             .width(28.dp)
229             .height(120.dp)
230             .clip(RoundedCornerShape(14.dp))
231             .background(theme.accentMuted.copy(alpha = 0.15f))
232             .border(1.dp, theme.panelStroke, RoundedCornerShape(14.dp))
233             .onSizeChanged { trackHeightPx = it.height.toFloat() }
234             .pointerInput(valueRange) {
235                 detectTapGestures { offset -> updateFromY(offset.y) }
236             }
237             .pointerInput(valueRange) {
238                 detectDragGestures { change, _ ->
239                     change.consume()
240                     updateFromY(change.position.y)
241                 }
242             }
243     ) {
244         Box(
245             Modifier
246                 .align(Alignment.BottomCenter)
247                 .fillMaxWidth()
248                 .fillMaxHeight(norm)
249                 .background(Brush.verticalGradient(listOf(theme.accentSoft, theme.accent)))
250         )
251         Box(
252             Modifier
253                 .align(Alignment.TopCenter)
254                 .offset {
255                     val thumbSizePx = with(density) { 22.dp.toPx() }
256                     IntOffset(0, ((1f - norm) * (trackHeightPx - thumbSizePx)).toInt())
257                 }
258                 .padding(2.dp)
259                 .size(22.dp)
260                 .clip(CircleShape)
261                 .background(theme.textPrimary)
262                 .border(1.dp, theme.panelStroke, CircleShape)
263         )
264     }
265 }
266
267 @Composable
268 private fun ToolbarIcon(
269     icon: ImageVector,
270     theme: AppTheme,
271     isSelected: Boolean = false,
272     enabled: Boolean = true,

```

```

273     contentDescription: String? = null,
274     iconSize: androidx.compose.ui.unit.Dp = 20.dp,
275     onClick: () -> Unit = {}
276 ) {
277     Box(
278         modifier = Modifier
279             .size(34.dp)
280             .clip(CircleShape)
281             .background(if (isSelected) theme.accentMuted.copy(alpha = 0.3f) else Color.Transparent)
282             .border(if (isSelected) 1.dp else 0.dp, if (isSelected) theme.accentSoft else Color.Transparent, CircleShape)
283             .clickable(enabled = enabled) { onClick() },
284         contentAlignment = Alignment.Center
285     ) {
286         Icon(
287             icon,
288             contentDescription = contentDescription,
289             tint = if (!enabled) theme.iconInactive.copy(alpha = 0.5f) else if (isSelected) theme.accent else theme.
textPrimary,
290             modifier = Modifier.size(iconSize)
291         )
292     }
293 }

```

111. app/src/main/java/com/wetinknext/ui/components/TransformMenuView.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.BorderStroke
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.shape.CircleShape
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.*
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.draw.rotate
17 import androidx.compose.ui.graphics.Color
18 import androidx.compose.ui.graphics.vector.ImageVector
19 import androidx.compose.ui.text.font.FontWeight
20 import androidx.compose.ui.unit.dp
21 import androidx.compose.ui.unit.sp
22 import com.wetinknext.ui.theme.AppTheme
23
24 enum class TransformModeUi {
25     FREEFORM,
26     UNIFORM,
27     DISTORT,
28     WARP
29 }
30
31 interface TransformActions {
32     fun reset()
33     fun cancel()
34     fun apply()
35     fun flipHorizontal()
36     fun flipVertical()
37     fun setMode(mode: TransformModeUi)
38 }
39
40 @Composable
41 fun TransformMenuView(
42     theme: AppTheme,
43     actions: TransformActions,
44     currentMode: TransformModeUi,
45     modifier: Modifier = Modifier
46 ) {
47     val panelBorder = BorderStroke(1.dp, theme.panelStroke)
48
49     Column(
50         modifier = modifier
51             .fillMaxWidth()
52             .navigationBarsPadding()
53             .padding(horizontal = 16.dp, vertical = 20.dp),
54         horizontalAlignment = Alignment.CenterHorizontally,
55         verticalArrangement = Arrangement.spacedBy(12.dp)
56     ) {
57         // Row 1: Modes
58         Surface(
59             color = theme.panelBg.copy(alpha = 0.94f),
60             shape = RoundedCornerShape(24.dp),
61             border = panelBorder
62         ) {
63             Row(
64                 modifier = Modifier
65                     .width(340.dp)
66                     .height(32.dp)
67                     .padding(4.dp),
68                 horizontalArrangement = Arrangement.spacedBy(4.dp)
69             ) {
70                 TransformModeButton("Свободно", TransformModeUi.FREEFORM, currentMode, theme, actions::setMode)
71                 TransformModeButton("Пропорц.", TransformModeUi.UNIFORM, currentMode, theme, actions::setMode)
72                 TransformModeButton("Исказить", TransformModeUi.DISTORT, currentMode, theme, actions::setMode)
73                 TransformModeButton("Деформ.", TransformModeUi.WARP, currentMode, theme, actions::setMode)
74             }
75         }
76
77         // Row 2: Actions
78         Surface(
79             color = theme.panelBg.copy(alpha = 0.94f),
80             shape = RoundedCornerShape(24.dp),
81             border = panelBorder
82         ) {
83             Row(
84                 modifier = Modifier.padding(horizontal = 4.dp, vertical = 4.dp),
85                 horizontalArrangement = Arrangement.spacedBy(4.dp),
86                 verticalAlignment = Alignment.CenterVertically
87             ) {
88                 ActionIconButton(Icons.Default.Flip, "Отразить Г", theme, enabled = false) { actions.flipHorizontal() }
89                 ActionIconButton(Icons.Default.Flip, "Отразить В", theme, iconModifier = Modifier.rotate(90f), enabled =
false) { actions.flipVertical() }

```



```

90         IconButton(Icons.Default.RotateRight, "Поворот 45", theme, enabled = false) { /* ... */ }
91         IconButton(Icons.Default.FitScreen, "Вписать", theme, enabled = false) { actions.reset() }
92
93         VerticalDivider(modifier = Modifier.height(24.dp).padding(horizontal = 4.dp), color = theme.panelStroke)
94
95         // Placeholder for interpolation/snapping
96         Box(modifier = Modifier.size(48.dp), contentAlignment = Alignment.Center) {
97             Icon(Icons.Default.LinkOff, null, tint = theme.textSecondary.copy(alpha = 0.4f), modifier = Modifier.size(
24.dp))
98         }
99     }
100 }
101
102 // Row 3: Main Controls
103 Row(
104     modifier = Modifier.fillMaxWidth(),
105     horizontalArrangement = Arrangement.spacedBy(16.dp, Alignment.CenterHorizontally),
106     verticalAlignment = Alignment.CenterVertically
107 ) {
108     // Reset
109     TransformRoundButton(Icons.Default.Refresh, "Сбросить", theme, enabled = false) { actions.reset() }
110
111     // Cancel
112     TransformRoundButton(Icons.Default.Close, "Отмена", theme) { actions.cancel() }
113
114     // Apply
115     Surface(
116         color = theme.accent.copy(alpha = 0.4f),
117         shape = CircleShape,
118         modifier = Modifier.size(48.dp)
119     ) {
120         Box(
121             modifier = Modifier.fillMaxSize(),
122             contentAlignment = Alignment.Center
123         ) {
124             Icon(Icons.Default.Check, "Применить", tint = Color.White.copy(alpha = 0.4f))
125         }
126     }
127 }
128 }
129 }
130
131 @Composable
132 private fun RowScope.TransformModeButton(
133     label: String,
134     mode: TransformModeUi,
135     currentMode: TransformModeUi,
136     theme: AppTheme,
137     onClick: (TransformModeUi) -> Unit
138 ) {
139     val isSelected = mode == currentMode
140     val enabled = false // Transform engine not ready
141     Box(
142         modifier = Modifier
143             .weight(1f)
144             .fillMaxHeight()
145             .clip(RoundedCornerShape(20.dp))
146             .background(if (isSelected) theme.accent.copy(alpha = 0.4f) else Color.Transparent)
147             .clickable(enabled = enabled) { onClick(mode) },
148         contentAlignment = Alignment.Center
149     ) {
150         Text(
151             text = label,
152             color = if (isSelected) Color.White.copy(alpha = 0.4f) else theme.textPrimary.copy(alpha = 0.4f),
153             fontSize = 13.sp,
154             fontWeight = if (isSelected) FontWeight.Bold else FontWeight.Normal
155         )
156     }
157 }
158
159 @Composable
160 private fun ActionIconButton(
161     icon: ImageVector,
162     description: String,
163     theme: AppTheme,
164     iconModifier: Modifier = Modifier,
165     enabled: Boolean = true,
166     onClick: () -> Unit
167 ) {
168     Box(
169         modifier = Modifier
170             .size(48.dp)
171             .clickable(enabled = enabled) { onClick() },
172         contentAlignment = Alignment.Center
173     ) {
174         Icon(
175             imageVector = icon,
176             contentDescription = description,
177             tint = if (enabled) theme.textPrimary else theme.textPrimary.copy(alpha = 0.4f),
178             modifier = iconModifier.size(24.dp)
179         )
180     }

```

```

181 }
182
183 @Composable
184 private fun TransformRoundButton(
185     icon: ImageVector,
186     description: String,
187     theme: AppTheme,
188     enabled: Boolean = true,
189     onClick: () -> Unit
190 ) {
191     Surface(
192         color = if (enabled) theme.panelBg.copy(alpha = 0.9f) else theme.panelBg.copy(alpha = 0.4f),
193         shape = CircleShape,
194         border = BorderStroke(1.dp, if (enabled) theme.panelStroke else theme.panelStroke.copy(alpha = 0.4f)),
195         modifier = Modifier.size(48.dp)
196     ) {
197         Box(
198             modifier = Modifier.fillMaxSize().clickable(enabled = enabled) { onClick() },
199             contentAlignment = Alignment.Center
200         ) {
201             Icon(icon, description, tint = if (enabled) theme.textPrimary else theme.textPrimary.copy(alpha = 0.4f))
202         }
203     }
204 }

```

112. app/src/main/java/com/wetinknext/ui/home/HomeScreen.kt

```

1  package com.wetinknext.ui.home
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.Image
7  import androidx.compose.foundation.layout.*
8  import androidx.compose.foundation.lazy.grid.GridCells
9  import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
10 import androidx.compose.foundation.lazy.grid.items
11 import androidx.compose.foundation.rememberScrollState
12 import androidx.compose.foundation.shape.RoundedCornerShape
13 import androidx.compose.foundation.verticalScroll
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.outlined.Add
16 import androidx.compose.material.icons.outlined.Delete
17 import androidx.compose.material.icons.outlined.ContentCopy
18 import androidx.compose.material.icons.outlined.Edit
19 import androidx.compose.material.icons.outlined.MoreVert
20 import androidx.compose.material3.*
21 import androidx.compose.runtime.*
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.draw.clip
25 import androidx.compose.ui.graphics.Color
26 import androidx.compose.ui.graphics.ImageBitmap
27 import androidx.compose.ui.graphics.asImageBitmap
28 import android.graphics.BitmapFactory
29 import android.util.LruCache
30 import androidx.compose.ui.graphics.graphicsLayer
31 import androidx.compose.ui.text.font.FontWeight
32 import androidx.compose.ui.text.style.TextAlign
33 import androidx.compose.ui.text.style.TextOverflow
34 import androidx.compose.ui.unit.dp
35 import androidx.compose.ui.unit.sp
36 import com.wetinknext.ui.theme.AppTheme
37 import java.text.SimpleDateFormat
38 import java.util.Date
39 import java.util.Locale
40 import kotlinx.coroutines.Dispatchers
41 import kotlinx.coroutines.withContext
42
43 // Minimal models for Home UI shell
44 data class HomeProject(
45     val id: String,
46     val name: String,
47     val canvasWidth: Int,
48     val canvasHeight: Int,
49     val updatedAtMs: Long,
50     val thumbnailBytes: ByteArray? = null,
51     val thumbnailVersion: Long = 0L,
52 )
53
54 data class DeletedHomeProject(
55     val project: HomeProject,
56     val deletedAtMs: Long,
57 )
58
59 private data class ProjectThumbnailKey(
60     val projectId: String,
61     val version: Long,
62 )
63
64 /** Bytes are primed from ProjectSummary; decoded bitmaps survive grid row recycling. */
65 private object ProjectThumbnailBitmapCache {
66     private val encoded = LruCache<ProjectThumbnailKey, ByteArray>(32)
67     private val decoded = LruCache<ProjectThumbnailKey, ImageBitmap>(32)
68
69     fun prime(key: ProjectThumbnailKey, bytes: ByteArray?) {
70         if (bytes != null) encoded.put(key, bytes)
71     }
72
73     fun bitmap(key: ProjectThumbnailKey): ImageBitmap? = decoded.get(key)
74
75     fun decode(key: ProjectThumbnailKey): ImageBitmap? {
76         decoded.get(key)?.let { return it }
77         val bytes = encoded.get(key) ?: return null
78         return BitmapFactory.decodeByteArray(bytes, 0, bytes.size)
79             ?.asImageBitmap()
80             ?.also { decoded.put(key, it) }
81     }
82 }
83
84 @Composable
85 private fun rememberProjectThumbnail(
86     projectId: String,
87     version: Long,
88 ): ImageBitmap? {
89     val key = remember(projectId, version) { ProjectThumbnailKey(projectId, version) }
90     // Preserve the old preview for this project until a newly saved WebP is decoded.

```

```

91     var bitmap by remember(projectId) { mutableStateOf(ProjectThumbnailBitmapCache.bitmap(key)) }
92
93     LaunchedEffect(key) {
94         withContext(Dispatchers.Default) {
95             ProjectThumbnailBitmapCache.decode(key)
96         }.let { bitmap = it }
97     }
98     return bitmap
99 }
100
101 @Composable
102 fun HomeScreen(
103     theme: AppTheme,
104     projects: List<HomeProject>,
105     deletedProjects: List<DeletedHomeProject> = emptyList(),
106     onOpenProject: (HomeProject) -> Unit,
107     onCreateProject: (name: String, width: Int, height: Int, dpi: Int, colorProfile: String) -> Unit,
108     onRenameProject: (HomeProject, String) -> Unit,
109     onDuplicateProject: (HomeProject) -> Unit,
110     onDeleteProject: (HomeProject) -> Unit,
111     onRestoreProject: (DeletedHomeProject) -> Unit,
112     onDeleteForever: (DeletedHomeProject) -> Unit,
113 ) {
114     var showNewDialog by remember { mutableStateOf(false) }
115     var renaming by remember { mutableStateOf<HomeProject?>(null) }
116     var deleting by remember { mutableStateOf<HomeProject?>(null) }
117     var showTrash by remember { mutableStateOf(false) }
118
119     Box(
120         modifier = Modifier
121             .fillMaxSize()
122             .background(theme.panelBgSolid),
123     ) {
124         Column(modifier = Modifier.fillMaxSize()) {
125             HomeTopBar(
126                 theme = theme,
127                 total = projects.size,
128                 onCreate = { showNewDialog = true },
129                 onTrash = { showTrash = true },
130             )
131             LazyVerticalGrid(
132                 columns = GridCells.Adaptive(minSize = 180.dp),
133                 contentPadding = PaddingValues(16.dp),
134                 horizontalArrangement = Arrangement.spacedBy(12.dp),
135                 verticalArrangement = Arrangement.spacedBy(12.dp),
136                 modifier = Modifier.fillMaxSize(),
137             ) {
138                 items(projects, key = { it.id }) { project ->
139                     ProjectTile(
140                         project = project,
141                         theme = theme,
142                         onOpen = { onOpenProject(project) },
143                         onRename = { renaming = project },
144                         onDuplicate = { onDuplicateProject(project) },
145                         onDelete = { deleting = project },
146                     )
147                 }
148             }
149         }
150     }
151
152     if (showNewDialog) {
153         NewCanvasDialog(
154             theme = theme,
155             defaultWidth = 2800,
156             defaultHeight = 1840,
157             onDismiss = { showNewDialog = false },
158             onCreate = { name, width, height, dpi, colorProfile ->
159                 showNewDialog = false
160                 onCreateProject(name, width, height, dpi, colorProfile)
161             },
162         )
163     }
164
165     renaming?.let { project ->
166         RenameDialog(
167             theme = theme,
168             initial = project.name,
169             onDismiss = { renaming = null },
170             onConfirm = { newName ->
171                 onRenameProject(project, newName)
172                 renaming = null
173             },
174         )
175     }
176
177     deleting?.let { project ->
178         AlertDialog(
179             onDismissRequest = { deleting = null },
180             confirmButton = {
181                 TextButton(
182                     onClick = {

```

```

183                 onDeleteProject(project)
184                 deleting = null
185             },
186         ) {
187             Text("Удалить", color = theme.danger, fontWeight = FontWeight.SemiBold)
188         }
189     },
190     dismissButton = {
191         TextButton(onClick = { deleting = null }) {
192             Text("Отмена", color = theme.textSecondary)
193         }
194     },
195     title = { Text("Удалить холст?", color = theme.textPrimary) },
196     text = { Text(project.name, color = theme.textSecondary) },
197     containerColor = theme.panelBgSolid,
198 )
199 }
200
201 if (showTrash) {
202     TrashDialog(
203         theme = theme,
204         deletedProjects = deletedProjects,
205         onRestore = { deleted ->
206             onRestoreProject(deleted)
207         },
208         onDeleteForever = { deleted ->
209             onDeleteForever(deleted)
210         },
211         onDismiss = { showTrash = false },
212     )
213 }
214 }
215
216 @Composable
217 private fun HomeTopBar(
218     theme: AppTheme,
219     total: Int,
220     onCreate: () -> Unit,
221     onTrash: () -> Unit,
222 ) {
223     Row(
224         modifier = Modifier
225             .fillMaxWidth()
226             .padding(horizontal = 16.dp, vertical = 14.dp),
227         verticalAlignment = Alignment.CenterVertically,
228     ) {
229         Column(modifier = Modifier.weight(1f)) {
230             Text(
231                 "Мои работы",
232                 color = theme.textPrimary,
233                 fontSize = 22.sp,
234                 fontWeight = FontWeight.SemiBold,
235             )
236             Spacer(Modifier.height(3.dp))
237             Text(
238                 "Всего: $total",
239                 color = theme.textSecondary,
240                 fontSize = 13.sp,
241                 fontWeight = FontWeight.Medium,
242             )
243         }
244         TopIconButton(theme = theme, onClick = onCreate) {
245             Icon(Icons.Outlined.Add, null, tint = theme.textPrimary, modifier = Modifier.size(22.dp))
246         }
247         Spacer(Modifier.size(10.dp))
248         TopIconButton(theme = theme, onClick = onTrash) {
249             Icon(Icons.Outlined.Delete, null, tint = theme.textPrimary, modifier = Modifier.size(22.dp))
250         }
251     }
252 }
253
254 @Composable
255 private fun TopIconButton(
256     theme: AppTheme,
257     onClick: () -> Unit,
258     content: @Composable () -> Unit,
259 ) {
260     Box(
261         modifier = Modifier
262             .size(44.dp)
263             .clip(RoundedCornerShape(14.dp))
264             .background(theme.panelBg)
265             .border(1.dp, theme.panelStroke, RoundedCornerShape(14.dp))
266             .clickable(onClick = onClick),
267         contentAlignment = Alignment.Center,
268     ) {
269         content()
270     }
271 }
272
273 @Composable
274 private fun ProjectTile(

```

```

275     project: HomeProject,
276     theme: AppTheme,
277     onOpen: () -> Unit,
278     onRename: () -> Unit,
279     onDuplicate: () -> Unit,
280     onDelete: () -> Unit,
281 ) {
282     var menuExpanded by remember { mutableStateOf(false) }
283
284     Column(
285         modifier = Modifier
286             .fillMaxWidth()
287             .clip(RoundedCornerShape(8.dp))
288             .background(theme.panelBg)
289             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(8.dp))
290             .clickable(onClick = onOpen)
291             .padding(10.dp),
292     ) {
293         ProjectPreview(project = project, theme = theme)
294         Spacer(Modifier.height(10.dp))
295         Row(verticalAlignment = Alignment.CenterVertically) {
296             Column(modifier = Modifier.weight(1f)) {
297                 Text(
298                     project.name,
299                     color = theme.textPrimary,
300                     fontSize = 14.sp,
301                     fontWeight = FontWeight.SemiBold,
302                     maxLines = 1,
303                     overflow = TextOverflow.Ellipsis,
304                 )
305                 Text(
306                     "${project.canvasWidth} x ${project.canvasHeight} px · ${formatDate(project.updatedAtMs)}",
307                     color = theme.textSecondary,
308                     fontSize = 11.sp,
309                     maxLines = 1,
310                     overflow = TextOverflow.Ellipsis,
311                 )
312             }
313             Box {
314                 Box(
315                     modifier = Modifier
316                         .size(34.dp)
317                         .clip(RoundedCornerShape(12.dp))
318                         .clickable { menuExpanded = true },
319                     contentAlignment = Alignment.Center,
320                 ) {
321                     Icon(Icons.Outlined.MoreVert, null, tint = theme.textSecondary, modifier = Modifier.size(20.dp))
322                 }
323                 DropdownMenu(
324                     expanded = menuExpanded,
325                     onDismissRequest = { menuExpanded = false },
326                     modifier = Modifier.background(theme.panelBg)
327                 ) {
328                     DropdownMenuItem(
329                         text = { Text("Переименовать", color = theme.textPrimary) },
330                         leadingIcon = { Icon(Icons.Outlined.Edit, null, tint = theme.textSecondary) },
331                         onClick = {
332                             menuExpanded = false
333                             onRename()
334                         },
335                     )
336                     DropdownMenuItem(
337                         text = { Text("Дублировать", color = theme.textPrimary) },
338                         leadingIcon = { Icon(Icons.Outlined.ContentCopy, null, tint = theme.textSecondary) },
339                         onClick = {
340                             menuExpanded = false
341                             onDuplicate()
342                         },
343                     )
344                     DropdownMenuItem(
345                         text = { Text("Удалить", color = theme.danger) },
346                         leadingIcon = { Icon(Icons.Outlined.Delete, null, tint = theme.danger) },
347                         onClick = {
348                             menuExpanded = false
349                             onDelete()
350                         },
351                     )
352                 }
353             }
354         }
355     }
356 }
357
358 @Composable
359 private fun ProjectPreview(
360     project: HomeProject,
361     theme: AppTheme,
362 ) {
363     // Placeholder for actual preview loading
364     Box(
365         modifier = Modifier
366             .fillMaxWidth()

```

```

367         .aspectRatio(
368             project.canvasWidth.toFloat() /
369             project.canvasHeight.coerceAtLeast(1).toFloat(),
370         )
371         .clip(RoundedCornerShape(6.dp))
372         .background(theme.panelInset)
373         .border(1.dp, theme.panelStroke, RoundedCornerShape(6.dp)),
374         contentAlignment = Alignment.Center,
375     ) {
376         ProjectThumbnailBitmapCache.prime(
377             key = ProjectThumbnailKey(project.id, project.thumbnailVersion),
378             bytes = project.thumbnailBytes,
379         )
380         val preview = rememberProjectThumbnail(
381             projectId = project.id,
382             version = project.thumbnailVersion,
383         )
384         if (preview != null) {
385             Image(bitmap = preview, contentDescription = "Превью ${project.name}", modifier = Modifier.fillMaxSize())
386             return@Box
387         }
388         EmptyProjectPlaceholder(theme)
389     }
390 }
391
392 @Composable
393 private fun EmptyProjectPlaceholder(theme: AppTheme) {
394     Box(
395         modifier = Modifier
396             .fillMaxSize()
397             .clip(RoundedCornerShape(4.dp))
398             .background(Color(0xFFFAFAFA))
399             .border(1.dp, theme.panelStroke, RoundedCornerShape(4.dp)),
400     )
401 }
402
403 @Composable
404 private fun RenameDialog(
405     theme: AppTheme,
406     initial: String,
407     onDismiss: () -> Unit,
408     onConfirm: (String) -> Unit,
409 ) {
410     var name by remember(initial) { mutableStateOf(initial) }
411     AlertDialog(
412         onDismissRequest = onDismiss,
413         confirmButton = {
414             TextButton(
415                 onClick = {
416                     val cleaned = name.trim()
417                     if (cleaned.isNotEmpty()) onConfirm(cleaned)
418                 },
419             ) {
420                 Text("Готово", color = theme.accent, fontWeight = FontWeight.SemiBold)
421             }
422         },
423         dismissButton = {
424             TextButton(onClick = onDismiss) {
425                 Text("Отмена", color = theme.textSecondary)
426             }
427         },
428         title = { Text("Название", color = theme.textPrimary) },
429         text = {
430             OutlinedTextField(
431                 value = name,
432                 onValueChange = { name = it.take(48) },
433                 singleLine = true,
434                 colors = OutlinedTextFieldDefaults.colors(
435                     focusedBorderColor = theme.accent,
436                     unfocusedBorderColor = theme.panelStroke,
437                     focusedTextColor = theme.textPrimary,
438                     unfocusedTextColor = theme.textPrimary
439                 )
440             )
441         },
442         containerColor = theme.panelBgSolid,
443     )
444 }
445
446 @Composable
447 private fun TrashDialog(
448     theme: AppTheme,
449     deletedProjects: List<DeletedHomeProject>,
450     onRestore: (DeletedHomeProject) -> Unit,
451     onDeleteForever: (DeletedHomeProject) -> Unit,
452     onDismiss: () -> Unit,
453 ) {
454     AlertDialog(
455         onDismissRequest = onDismiss,
456         confirmButton = {
457             TextButton(onClick = onDismiss) {
458                 Text("Готово", color = theme.accent, fontWeight = FontWeight.SemiBold)

```

```

459     }
460 },
461 title = { Text("Корзина", color = theme.textPrimary) },
462 text = {
463     if (deletedProjects.isEmpty()) {
464         Text("Удалённые работы хранятся здесь 3 дня.", color = theme.textSecondary, fontSize = 13.sp)
465     } else {
466         Column(
467             modifier = Modifier
468                 .fillMaxWidth()
469                 .height(360.dp)
470                 .verticalScroll(rememberScrollState()),
471             verticalArrangement = Arrangement.spacedBy(10.dp),
472         ) {
473             Text(
474                 "Работы хранятся 3 дня.",
475                 color = theme.textSecondary,
476                 fontSize = 12.sp,
477             )
478             deletedProjects.sortedByDescending { it.deletedAtMs }.forEach { deleted ->
479                 DeletedProjectRow(
480                     deleted = deleted,
481                     theme = theme,
482                     onRestore = { onRestore(deleted) },
483                     onDeleteForever = { onDeleteForever(deleted) },
484                 )
485             }
486         }
487     }
488 },
489 containerColor = theme.panelBgSolid,
490 )
491 }
492
493 @Composable
494 private fun DeletedProjectRow(
495     deleted: DeletedHomeProject,
496     theme: AppTheme,
497     onRestore: () -> Unit,
498     onDeleteForever: () -> Unit,
499 ) {
500     val project = deleted.project
501     Row(
502         modifier = Modifier
503             .fillMaxWidth()
504             .clip(RoundedCornerShape(12.dp))
505             .background(theme.panelBg)
506             .border(1.dp, theme.panelStroke, RoundedCornerShape(12.dp))
507             .padding(10.dp),
508         verticalAlignment = Alignment.CenterVertically,
509     ) {
510         Box(
511             modifier = Modifier
512                 .size(54.dp)
513                 .clip(RoundedCornerShape(6.dp))
514                 .background(theme.panelInset)
515                 .border(1.dp, theme.panelStroke, RoundedCornerShape(6.dp)),
516             contentAlignment = Alignment.Center,
517         ) {
518             Box(
519                 modifier = Modifier
520                     .fillMaxWidth(0.5f)
521                     .aspectRatio(project.canvasWidth.toFloat() / project.canvasHeight.coerceAtLeast(1).toFloat())
522                     .background(Color(0xFFFAFAFA)),
523             )
524         }
525         Column(
526             modifier = Modifier
527                 .weight(1f)
528                 .padding(start = 10.dp),
529         ) {
530             Text(
531                 project.name,
532                 color = theme.textPrimary,
533                 fontSize = 13.sp,
534                 fontWeight = FontWeight.SemiBold,
535                 maxLines = 1,
536                 overflow = TextOverflow.Ellipsis,
537             )
538             Text(
539                 "Осталось: ${trashDaysLeft(deleted.deletedAtMs)} дн.",
540                 color = theme.textSecondary,
541                 fontSize = 11.sp,
542             )
543             Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
544                 TextButton(onClick = onRestore) {
545                     Text("Восстановить", color = theme.accent, fontSize = 12.sp)
546                 }
547                 TextButton(onClick = onDeleteForever) {
548                     Text("Удалить", color = theme.textSecondary, fontSize = 12.sp)
549                 }
550             }
551         }
552     }
553 }

```



```

551     }
552 }
553 }
554
555 private fun formatDate(timeMs: Long): String =
556     SimpleDateFormat("d MMM, HH:mm", Locale.forLanguageTag("ru")).format(Date(timeMs))
557
558 private fun trashDaysLeft(deletedAtMs: Long): Int {
559     val retentionMs = 3L * 24L * 60L * 60L * 1000L
560     val remainingMs = (retentionMs - (System.currentTimeMillis() - deletedAtMs)).coerceAtLeast(0L)
561     return ((remainingMs + 24L * 60L * 60L * 1000L - 1L) / (24L * 60L * 60L * 1000L)).toInt().coerceAtLeast(1)
562 }

```

113. app/src/main/java/com/wetinknext/ui/home/NewCanvasDialog.kt

```

1 package com.wetinknext.ui.home
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.horizontalScroll
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.foundation.rememberScrollState
9 import androidx.compose.foundation.shape.RoundedCornerShape
10 import androidx.compose.foundation.text.KeyboardOptions
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.*
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.graphics.Color
17 import androidx.compose.ui.text.font.FontWeight
18 import androidx.compose.ui.text.input.KeyboardType
19 import androidx.compose.ui.text.style.TextAlign
20 import androidx.compose.ui.unit.dp
21 import androidx.compose.ui.unit.sp
22 import com.wetinknext.ui.theme.AppTheme
23 import com.wetinknext.domain.document.ProjectDocument
24 import kotlin.math.max
25
26 private data class CanvasPreset(
27     val label: String,
28     val width: Int,
29     val height: Int,
30     val description: String,
31 )
32
33 private val QuickPresets = listOf(
34     CanvasPreset("Экран", 2800, 1840, "2800 x 1840 px"),
35     CanvasPreset("Квадрат", 2048, 2048, "2048 x 2048 px"),
36     CanvasPreset("4K", 4096, 2160, "4096 x 2160 px"),
37     CanvasPreset("A4", 2480, 3508, "210 x 297 мм"),
38     CanvasPreset("1080P", 1920, 1080, "1920 x 1080 px"),
39     CanvasPreset("9:16", 1080, 1920, "1080 x 1920 px"),
40 )
41
42 private val DpiOptions = listOf(72, 144, 300, 350, 600, 1200)
43 private val ColorProfiles = listOf(
44     ProjectDocument.SRGB_PROFILE,
45     ProjectDocument.DISPLAY_P3_PROFILE,
46 )
47
48 @Composable
49 fun NewCanvasDialog(
50     theme: AppTheme,
51     defaultWidth: Int,
52     defaultHeight: Int,
53     onDismiss: () -> Unit,
54     onConfirm: (name: String, width: Int, height: Int, dpi: Int, colorProfile: String) -> Unit,
55 ) {
56     val initialIndex = QuickPresets.indexOfFirst {
57         it.width == defaultWidth && it.height == defaultHeight
58     }
59     var selected by remember { mutableIntStateOf(initialIndex) }
60     var customW by remember { mutableStateOf(defaultWidth.toString()) }
61     var customH by remember { mutableStateOf(defaultHeight.toString()) }
62     var name by remember { mutableStateOf("Новый холст") }
63     var dpi by remember { mutableIntStateOf(300) }
64     var colorProfile by remember { mutableStateOf(ProjectDocument.SRGB_PROFILE) }
65     var dpiExpanded by remember { mutableStateOf(false) }
66     var profileExpanded by remember { mutableStateOf(false) }
67     var validationError by remember { mutableStateOf<String?>(null) }
68     val isCustom = selected !in QuickPresets.indices
69
70     AlertDialog(
71         onDismissRequest = onDismiss,
72         title = {
73             Text("Новый холст", color = theme.textPrimary, fontWeight = FontWeight.SemiBold)
74         },
75         text = {
76             Column {
77                 OutlinedTextField(
78                     value = name,
79                     onValueChange = { name = it.take(48) },
80                     label = { Text("Имя") },
81                     singleLine = true,
82                     modifier = Modifier.fillMaxWidth(),
83                     colors = OutlinedTextFieldDefaults.colors(
84                         focusedBorderColor = theme.accent,
85                         unfocusedBorderColor = theme.panelStroke,
86                         focusedLabelColor = theme.accent,
87                         unfocusedLabelColor = theme.textSecondary
88                     )
89                 )
90                 Spacer(Modifier.height(12.dp))

```

```

91 Text("Размеры", color = theme.textSecondary, fontSize = 13.sp)
92 Spacer(Modifier.height(8.dp))
93 Row(
94     horizontalArrangement = Arrangement.spacedBy(8.dp),
95     modifier = Modifier.horizontalScroll(rememberScrollState()),
96 ) {
97     QuickPresets.forEachIndexed { index, preset ->
98         QuickPresetCard(
99             preset = preset,
100             selected = selected == index,
101             theme = theme,
102             onClick = {
103                 selected = index
104                 customW = preset.width.toString()
105                 customH = preset.height.toString()
106             },
107         )
108     }
109 }
110
111 Spacer(Modifier.height(12.dp))
112 TextButton(onClick = { selected = -1 }) {
113     Text(
114         "Свой размер",
115         color = if (isCustom) theme.accent else theme.textSecondary,
116         fontWeight = if (isCustom) FontWeight.SemiBold else FontWeight.Normal,
117         fontSize = 13.sp,
118     )
119 }
120
121 if (isCustom) {
122     Spacer(Modifier.height(4.dp))
123     Row(verticalAlignment = Alignment.CenterVertically) {
124         OutlinedTextField(
125             value = customW,
126             onValueChange = { v -> customW = v.filter { it.isDigit() }.take(5) },
127             label = { Text("Ширина") },
128             singleLine = true,
129             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
130             modifier = Modifier.weight(1f),
131             colors = OutlinedTextFieldDefaults.colors(
132                 focusedBorderColor = theme.accent,
133                 unfocusedBorderColor = theme.panelStroke,
134                 focusedLabelColor = theme.accent,
135                 unfocusedLabelColor = theme.textSecondary
136             )
137         )
138         Spacer(Modifier.width(8.dp))
139         Text("x", color = theme.textSecondary)
140         Spacer(Modifier.width(8.dp))
141         OutlinedTextField(
142             value = customH,
143             onValueChange = { v -> customH = v.filter { it.isDigit() }.take(5) },
144             label = { Text("Высота") },
145             singleLine = true,
146             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
147             modifier = Modifier.weight(1f),
148             colors = OutlinedTextFieldDefaults.colors(
149                 focusedBorderColor = theme.accent,
150                 unfocusedBorderColor = theme.panelStroke,
151                 focusedLabelColor = theme.accent,
152                 unfocusedLabelColor = theme.textSecondary
153             )
154         )
155     }
156 }
157
158 Spacer(Modifier.height(14.dp))
159 OptionRow(label = "DPI", value = "$dpi dpi", theme = theme, expanded = dpiExpanded, onExpand = { dpiExpanded
= true }) {
160     DpiOptions.forEach { option ->
161         DropdownMenuItem(
162             text = { Text("$option dpi") },
163             onClick = {
164                 dpi = option
165                 dpiExpanded = false
166             },
167         )
168     }
169 }
170 OptionRow(label = "Цветовой профиль", value = colorProfile, theme = theme, expanded = profileExpanded,
onExpand = { profileExpanded = true }) {
171     ColorProfiles.forEach { option ->
172         DropdownMenuItem(
173             text = { Text(option) },
174             onClick = {
175                 colorProfile = option
176                 profileExpanded = false
177             },
178         )
179     }
180 }

```

```

181         validationError?.let { message ->
182             Spacer(Modifier.height(8.dp))
183             Text(message, color = MaterialTheme.colorScheme.error, fontSize = 12.sp)
184         }
185     },
186 },
187     confirmButton = {
188         TextButton(
189             onClick = {
190                 val (w, h) = if (isCustom) {
191                     val cw = customW.toIntOrNull()?.coerceIn(MinDimension, MaxDimension) ?: return@TextButton
192                     val ch = customH.toIntOrNull()?.coerceIn(MinDimension, MaxDimension) ?: return@TextButton
193                     cw to ch
194                 } else {
195                     val preset = QuickPresets[selected]
196                     preset.width to preset.height
197                 }
198                 validationError = runCatching {
199                     ProjectDocument.validateNewCanvas(w, h, dpi, colorProfile)
200                 }.exceptionOrNull()?.message
201                 if (validationError == null) {
202                     onConfirm(name.trim().ifBlank { "Новый холст" }, w, h, dpi, colorProfile)
203                 }
204             },
205         ) {
206             Text("Создать", color = theme.accent, fontWeight = FontWeight.SemiBold)
207         }
208     },
209     dismissButton = {
210         TextButton(onClick = onDismiss) {
211             Text("Отмена", color = theme.textSecondary)
212         }
213     },
214     containerColor = theme.panelBgSolid,
215 )
216 }
217
218 @Composable
219 private fun OptionRow(
220     label: String,
221     value: String,
222     theme: AppTheme,
223     expanded: Boolean,
224     onExpand: () -> Unit,
225     menu: @Composable () -> Unit,
226 ) {
227     Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth()) {
228         Text(label, color = theme.textPrimary, fontSize = 13.sp)
229         Spacer(Modifier.weight(1f))
230         Box {
231             TextButton(onClick = onExpand) {
232                 Text(value, color = theme.accent, fontWeight = FontWeight.SemiBold)
233             }
234             DropdownMenu(
235                 expanded = expanded,
236                 onDismissRequest = { /* onExpand is handled by parent */ },
237                 modifier = Modifier.background(theme.panelBg)
238             ) {
239                 menu()
240             }
241         }
242     }
243 }
244
245 @Composable
246 private fun QuickPresetCard(
247     preset: CanvasPreset,
248     selected: Boolean,
249     theme: AppTheme,
250     onClick: () -> Unit,
251 ) {
252     Column(
253         horizontalAlignment = Alignment.CenterHorizontally,
254         modifier = Modifier
255             .width(100.dp)
256             .clip(RoundedCornerShape(12.dp))
257             .background(if (selected) theme.accent.copy(alpha = 0.15f) else theme.panelBg)
258             .border(1.5.dp, if (selected) theme.accent else theme.panelStroke, RoundedCornerShape(12.dp))
259             .clickable(onClick = onClick)
260             .padding(vertical = 12.dp, horizontal = 8.dp),
261     ) {
262         PresetSwatch(preset.width, preset.height, selected, theme)
263         Spacer(Modifier.height(8.dp))
264         Text(preset.label, color = theme.textPrimary, fontWeight = FontWeight.Medium, fontSize = 13.sp, textAlign = TextAlign.Center)
265         Text(preset.description, color = theme.textSecondary, fontSize = 10.sp, textAlign = TextAlign.Center, maxLines = 1)
266     }
267 }
268
269 @Composable
270 private fun PresetSwatch(width: Int, height: Int, selected: Boolean, theme: AppTheme) {
271     val scale = 40f / max(width.coerceAtLeast(1), height.coerceAtLeast(1))

```

```

272     val previewW = (width * scale).coerceAtLeast(8f)
273     val previewH = (height * scale).coerceAtLeast(8f)
274     Box(modifier = Modifier.size(48.dp), contentAlignment = Alignment.Center) {
275         Box(
276             modifier = Modifier
277                 .width(previewW.dp)
278                 .height(previewH.dp)
279                 .clip(RoundedCornerShape(4.dp))
280                 .background(if (selected) theme.accent else Color(0xFF5A5A60))
281                 .border(1.dp, if (selected) Color.White else theme.panelStroke, RoundedCornerShape(4.dp)),
282             )
283     }
284 }
285
286 private const val MinDimension = 256
287 private const val MaxDimension = 8192

```

114. app/src/main/java/com/wetinknext/ui/state/EditorActions.kt

```
1 package com.wetinknext.ui.state
2
3 /**
4  * Interface defining all possible user actions from the UI that affect the paint engine state.
5  */
6 interface EditorActions {
7     fun undo()
8     fun redo()
9     fun setBrushSize(px: Float)
10    fun setBrushOpacity(opacity: Float)
11
12    fun addLayer()
13    fun removeLayer(id: Long)
14    fun setActiveLayer(id: Long)
15    fun setLayerVisible(id: Long, visible: Boolean)
16    fun setLayerOpacity(id: Long, opacity: Float)
17 }
```

115. app/src/main/java/com/wetinknext/ui/state/LayerState.kt

```

1 package com.wetinknext.ui.state
2
3 import androidx.compose.runtime.*
4 import com.wetinknext.engine.core.EditorUiState
5
6 class LayerItem(
7     val id: Long,
8     name: String,
9     visible: Boolean,
10    opacity: Float,
11    locked: Boolean,
12    thumbnailPath: String? = null,
13    thumbnailVersion: Long = 0L,
14 ) {
15     var name by mutableStateOf(name)
16     var visible by mutableStateOf(visible)
17     var opacity by mutableFloatStateOf(opacity)
18     var locked by mutableStateOf(locked)
19     var thumbnailPath by mutableStateOf(thumbnailPath)
20     var thumbnailVersion by mutableLongStateOf(thumbnailVersion)
21 }
22
23 class LayerState {
24     val layers = mutableStateListOf<LayerItem>()
25     var activeLayerId by mutableStateOf(-1L)
26
27     fun syncFromEditor(state: EditorUiState) {
28         val ids = state.layers.map { it.id }.toSet()
29         layers.removeAll { it.id !in ids }
30
31         state.layers.forEachIndexed { index, model ->
32             val item = layers.firstOrNull { it.id == model.id }
33
34             if (item == null) {
35                 layers.add(
36                     index.coerceAtMost(layers.size),
37                     LayerItem(
38                         id = model.id,
39                         name = model.name,
40                         visible = model.isVisible,
41                         opacity = model.opacity,
42                         locked = model.isLocked,
43                         thumbnailPath = model.thumbnailPath,
44                         thumbnailVersion = model.thumbnailVersion,
45                     ),
46                 )
47             } else {
48                 item.name = model.name
49                 item.visible = model.isVisible
50                 item.opacity = model.opacity
51                 item.locked = model.isLocked
52                 item.thumbnailPath = model.thumbnailPath
53                 item.thumbnailVersion = model.thumbnailVersion
54
55                 // If the item needs to be moved to match the index in state
56                 val currentIndex = layers.indexOf(item)
57                 if (currentIndex != index && index < layers.size) {
58                     layers.removeAt(currentIndex)
59                     layers.add(index, item)
60                 }
61             }
62         }
63
64         activeLayerId = state.activeLayerId
65     }
66 }

```

116. app/src/main/java/com/wetinknext/ui/theme/AppThemes.kt

```

1 package com.wetinknext.ui.theme
2
3 import androidx.compose.runtime.Immutable
4 import androidx.compose.ui.graphics.Color
5
6 @Immutable
7 data class AppTheme(
8     val id: String,
9     val displayName: String,
10    val isLight: Boolean,
11
12    // Core Layout
13    val appBg: Color,
14    val gridLine: Color,
15    val panelBg: Color,
16    val panelBgSolid: Color,
17    val panelStroke: Color,
18    val panelInset: Color,
19    val panelInsetSoft: Color,
20
21    // Canvas
22    val canvasBackdrop: Color,
23    val canvasGrid: Color,
24
25    // Typography & Icons
26    val textPrimary: Color,
27    val textSecondary: Color,
28    val iconInactive: Color,
29
30    // Accents
31    val accent: Color,
32    val accentSoft: Color,
33    val accentMuted: Color,
34    val danger: Color
35 ) {
36     val panelBgVariant: Color get() = panelInset
37 }
38
39 object AppThemes {
40     val Dark = AppTheme(
41         id = "dark", displayName = "Dark", isLight = false,
42         appBg = Color(0xFF1A0C12), gridLine = Color(0xFF2A1418),
43         panelBg = Color(0xFF2A141C), panelBgSolid = Color(0xFF2D1620),
44         panelStroke = Color(0xFF3D1F2A), panelInset = Color(0xFF1F1015), panelInsetSoft = Color(0xFF2A1820),
45         canvasBackdrop = Color(0xFF1A0C12), canvasGrid = Color(0xFF3A1C24),
46         textPrimary = Color(0xFFFF5E8E), textSecondary = Color(0xFF9A7F8A), iconInactive = Color(0xFFC8B0B8),
47         accent = Color(0xFFFF4D7A), accentSoft = Color(0xFFFF7AA0), accentMuted = Color(0x55FF4D7A), danger = Color(
0xFFFF3B5C)
48     )
49
50     val Light = AppTheme(
51         id = "light", displayName = "Light", isLight = true,
52         appBg = Color(0xFFFFE4E9), gridLine = Color(0xFF5D0D7),
53         panelBg = Color(0xFFFFF1F4), panelBgSolid = Color(0xFFFFFFFF),
54         panelStroke = Color(0xFFFF0C8D0), panelInset = Color(0xFFFFE0E6), panelInsetSoft = Color(0xFFFFF0DA),
55         canvasBackdrop = Color(0xFFFFE4E9), canvasGrid = Color(0xFF5D0D7),
56         textPrimary = Color(0xFF3D1B26), textSecondary = Color(0xFF8A5A6A), iconInactive = Color(0xFFB07A88),
57         accent = Color(0xFFFF4D7A), accentSoft = Color(0xFFFFA0B7), accentMuted = Color(0x33FF4D7A), danger = Color(
0xFFFF3B5C)
58     )
59
60     val Red = AppTheme(
61         id = "red", displayName = "Red", isLight = false,
62         appBg = Color(0xFF1A0606), gridLine = Color(0xFF2A0A0A),
63         panelBg = Color(0xFF2A0C0C), panelBgSolid = Color(0xFF2D0E0E),
64         panelStroke = Color(0xFF3D1414), panelInset = Color(0xFF1F0808), panelInsetSoft = Color(0xFF2A1010),
65         canvasBackdrop = Color(0xFF1A0606), canvasGrid = Color(0xFF3A1414),
66         textPrimary = Color(0xFFFF5E8E), textSecondary = Color(0xFF9A7F7F), iconInactive = Color(0xFFC8B0B0),
67         accent = Color(0xFFFF3D3D), accentSoft = Color(0xFFFF7A7A), accentMuted = Color(0x55FF3D3D), danger = Color(
0xFFFF8888)
68     )
69
70     val Vampire = AppTheme(
71         id = "vampire", displayName = "Vampire", isLight = false,
72         appBg = Color(0xFF1B1C1B), gridLine = Color(0xFF252525),
73         panelBg = Color(0xFF262626), panelBgSolid = Color(0xFF2D2D2D),
74         panelStroke = Color(0xFF4B2529), panelInset = Color(0xFF171717), panelInsetSoft = Color(0xFF2A1B1E),
75         canvasBackdrop = Color(0xFF1B1C1B), canvasGrid = Color(0xFF2B2B2B),
76         textPrimary = Color(0xFFFF2ECC), textSecondary = Color(0xFFCDA2A6), iconInactive = Color(0xFFE15761),
77         accent = Color(0xFFFF3547), accentSoft = Color(0xFFFF3A38), accentMuted = Color(0xAFF3547), danger = Color(
0xFFFF4A5F)
78     )
79
80     val Olive = AppTheme(
81         id = "olive", displayName = "Olive", isLight = false,
82         appBg = Color(0xFF171B0D), gridLine = Color(0xFF252A16),
83         panelBg = Color(0xFF202711), panelBgSolid = Color(0xFF263014),
84         panelStroke = Color(0xFF566032), panelInset = Color(0xFF121806), panelInsetSoft = Color(0xFF2B3418),
85         canvasBackdrop = Color(0xFF171B0D), canvasGrid = Color(0xFF2F31C),
86         textPrimary = Color(0xFF1E7D0), textSecondary = Color(0xFFC7A77E), iconInactive = Color(0xFFE3A15E),

```



```

87         accent = Color(0xFFFFF982F), accentSoft = Color(0xFFFFC46A), accentMuted = Color(0x88FF982F), danger = Color(
0xFFFF624D)
88     )
89
90     val Blue = AppTheme(
91         id = "blue", displayName = "Blue", isLight = false,
92         appBg = Color(0xFF06101A), gridLine = Color(0xFF0A1828),
93         panelBg = Color(0xFF0C1A2A), panelBgSolid = Color(0xFF0E1D2E),
94         panelStroke = Color(0xFF14283D), panelInset = Color(0xFF08141F), panelInsetSoft = Color(0xFF101F2A),
95         canvasBackdrop = Color(0xFF06101A), canvasGrid = Color(0xFF14283A),
96         textPrimary = Color(0xFFE8EEF5), textSecondary = Color(0xFF7F8C9A), iconInactive = Color(0xFFB0BBC8),
97         accent = Color(0xFF4D7AFF), accentSoft = Color(0xFF7AA0FF), accentMuted = Color(0x554D7AFF), danger = Color(
0xFFFF3B5C)
98     )
99
100    val Studio = AppTheme(
101        id = "studio", displayName = "Studio", isLight = true,
102        appBg = Color(0xFFE1E1E1), gridLine = Color(0xFFD1D6DA),
103        panelBg = Color(0xFFFFF0F2F4), panelBgSolid = Color(0xFFFFBFCFD),
104        panelStroke = Color(0xFFD4DAE0), panelInset = Color(0xFFE7EBEF), panelInsetSoft = Color(0xFFDCE8F3),
105        canvasBackdrop = Color(0xFFDADADA), canvasGrid = Color(0xFFCFCFCF),
106        textPrimary = Color(0xFF20252A), textSecondary = Color(0xFF65737F), iconInactive = Color(0xFF5C8DB3),
107        accent = Color(0xFF4A9AD8), accentSoft = Color(0xFF7FBCB8), accentMuted = Color(0x334A9AD8), danger = Color(
0xFFE65C73)
108    )
109
110    val Neon = AppTheme(
111        id = "neon", displayName = "Neon", isLight = false,
112        appBg = Color(0xFF2B123E), gridLine = Color(0xFF32184A),
113        panelBg = Color(0xFF4A2366), panelBgSolid = Color(0xFF522A70),
114        panelStroke = Color(0xFF6C3A8D), panelInset = Color(0xFF321344), panelInsetSoft = Color(0xFF3C1853),
115        canvasBackdrop = Color(0xFF2B123E), canvasGrid = Color(0xFF3C1D55),
116        textPrimary = Color(0xFF3EAFE), textSecondary = Color(0xFFC6A8D8), iconInactive = Color(0xFF2FE0EA),
117        accent = Color(0xFF25DDE8), accentSoft = Color(0xFF73F3FA), accentMuted = Color(0x5525DDE8), danger = Color(
0xFFFF6E9C)
118    )
119
120    val Cyan = AppTheme(
121        id = "cyan", displayName = "Cyan", isLight = false,
122        appBg = Color(0xFF06181A), gridLine = Color(0xFF0A2628),
123        panelBg = Color(0xFF0C282A), panelBgSolid = Color(0xFF0E2C2E),
124        panelStroke = Color(0xFF143C3D), panelInset = Color(0xFF081F20), panelInsetSoft = Color(0xFF102A2A),
125        canvasBackdrop = Color(0xFF06181A), canvasGrid = Color(0xFF143A3A),
126        textPrimary = Color(0xFFE8F5F5), textSecondary = Color(0xFF7F9A9A), iconInactive = Color(0xFFB0C8C8),
127        accent = Color(0xFF26D9D9), accentSoft = Color(0xFF7AE0E0), accentMuted = Color(0x5526D9D9), danger = Color(
0xFFFF3B5C)
128    )
129
130    val Green = AppTheme(
131        id = "green", displayName = "Green", isLight = false,
132        appBg = Color(0xFF071A0C), gridLine = Color(0xFF0A2814),
133        panelBg = Color(0xFF0C2A14), panelBgSolid = Color(0xFF0E2D16),
134        panelStroke = Color(0xFF143D1F), panelInset = Color(0xFF081F0C), panelInsetSoft = Color(0xFF102A14),
135        canvasBackdrop = Color(0xFF071A0C), canvasGrid = Color(0xFF143A1F),
136        textPrimary = Color(0xFFE8F5EC), textSecondary = Color(0xFF7F9A88), iconInactive = Color(0xFFB0C8B8),
137        accent = Color(0xFF3DDB6E), accentSoft = Color(0xFF7AE0A0), accentMuted = Color(0x553DDB6E), danger = Color(
0xFFFF3B5C)
138    )
139
140    val Yellow = AppTheme(
141        id = "yellow", displayName = "Yellow", isLight = true,
142        appBg = Color(0xFFFFF8E0), gridLine = Color(0xFFFF5E5A0),
143        panelBg = Color(0xFFFFF4D0), panelBgSolid = Color(0xFFFFF0CF0),
144        panelStroke = Color(0xFFE6CC60), panelInset = Color(0xFFFFF0B8), panelInsetSoft = Color(0xFFFFF890),
145        canvasBackdrop = Color(0xFFFFF8E0), canvasGrid = Color(0xFFFF0DA80),
146        textPrimary = Color(0xFF3D2E08), textSecondary = Color(0xFF8A7A40), iconInactive = Color(0xFFB09860),
147        accent = Color(0xFFFF5B800), accentSoft = Color(0xFFFFD050), accentMuted = Color(0x55F5B800), danger = Color(
0xFFFF3B5C)
148    )
149
150    val Orange = AppTheme(
151        id = "orange", displayName = "Orange", isLight = true,
152        appBg = Color(0xFFFFF8D5), gridLine = Color(0xFFFF5C898),
153        panelBg = Color(0xFFFFF0C5), panelBgSolid = Color(0xFFFFF1E5),
154        panelStroke = Color(0xFFE69C58), panelInset = Color(0xFFFFD8B5), panelInsetSoft = Color(0xFFFFF890),
155        canvasBackdrop = Color(0xFFFFF8D5), canvasGrid = Color(0xFFFF0BC80),
156        textPrimary = Color(0xFF3D1E08), textSecondary = Color(0xFF8A5A30), iconInactive = Color(0xFFB07A50),
157        accent = Color(0xFFFFF8800), accentSoft = Color(0xFFFFD050), accentMuted = Color(0x55FF8800), danger = Color(
0xFFFF3B5C)
158    )
159
160    val Gray = AppTheme(
161        id = "gray", displayName = "Gray", isLight = false,
162        appBg = Color(0xFF1A1A1A), gridLine = Color(0xFF2A2A2A),
163        panelBg = Color(0xFF2A2A2A), panelBgSolid = Color(0xFF303030),
164        panelStroke = Color(0xFF404040), panelInset = Color(0xFF1F1F1F), panelInsetSoft = Color(0xFF2B2B2B),
165        canvasBackdrop = Color(0xFF1A1A1A), canvasGrid = Color(0xFF3A3A3A),
166        textPrimary = Color(0xFFFFF0F0), textSecondary = Color(0xFF909090), iconInactive = Color(0xFFB8B8B8),
167        accent = Color(0xFFB0B0B0), accentSoft = Color(0xFFD0D0D0), accentMuted = Color(0x55B0B0B0), danger = Color(
0xFFFF3B5C)
168    )
169

```

```

170     val White = AppTheme(
171         id = "white", displayName = "White", isLight = true,
172         appBg = Color(0xFFFFFFFF), gridLine = Color(0xFFE8E8E8),
173         panelBg = Color(0xFFF5F5F5), panelBgSolid = Color(0xFFFFFFFF),
174         panelStroke = Color(0xFFD8D8D8), panelInset = Color(0xFFEEEEEE), panelInsetSoft = Color(0xFFE0E0E0),
175         canvasBackdrop = Color(0xFFFFFFFF), canvasGrid = Color(0xFFE0E0E0),
176         textPrimary = Color(0xFF202020), textSecondary = Color(0xFF707070), iconInactive = Color(0xFF989898),
177         accent = Color(0xFF606060), accentSoft = Color(0xFF888888), accentMuted = Color(0x55606060), danger = Color(
0xFFFF3B5C)
178     )
179
180     val all = listOf(
181         Dark, Light, Red, Vampire, Olive, Blue, Studio, Neon, Cyan, Green, Yellow, Orange, Gray, White
182     )
183
184     val default = Gray
185 }
186
187 enum class UIFont { Classic, Italic }

```

117. app/src/main/java/com/wetinknext/ui/theme/CustomThemeEditor.kt

```

1  package com.wetinknext.ui.theme
2
3  import android.graphics.Color as AndroidColor
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.border
6  import androidx.compose.foundation.clickable
7  import androidx.compose.foundation.layout.*
8  import androidx.compose.foundation.shape.CircleShape
9  import androidx.compose.foundation.shape.RoundedCornerShape
10 import androidx.compose.foundation.text.BasicTextField
11 import androidx.compose.material3.Slider
12 import androidx.compose.material3.SliderDefaults
13 import androidx.compose.material3.Text
14 import androidx.compose.runtime.*
15 import androidx.compose.ui.Alignment
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.draw.clip
18 import androidx.compose.ui.graphics.Color
19 import androidx.compose.ui.graphics.SolidColor
20 import androidx.compose.ui.text.TextStyle
21 import androidx.compose.ui.text.font.FontWeight
22 import androidx.compose.ui.unit.dp
23 import androidx.compose.ui.unit.sp
24 import com.wetinknext.engine.core.CanvasBackdropMode
25 import kotlin.random.Random
26
27 private enum class ThemeColorSlot(val label: String) {
28     AppBackground("Рабочая область"), CanvasBackground("За холстом"), Grid("Сетка"),
29     Panel("Панели"), Accent("Акцент"), Text("Текст"), Danger("Удаление")
30 }
31
32 @Composable
33 fun CustomThemeEditor(
34     colors: CustomThemeColors,
35     theme: AppTheme,
36     backdropMode: CanvasBackdropMode,
37     onChange: (CustomThemeColors) -> Unit,
38     onBackdropModeChange: (CanvasBackdropMode) -> Unit,
39     onDismiss: () -> Unit,
40 ) {
41     var slot by remember { mutableStateOf(ThemeColorSlot.Accent) }
42     val current = colors.color(slot)
43     Column(Modifier.fillMaxWidth(), verticalArrangement = Arrangement.spacedBy(10.dp)) {
44         Row(verticalAlignment = Alignment.CenterVertically) {
45             Text("Своя тема", color = theme.textPrimary, style = androidx.compose.material3.MaterialTheme.typography.
titleLarge)
46             Spacer(Modifier.weight(1f))
47             Text("Случайная", color = theme.textSecondary, fontSize = 12.sp,
48                 modifier = Modifier.clickable { onChange(randomThemeColors()) }.padding(6.dp))
49             Text("Готово", color = theme.accent, fontWeight = FontWeight.SemiBold,
50                 modifier = Modifier.clickable(onClick = onDismiss).padding(6.dp))
51         }
52         ThemeLivePreview(colors)
53         Text("Фон за холстом", color = theme.textSecondary, fontSize = 12.sp)
54         Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(6.dp)) {
55             listOf(
56                 CanvasBackdropMode.GRID to "Сетка",
57                 CanvasBackdropMode.CHECKERBOARD to "Клетки",
58                 CanvasBackdropMode.SOLID to "Сплошной",
59             ).forEach { (mode, label) ->
60                 val selected = mode == backdropMode
61                 Box(
62                     Modifier.weight(1f).clip(RoundedCornerShape(8.dp))
63                     .background(if (selected) theme.accent.copy(alpha = .16f) else theme.panelInset)
64                     .border(1.dp, if (selected) theme.accent else Color.Transparent, RoundedCornerShape(8.dp))
65                     .clickable { onBackdropModeChange(mode) }.padding(vertical = 7.dp),
66                     contentAlignment = Alignment.Center,
67                 ) { Text(label, color = theme.textPrimary, fontSize = 10.sp, maxLines = 1) }
68             }
69         }
70         ThemeColorSlot.entries.chunked(2).forEach { row ->
71             Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
72                 row.forEach { item ->
73                     val active = item == slot
74                     Row(
75                         Modifier.weight(1f).clip(RoundedCornerShape(10.dp))
76                         .background(if (active) theme.accent.copy(alpha = .14f) else theme.panelInset)
77                         .border(1.dp, if (active) theme.accent else Color.Transparent, RoundedCornerShape(10.dp))
78                         .clickable { slot = item }.padding(8.dp),
79                         verticalAlignment = Alignment.CenterVertically,
80                     ) {
81                         Box(Modifier.size(20.dp).clip(CircleShape).background(Color(colors.color(item))).border(1.dp, theme.
panelStroke, CircleShape))
82                         Spacer(Modifier.width(7.dp))
83                         Text(item.label, color = theme.textPrimary, fontSize = 11.sp, maxLines = 1)
84                     }
85                 }
86                 if (row.size == 1) Spacer(Modifier.weight(1f))
87             }
88         }

```

```

89         ColorControls(slot.label, current, theme) { onChange(colors.withColor(slot, it)) }
90         Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
91             Text("Сбросить", color = theme.textSecondary, modifier = Modifier.weight(1f).clickable { onChange(
CustomThemeColors()) }.padding(8.dp))
92             Text("Применяется сразу", color = theme.textSecondary, fontSize = 11.sp, modifier = Modifier.padding(8.dp))
93         }
94     }
95 }
96
97 @Composable private fun ColorControls(label: String, color: Int, theme: AppTheme, update: (Int) -> Unit) {
98     var hex by remember(color) { mutableStateOf("%06X".format(color and 0xFFFFFF)) }
99     val r = color ushr 16 and 0xFF; val g = color ushr 8 and 0xFF; val b = color and 0xFF
100    val hsv = FloatArray(3).also { AndroidColor.RGBToHSV(r, g, b, it) }
101    Column(Modifier.clip(RoundedCornerShape(12.dp)).background(theme.panelInset).padding(10.dp), verticalArrangement =
Arrangement.spacedBy(4.dp)) {
102        Text("Цвет: $label", color = theme.textPrimary, fontWeight = FontWeight.Medium, fontSize = 13.sp)
103        BasicTextField(hex, { raw ->
104            val clean = raw.removePrefix("#").filter { it.isDigit() || it.lowercaseChar() in 'a'..'f' }.take(6).uppercase()
105            hex = clean; clean.toIntOrNull(16)?.takeIf { clean.length == 6 }?.let { update(0xFF000000.toInt() or it) }
106        }, textStyle = TextStyle(color = theme.textPrimary, fontSize = 14.sp), cursorBrush = SolidColor(theme.accent),
107        decorationBox = { inner -> Row(Modifier.fillMaxWidth().clip(RoundedCornerShape(7.dp)).background(theme.panelBg).
padding(8.dp)) { Text("#", color = theme.textSecondary); inner() } })
108        Channel("R", r, Color.Red, theme) { update(argb(it, g, b)) }
109        Channel("G", g, Color(0xFF38C878), theme) { update(argb(r, it, b)) }
110        Channel("B", b, Color(0xFF568DFF), theme) { update(argb(r, g, it)) }
111        PercentageChannel("H", hsv[1], theme.accent, theme) { saturation ->
112            hsv[1] = saturation
113            update(AndroidColor.HSVToColor(hsv))
114        }
115    }
116 }
117
118 @Composable private fun Channel(name: String, value: Int, color: Color, theme: AppTheme, update: (Int) -> Unit) =
119     Row(verticalAlignment = Alignment.CenterVertically) {
120         Text(name, color = theme.textSecondary, modifier = Modifier.width(18.dp), fontSize = 11.sp)
121         Slider(value.toFloat(), { update(it.toInt()) }, valueRange = 0f..255f, modifier = Modifier.weight(1f), colors =
SliderDefaults.colors(thumbColor = color, activeTrackColor = color, inactiveTrackColor = theme.panelBg))
122         Text(value.toString(), color = theme.textSecondary, modifier = Modifier.width(28.dp), fontSize = 11.sp)
123     }
124
125 @Composable private fun PercentageChannel(name: String, value: Float, color: Color, theme: AppTheme, update: (Float) -> Unit)
=
126     Row(verticalAlignment = Alignment.CenterVertically) {
127         Text(name, color = theme.textSecondary, modifier = Modifier.width(18.dp), fontSize = 11.sp)
128         Slider(value, update, valueRange = 0f..1f, modifier = Modifier.weight(1f), colors = SliderDefaults.colors(thumbColor
= color, activeTrackColor = color, inactiveTrackColor = theme.panelBg))
129         Text("${(value * 100).toInt()}%", color = theme.textSecondary, modifier = Modifier.width(28.dp), fontSize = 11.sp)
130     }
131
132 @Composable private fun ThemeLivePreview(colors: CustomThemeColors) =
133     Row(Modifier.fillMaxWidth().height(56.dp).clip(RoundedCornerShape(12.dp)).background(Color(colors.appBackground)).padding(
9.dp), verticalAlignment = Alignment.CenterVertically) {
134         Box(Modifier.size(38.dp).clip(RoundedCornerShape(7.dp)).background(Color(colors.panel)))
135         Spacer(Modifier.width(9.dp)); Column { Box(Modifier.width(112.dp).height(7.dp).clip(CircleShape).background(Color(
colors.text))); Spacer(Modifier.height(6.dp)); Box(Modifier.width(72.dp).height(5.dp).clip(CircleShape).background(Color(colors.
text).copy(alpha = .5f))) }
136         Spacer(Modifier.weight(1f)); Box(Modifier.size(27.dp).clip(CircleShape).background(Color(colors.accent)))
137     }
138
139 private fun CustomThemeColors.color(slot: ThemeColorSlot) = when (slot) { ThemeColorSlot.AppBackground -> appBackground;
ThemeColorSlot.CanvasBackground -> canvasBackground; ThemeColorSlot.Grid -> grid; ThemeColorSlot.Panel -> panel; ThemeColorSlot.
Accent -> accent; ThemeColorSlot.Text -> text; ThemeColorSlot.Danger -> danger }
140 private fun CustomThemeColors.withColor(slot: ThemeColorSlot, value: Int) = when (slot) { ThemeColorSlot.AppBackground ->
copy(appBackground = value); ThemeColorSlot.CanvasBackground -> copy(canvasBackground = value); ThemeColorSlot.Grid -> copy(grid =
value); ThemeColorSlot.Panel -> copy(panel = value); ThemeColorSlot.Accent -> copy(accent = value); ThemeColorSlot.Text -> copy(
text = value); ThemeColorSlot.Danger -> copy(danger = value) }
141 private fun argb(r: Int, g: Int, b: Int) = (0xFF shl 24) or (r shl 16) or (g shl 8) or b
142
143 /** Creates a coherent palette rather than seven unrelated random colours. */
144 private fun randomThemeColors(): CustomThemeColors {
145     val hue = Random.nextInt(360).toFloat()
146     val dark = Random.nextBoolean()
147     fun hsv(h: Float, s: Float, v: Float) = AndroidColor.HSVToColor(floatArrayOf((h + 360f) % 360f, s, v))
148     val baseValue = if (dark) .12f else .94f
149     val panelValue = if (dark) .18f else .99f
150     val textValue = if (dark) .94f else .16f
151     return CustomThemeColors(
152         appBackground = hsv(hue, .22f, baseValue),
153         canvasBackground = hsv(hue, .16f, if (dark) .09f else .89f),
154         grid = hsv(hue, .28f, if (dark) .27f else .78f),
155         panel = hsv(hue, .24f, panelValue),
156         accent = hsv(hue + 28f, .72f, .92f),
157         text = hsv(hue, .12f, textValue),
158         danger = hsv(2f, .72f, .92f),
159     )
160 }

```

118. app/src/main/java/com/wetinknext/ui/theme/Theme.kt

```

1 package com.wetinknext.ui.theme
2
3 import androidx.compose.material3.MaterialTheme
4 import androidx.compose.material3.Typography
5 import androidx.compose.material3.darkColorScheme
6 import androidx.compose.material3.lightColorScheme
7 import androidx.compose.runtime.Composable
8 import androidx.compose.runtime.CompositionLocalProvider
9 import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.text.TextStyle
11 import androidx.compose.ui.text.font.Font
12 import androidx.compose.ui.text.font.FontFamily
13 import androidx.compose.ui.text.font.FontWeight
14 import androidx.compose.ui.unit.sp
15 import com.wetinknext.R
16
17 private val ClassicFontFamily = FontFamily(
18     Font(R.font.lora_variable, FontWeight.Normal),
19     Font(R.font.lora_variable, FontWeight.Bold),
20 )
21
22 private val HandwrittenFontFamily = FontFamily(
23     Font(R.font.caveat_variable, FontWeight.Normal),
24     Font(R.font.caveat_variable, FontWeight.Bold),
25 )
26
27 @Composable
28 fun WetInkTheme(
29     theme: AppTheme,
30     fontMode: UiFont,
31     content: @Composable () -> Unit
32 ) {
33     val fontFamily = if (fontMode == UiFont.Italic) HandwrittenFontFamily else ClassicFontFamily
34
35     /*
36      * One compact editorial scale for every editor panel. Individual controls
37      * may still use a title style, but labels and segmented toolbars now share
38      * the same readable base instead of each panel inventing its own scale.
39      */
40     val typography = Typography(
41         bodyLarge = TextStyle(fontFamily = fontFamily, fontSize = 14.sp, lineHeight = 18.sp),
42         bodyMedium = TextStyle(fontFamily = fontFamily, fontSize = 13.sp, lineHeight = 17.sp),
43         bodySmall = TextStyle(fontFamily = fontFamily, fontSize = 12.sp, lineHeight = 16.sp),
44         titleLarge = TextStyle(fontFamily = fontFamily, fontSize = 16.sp, lineHeight = 20.sp, fontWeight = FontWeight.SemiBold),
45         titleMedium = TextStyle(fontFamily = fontFamily, fontSize = 15.sp, lineHeight = 19.sp, fontWeight = FontWeight.Medium),
46         titleSmall = TextStyle(fontFamily = fontFamily, fontSize = 14.sp, lineHeight = 18.sp, fontWeight = FontWeight.Medium),
47         labelLarge = TextStyle(fontFamily = fontFamily, fontSize = 13.sp, lineHeight = 16.sp, fontWeight = FontWeight.Medium),
48         labelMedium = TextStyle(fontFamily = fontFamily, fontSize = 12.sp, lineHeight = 15.sp, fontWeight = FontWeight.Medium),
49         labelSmall = TextStyle(fontFamily = fontFamily, fontSize = 11.sp, lineHeight = 14.sp, fontWeight = FontWeight.Medium),
50     )
51
52     val scheme = if (!theme.isLight) {
53         darkColorScheme(
54             primary = theme.accent,
55             onPrimary = theme.textPrimary,
56             background = theme.appBg, // Цвет стола
57             onBackground = theme.textPrimary,
58             surface = theme.panelBg, // Цвет панелей
59             onSurface = theme.textPrimary,
60             surfaceVariant = theme.panelBgVariant,
61             onSurfaceVariant = theme.textSecondary,
62         )
63     } else {
64         lightColorScheme(
65             primary = theme.accent,
66             onPrimary = Color.White,
67             background = theme.appBg,
68             onBackground = theme.textPrimary,
69             surface = theme.panelBg,
70             onSurface = theme.textPrimary,
71             surfaceVariant = theme.panelBgVariant,
72             onSurfaceVariant = theme.textSecondary,
73         )
74     }
75
76     CompositionLocalProvider(LocalAppTheme provides theme) {
77         MaterialTheme(
78             colorScheme = scheme,
79             typography = typography,
80             content = content,
81         )
82     }
83 }

```

119. app/src/main/java/com/wetinknext/ui/theme/ThemeController.kt

```

1  package com.wetinknext.ui.theme
2
3  import android.content.Context
4  import androidx.compose.runtime.*
5  import androidx.compose.runtime.saveable.Saver
6  import androidx.compose.runtime.saveable.rememberSaveable
7  import androidx.compose.ui.graphics.Color
8  import androidx.compose.ui.platform.LocalContext
9  import com.wetinknext.engine.core.CanvasBackdropMode
10 import kotlin.math.pow
11
12 val LocalAppTheme = compositionLocalOf { AppThemes.Gray }
13
14 data class CustomThemeColors(
15     val appBackground: Int = 0xFF101826.toInt(),
16     val canvasBackground: Int = 0xFF101826.toInt(),
17     val grid: Int = 0xFF38424C.toInt(),
18     val panel: Int = 0xFF172033.toInt(),
19     val accent: Int = 0xFF58A6FF.toInt(),
20     val text: Int = 0xFFF4F7FB.toInt(),
21     val danger: Int = 0xFFFF4A5F.toInt(),
22 )
23
24 class ThemeController(
25     initialThemeId: String = "gray",
26     initialFontMode: String = "Classic",
27     initialCustomColors: CustomThemeColors = CustomThemeColors(),
28     initialBackdropMode: String = CanvasBackdropMode.GRID.name,
29 ) {
30     var current by mutableStateOf(resolveTheme(initialThemeId, initialCustomColors))
31     private set
32     var font by mutableStateOf(if (initialFontMode == "Italic") UiFont.Italic else UiFont.Classic)
33     private set
34     var customColors by mutableStateOf(initialCustomColors)
35     private set
36     var backdropMode by mutableStateOf(runCatching { CanvasBackdropMode.valueOf(initialBackdropMode) }.getOrElse(
CanvasBackdropMode.GRID))
37     private set
38     val customTheme: AppTheme
39     get() = createCustomTheme(customColors)
40
41     fun select(theme: AppTheme) {
42         current = if (theme.id == "custom") createCustomTheme(customColors) else theme
43     }
44
45     fun selectCustom() {
46         current = createCustomTheme(customColors)
47     }
48
49     fun selectFont(f: UiFont) { font = f }
50     fun selectBackdropMode(mode: CanvasBackdropMode) { backdropMode = mode }
51
52     fun updateCustomColors(newColors: CustomThemeColors) {
53         customColors = newColors
54         if (current.id == "custom") current = createCustomTheme(newColors)
55     }
56
57     private fun resolveTheme(id: String, colors: CustomThemeColors): AppTheme {
58         if (id == "custom") return createCustomTheme(colors)
59         return AppThemes.all.find { it.id == id } ?: AppThemes.Gray
60     }
61
62     private fun createCustomTheme(colors: CustomThemeColors): AppTheme {
63         val appBg = Color(colors.appBackground)
64         val panel = Color(colors.panel)
65         val accent = Color(colors.accent)
66         val text = Color(colors.text)
67         val grid = Color(colors.grid)
68         val isDark = luminanceOf(appBg) < 0.5f
69
70         return AppTheme(
71             id = "custom", displayName = "Своя", isLight = !isDark,
72             appBg = appBg, gridLine = grid,
73             panelBg = panel, panelBgSolid = panel.mix(if (isDark) Color.White else Color.Black, 0.08f),
74             panelStroke = panel.mix(accent, 0.3f),
75             panelInset = panel.mix(if (isDark) Color.Black else Color.White, 0.2f),
76             panelInsetSoft = panel.mix(accent, 0.1f),
77             canvasBackdrop = Color(colors.canvasBackground), canvasGrid = grid,
78             textPrimary = text, textSecondary = text.copy(alpha = 0.6f),
79             iconInactive = text.copy(alpha = 0.4f),
80             accent = accent, accentSoft = accent.copy(alpha = 0.7f),
81             accentMuted = accent.copy(alpha = 0.3f),
82             danger = Color(colors.danger)
83         )
84     }
85
86     private fun Color.mix(target: Color, amount: Float): Color {
87         return Color(
88             red = red * (1f - amount) + target.red * amount,
89             green = green * (1f - amount) + target.green * amount,

```

```

90         blue = blue * (1f - amount) + target.blue * amount,
91         alpha = alpha
92     )
93 }
94
95 private fun luminanceOf(color: Color): Float {
96     fun channel(value: Float): Float =
97         if (value <= 0.03928f) value / 12.92f else ((value + 0.055f) / 1.055f).pow(2.4f)
98     return 0.2126f * channel(color.red) + 0.7152f * channel(color.green) + 0.0722f * channel(color.blue)
99 }
100
101 companion object {
102     val Saver: Saver<ThemeController, *> = Saver(
103         save = { listOf(it.current.id, it.font.name, it.backdropMode.name, it.customColors.appBackground, it.customColors.
104 canvasBackground, it.customColors.grid, it.customColors.panel, it.customColors.accent, it.customColors.text, it.customColors.
105 danger) },
106         restore = { list ->
107             val l = list as List<*>
108             ThemeController(
109                 initialThemeId = l[0] as String,
110                 initialFontMode = l[1] as String,
111                 initialBackdropMode = if (l.size >= 10) l[2] as String else CanvasBackdropMode.GRID.name,
112                 initialCustomColors = if (l.size >= 10) CustomThemeColors(l[3] as Int, l[4] as Int, l[5] as Int, l[6] as
113 Int, l[7] as Int, l[8] as Int, l[9] as Int)
114                 else if (l.size >= 9) CustomThemeColors(l[2] as Int, l[3] as Int, l[4] as Int, l[5] as Int, l[6] as Int,
115 l[7] as Int, l[8] as Int)
116                 else CustomThemeColors(l[2] as Int, l[2] as Int, l[3] as Int, l[4] as Int, l[5] as Int, l[6] as Int)
117             )
118         }
119     )
120 }
121
122 @Composable
123 fun rememberThemeController(): ThemeController {
124     val context = LocalContext.current
125     val prefs = remember { context.getSharedPreferences("wetinknext_theme_prefs", Context.MODE_PRIVATE) }
126
127     val controller = rememberSaveable(saver = ThemeController.Saver) {
128         val themeId = prefs.getString("theme_id", "gray") ?: "gray"
129         val fontMode = prefs.getString("font_mode", "Classic") ?: "Classic"
130         val backdropMode = prefs.getString("backdrop_mode", CanvasBackdropMode.GRID.name) ?: CanvasBackdropMode.GRID.name
131         val customColors = CustomThemeColors(
132             appBackground = prefs.getInt("custom_bg", 0xFF101826.toInt()),
133             canvasBackground = prefs.getInt("custom_canvas_bg", prefs.getInt("custom_bg", 0xFF101826.toInt())),
134             grid = prefs.getInt("custom_grid", 0xFF38424C.toInt()),
135             panel = prefs.getInt("custom_panel", 0xFF172033.toInt()),
136             accent = prefs.getInt("custom_accent", 0xFF58A6FF.toInt()),
137             text = prefs.getInt("custom_text", 0xFFFF4F7FB.toInt()),
138             danger = prefs.getInt("custom_danger", 0xFFFF4A5F.toInt()),
139         )
140         ThemeController(themeId, fontMode, customColors, backdropMode)
141     }
142
143     LaunchedEffect(controller.current.id, controller.font, controller.customColors, controller.backdropMode) {
144         prefs.edit()
145             .putString("theme_id", controller.current.id)
146             .putString("font_mode", controller.font.name)
147             .putString("backdrop_mode", controller.backdropMode.name)
148             .putInt("custom_bg", controller.customColors.appBackground)
149             .putInt("custom_canvas_bg", controller.customColors.canvasBackground)
150             .putInt("custom_grid", controller.customColors.grid)
151             .putInt("custom_panel", controller.customColors.panel)
152             .putInt("custom_accent", controller.customColors.accent)
153             .putInt("custom_text", controller.customColors.text)
154             .putInt("custom_danger", controller.customColors.danger)
155             .apply()
156     }
157
158     return controller
159 }

```

120. app/src/main/java/com/wetinknext/ui/theme/WetInkNextPalette.kt

```
1 package com.wetinknext.ui.theme
2
3 import androidx.compose.ui.graphics.Color
4
5 data class WetInkNextPalette(
6     val appBg: Color,
7     val panelBg: Color,
8     val panelStroke: Color,
9     val textPrimary: Color,
10    val textSecondary: Color,
11    val accent: Color,
12 ) {
13     companion object {
14         val default = WetInkNextPalette(
15             appBg = Color(0xFF17181C),
16             panelBg = Color(0xFF25272D),
17             panelStroke = Color(0xFF3A3D46),
18             textPrimary = Color(0xFFF3F4F6),
19             textSecondary = Color(0xFFA4A8B3),
20             accent = Color(0xFF4D7AFF),
21         )
22     }
23 }
```


121. app/src/main/java/com/wetinknext/ui/thumbnail/LayerThumbnail.kt

```

1 package com.wetinknext.ui.thumbnail
2
3 import android.graphics.BitmapFactory
4 import android.util.LruCache
5 import androidx.compose.foundation.Image
6 import androidx.compose.foundation.layout.Box
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.runtime.Composable
9 import androidx.compose.runtime.LaunchedEffect
10 import androidx.compose.runtime.getValue
11 import androidx.compose.runtime.mutableStateOf
12 import androidx.compose.runtime.remember
13 import androidx.compose.runtime.setValue
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.graphics.ImageBitmap
16 import androidx.compose.ui.graphics.asImageBitmap
17 import androidx.compose.ui.layout.ContentScale
18 import androidx.compose.foundation.layout.aspectRatio
19 import java.io.File
20 import kotlinx.coroutines.Dispatchers
21 import kotlinx.coroutines.withContext
22
23 private data class LayerThumbnailKey(
24     val layerId: Long,
25     val version: Long,
26 )
27
28 /** Process-local cache; the layer version prevents recycled rows showing stale previews. */
29 private object LayerThumbnailBitmapCache {
30     private val bitmaps = LruCache<LayerThumbnailKey, ImageBitmap>(48)
31
32     fun get(key: LayerThumbnailKey): ImageBitmap? = bitmaps.get(key)
33
34     fun put(key: LayerThumbnailKey, bitmap: ImageBitmap) {
35         bitmaps.put(key, bitmap)
36     }
37 }
38
39 /**
40  * A persisted WebP preview with alpha and the document's native aspect ratio.
41  * The last decoded bitmap remains visible while a newer version is read from disk.
42  */
43 @Composable
44 fun LayerThumbnail(
45     layerId: Long,
46     path: String?,
47     thumbnailVersion: Long,
48     modifier: Modifier = Modifier,
49 ) {
50     val key = remember(layerId, thumbnailVersion) {
51         LayerThumbnailKey(layerId, thumbnailVersion)
52     }
53     var displayedBitmap by remember(layerId) {
54         mutableStateOf(LayerThumbnailBitmapCache.get(key))
55     }
56
57     LaunchedEffect(key, path) {
58         val bitmap = LayerThumbnailBitmapCache.get(key) ?: withContext(Dispatchers.IO) {
59             path
60                 ?.let(::File)
61                 ?.takeIf(File::isFile)
62                 ?.let { BitmapFactory.decodeFile(it.absolutePath) }
63                 ?.asImageBitmap()
64                 ?.also { LayerThumbnailBitmapCache.put(key, it) }
65         }
66         if (bitmap != null) {
67             displayedBitmap = bitmap
68         }
69     }
70
71     val previewModifier = displayedBitmap?.let { bitmap ->
72         modifier.aspectRatio(
73             ratio = bitmap.width.toFloat() / bitmap.height.coerceAtLeast(1).toFloat(),
74             matchHeightConstraintsFirst = true,
75         )
76     } ?: modifier
77
78     // No panel-coloured backing: alpha in the layer WebP stays alpha in UI.
79     // The dimensions come from the image itself, so a portrait document does
80     // not acquire horizontal bars just because its row is wider than it is.
81     Box(modifier = previewModifier) {
82         displayedBitmap?.let { bitmap ->
83             Image(
84                 bitmap = bitmap,
85                 contentDescription = "Layer preview",
86                 modifier = Modifier.fillMaxSize(),
87                 contentScale = ContentScale.Fit,
88             )
89         }
90     }

```


122. app/src/main/res/drawable/ic_eraser.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
3      android:width="24dp"
4      android:height="24dp"
5      android:viewportWidth="24"
6      android:viewportHeight="24">
7      <path
8          android:fillColor="#FFFFFF"
9          android:pathData="M14.59,3.41a2,2 0,0 1,2.83 0l3.17,3.17a2,2 0,0 1,0 2.83l11.41,18.59a2,2 0,0 1,-2.83 0l4.41,14.41a2,
2 0,0 1,0 -2.83l10.18,-8.17zM3,20h18v2H3z" />
10     <path
11         android:fillColor="#FFFFFF"
12         android:fillAlpha="0.55"
13         android:pathData="M7.05,11l4.95,4.95l3.18,-3.18l-4.95,-4.95z" />
14 </vector>
```

123. app/src/main/res/drawable/ic_flip_horizontal.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
3      android:width="24dp"
4      android:height="24dp"
5      android:viewportWidth="24"
6      android:viewportHeight="24">
7      <!-- Central Axis (Centered at 12) -->
8      <path
9          android:fillColor="#FFFFFF"
10         android:pathData="M11.25,4h1.5v16h-1.5V4z"/>
11
12     <!-- Left solid bracket (Symmetrical) -->
13     <path
14         android:fillColor="#FFFFFF"
15         android:pathData="M9,6.5H5v11h4V16H6.5V8H9V6.5z"/>
16
17     <!-- Right dashed bracket (Mirror of left, perfectly aligned) -->
18     <!-- Top dash -->
19     <path android:fillColor="#FFFFFF" android:pathData="M15,6.5h2.5V8H15V6.5z"/>
20     <path android:fillColor="#FFFFFF" android:pathData="M18.5,6.5h0.5V8h-0.5V6.5z"/>
21     <!-- Side dashes -->
22     <path android:fillColor="#FFFFFF" android:pathData="M17.5,9.5h1.5v2h-1.5V9.5z"/>
23     <path android:fillColor="#FFFFFF" android:pathData="M17.5,12.5h1.5v2h-1.5V12.5z"/>
24     <!-- Bottom dash -->
25     <path android:fillColor="#FFFFFF" android:pathData="M15,16h2.5v1.5H15V16z"/>
26     <path android:fillColor="#FFFFFF" android:pathData="M18.5,16h0.5v1.5h-0.5V16z"/>
27 </vector>
```

124. app/src/main/res/drawable/ic_launcher_background.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <vector xmlns:android="http://schemas.android.com/apk/res/android"
3     android:width="108dp"
4     android:height="108dp"
5     android:viewportWidth="108"
6     android:viewportHeight="108">
7     <path
8         android:fillColor="#FF211019"
9         android:pathData="M0,0h108v108h-108z" />
10 </vector>
```

125. app/src/main/res/drawable/ic_launcher_foreground.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:aapt="http://schemas.android.com/aapt"
4      android:width="108dp"
5      android:height="108dp"
6      android:viewportWidth="108"
7      android:viewportHeight="108">
8      <group
9          android:pivotX="54"
10         android:pivotY="54"
11         android:scaleX="0.6"
12         android:scaleY="0.6">
13         <path
14             android:pathData="M14,88h80"
15             android:strokeWidth="5"
16             android:strokeLineCap="round">
17             <aapt:attr name="android:strokeColor">
18                 <gradient
19                     android:startX="78"
20                     android:startY="20"
21                     android:endX="30"
22                     android:endY="80"
23                     android:type="linear">
24                     <item android:offset="0" android:color="#FFFFB1C5" />
25                     <item android:offset="1" android:color="#FFFF3D70" />
26                 </gradient>
27             </aapt:attr>
28         </path>
29         <path
30             android:pathData="M66,16c3.3,-3.3 8.7,-3.3 12,0l4,4c3.3,3.3 3.3,8.7 0,12l34,80l-16,4l4,-16l66,16z"
31             android:fillType="nonZero">
32             <aapt:attr name="android:fillColor">
33                 <gradient
34                     android:startX="78"
35                     android:startY="20"
36                     android:endX="30"
37                     android:endY="80"
38                     android:type="linear">
39                     <item android:offset="0" android:color="#FFFFB1C5" />
40                     <item android:offset="1" android:color="#FFFF3D70" />
41                 </gradient>
42             </aapt:attr>
43         </path>
44     </group>
45 </vector>
```

126. app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
3     <background android:drawable="@drawable/ic_launcher_background" />
4     <foreground android:drawable="@drawable/ic_launcher_foreground" />
5 </adaptive-icon>
```

127. app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
3     <background android:drawable="@drawable/ic_launcher_background" />
4     <foreground android:drawable="@drawable/ic_launcher_foreground" />
5 </adaptive-icon>
```


128. app/src/main/res/mipmap-anydpi/ic_launcher.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <layer-list xmlns:android="http://schemas.android.com/apk/res/android">
3     <item android:drawable="@drawable/ic_launcher_background" />
4     <item android:drawable="@drawable/ic_launcher_foreground" />
5 </layer-list>
```

129. app/src/main/res/mipmap-anydpi/ic_launcher_round.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <layer-list xmlns:android="http://schemas.android.com/apk/res/android">
3     <item android:drawable="@drawable/ic_launcher_background" />
4     <item android:drawable="@drawable/ic_launcher_foreground" />
5 </layer-list>
```

130. app/src/main/res/values/strings.xml

```
1 <resources>
2     <string name="app_name">WetInk Next</string>
3 </resources>
```

131. app/src/main/res/values/themes.xml

```
1 <resources>
2     <style name="Theme.WetInkNext" parent="android:style/Theme.Material.NoActionBar" />
3 </resources>
```

132. app/src/test/java/com/wetinknext/data/project/AutosaveCoordinatorTest.kt

```
1 package com.wetinknext.data.project
2
3 import kotlinx.coroutines.CoroutineScope
4 import kotlinx.coroutines.Dispatchers
5 import kotlinx.coroutines.SupervisorJob
6 import kotlinx.coroutines.cancel
7 import org.junit.Assert.assertEquals
8 import org.junit.Assert.assertTrue
9 import org.junit.Test
10 import java.util.concurrent.CopyOnWriteArrayList
11 import java.util.concurrent.CountDownLatch
12 import java.util.concurrent.TimeUnit
13
14 class AutosaveCoordinatorTest {
15     @Test
16     fun laterCompletedEditReplacesEarlierPendingSave() {
17         val scope = CoroutineScope(SupervisorJob() + Dispatchers.Default)
18         val coordinator = AutosaveCoordinator(scope, debounceMillis = 30L)
19         val saved = CopyOnWriteArrayList<String>()
20         val latch = CountDownLatch(1)
21
22         coordinator.schedule { saved += "first" }
23         coordinator.schedule {
24             saved += "second"
25             latch.countDown()
26         }
27
28         assertTrue(latch.await(2, TimeUnit.SECONDS))
29         assertEquals(listOf("second"), saved)
30         scope.cancel()
31     }
32 }
```

133. app/src/test/java/com/wetinknext/data/project/FileProjectRepositoryTest.kt

```

1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.ProjectAssets
4 import com.wetinknext.domain.document.ProjectDocument
5 import com.wetinknext.domain.document.WetInkProjectContainer
6 import org.junit.Assert.assertArrayEquals
7 import org.junit.Assert.assertEquals
8 import org.junit.Assert.assertFalse
9 import org.junit.Assert.assertTrue
10 import org.junit.Test
11 import java.io.File
12 import java.nio.file.Files
13 import java.util.concurrent.atomic.AtomicLong
14 import kotlin.coroutines.Continuation
15 import kotlin.coroutines.EmptyCoroutineContext
16 import kotlin.coroutines.startCoroutine
17
18 class FileProjectRepositoryTest {
19     @Test
20     fun projectLifecycleKeepsGalleryMetadataCompact() {
21         val root = Files.createTempDirectory("wetink-repository").toFile()
22         val time = AtomicLong(100L)
23         val repository = FileProjectRepository(
24             projectsDirectory = root,
25             container = WetInkProjectContainer(clockMillis = { time.incrementAndGet() }),
26         )
27
28         val first = runSuspend {
29             repository.create(CreateProjectRequest(name = "First", width = 800, height = 600))
30         }
31         val second = runSuspend {
32             repository.create(CreateProjectRequest(name = "Second", width = 1200, height = 900))
33         }
34
35         val listed = runSuspend { repository.list() }
36         assertEquals(listOf(second.id, first.id), listed.map(ProjectSummary::id))
37         assertEquals("Second", listed.first().name)
38         assertEquals(1200, listed.first().width)
39         assertEquals(900, listed.first().height)
40
41         val opened = runSuspend { repository.open(first.id) }
42         assertEquals("First", opened.name)
43         assertEquals(800, opened.width)
44
45         runSuspend { repository.rename(first.id, "Renamed") }
46         assertEquals("Renamed", runSuspend { repository.open(first.id) }.name)
47
48         runSuspend { repository.moveToTrash(first.id) }
49         assertFalse(runSuspend { repository.list() }.any { it.id == first.id })
50         assertEquals(first.id, runSuspend { repository.listTrash() }.single().project.id)
51         runSuspend { repository.restore(first.id) }
52         assertTrue(runSuspend { repository.list() }.any { it.id == first.id })
53         runSuspend { repository.moveToTrash(first.id) }
54         runSuspend { repository.deleteForever(first.id) }
55         assertFalse(File(root, ".trash/${first.id}.wetink").exists())
56     }
57
58     @Test
59     fun metadataSavePreservesExistingTilePayloads() {
60         val root = Files.createTempDirectory("wetink-repository-save").toFile()
61         val container = WetInkProjectContainer(clockMillis = { 500L })
62         val repository = FileProjectRepository(root, container)
63         val document = ProjectDocument.newUntitled(id = "project-1", nowMillis = 100L)
64         val file = File(root, "project-1.wetink")
65         container.save(
66             file,
67             document,
68             ProjectAssets(mapOf(1L to byteArrayOf(1, 2), 2L to byteArrayOf(3, 4))),
69         )
70
71         runSuspend { repository.save(document.copy(name = "Saved name")) }
72
73         // Saving a legacy ZIP is the migration point: the same project is now
74         // represented by projects/project-1/{manifest,document,layers}.
75         assertTrue(File(root, "project-1").isDirectory)
76         val loaded = runSuspend { repository.openForEditor("project-1") }
77         assertEquals("Saved name", loaded.document.name)
78         assertArrayEquals(byteArrayOf(1, 2), loaded.assets.layerTiles.getValue(1L))
79         assertArrayEquals(byteArrayOf(3, 4), loaded.assets.layerTiles.getValue(2L))
80     }
81
82     @Test
83     fun recoverySnapshotPreservesNewestTilePayloadUntilPrimarySave() {
84         val root = Files.createTempDirectory("wetink-recovery").toFile()
85         val repository = FileProjectRepository(root, WetInkProjectContainer(clockMillis = { 700L }))
86         val summary = runSuspend { repository.create(CreateProjectRequest(name = "Recovery")) }
87         val document = runSuspend { repository.open(summary.id) }
88
89         runSuspend { repository.saveRecovery(document, mapOf(2L to byteArrayOf(9, 8, 7))) }
90         val recovered = runSuspend { repository.openRecovery(summary.id) }

```

```

91         assertEquals(byteArrayOf(9, 8, 7), checkNotNull(recovered).assets.layerTiles.getValue(2L))
92     }
93     runSuspend { repository.save(recovered.document, recovered.assets.layerTiles) }
94     runSuspend { repository.clearRecovery(summary.id) }
95     assertTrue(runSuspend { repository.openRecovery(summary.id) } == null)
96     assertEquals(byteArrayOf(9, 8, 7), runSuspend { repository.openForEditor(summary.id) }.assets.layerTiles.
getValue(2L))
97     }
98 }
99 @Test
100 fun thumbnailSaveIsAtomicAndGalleryReadsThePublishedPreview() {
101     val root = Files.createTempDirectory("wetink-thumbnail").toFile()
102     val repository = FileProjectRepository(root, WetInkProjectContainer(clockMillis = { 800L }))
103     val summary = runSuspend { repository.create(CreateProjectRequest(name = "Preview")) }
104     val document = runSuspend { repository.open(summary.id) }
105     val webp = byteArrayOf(0x52, 0x49, 0x46, 0x46, 0x57, 0x45, 0x42, 0x50)
106
107     runSuspend { repository.save(document, emptyMap(), webp) }
108
109     val galleryItem = runSuspend { repository.list() }.single()
110     assertEquals(webp, galleryItem.thumbnailBytes)
111     assertEquals(webp, runSuspend { repository.openForEditor(summary.id) }.assets.thumbnailWebp)
112 }
113
114 @Test
115 fun duplicateCreatesIndependentDirectoryProject() {
116     val root = Files.createTempDirectory("wetink-duplicate").toFile()
117     val repository = FileProjectRepository(root, WetInkProjectContainer(clockMillis = { 900L }))
118     val original = runSuspend { repository.create(CreateProjectRequest(name = "Original")) }
119
120     val duplicate = runSuspend { repository.duplicate(original.id) }
121
122     assertFalse(original.id == duplicate.id)
123     assertEquals("Original copy", duplicate.name)
124     assertTrue(File(root, duplicate.id).isDirectory)
125     val originalTiles = runSuspend { repository.openForEditor(original.id) }.assets.layerTiles
126     val duplicateTiles = runSuspend { repository.openForEditor(duplicate.id) }.assets.layerTiles
127     assertEquals(originalTiles.keys, duplicateTiles.keys)
128     originalTiles.forEach { (layerId, bytes) ->
129         assertEquals(bytes, duplicateTiles.getValue(layerId))
130     }
131 }
132
133 private fun <T> runSuspend(block: suspend () -> T): T {
134     var result: Result<T>? = null
135     block.startCoroutine(object : Continuation<T> {
136         override val context = EmptyCoroutineContext
137
138         override fun resumeWith(value: Result<T>) {
139             result = value
140         }
141     })
142     return checkNotNull(result).getOrNull()
143 }
144 }

```

134. app/src/test/java/com/wetinknext/domain/document/ProjectDocumentTest.kt

```

1  package com.wetinknext.domain.document
2
3  import kotlinx.serialization.encodeToString
4  import kotlinx.serialization.json.Json
5  import org.junit.Assert.assertEquals
6  import org.junit.Assert.assertFalse
7  import org.junit.Assert.assertTrue
8  import org.junit.Assert.fail
9  import org.junit.Test
10
11 class ProjectDocumentTest {
12     private val json = Json {
13         encodeDefaults = true
14         ignoreUnknownKeys = true
15     }
16
17     @Test
18     fun untitledProjectRoundTripsWithoutEmbeddingLayerPixels() {
19         val document = ProjectDocument.newUntitled(
20             id = "project-1",
21             name = "Sketch",
22             width = 2048,
23             height = 1536,
24             dpi = 300,
25             nowMillis = 1234L,
26         )
27
28         val encoded = json.encodeToString(document)
29         val decoded = json.decodeFromString<ProjectDocument>(encoded)
30
31         assertEquals(document, decoded)
32         assertTrue(encoded.contains("\pixelFile\":"layers/1.tiles\"))
33         assertFalse(encoded.contains("rawBytes"))
34         assertFalse(encoded.contains("textureId"))
35     }
36
37     @Test(expected = IllegalArgumentException::class)
38     fun projectRejectsAnUnknownActiveLayer() {
39         val document = ProjectDocument.newUntitled(id = "project-1", nowMillis = 1234L)
40         document.copy(activeLayerId = 999L)
41     }
42
43     @Test
44     fun newCanvasRejectsAnUnsafeInitialGpuAllocation() {
45         try {
46             ProjectDocument.newUntitled(width = 8192, height = 8192)
47             fail("Expected memory-budget validation to reject an 8192px canvas")
48         } catch (_: IllegalArgumentException) {
49             // Expected: two initial layers and the stroke target would exceed the budget.
50         }
51     }
52 }

```


135. app/src/test/java/com/wetinknext/domain/document/WetInkProjectContainerTest.kt

```

1 package com.wetinknext.domain.document
2
3 import org.junit.Assert.assertArrayEquals
4 import org.junit.Assert.assertEquals
5 import org.junit.Assert.assertFalse
6 import org.junit.Assert.fail
7 import org.junit.Test
8 import java.io.File
9 import java.nio.file.Files
10 import java.util.zip.ZipFile
11
12 class WetInkProjectContainerTest {
13     @Test
14     fun savePublishesOneContainerWithSeparateLayerPayloads() {
15         val root = Files.createTempDirectory("wetink-project-test").toFile()
16         val projectFile = File(root, "sketch.wetink")
17         val document = ProjectDocument.newUntitled(
18             id = "project-1",
19             nowMillis = 100L,
20         )
21         val assets = ProjectAssets(
22             layerTiles = mapOf(
23                 1L to byteArrayOf(1, 2, 3),
24                 2L to byteArrayOf(4, 5, 6),
25             ),
26             thumbnailWebp = byteArrayOf(7, 8),
27             layerPreviewsWebp = mapOf(2L to byteArrayOf(9)),
28         )
29         val container = WetInkProjectContainer(clockMillis = { 200L })
30
31         val saved = container.save(projectFile, document, assets)
32         val loaded = container.load(projectFile)
33
34         assertEquals(100L, document.updatedAt)
35         assertEquals(200L, saved.document.updatedAt)
36         assertEquals(saved.document, loaded.document)
37         assertArrayEquals(byteArrayOf(1, 2, 3), loaded.assets.layerTiles.getValue(1L))
38         assertArrayEquals(byteArrayOf(4, 5, 6), loaded.assets.layerTiles.getValue(2L))
39         assertArrayEquals(byteArrayOf(7, 8), loaded.assets.thumbnailWebp)
40         assertArrayEquals(byteArrayOf(9), loaded.assets.layerPreviewsWebp.getValue(2L))
41         assertFalse(File(root, "sketch.wetink.tmp").exists())
42
43         ZipFile(projectFile).use { zip ->
44             assertEquals(true, zip.getEntry("manifest.json") != null)
45             assertEquals(true, zip.getEntry("document.json") != null)
46             assertEquals(true, zip.getEntry("layers/1.tiles") != null)
47             assertEquals(true, zip.getEntry("layers/2.tiles") != null)
48             assertEquals(true, zip.getEntry("thumbnail.webp") != null)
49             assertEquals(true, zip.getEntry("previews/2.webp") != null)
50         }
51     }
52
53     @Test
54     fun invalidSecondSaveLeavesPublishedProjectUntouched() {
55         val root = Files.createTempDirectory("wetink-project-atomic").toFile()
56         val projectFile = File(root, "sketch.wetink")
57         val document = ProjectDocument.newUntitled(
58             id = "project-1",
59             nowMillis = 100L,
60         )
61         val container = WetInkProjectContainer(clockMillis = { 200L })
62         container.save(
63             projectFile,
64             document,
65             ProjectAssets(mapOf(1L to byteArrayOf(1), 2L to byteArrayOf(2))),
66         )
67
68         try {
69             container.save(
70                 projectFile,
71                 document,
72                 ProjectAssets(mapOf(1L to byteArrayOf(99))),
73             )
74             fail("Missing layer payload must reject the save")
75         } catch (_: IllegalArgumentException) {
76             // Expected: validation happens before replacing the published file.
77         }
78
79         val loaded = container.load(projectFile)
80         assertArrayEquals(byteArrayOf(1), loaded.assets.layerTiles.getValue(1L))
81         assertArrayEquals(byteArrayOf(2), loaded.assets.layerTiles.getValue(2L))
82     }
83 }

```

136. app/src/test/java/com/wetinknext/engine/brush/BrushDynamicsTest.kt

```

1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertNotEquals
5 import org.junit.Test
6 import kotlin.math.PI
7
8 class BrushDynamicsTest {
9     @Test
10    fun resolve_separatesLocalCoverageFromGlobalOpacity() {
11        val settings = BrushSettings(
12            baseRadiusPx = 10f,
13            opacity = 0.2f,
14            pressureGamma = 2f,
15            minSizeRatio = 0.1f,
16            tiltToSize = 0.5f,
17            tiltToOpacity = 1f,
18        )
19        val result = ResolvedDab()
20
21        BrushDynamics.resolve(
22            settings = settings,
23            pressure = 0.5f,
24            tiltX = 1f,
25            tiltY = 0f,
26            velocityPxPerSecond = 0f,
27            random01 = 0.5f,
28            out = result,
29        )
30
31        // p = 0.5 ^ 2 = 0.25; pressure size = .325; tilt size = 1.5.
32        assertEquals(4.875f, result.radius, 0.0001f)
33        // Tilt attenuates, rather than amplifies, local coverage.
34        assertEquals(0.1375f, result.coverage, 0.0001f)
35        assertEquals(1f, result.flow, 0.0001f)
36        assertEquals(1f, result.hardness, 0.0001f)
37    }
38
39    @Test
40    fun resolve_appliesVelocityAndTiltRotation() {
41        val result = ResolvedDab()
42        BrushDynamics.resolve(
43            settings = BrushSettings(
44                baseRadiusPx = 10f,
45                pressureToSize = false,
46                pressureToOpacity = false,
47                velocityToSize = .5f,
48                velocityToOpacity = .5f,
49                tiltToRotation = .5f,
50            ),
51            pressure = 1f,
52            tiltX = 0f,
53            tiltY = 1f,
54            velocityPxPerSecond = 1_200f,
55            random01 = 0.5f,
56            out = result,
57        )
58
59        assertEquals(7.5f, result.radius, .0001f)
60        assertEquals(.75f, result.coverage, .0001f)
61        assertEquals(PI.toFloat() * .25f, result.rotationRad, .0001f)
62    }
63
64    @Test
65    fun resolve_usesProvidedRandomValueDeterministically() {
66        val settings = BrushSettings(
67            baseRadiusPx = 10f,
68            pressureToSize = false,
69            pressureToOpacity = false,
70            sizeJitter = .2f,
71            opacityJitter = .2f,
72            scatter = .5f,
73        )
74        val first = ResolvedDab()
75        val repeated = ResolvedDab()
76        val other = ResolvedDab()
77
78        BrushDynamics.resolve(settings, 1f, 0f, 0f, 0f, .75f, first)
79        BrushDynamics.resolve(settings, 1f, 0f, 0f, 0f, .75f, repeated)
80        BrushDynamics.resolve(settings, 1f, 0f, 0f, 0f, .25f, other)
81
82        assertEquals(first.radius, repeated.radius, 0f)
83        assertEquals(first.coverage, repeated.coverage, 0f)
84        assertEquals(first.scatterX, repeated.scatterX, 0f)
85        assertEquals(first.scatterY, repeated.scatterY, 0f)
86        assertEquals(first.radius, other.radius, 0.0001f)
87    }
88 }

```

137. app/src/test/java/com/wetinknext/engine/brush/BrushLibraryTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertFalse
5 import org.junit.Assert.assertTrue
6 import org.junit.Test
7
8 class BrushLibraryTest {
9     @Test
10     fun penPresetDisablesBothTapers() {
11         assertEquals(0f, BrushLibrary.pen.settings.ribbon.taperStartPx, 0f)
12         assertEquals(0f, BrushLibrary.pen.settings.ribbon.taperEndPx, 0f)
13     }
14
15     @Test
16     fun airbrushIsSoftLowFlowStampInPaintCategory() {
17         val settings = BrushLibrary.airbrush.settings
18
19         assertEquals("Paint", settings.category)
20         assertEquals(BrushRenderMode.STAMP, settings.renderMode)
21         assertEquals(80f, settings.baseRadiusPx, 0f)
22         assertEquals(0.18f, settings.opacity, 0f)
23         assertEquals(0.08f, settings.flow, 0f)
24         assertEquals(0.035f, settings.spacing, 0f)
25         assertTrue(settings.spacingUsesDiameter)
26         assertFalse(settings.pressureToSize)
27         assertTrue(settings.pressureToOpacity)
28         assertEquals(1.15f, settings.pressureGamma, 0f)
29         assertEquals(0f, settings.hardness, 0f)
30         assertEquals(3, settings.antiAliasLevel)
31         assertEquals(BlendPolicy.NORMAL_BUILDUP, settings.blendPolicy)
32         assertTrue(settings.useTempStrokeBuffer)
33         assertTrue(BrushLibrary.allCategories.first { it.name == "Paint" }.brushes.contains(BrushLibrary.airbrush))
34     }
35 }
```

138. app/src/test/java/com/wetinknext/engine/brush/BrushSerializationTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlinx.serialization.encodeToString
4 import org.junit.Assert.assertEquals
5 import org.junit.Test
6
7 class BrushSerializationTest {
8
9     @Test
10     fun settingsRoundTripKeepsRenderFields() {
11         val json = BrushRepository.defaultJson
12         val original = BrushSettings(
13             name = "Pencil 6B",
14             grainAssetPath = "asset:brush/pencil_6b.png",
15             grainScale = 5.5f,
16             textureDepth = .65f,
17             tiltToSize = .55f,
18             blendPolicy = BlendPolicy.NORMAL_BUILDUP,
19         )
20
21         val encoded = json.encodeToString(original)
22         val restored = json.decodeFromString<BrushSettings>(encoded)
23
24         assertEquals(original, restored)
25     }
26
27     @Test
28     fun presetRoundTrip() {
29         val repo = BrushRepository()
30         val original = BrushPreset(
31             id = "test-preset",
32             settings = BrushSettings(name = "Test Brush")
33         )
34
35         val encoded = repo.encode(original)
36         val restored = repo.decode(encoded)
37
38         assertEquals(original, restored)
39     }
40 }
```

139. app/src/test/java/com/wetinknext/engine/brush/BrushSettingsTest.kt

```

1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class BrushSettingsTest {
7     @Test
8     fun resolved_clampsBrushValues() {
9         val result = BrushSettings(
10             baseRadiusPx = -10f,
11             opacity = 3f,
12             flow = -1f,
13             spacing = 100f,
14             pressureGamma = 0f,
15             minSizeRatio = 4f,
16         ).resolved()
17
18         assertEquals(0.25f, result.baseRadiusPx, 0.001f)
19         assertEquals(1f, result.opacity, 0.001f)
20         assertEquals(0f, result.flow, 0.001f)
21         assertEquals(4f, result.spacing, 0.001f)
22         assertEquals(0.05f, result.pressureGamma, 0.001f)
23         assertEquals(1f, result.minSizeRatio, 0.001f)
24     }
25
26     @Test
27     fun strokeRenderModeMapsPersistedBlendPolicyExplicitly() {
28         assertEquals(
29             StrokeRenderMode.NORMAL_BUILDUP,
30             BrushSettings(blendPolicy = BlendPolicy.NORMAL_BUILDUP).strokeRenderMode,
31         )
32         assertEquals(
33             StrokeRenderMode.NON_BUILDUP,
34             BrushSettings(blendPolicy = BlendPolicy.NON_BUILDUP).strokeRenderMode,
35         )
36     }
37
38     @Test
39     fun ribbonUsesNonBuildupAfterCoveragePassIsAvailable() {
40         val ribbon = BrushSettings(
41             renderMode = BrushRenderMode.RIBBON,
42             blendPolicy = BlendPolicy.NON_BUILDUP,
43         )
44
45         assertEquals(StrokeRenderMode.NON_BUILDUP, ribbon.strokeRenderMode)
46         assertEquals(StrokeRenderMode.NON_BUILDUP, ribbon.effectiveStrokeRenderMode)
47     }
48 }

```

140. app/src/test/java/com/wetinknext/engine/brush/CapsuleEmitterTest.kt

```

1  package com.wetinknext.engine.brush
2
3  import com.wetinknext.engine.input.InputAction
4  import com.wetinknext.engine.input.InputBatch
5  import com.wetinknext.engine.input.PointerTool
6  import org.junit.Assert.assertTrue
7  import org.junit.Test
8
9  class CapsuleEmitterTest {
10
11     @Test
12     fun finishFlushesTheLastInterpolatedSection() {
13         val settings = BrushSettings(
14             renderMode = BrushRenderMode.RIBBON,
15             baseRadiusPx = 10f,
16             smoothing = 0.5f,
17             streamline = 0f,
18         )
19         val emitter = CapsuleEmitter(settings)
20         // We need a dummy renderer to receive segments
21         // Since we can't easily mock GL context here, we just use a small maxSegments
22         val renderer = CapsuleStrokeRenderer(maxSegments = 128)
23
24         emitter.begin(batch(InputAction.DOWN, 0f, 0f, 0L), renderer)
25         emitter.append(batch(InputAction.MOVE, 100f, 0f, 16_000_000L), renderer)
26         emitter.append(batch(InputAction.MOVE, 200f, 30f, 32_000_000L), renderer)
27
28         val segmentsBeforeFinish = renderer.segmentCount
29         emitter.finish(renderer, cancel = false)
30         val segmentsAfterFinish = renderer.segmentCount
31
32         assertTrue("Should have some segments after append", segmentsBeforeFinish > 0)
33         assertTrue("Finish should have flushed more segments", segmentsAfterFinish > segmentsBeforeFinish)
34         assertTrue(emitter.hasStroke)
35     }
36
37     private fun batch(action: InputAction, x: Float, y: Float, t: Long) = InputBatch(8).apply {
38         begin(action)
39         addSample(x, y, .5f, 0f, 0f, 0f, t, 0, PointerTool.STYLUS, false)
40     }
41 }

```

141. app/src/test/java/com/wetinknext/engine/brush/CapsuleStrokeRendererTest.kt

```

1  package com.wetinknext.engine.brush
2
3  import org.junit.Assert.assertEquals
4  import org.junit.Assert.assertFalse
5  import org.junit.Assert.assertTrue
6  import org.junit.Test
7  import com.wetinknext.engine.gl.ShaderLib
8
9  class CapsuleStrokeRendererTest {
10
11     @Test
12     fun capsuleShaderAppliesFlowInsideTheStrokeTarget() {
13         assertTrue(ShaderLib.capsuleFragment.contains("uniform float uFlow"))
14         assertTrue(ShaderLib.capsuleFragment.contains("grainFactor * uFlow"))
15     }
16
17     @Test
18     fun initiallyEmpty() {
19         val renderer = CapsuleStrokeRenderer(maxSegments = 100)
20         assertTrue(renderer.isEmpty)
21         assertEquals(0, renderer.segmentCount)
22     }
23
24     @Test
25     fun addsSegmentsWithinCapacity() {
26         val renderer = CapsuleStrokeRenderer(maxSegments = 10)
27         renderer.beginStroke()
28
29         for (i in 0 until 10) {
30             val added = renderer.addSegment(0f, 0f, 5f, 1f, 10f, 10f, 5f, 1f)
31             assertTrue("Segment $i should be added", added)
32         }
33
34         assertEquals(10, renderer.segmentCount)
35         assertFalse(renderer.isEmpty)
36
37         val addedOverflow = renderer.addSegment(0f, 0f, 5f, 1f, 10f, 10f, 5f, 1f)
38         assertFalse("Overflow segment should not be added", addedOverflow)
39         assertEquals(1, renderer.overflowCount)
40     }
41
42     @Test
43     fun clearResetsStateButNotOverflow() {
44         val renderer = CapsuleStrokeRenderer(maxSegments = 10)
45         renderer.beginStroke()
46         renderer.addSegment(0f, 0f, 5f, 1f, 10f, 10f, 5f, 1f)
47
48         // Overflow it
49         for (i in 0 until 10) renderer.addSegment(0f, 0f, 1f, 1f, 1f, 1f, 1f, 1f)
50         assertEquals(1, renderer.overflowCount)
51
52         renderer.clearStrokeData()
53         assertTrue(renderer.isEmpty)
54         assertEquals(0, renderer.segmentCount)
55         assertEquals(1, renderer.overflowCount)
56     }
57 }

```

142. app/src/test/java/com/wetinknext/engine/brush/ColorSpacesTest.kt

```
1 package com.wetinknext.engine.brush
2 import org.junit.Assert.*
3 import org.junit.Test
4 class ColorSpacesTest { @Test fun blackAndWhiteAreStable(){val b=FloatArray(3);val w=FloatArray(3);ColorSpaces.srgb8ToLinear(
0xFF000000L,b);ColorSpaces.srgb8ToLinear(0xFFFFFFFFL,w);assertEquals(0f,b[0],.0001f);assertEquals(1f,w[0],.0001f)} @Test fun
roundTrip(){assertEquals(.25f,ColorSpaces.srgbToLinear(ColorSpaces.linearToSrgb(.25f)),.0001f)} }
```


143. app/src/test/java/com/wetinknext/engine/brush/DabBufferTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import com.wetinknext.engine.core.DirtyRect
4 import org.junit.Assert.*
5 import org.junit.Test
6
7 class DabBufferTest {
8     @Test fun overflowIsCounted(){val b=DabBuffer(2);assertTrue(b.add(0f,0f,1f,0f,1f,1f,1f));assertTrue(b.add(1f,1f,1f,0f,1f,
1f,1f));assertFalse(b.add(2f,2f,1f,0f,1f,1f,1f));assertEquals(1L,b.overflowCount)}
9     @Test fun clearResets(){val b=DabBuffer(1);b.add(0f,0f,1f,0f,1f,1f,1f);b.clear();assertEquals(0,b.count)}
10    @Test fun dirtyRectIncludesTextureMargin(){
11        val buffer = DabBuffer(1)
12        buffer.add(10f, 20f, 5f, 0f, 1f, 1f, 1f)
13        val rect = DirtyRect()
14        buffer.includeDirtyRect(rect, extraMargin = 4f)
15        assertEquals(1f, rect.left, 0.001f)
16        assertEquals(19f, rect.right, 0.001f)
17        assertEquals(11f, rect.top, 0.001f)
18        assertEquals(29f, rect.bottom, 0.001f)
19    }
20 }
```

144. app/src/test/java/com/wetinknext/engine/brush/DabBufferUploadTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class DabBufferUploadTest {
7     @Test
8     fun uploadRangePositionsOnlyTheRequestedPendingDabs() {
9         val buffer = DabBuffer(capacity = 4)
10        repeat(3) { index ->
11            buffer.add(index.toFloat(), 0f, 1f, 0f, 1f, 1f, 1f)
12        }
13
14        buffer.prepareForUpload(firstDab = 1, dabCount = 2)
15
16        assertEquals(DabBuffer.FLOATS_PER_DAB, buffer.floats.position())
17        assertEquals(DabBuffer.FLOATS_PER_DAB * 3, buffer.floats.limit())
18        buffer.finishUpload()
19    }
20 }
```

145. app/src/test/java/com/wetinknext/engine/brush/OneEuroFilterTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertFalse
5 import org.junit.Test
6
7 class OneEuroFilterTest {
8     @Test fun firstSamplePassesThrough() { val f=OneEuroFilter();val out=FloatArray(2);f.filter(0,10f,20f,out);assertEquals(
10f,out[0],.0001f);assertEquals(20f,out[1],.0001f) }
9     @Test fun samplesRemainFinite() { val f=OneEuroFilter();val out=FloatArray(2);f.filter(0,0f,0f,out);f.filter(8_000_000,
10f,5f,out);assertFalse(out[0].isNaN());assertFalse(out[1].isInfinite()) }
10 }
```

146. app/src/test/java/com/wetinknext/engine/brush/PressureFilterTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertTrue
4 import org.junit.Test
5
6 class PressureFilterTest {
7     @Test fun singleZeroPressureGlitchDoesNotCollapseWidth() {
8         val filter = PressureFilter()
9         filter.filter(0L, .6f)
10        val filtered = filter.filter(16_000_000L, 0f)
11        assertTrue("zero glitch must be ignored while stroke is active", filtered > .55f)
12    }
13
14    @Test fun realPressureChangeIsSmoothed() {
15        val filter = PressureFilter()
16        filter.filter(0L, .8f)
17        val filtered = filter.filter(16_000_000L, .2f)
18        assertTrue(filtered in .2f.. .8f)
19    }
20 }
```

147. app/src/test/java/com/wetinknext/engine/brush/RibbonGeometryBuilderTest.kt

```

1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertTrue
5 import org.junit.Test
6
7 class RibbonGeometryBuilderTest {
8     @Test
9     fun buildsWeldedBanksWithAaAndRoundCaps() {
10         val builder = RibbonGeometryBuilder()
11         builder.addPoint(0f, 0f, 8f, 1f)
12         builder.addPoint(50f, 0f, 8f, 1f)
13         builder.addPoint(50f, 50f, 8f, 1f)
14
15         val mesh = requireNotNull(builder.build(RibbonSettings(cap = RibbonCap.ROUND, join = RibbonJoin.ROUND, aaWidthPx =
16 1f)))
17
18         assertTrue(mesh.indices.isNotEmpty())
19         assertEquals(0, mesh.vertices.size % 3)
20         assertTrue("round caps and join must add more than strip triangles", mesh.indices.size / 3 > 12)
21         assertTrue(mesh.bounds[0] < 0f)
22         assertTrue(mesh.bounds[3] > 50f)
23     }
24
25     @Test
26     fun miterIsClampedAtSharpCorner() {
27         val builder = RibbonGeometryBuilder()
28         builder.addPoint(0f, 0f, 10f, 1f)
29         builder.addPoint(30f, 0f, 10f, 1f)
30         builder.addPoint(30.2f, 30f, 10f, 1f)
31         val mesh = requireNotNull(builder.build(RibbonSettings(join = RibbonJoin.MITER, miterLimit = 2f)))
32         val width = mesh.bounds[2] - mesh.bounds[0]
33         assertTrue("unbounded miter spike", width < 80f)
34     }
35 }

```

148. app/src/test/java/com/wetinknext/engine/brush/StabilizerTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertTrue
5 import org.junit.Test
6
7 class StabilizerTest {
8     @Test fun strengthZeroPassesRawThrough(){val s=Stabilizer();s.process(0,100f,200f);assertEquals(100f,s.x,.0001f);
assertEquals(200f,s.y,.0001f)}
9     @Test fun velocityIsPositiveAfterMovement(){val s=Stabilizer();s.process(0,0f,0f);s.process(1_000_000_000,100f,0f);
assertTrue(s.velocity>0f)}
10 }
```

149. app/src/test/java/com/wetinknext/engine/brush/StampEmitterTest.kt

```

1  package com.wetinknext.engine.brush
2
3  import com.wetinknext.engine.input.*
4  import org.junit.Assert.assertEquals
5  import org.junit.Test
6
7  class StampEmitterTest {
8      @Test fun movingStrokeHasNoExtraDownDab(){val e=StampEmitter(BrushSettings(baseRadiusPx=10f,spacing=1f,
spacingUsesDiameter=false,pressureToSize=false,pressureToOpacity=false));val d=DabBuffer(16);val down=InputBatch(1).apply{begin(
InputAction.DOWN);addSample(0f,0f,1f,0f,0f,0f,0,0,PointerTool.STYLUS,false));val move=InputBatch(1).apply{begin(InputAction.MOVE);
addSample(20f,0f,1f,0f,0f,0f,1,0,PointerTool.STYLUS,false));e.begin(down,d);e.append(move,d);e.finish(d,false);assertEquals(2,d.
count)}}
9
10     @Test fun tapProducesExactlyOneDab(){val e=StampEmitter(BrushSettings());val d=DabBuffer(16);val down=InputBatch(1).apply{
begin(InputAction.DOWN);addSample(5f,5f,1f,0f,0f,0f,0,0,PointerTool.STYLUS,false));e.begin(down,d);e.finish(d,false);assertEquals(1,
d.count)}}
11
12     @Test
13     fun activeStrokeKeepsItsSettingsAfterUiBrushChanges() {
14         val strokeSettings = BrushSettings(
15             baseRadiusPx = 5f,
16             spacing = 1f,
17             spacingUsesDiameter = false,
18             smoothing = 0f,
19             pressureToSize = false,
20             pressureToOpacity = false,
21         )
22         val emitter = StampEmitter(strokeSettings)
23         val dabs = DabBuffer(16)
24         val down = InputBatch(1).apply {
25             begin(InputAction.DOWN)
26             addSample(0f, 0f, 1f, 0f, 0f, 0f, 0L, 0, PointerTool.STYLUS, false)
27         }
28         val move = InputBatch(1).apply {
29             begin(InputAction.MOVE)
30             addSample(20f, 0f, 1f, 0f, 0f, 0f, 1L, 0, PointerTool.STYLUS, false)
31         }
32
33         emitter.begin(down, dabs, strokeSettings)
34         emitter.updateSettings(strokeSettings.copy(baseRadiusPx = 20f))
35         emitter.append(move, dabs)
36
37         assertEquals(5f, dabs.floats.get(2), 0.001f)
38     }
39 }

```

150. app/src/test/java/com/wetinknext/engine/core/DirtyRectTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertTrue
5 import org.junit.Test
6
7 class DirtyRectTest {
8     @Test fun includesDabsAndConvertsToPixelBounds() {
9         val rect = DirtyRect()
10        rect.include(100f, 120f, 10f)
11        rect.include(150f, 140f, 5f)
12        val pixels = IntArray(4)
13        rect.toPixelBounds(pixels)
14        assertEquals(intArrayOf(90, 110, 155, 145), pixels)
15    }
16
17    @Test fun clampCanMakeRectEmpty() {
18        val rect = DirtyRect()
19        rect.include(-100f, -100f, -10f)
20        rect.clamp(100f, 100f)
21        assertTrue(rect.isEmpty)
22    }
23
24    @Test fun clearResetsRect() {
25        val rect = DirtyRect()
26        rect.include(0f, 0f, 10f)
27        rect.clear()
28        assertTrue(rect.isEmpty)
29    }
30 }
```


151. app/src/test/java/com/wetinknext/engine/core/DocumentSessionTest.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.domain.document.ProjectDocument
4 import com.wetinknext.engine.canvas.LayerStack
5 import com.wetinknext.engine.undo.UndoManager
6 import org.junit.Assert.assertEquals
7 import org.junit.Assert.assertFalse
8 import org.junit.Assert.assertTrue
9 import org.junit.Test
10
11 class DocumentSessionTest {
12     @Test
13     fun committedLayersRemainDirtyUntilTheirSaveIsAcknowledged() {
14         val session = DocumentSession(
15             ProjectDocument.newUntitled(id = "session-test"),
16             LayerStack(),
17             UndoManager(),
18             emptyMap(),
19         )
20
21         session.markLayerDirty(2L)
22         session.markLayerDirty(2L)
23
24         assertEquals(setOf(2L), session.dirtyLayerIds)
25         assertTrue(session.projectDirty)
26
27         session.markSaved(setOf(2L))
28
29         assertTrue(session.dirtyLayerIds.isEmpty())
30         assertFalse(session.projectDirty)
31     }
32 }
```

152. app/src/test/java/com/wetinknext/engine/core/LatestValueDeliveryTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class LatestValueDeliveryTest {
7     @Test
8     fun captureBeforeListenerIsDeliveredAfterListenerIsInstalled() {
9         val delivery = LatestValueDelivery<String>()
10        val delivered = mutableListOf<String>()
11
12        delivery.offer("first-thumbnail")
13        delivery.dispatchTo(null)
14        delivery.dispatchTo(delivered::add)
15
16        assertEquals(listOf("first-thumbnail"), delivered)
17    }
18
19    @Test
20    fun newestCaptureReplacesAnUndeliveredOlderCapture() {
21        val delivery = LatestValueDelivery<String>()
22        val delivered = mutableListOf<String>()
23
24        delivery.offer("outdated")
25        delivery.offer("current")
26        delivery.dispatchTo(delivered::add)
27
28        assertEquals(listOf("current"), delivered)
29    }
30 }
```

153. app/src/test/java/com/wetinknext/engine/core/LayerTileCodecTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertEquals
5 import org.junit.Assert.assertNull
6 import org.junit.Test
7
8 class LayerTileCodecTest {
9     @Test
10     fun roundTripPreservesRgba8LayerPayload() {
11         val pixels = byteArrayOf(1, 2, 3, 4, 5, 6, 7, 8)
12         val decoded = LayerTileCodec.decode(LayerTileCodec.encode(2, 1, 4, pixels))
13
14         checkNotNull(decoded)
15         assertEquals(2, decoded.width)
16         assertEquals(1, decoded.height)
17         assertEquals(4, decoded.bytesPerPixel)
18         assertEquals(pixels, decoded.pixels)
19     }
20
21     @Test
22     fun emptyPayloadMeansNewTransparentLayer() {
23         assertNull(LayerTileCodec.decode(ByteArray(0)))
24     }
25 }
```

154. app/src/test/java/com/wetinknext/engine/core/PersistentLayerTilesTest.kt

```

1  package com.wetinknext.engine.persistence
2
3  import com.wetinknext.engine.undo.TileCoord
4  import com.wetinknext.engine.undo.RawTileSnapshot
5  import java.nio.ByteBuffer
6  import java.nio.ByteOrder
7  import org.junit.Assert.assertArrayEquals
8  import org.junit.Assert.assertEquals
9  import org.junit.Assert.assertSame
10 import org.junit.Assert.assertTrue
11 import org.junit.Test
12
13 class PersistentLayerTilesTest {
14     @Test
15     fun ownedSnapshotRetainsCapturedBytesWithoutCopy() {
16         val pixels = ByteArray(16)
17         val raw = RawTileSnapshot(
18             coord = TileCoord(0, 0),
19             pixelLeft = 0,
20             pixelTop = 0,
21             pixelWidth = 2,
22             pixelHeight = 2,
23             bytesPerPixel = 4,
24             rawBytes = pixels,
25         )
26
27         val retained = PersistentLayerTiles.fromRawOwned(raw)
28
29         assertEquals(pixels, retained.bytes)
30     }
31
32     @Test
33     fun compressedPayloadRoundTripsTilePixels() {
34         val pixels = ByteArray(256 * 256 * 4)
35         val tile = PersistentLayerTiles.Tile(
36             coord = TileCoord(1, 2),
37             pixelLeft = 256,
38             pixelTop = 512,
39             pixelWidth = 256,
40             pixelHeight = 256,
41             bytesPerPixel = 4,
42             bytes = pixels,
43         )
44
45         val payload = PersistentLayerTiles.encode(listOf(tile))
46         val restored = PersistentLayerTiles.decode(payload).single()
47
48         assertTrue("transparent tile should compress", payload.size < pixels.size / 10)
49         assertEquals(tile.coord, restored.coord)
50         assertEquals(pixels, restored.bytes)
51     }
52
53     @Test
54     fun versionOnePayloadRemainsReadableForMigration() {
55         val pixels = ByteArray(16) { it.toByte() }
56         val payload = ByteBuffer.allocate(12 + 32 + pixels.size)
57             .order(ByteOrder.BIG_ENDIAN)
58             .putInt(0x57544C53)
59             .putInt(1)
60             .putInt(1)
61             .putInt(0).putInt(0)
62             .putInt(0).putInt(0)
63             .putInt(2).putInt(2)
64             .putInt(4).putInt(pixels.size)
65             .put(pixels)
66             .array()
67
68         val restored = PersistentLayerTiles.decode(payload).single()
69
70         assertEquals(pixels, restored.bytes)
71     }
72 }

```

155. app/src/test/java/com/wetinknext/engine/core/UndoCompressionPipelineTest.kt

```

1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.undo.IdentityTileCompressor
4 import com.wetinknext.engine.undo.RawTileSnapshot
5 import com.wetinknext.engine.undo.TileCompressor
6 import com.wetinknext.engine.undo.TileCoord
7 import com.wetinknext.engine.undo.UndoManager
8 import org.junit.Assert.assertEquals
9 import org.junit.Assert.assertFalse
10 import org.junit.Assert.assertTrue
11 import org.junit.Test
12 import java.util.concurrent.CountDownLatch
13 import java.util.concurrent.Executors
14 import java.util.concurrent.TimeUnit
15 import java.util.concurrent.atomic.AtomicInteger
16
17 class UndoCompressionPipelineTest {
18     @Test
19     fun synchronousFallbackUsesTheSameOrderedHistoryPath() {
20         val manager = UndoManager()
21         val pipeline = UndoCompressionPipeline(
22             requestRender = {},
23             compressor = IdentityTileCompressor,
24             maxPendingJobs = 1,
25         )
26
27         pipeline.enqueue(
28             undoManager = manager,
29             layerId = 7L,
30             beforeRaw = listOf(raw(byteArrayOf(1, 2, 3, 4))),
31             afterRaw = listOf(raw(byteArrayOf(5, 6, 7, 8))),
32         )
33
34         assertEquals(0, pipeline.pendingCount)
35         assertEquals(1, manager.undoCount)
36         assertEquals(0, manager.redoCount)
37         assertTrue(manager.canUndo)
38         pipeline.shutdown()
39     }
40
41     @Test
42     fun laterWorkerResultWaitsForTheEarlierSequence() {
43         val firstCompressionStarted = CountDownLatch(1)
44         val allowFirstCompression = CountDownLatch(1)
45         val secondResultReady = CountDownLatch(1)
46         val allResultsReady = CountDownLatch(2)
47         val resultCount = AtomicInteger(0)
48         val executor = Executors.newFixedThreadPool(2)
49         val compressor = object : TileCompressor {
50             override fun compress(data: ByteArray): ByteArray {
51                 if (data.first() == 0.toByte()) {
52                     firstCompressionStarted.countDown()
53                     check(allowFirstCompression.await(2, TimeUnit.SECONDS))
54                 }
55                 return data
56             }
57
58             override fun decompress(data: ByteArray, expectedSize: Int): ByteArray = data
59         }
60         val manager = UndoManager()
61         val pipeline = UndoCompressionPipeline(
62             requestRender = {
63                 if (resultCount.incrementAndGet() == 1) secondResultReady.countDown()
64                 allResultsReady.countDown()
65             },
66             compressor = compressor,
67             executorFactory = { executor },
68         )
69
70         pipeline.enqueue(
71             undoManager = manager,
72             layerId = 1L,
73             beforeRaw = listOf(raw(byteArrayOf(0, 0, 0, 0))),
74             afterRaw = listOf(raw(byteArrayOf(2, 0, 0, 0))),
75         )
76         assertTrue(firstCompressionStarted.await(1, TimeUnit.SECONDS))
77
78         pipeline.enqueue(
79             undoManager = manager,
80             layerId = 2L,
81             beforeRaw = listOf(raw(byteArrayOf(1, 0, 0, 0))),
82             afterRaw = listOf(raw(byteArrayOf(3, 0, 0, 0))),
83         )
84         assertTrue(secondResultReady.await(1, TimeUnit.SECONDS))
85
86         assertFalse(pipeline.process(manager))
87         assertEquals(0, manager.undoCount)
88         assertEquals(2, pipeline.pendingCount)
89
90         allowFirstCompression.countDown()

```

```

91         assertTrue(allResultsReady.await(1, TimeUnit.SECONDS))
92
93         assertTrue(pipeline.process(manager))
94         assertEquals(2, manager.undoCount)
95         assertEquals(0, pipeline.pendingCount)
96         pipeline.shutdown()
97     }
98
99     private fun raw(bytes: ByteArray) = RawTileSnapshot(
100         coord = TileCoord(0, 0),
101         pixelLeft = 0,
102         pixelTop = 0,
103         pixelWidth = 1,
104         pixelHeight = 1,
105         bytesPerPixel = 4,
106         rawBytes = bytes,
107     )
108 }

```

156. app/src/test/java/com/wetinknext/engine/core/UndoHistoryTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertFalse
5 import org.junit.Assert.assertTrue
6 import org.junit.Test
7
8 class UndoHistoryTest {
9     @Test
10     fun staleResultIsRejected() {
11         assertFalse(isUndoResultCurrent(resultEpoch = 3L, currentEpoch = 4L))
12     }
13
14     @Test
15     fun failedCompressionDoesNotLeavePendingCountLocked() {
16         var pending = 1
17         val result: UndoJobResult =
18             UndoJobResult.Failed(epoch = 1L, sequence = 0L, error = IllegalStateException())
19
20         pending = (pending - 1).coerceAtLeast(0)
21
22         assertEquals(0, pending)
23         assertTrue(result is UndoJobResult.Failed)
24     }
25 }
```

157. app/src/test/java/com/wetinknext/engine/core/ViewTransformFboTest.kt

```
1 package com.wetinknext.engine.core
2 import org.junit.Assert.assertEquals
3 import org.junit.Test
4 class ViewTransformFboTest { @Test fun orthoValues(){val m=FloatArray(16);ViewTransform.buildCanvasToFbo(100f,200f,m);
assertEquals(.02f,m[0],.0001f);assertEquals(.01f,m[5],.0001f);assertEquals(-1f,m[12],.0001f);assertEquals(-1f,m[13],.0001f)} }
```


158. app/src/test/java/com/wetinknext/engine/core/ViewTransformTest.kt

```

1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class ViewTransformTest {
7     @Test fun canvasAndScreenCoordinatesAreInverse() {
8         val transform = ViewTransform(120f, 80f, 2.5f, 0.35f, true)
9         val screen = FloatArray(2)
10        val canvas = FloatArray(2)
11        transform.canvasToScreen(240f, 310f, screen)
12        transform.screenToCanvas(screen[0], screen[1], canvas)
13        assertEquals(240f, canvas[0], 0.001f)
14        assertEquals(310f, canvas[1], 0.001f)
15    }
16
17    @Test fun zoomAroundAnchorKeepsAnchorStable() {
18        val initial = ViewTransform(translateX = 20f, translateY = 30f)
19        val before = FloatArray(2)
20        val after = FloatArray(2)
21        initial.screenToCanvas(400f, 300f, before)
22        initial.zoomAround(400f, 300f, 3f).screenToCanvas(400f, 300f, after)
23        assertEquals(before[0], after[0], 0.001f)
24        assertEquals(before[1], after[1], 0.001f)
25    }
26
27    @Test fun gestureDeltaKeepsTheAnchorStableWithoutPan() {
28        val initial = ViewTransform(translateX = 30f, translateY = -10f, scale = 1.4f, rotationRad = .2f)
29        val before = FloatArray(2)
30        val after = FloatArray(2)
31        initial.screenToCanvas(400f, 300f, before)
32        initial.applyGestureDelta(
33            panX = 0f,
34            panY = 0f,
35            anchorX = 400f,
36            anchorY = 300f,
37            zoomFactor = 1.8f,
38            rotationDelta = .3f,
39        ).screenToCanvas(400f, 300f, after)
40        assertEquals(before[0], after[0], .001f)
41        assertEquals(before[1], after[1], .001f)
42    }
43 }

```

159. app/src/test/java/com/wetinknext/engine/thumbnail/ThumbnailPreviewFilesTest.kt

```
1 package com.wetinknext.engine.thumbnail
2
3 import java.nio.file.Files
4 import org.junit.Assert.assertArrayEquals
5 import org.junit.Assert.assertEquals
6 import org.junit.Test
7
8 class ThumbnailPreviewFilesTest {
9     @Test
10     fun restoredLayerPreviewKeepsItsBytesAndStablePath() {
11         val directory = Files.createTempDirectory("wetink-preview-test").toFile()
12         try {
13             val bytes = arrayOf(0x52, 0x49, 0x46, 0x46, 0x01)
14
15             val restored = ThumbnailPreviewFiles.restoreLayerPreviews(
16                 outputDirectory = directory,
17                 previews = mapOf(42L to bytes),
18             )
19
20             val preview = requireNotNull(restored[42L])
21             assertEquals("42.webp", preview.name)
22             assertArrayEquals(bytes, preview.readBytes())
23         } finally {
24             directory.deleteRecursively()
25         }
26     }
27 }
```

160. app/src/test/java/com/wetinknext/engine/undo/TileCoordinateContractTest.kt

```
1 package com.wetinknext.engine.undo
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class TileCoordinateContractTest {
7     @Test
8     fun topCanvasTileStartsAtBottomGlRegion() {
9         val canvasTop = 0
10        val glBottom = canvasTop
11        assertEquals(0, glBottom)
12    }
13
14    @Test
15    fun bottomCanvasTileUsesItsCanvasYAsGlY() {
16        val canvasTop = 1792
17        val glBottom = canvasTop
18        assertEquals(1792, glBottom)
19    }
20 }
```

161. app/src/test/java/com/wetinknext/engine/undo/TileSnapshotTest.kt

```
1 package com.wetinknext.engine.undo
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertEquals
5 import org.junit.Test
6
7 class TileSnapshotTest {
8     @Test
9     fun rawIdentityTileKeepsItsExactBytes() {
10         val bytes = byteArrayOf(0, 10, 20, 30, 40, 50, 60, 70)
11         val tile = TileSnapshot(
12             coord = TileCoord(2, 3),
13             pixelLeft = 512,
14             pixelTop = 768,
15             pixelWidth = 1,
16             pixelHeight = 2,
17             bytesPerPixel = TileSnapshot.BYTES_PER_PIXEL_RGBA8,
18             storedData = bytes,
19         )
20
21         assertEquals(bytes.size, tile.memorySize)
22         assertEquals(bytes.size, tile.rawSize)
23         assertEquals(bytes, tile.decompress())
24     }
25
26     @Test
27     fun tileSizeIsTheSpecified256Pixels() {
28         assertEquals(256, TileSnapshot.TILE_SIZE)
29     }
30 }
```

162. app/src/test/java/com/wetinknext/engine/undo/UndoManagerTest.kt

```

1 package com.wetinknext.engine.undo
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertFalse
5 import org.junit.Assert.assertNotNull
6 import org.junit.Assert.assertSame
7 import org.junit.Assert.assertTrue
8 import org.junit.Test
9
10 class UndoManagerTest {
11     @Test
12     fun undoAndRedoMoveTheSameEntryBetweenStacks() {
13         val manager = UndoManager()
14         val entry = entry(1L)
15         manager.push(entry)
16
17         assertTrue(manager.canUndo)
18         assertFalse(manager.canRedo)
19         assertSame(entry, manager.peekUndo())
20         assertTrue(manager.commitUndo(entry))
21         assertFalse(manager.canUndo)
22         assertTrue(manager.canRedo)
23         assertSame(entry, manager.peekRedo())
24         assertTrue(manager.commitRedo(entry))
25         assertTrue(manager.canUndo)
26         assertFalse(manager.canRedo)
27     }
28
29     @Test
30     fun aNewEntryClearsRedoHistory() {
31         val manager = UndoManager()
32         val first = entry(1L)
33         manager.push(first)
34         val undo = manager.peekUndo()
35         assertNotNull(undo)
36         assertTrue(manager.commitUndo(checkNotNull(undo)))
37         assertTrue(manager.canRedo)
38
39         manager.push(entry(1L))
40
41         assertFalse(manager.canRedo)
42         assertEquals(1, manager.undoCount)
43         assertEquals(0, manager.redoCount)
44     }
45
46     @Test
47     fun stepLimitEvictsTheOldestHistoryEntry() {
48         val manager = UndoManager(maxSteps = 2)
49         manager.push(entry(1L))
50         manager.push(entry(2L))
51         manager.push(entry(3L))
52
53         assertEquals(2, manager.undoCount)
54     }
55
56     @Test
57     fun rejectedCommitLeavesUndoHistoryUntouched() {
58         val manager = UndoManager()
59         val stored = entry(1L)
60         manager.push(stored)
61
62         assertFalse(manager.commitUndo(entry(2L)))
63         assertSame(stored, manager.peekUndo())
64         assertTrue(manager.canUndo)
65         assertFalse(manager.canRedo)
66     }
67
68     @Test
69     fun removingLayerPurgesOnlyThatLayersUndoAndRedoEntries() {
70         val manager = UndoManager()
71         val firstLayer = entry(1L)
72         val secondLayer = entry(2L)
73         manager.push(firstLayer)
74         manager.push(secondLayer)
75         assertTrue(manager.commitUndo(secondLayer))
76
77         manager.removeEntriesForLayer(1L)
78
79         assertFalse(manager.canUndo)
80         assertTrue(manager.canRedo)
81         assertSame(secondLayer, manager.peekRedo())
82
83         manager.removeEntriesForLayer(2L)
84
85         assertFalse(manager.canRedo)
86     }
87
88     @Test
89     fun entryKeepsItsOperationMetadata() {
90         val entry = UndoEntry(

```

```

91         layerId = 1L,
92         beforeTiles = emptyList(),
93         afterTiles = emptyList(),
94         operation = UndoOperationType.CLEAR_LAYER,
95         tag = "clear_layer",
96     )
97
98     assertEquals(UndoOperationType.CLEAR_LAYER, entry.operation)
99     assertEquals("clear_layer", entry.tag)
100 }
101
102 @Test
103 fun removingLayerEntriesDoesNotBlockOtherHistory() {
104     val manager = UndoManager()
105     val removedLayerEntry = entry(layerId = 10L)
106     val validEntry = entry(layerId = 20L)
107     manager.push(removedLayerEntry)
108     manager.push(validEntry)
109
110     manager.removeEntriesForLayer(10L)
111
112     assertTrue(manager.canUndo)
113     assertEquals(validEntry, manager.peekUndo())
114 }
115
116 @Test
117 fun fiveEntriesCanBeCommittedInOrder() {
118     val manager = UndoManager()
119     repeat(5) { index -> manager.push(entry(layerId = index.toLong())) }
120
121     assertEquals(5, manager.undoCount)
122     repeat(5) {
123         val entry = manager.peekUndo()
124         assertNotNull(entry)
125         assertTrue(manager.commitUndo(checkNotNull(entry)))
126     }
127
128     assertEquals(0, manager.undoCount)
129     assertEquals(5, manager.redoCount)
130 }
131
132 private fun entry(layerId: Long): UndoEntry = UndoEntry(
133     layerId = layerId,
134     beforeTiles = listOf(tile(0)),
135     afterTiles = listOf(tile(1)),
136 )
137
138 private fun tile(x: Int): TileSnapshot = TileSnapshot(
139     coord = TileCoord(x, 0),
140     pixelLeft = x,
141     pixelTop = 0,
142     pixelWidth = 1,
143     pixelHeight = 1,
144     bytesPerPixel = TileSnapshot.BYTES_PER_PIXEL_RGBA8,
145     storedData = arrayOf(1, 2, 3, 4),
146 )
147 }

```

163. build.gradle.kts

```
1 plugins {  
2     id("com.android.application") version "8.6.1" apply false  
3     id("org.jetbrains.kotlin.android") version "2.0.21" apply false  
4     id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false  
5     id("org.jetbrains.kotlin.plugin.serialization") version "2.0.21" apply false  
6 }
```

164. gradle.properties

```
1 org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
2 android.useAndroidX=true
3 android.overridePathCheck=true
4 kotlin.code.style=official
5 org.gradle.daemon=true
6 org.gradle.caching=true
```


165. gradle/wrapper/gradle-wrapper.properties

```
1 distributionBase=GRADLE_USER_HOME
2 distributionPath=wrapper/dists
3 distributionUrl=https\://services.gradle.org/distributions/gradle-8.7-bin.zip
4 networkTimeout=10000
5 validateDistributionUrl=true
6 zipStoreBase=GRADLE_USER_HOME
7 zipStorePath=wrapper/dists
```

166. gradlew

```
1 #!/bin/sh
2
3 APP_HOME=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd -P) || exit 1
4 JAVA_CMD="{JAVA_HOME:+$JAVA_HOME/bin/}java"
5 exec "$JAVA_CMD" -classpath "$APP_HOME/gradle/wrapper/gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain "$@"
```

167. gradlew.bat

```
1 @rem Minimal Gradle startup script for Windows.
2 @echo off
3 setlocal
4 set APP_HOME=%~dp0
5 if defined JAVA_HOME goto javaHome
6 java -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
7 goto end
8 :javaHome
9 "%JAVA_HOME%\bin\java.exe" -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
10 :end
```

168. local.properties

```
1 sdk.dir=C:\\Users\\iick\\AppData\\Local\\Android\\Sdk
```

169. settings.gradle.kts

```
1  pluginManagement {
2      repositories {
3          google()
4          mavenCentral()
5          gradlePluginPortal()
6      }
7  }
8
9  dependencyResolutionManagement {
10     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
11     repositories {
12         google()
13         mavenCentral()
14     }
15 }
16
17 rootProject.name = "WetInk Next"
18 include(":app")
```